

Data Structures and Algorithms in C++

Algorithmic Reasoning, Efficient Data Representation, and Performance-Aware Problem Solving

Dr. Charles Dorner

2026

Data Structures and Algorithms in C++

1. [About this expanded volume](#)
 1. [Audience and progression](#)
 2. [Technical scope and evidence](#)
2. [How to use this book](#)
 1. [Read for a decision, then trace the mechanism](#)
 2. [Build the supplied examples](#)
 3. [Use the source and the tests together](#)
 4. [Practice without confusing failure with corruption](#)
 5. [Interpret the deliverables correctly](#)
3. [Chapter 1 — Developing Algorithmic Problem-Solving Skills](#)
 1. [1.1 Translating Problems into Algorithms](#)
 1. [1.1.1 Identifying Inputs, Outputs, and Constraints](#)
 2. [1.1.2 Expressing Solutions as Ordered Procedures](#)
 2. [1.2 Establishing Algorithm Correctness](#)
 1. [1.2.1 Defining Preconditions and Postconditions](#)
 2. [1.2.2 Reasoning with Invariants](#)
 3. [Worked example — Find the first maximum and compare against an independent library operation](#)
 1. [Requirement and design](#)
 2. [Trace the requirement into the test oracle](#)
 3. [Complete implementation](#)
 4. [Build, run, and inspect](#)
 5. [Evidence and interpretation](#)
 6. [Failure clinic — Correct value, wrong record](#)
 4. [Practice and implementation lab](#)
 1. [Review answers and solution notes](#)
 2. [Guided lab — Specify, mutate, and explain](#)
 3. [Primary reference notes](#)
4. [Chapter 2 — Measuring Time and Space Complexity](#)
 1. [2.1 Understanding Asymptotic Complexity](#)
 1. [2.1.1 Interpreting Big-O Notation](#)
 2. [2.1.2 Comparing Common Growth Rates](#)
 2. [2.2 Analyzing Practical Complexity](#)
 1. [2.2.1 Distinguishing Best, Average, and Worst Cases](#)
 2. [2.2.2 Understanding Amortized Complexity](#)
 3. [Worked example — Count search comparisons and simulate geometric growth](#)
 1. [Requirement and design](#)
 2. [Establish the counting boundary](#)
 3. [Complete implementation](#)
 4. [Build, run, and inspect](#)

- 5. [Evidence and interpretation](#)
 - 6. [Failure clinic — A valid measurement answering the wrong question](#)
- 4. [Practice and implementation lab](#)
 - 1. [Review answers and solution notes](#)
 - 2. [Guided lab — Build an operation ledger](#)
 - 3. [Primary reference notes](#)
- 5. [Chapter 3 — Understanding Arrays and Contiguous Dynamic Storage](#)
 - 1. [3.1 Understanding Contiguous Memory Layout](#)
 - 1. [3.1.1 Mapping Indices to Addresses](#)
 - 2. [3.1.2 Exploiting Cache Locality](#)
 - 2. [3.2 Implementing Dynamic Array Growth](#)
 - 1. [3.2.1 Allocating Expanded Storage](#)
 - 2. [3.2.2 Moving Elements During Reallocation](#)
 - 3. [Worked example — Implement and differentially test a growing integer sequence](#)
 - 1. [Requirement and design](#)
 - 2. [Inspect the representation before the loop](#)
 - 3. [Complete implementation](#)
 - 4. [Build, run, and inspect](#)
 - 5. [Evidence and interpretation](#)
 - 6. [Failure clinic — A count survived after its storage moved](#)
 - 4. [Practice and implementation lab](#)
 - 1. [Review answers and solution notes](#)
 - 2. [Guided lab — Make state transitions observable](#)
 - 3. [Primary reference notes](#)
- 6. [Chapter 4 — Building Linked Data Structures](#)
 - 1. [4.1 Implementing Singly Linked Lists](#)
 - 1. [4.1.1 Creating and Connecting Nodes](#)
 - 2. [4.1.2 Inserting and Removing Elements](#)
 - 2. [4.2 Implementing Doubly Linked Lists](#)
 - 1. [4.2.1 Maintaining Previous and Next Links](#)
 - 2. [4.2.2 Traversing in Both Directions](#)
 - 3. [Worked example — Insert and erase nodes while preserving both traversal directions](#)
 - 1. [Requirement and design](#)
 - 2. [Draw every affected link](#)
 - 3. [Complete implementation](#)
 - 4. [Build, run, and inspect](#)
 - 5. [Evidence and interpretation](#)
 - 6. [Failure clinic — Forward output hid a broken reverse chain](#)
 - 4. [Practice and implementation lab](#)
 - 1. [Review answers and solution notes](#)
 - 2. [Guided lab — Validate operation sequences, not isolated calls](#)
 - 3. [Primary reference notes](#)
- 7. [Chapter 5 — Implementing Last-In, First-Out Structures](#)
 - 1. [5.1 Understanding Stack Behavior](#)
 - 1. [5.1.1 Performing Push and Pop Operations](#)
 - 2. [5.1.2 Examining the Top Element](#)

2. [5.2 Applying Stacks to Real Problems](#)
 1. [5.2.1 Evaluating Expressions](#)
 2. [5.2.2 Managing Traversal State](#)
3. [Worked example — Validate nested brackets and evaluate bounded postfix expressions](#)
 1. [Requirement and design](#)
 2. [Trace the pending state](#)
 3. [Complete implementation](#)
 4. [Build, run, and inspect](#)
 5. [Evidence and interpretation](#)
 6. [Failure clinic — The arithmetic was right for the wrong operands](#)
4. [Practice and implementation lab](#)
 1. [Review answers and solution notes](#)
 2. [Guided lab — Build a diagnostic evaluator](#)
 3. [Primary reference notes](#)
8. [Chapter 6 — Implementing Queues and Double-Ended Queues](#)
 1. [6.1 Understanding First-In, First-Out Behavior](#)
 1. [6.1.1 Enqueuing and Dequeuing Values](#)
 2. [6.1.2 Implementing Circular Buffers](#)
 2. [6.2 Using Deques for Flexible Access](#)
 1. [6.2.1 Adding Elements at Both Ends](#)
 2. [6.2.2 Removing Elements at Both Ends](#)
 3. [Worked example — Wrap a bounded FIFO and compare every operation with a reference queue](#)
 1. [Requirement and design](#)
 2. [Build a model with the same acceptance rule](#)
 3. [Complete implementation](#)
 4. [Build, run, and inspect](#)
 5. [Evidence and interpretation](#)
 6. [Failure clinic — Full and empty looked identical](#)
 4. [Practice and implementation lab](#)
 1. [Review answers and solution notes](#)
 2. [Guided lab — Exhaust small histories](#)
 3. [Primary reference notes](#)
9. [Chapter 7 — Implementing Hash Tables](#)
 1. [7.1 Understanding Hash Functions](#)
 1. [7.1.1 Mapping Keys to Buckets](#)
 2. [7.1.2 Evaluating Hash Distribution](#)
 2. [7.2 Resolving Hash Collisions](#)
 1. [7.2.1 Implementing Separate Chaining](#)
 2. [7.2.2 Implementing Open Addressing](#)
 3. [Worked example — Compare two collision strategies through updates and deletions](#)
 1. [Requirement and design](#)
 2. [Follow a delete-update chain](#)
 3. [Complete implementation](#)
 4. [Build, run, and inspect](#)
 5. [Evidence and interpretation](#)

6. [Failure clinic — Deletion made an unrelated key disappear](#)
4. [Practice and implementation lab](#)
 1. [Review answers and solution notes](#)
 2. [Guided lab — Force collisions and test the model](#)
 3. [Primary reference notes](#)
10. [Chapter 8 — Organizing Unique and Keyed Data](#)
 1. [8.1 Understanding Set Abstractions](#)
 1. [8.1.1 Maintaining Unique Values](#)
 2. [8.1.2 Performing Set Operations](#)
 2. [8.2 Understanding Map Abstractions](#)
 1. [8.2.1 Associating Keys with Values](#)
 2. [8.2.2 Comparing Ordered and Hash-Based Maps](#)
 3. [Worked example — Compute set relationships and verify equivalent word-count maps](#)
 1. [Requirement and design](#)
 2. [Read the example as two different applications](#)
 3. [Complete implementation](#)
 4. [Build, run, and inspect](#)
 5. [Evidence and interpretation](#)
 6. [Failure clinic — A report changed the data it inspected](#)
 4. [Practice and implementation lab](#)
 1. [Review answers and solution notes](#)
 2. [Guided lab — Reconcile an inventory and produce a stable report](#)
 3. [Primary reference notes](#)
11. [Chapter 9 — Representing Hierarchical Data with Trees](#)
 1. [9.1 Understanding Tree Terminology](#)
 1. [9.1.1 Identifying Roots, Parents, Children, and Leaves](#)
 2. [9.1.2 Measuring Height and Depth](#)
 2. [9.2 Traversing Trees](#)
 1. [9.2.1 Performing Depth-First Traversals](#)
 2. [9.2.2 Performing Breadth-First Traversals](#)
 3. [Worked example — Traverse one hierarchy in three meaningful orders](#)
 1. [Requirement and design](#)
 2. [Make node identity and child order visible](#)
 3. [Complete implementation](#)
 4. [Build, run, and inspect](#)
 5. [Evidence and interpretation](#)
 6. [Failure clinic — The output was complete but in the wrong order](#)
 4. [Practice and implementation lab](#)
 1. [Review answers and solution notes](#)
 2. [Guided lab — Compare shapes and traversal strategies](#)
 3. [Primary reference notes](#)
12. [Chapter 10 — Building Binary Search Trees](#)
 1. [10.1 Maintaining the Binary Search Property](#)
 1. [10.1.1 Searching for Keys](#)
 2. [10.1.2 Inserting New Nodes](#)

2. [10.2 Removing Nodes Safely](#)
 1. [10.2.1 Removing Nodes with Zero or One Child](#)
 2. [10.2.2 Removing Nodes with Two Children](#)
3. [Worked example — Differentially test insertion, lookup, and every deletion shape](#)
 1. [Requirement and design](#)
 2. [Track the owning link through deletion](#)
 3. [Complete implementation](#)
 4. [Build, run, and inspect](#)
 5. [Evidence and interpretation](#)
 6. [Failure clinic — A local ordering check missed an ancestor violation](#)
4. [Practice and implementation lab](#)
 1. [Review answers and solution notes](#)
 2. [Guided lab — Validate whole-subtree ordering](#)
 3. [Primary reference notes](#)
13. [Chapter 11 — Keeping Search Trees Balanced](#)
 1. [11.1 Understanding Tree Imbalance](#)
 1. [11.1.1 Recognizing Degenerate Trees](#)
 2. [11.1.2 Measuring Balance](#)
 2. [11.2 Understanding Self-Balancing Trees](#)
 1. [11.2.1 Understanding AVL Rotations](#)
 2. [11.2.2 Understanding Red-Black Tree Properties](#)
 3. [Worked example — Keep an ordered insertion workload shallow](#)
 1. [Requirement and design](#)
 2. [Make each rotation branch unavoidable](#)
 3. [Complete implementation](#)
 4. [Build, run, and inspect](#)
 5. [Evidence and interpretation](#)
 6. [Failure clinic — The keys were right but the next rotation was wrong](#)
 4. [Practice and implementation lab](#)
 1. [Review answers and solution notes](#)
 2. [Guided lab — Validate topology, metadata, and order independently](#)
 3. [Primary reference notes](#)
14. [Chapter 12 — Prioritizing Data with Heaps](#)
 1. [12.1 Building Binary Heaps](#)
 1. [12.1.1 Maintaining Heap Properties](#)
 2. [12.1.2 Performing Heapify Operations](#)
 2. [12.2 Implementing Priority Queues](#)
 1. [12.2.1 Inserting Prioritized Values](#)
 2. [12.2.2 Removing Highest-Priority Values](#)
 3. [Worked example — Implement and differentially test a minimum heap](#)
 1. [Requirement and design](#)
 2. [Separate construction, mutation, and validation](#)
 3. [Complete implementation](#)
 4. [Build, run, and inspect](#)
 5. [Evidence and interpretation](#)
 6. [Failure clinic — The smaller sibling stayed below a larger parent](#)

4. [Practice and implementation lab](#)
 1. [Review answers and solution notes](#)
 2. [Guided lab — Validate a heap as a multiset and an order rule](#)
 3. [Primary reference notes](#)
15. [Chapter 13 — Representing Networks with Graphs](#)
 1. [13.1 Understanding Graph Models](#)
 1. [13.1.1 Distinguishing Directed and Undirected Graphs](#)
 2. [13.1.2 Distinguishing Weighted and Unweighted Graphs](#)
 2. [13.2 Choosing Graph Representations](#)
 1. [13.2.1 Building Adjacency Matrices](#)
 2. [13.2.2 Building Adjacency Lists](#)
 3. [Worked example — Preserve edge meaning across two representations](#)
 1. [Requirement and design](#)
 2. [Compare three different meanings of “missing”](#)
 3. [Complete implementation](#)
 4. [Build, run, and inspect](#)
 5. [Evidence and interpretation](#)
 6. [Failure clinic — A representation erased a valid edge](#)
 4. [Practice and implementation lab](#)
 1. [Review answers and solution notes](#)
 2. [Guided lab — Specify a graph before choosing storage](#)
 3. [Primary reference notes](#)
16. [Chapter 14 — Traversing and Analyzing Graphs](#)
 1. [14.1 Performing Breadth-First Search](#)
 1. [14.1.1 Traversing by Distance Layers](#)
 2. [14.1.2 Finding Unweighted Shortest Paths](#)
 2. [14.2 Performing Depth-First Search](#)
 1. [14.2.1 Traversing Recursive Graph Structure](#)
 2. [14.2.2 Detecting Cycles and Connected Components](#)
 3. [Worked example — Answer three distinct topology questions](#)
 1. [Requirement and design](#)
 2. [Keep three result contracts separate](#)
 3. [Complete implementation](#)
 4. [Build, run, and inspect](#)
 5. [Evidence and interpretation](#)
 6. [Failure clinic — Visited did not mean finished](#)
 4. [Practice and implementation lab](#)
 1. [Review answers and solution notes](#)
 2. [Guided lab — Produce and verify path witnesses](#)
 3. [Primary reference notes](#)
17. [Chapter 15 — Designing Efficient Search Algorithms](#)
 1. [15.1 Performing Linear Search](#)
 1. [15.1.1 Searching Unordered Data](#)
 2. [15.1.2 Understanding Linear Complexity](#)
 2. [15.2 Performing Binary Search](#)
 1. [15.2.1 Halving Ordered Search Spaces](#)

2. [15.2.2 Avoiding Boundary Errors](#)
3. [Worked example — Find first occurrences and insertion positions without boundary defects](#)
 1. [Requirement and design](#)
 2. [Use two independent answers for two different questions](#)
 3. [Complete implementation](#)
 4. [Build, run, and inspect](#)
 5. [Evidence and interpretation](#)
 6. [Failure clinic — A valid boundary was treated as an element](#)
4. [Practice and implementation lab](#)
 1. [Review answers and solution notes](#)
 2. [Guided lab — Search for a boundary, then derive membership](#)
 3. [Primary reference notes](#)
18. [Chapter 16 — Designing and Comparing Sorting Algorithms](#)
 1. [16.1 Understanding Elementary Sorting Algorithms](#)
 1. [16.1.1 Implementing Insertion and Selection Sort](#)
 2. [16.1.2 Understanding Quadratic Complexity](#)
 2. [16.2 Understanding Efficient Sorting Algorithms](#)
 1. [16.2.1 Implementing Merge Sort](#)
 2. [16.2.2 Implementing Quick Sort](#)
 3. [Worked example — Compare four sorting mechanisms against one contract](#)
 1. [Requirement and design](#)
 2. [Test one semantic contract across four mechanisms](#)
 3. [Complete implementation](#)
 4. [Build, run, and inspect](#)
 5. [Evidence and interpretation](#)
 6. [Failure clinic — Sorted output lost a value](#)
 4. [Practice and implementation lab](#)
 1. [Review answers and solution notes](#)
 2. [Guided lab — Verify order, preservation, and stability separately](#)
 3. [Primary reference notes](#)
19. [Chapter 17 — Solving Recursive and Backtracking Problems](#)
 1. [17.1 Designing Recursive Solutions](#)
 1. [17.1.1 Defining Base Cases](#)
 2. [17.1.2 Reducing Problems Recursively](#)
 2. [17.2 Exploring Search Spaces with Backtracking](#)
 1. [17.2.1 Making and Reversing Choices](#)
 2. [17.2.2 Pruning Impossible Solutions](#)
 3. [Worked example — Count legal placements with reversible state](#)
 1. [Requirement and design](#)
 2. [Read the search as a sequence of reversible states](#)
 3. [Complete implementation](#)
 4. [Build, run, and inspect](#)
 5. [Evidence and interpretation](#)
 6. [Failure clinic — A branch contaminates its sibling](#)

4. [Practice and implementation lab](#)
 1. [Review answers and solution notes](#)
 2. [Guided lab — Separate exploration from evidence](#)
 3. [Review and solution reasoning](#)
 4. [Primary reference notes](#)
20. [Chapter 18 — Solving Optimization Problems with Greedy Algorithms](#)
 1. [18.1 Understanding Greedy Choice](#)
 1. [18.1.1 Selecting Locally Optimal Decisions](#)
 2. [18.1.2 Recognizing Greedy-Solvable Problems](#)
 2. [18.2 Applying Greedy Techniques](#)
 1. [18.2.1 Solving Scheduling Problems](#)
 2. [18.2.2 Applying Greedy Graph Algorithms](#)
 3. [Worked example — Select a schedule and build a minimum-cost connection forest](#)
 1. [Requirement and design](#)
 2. [Make the certificate visible before optimizing](#)
 3. [Complete implementation](#)
 4. [Build, run, and inspect](#)
 5. [Evidence and interpretation](#)
 6. [Failure clinic — Solving the wrong optimization problem](#)
 4. [Practice and implementation lab](#)
 1. [Review answers and solution notes](#)
 2. [Guided lab — Challenge the greedy rule with exhaustive evidence](#)
 3. [Review and solution reasoning](#)
 4. [Primary reference notes](#)
21. [Chapter 19 — Solving Repeated Subproblems with Dynamic Programming](#)
 1. [19.1 Recognizing Dynamic-Programming Structure](#)
 1. [19.1.1 Identifying Overlapping Subproblems](#)
 2. [19.1.2 Identifying Optimal Substructure](#)
 2. [19.2 Implementing Dynamic Programming](#)
 1. [19.2.1 Solving Top-Down with Memoization](#)
 2. [19.2.2 Solving Bottom-Up with Tabulation](#)
 3. [Worked example — Compute an optimum and reconstruct evidence for it](#)
 1. [Requirement and design](#)
 2. [Derive a table before reading the implementation](#)
 3. [Complete implementation](#)
 4. [Build, run, and inspect](#)
 5. [Evidence and interpretation](#)
 6. [Failure clinic — A table hides an invalid state model](#)
 4. [Practice and implementation lab](#)
 1. [Review answers and solution notes](#)
 2. [Guided lab — Test the recurrence from three directions](#)
 3. [Review and solution reasoning](#)
 4. [Primary reference notes](#)
22. [Chapter 20 — Designing Algorithms for Real Hardware](#)
 1. [20.1 Understanding Cache-Aware Performance](#)
 1. [20.1.1 Improving Spatial Locality](#)

2. [20.1.2 Improving Temporal Locality](#)
2. [20.2 Measuring Algorithm Performance](#)
 1. [20.2.1 Benchmarking Competing Implementations](#)
 2. [20.2.2 Separating Algorithmic and Hardware Costs](#)
3. [Worked example — Investigate locality without turning timings into correctness tests](#)
 1. [Requirement and design](#)
 2. [Establish equality before comparing durations](#)
 3. [Complete implementation](#)
 4. [Build, run, and inspect](#)
 5. [Evidence and interpretation](#)
 6. [Failure clinic — A faster benchmark computes less work](#)
4. [Practice and implementation lab](#)
 1. [Review answers and solution notes](#)
 2. [Guided lab — Write a reproducible performance note](#)
 3. [Review and solution reasoning](#)
 4. [Primary reference notes](#)
23. [Appendix A — An Algorithmic Diagnostic Desk Reference](#)
 1. [Start with the observed disagreement](#)
 2. [Locate the broken layer](#)
 3. [Use operation histories, not only snapshots](#)
 4. [Distinguish missing results from exceptional failures](#)
 5. [Keep four forms of evidence separate](#)
 6. [Decide whether to keep a custom structure](#)
24. [Appendix B — Executable Invariant and Witness Laboratories](#)
 1. [Force a collision history rather than hoping for one](#)
 2. [Return a BFS witness and replace recursive DFS frames explicitly](#)
 3. [Make stability observable with record identities](#)
 4. [Compare a recurrence with an independent state-space search](#)
25. [Appendix C — HarborRoutes: An Integrated Algorithmic Application](#)
 1. [Requirement and deliberate boundaries](#)
 2. [Select the algorithm from the weight contract](#)
 3. [Define ownership and the commit point](#)
 1. [Public interface](#)
 2. [Parsing, validation, shortest-route calculation, and rendering](#)
 3. [Command-line boundary](#)
 4. [Validate a path independently of the priority queue](#)
 5. [Build, exercise, and interpret](#)
26. [Appendix D — Working Glossary and Acceptance Review](#)
 1. [Vocabulary for reasoning about algorithms](#)
 2. [Review an implementation before relying on it](#)
27. [Primary references and further reading](#)
 1. [C++20 working draft N4861 — Container adaptors \[container.adaptors\]](#)
 2. [C++20 working draft N4861 — Algorithms library \[algorithms\]](#)
 3. [Clang AddressSanitizer documentation](#)
 4. [C++20 working draft N4861 — Associative-container requirements \[associative.reqmts\]](#)
 5. [C++20 working draft N4861 — Binary-search operations \[alg.binary.search\]](#)

6. [C++20 working draft N4861 — Time library \[time\]](#)
7. [CMake build-system manual](#)
8. [C++20 working draft N4861 — Deque \[deque\]](#)
9. [C++20 working draft N4861 — Construction, destruction, and exception cleanup \[except.ctor\]](#)
10. [C++20 working draft N4861 — Numeric parsing with from_chars \[charconv.from.chars\]](#)
11. [C++20 working draft N4861 — Function declarations \[dcl.fct\]](#)
12. [C++20 working draft N4861 — Fundamental types \[basic.fundamental\]](#)
13. [GCC: C dialect and language-standard options](#)
14. [GCC: optimization options](#)
15. [GCC: options controlling the kind of output](#)
16. [C++20 working draft N4861 — Hash requirements \[unord.hash\]](#)
17. [C++20 working draft N4861 — General principles \[intro\]](#)
18. [C++20 working draft N4861 — Iterator operations \[iterator.operations\]](#)
19. [C++20 working draft N4861 — Lambda expressions \[expr.prim.lambda\]](#)
20. [C++20 working draft N4861 — Object lifetime \[basic.life\]](#)
21. [C++20 working draft N4861 — Numeric limits \[numeric.limits\]](#)
22. [C++20 working draft N4861 — List \[list\]](#)
23. [C++20 working draft N4861 — Minimum and maximum operations \[alg.min.max\]](#)
24. [MIT 6.006, Spring 2020: algorithm lecture directory \(further reading\)](#)
25. [Microsoft C++: selecting the language-standard version](#)
26. [C++20 working draft N4861 — Nonmodifying sequence operations \[alg.nonmodifying\]](#)
27. [C++20 working draft N4861 — Generalized numeric operations \[numeric.ops\]](#)
28. [C++20 working draft N4861 — Optional objects \[optional\]](#)
29. [C++20 working draft N4861 — Queue and priority-queue adaptors \[queue\]](#)
30. [C++20 working draft N4861 — Set operations on sorted structures \[alg.set.operations\]](#)
31. [C++20 working draft N4861 — Sorting and related operations \[alg.sorting\]](#)
32. [C++20 working draft N4861 — Tuples \[tuple\]](#)
33. [Clang UndefinedBehaviorSanitizer documentation](#)
34. [C++20 working draft N4861 — Unique-ownership pointers \[unique.ptr\]](#)
35. [C++20 working draft N4861 — Unordered associative requirements \[unord.req\]](#)
36. [C++20 working draft N4861 — Utility components \[utility\]](#)
37. [C++20 working draft N4861 — Vector \[vector\]](#)
38. [C++20 working draft N4861 — Vector capacity \[vector.capacity\]](#)
39. [C++20 working draft N4861 — Vector modifiers \[vector.modifiers\]](#)

About this expanded volume

Data Structures and Algorithms in C++

Algorithmic Reasoning, Efficient Data Representation, and Performance-Aware Problem Solving

Dr. Charles Dörner

Book III of *C++ From Beginner to Expert* · Expanded manuscript · 2026

A program that returns the expected answer on one example has demonstrated one observation. An algorithm becomes dependable when its input domain, state transitions, invariants, output guarantees, and resource costs can be explained together. This volume develops that discipline through complete C++ implementations rather than treating data structures as names to memorize or code to copy without understanding.

The original twenty-chapter curriculum is preserved, including all forty numbered sections and eighty numbered subsections. The expanded explanations connect abstract reasoning to object lifetime, ownership, failure handling, tests, and the memory behavior of actual programs. Every chapter retains a complete worked program and adds deeper traces, failure investigation, guided practice, and solution reasoning. The appendices supply additional executable laboratories and an integrated routing application that combines several of the book's techniques.

HarborWorks and its examples are fictional teaching scenarios. The data, operational requirements, and acceptance thresholds are invented for instruction. They do not describe an employer's systems or contain confidential infrastructure information. HarborRoutes, the supplemental application, operates on small local graph descriptions; it is not a live network control system or a production routing service.

Audience and progression

This book assumes the C++ fundamentals and resource-management concepts taught in Books I and II. You should be able to read functions, classes, standard containers, references, smart pointers, lambdas, and basic templates. The common language baseline is C++20. The book does not require competitive-programming experience or an advanced mathematics background, but it asks you to follow precise arguments and to distinguish examples from proofs.

Chapters 1 and 2 establish contracts, invariants, and complexity. Chapters 3–12 examine sequence, associative, hierarchical, and priority structures. Chapters 13–19 develop graph traversal, searching, sorting, recursion, greedy reasoning, and dynamic programming. Chapter 20 connects these abstractions to measurement on real hardware. The supplementary application is best read after graph traversal, heaps, and algorithmic correctness are familiar.

The custom containers deliberately have narrower contracts than their standard-library counterparts. An integer-only dynamic array is not a drop-in `std::vector`; a fixed open-addressing table is not a general `std::unordered_map`; a color-property checker is not a complete red-black tree implementation. These boundaries are stated where they matter. Standard containers remain the default practical choice unless a measured requirement justifies specialized storage.

Technical scope and evidence

The examples use standard C++20 and the standard library, with CMake and Python for building and fixture execution. The companion does not download third-party C++ dependencies. Language references point to the C++20 working draft N4861 rather than silently applying a future draft to the selected baseline. Compiler and build-tool documentation is separate because tool options and diagnostics can change independently of the language standard.

The algorithm explanations, proofs, scenarios, exercises, and added implementations are original teaching material. The primary reference directory supports standard-library and tool behavior and provides further reading. A library complexity guarantee, a mathematical proof for a custom algorithm, and a timing observation are different kinds of evidence. The prose keeps them separate rather than presenting one as a substitute for another.

`TEST_REPORT.md` records the configurations actually executed for this release and the checks actually performed on the EPUB and PDF. A successful compiler run does not establish correctness for all possible inputs. Sanitizers inspect exercised executions; they do not prove the absence of every lifetime, arithmetic, or logic defect. Portability provisions in CMake are not evidence that every listed platform was tested.

How to use this book

Read for a decision, then trace the mechanism

Begin each example by stating the required result and the accepted input domain. Then identify the representation and the invariant that gives its fields meaning. Trace one small success, one boundary, and one failure before studying performance. This sequence prevents a fast but incorrectly modeled implementation from becoming the baseline.

Read the text between code blocks. It explains assumptions that a short listing cannot express by syntax alone: whether duplicates are allowed, whether an index is borrowed from a mutable sequence, whether a graph is directed, whether a result is one of several optima, and what remains unchanged after an operation fails. Those decisions determine which tests are meaningful.

The recurring working rule is: **define the contract, preserve the invariant, verify the result, and measure only the work you intend to compare.** In a proof, identify initialization, preservation, and termination. In a test, identify setup, action, and observation. In a benchmark, identify preparation, timed work, and result consumption.

Build the supplied examples

From the extracted package root, configure an out-of-source build and execute its tests:

```
cmake -S source -B build/debug -DCMAKE_BUILD_TYPE=Debug
cmake --build build/debug --parallel 2
ctest --test-dir build/debug --output-on-failure
```

A recent compiler with the used C++20 features, CMake, and Python 3 are required. The README records the versions used for this release. On a multi-configuration generator, select the matching configuration with `--config Debug` for the build and `-C Debug` for CTest. The commands above do not require administrator privileges.

For a focused chapter, use its independent CMake project as shown after the worked listing, or run `python tools/run_chapter.py 15` to build and test Book III, Chapter 15. Read the helper's `--help` output before selecting compiler or sanitizer options. The expanded supplemental targets are built by the root source project.

Use the source and the tests together

The `source/book3/chNN` directories contain the chapter programs and fixtures. `source/supplemental` contains the additional laboratories and HarborRoutes. Shared checks remain active in optimized builds, unlike assertions that may disappear when `NDEBUG` is defined. A test executable can perform many internal checks while still constituting one CTest case; reports distinguish those quantities.

Code listings in the assembled manuscript are inserted from the actual source files. Their inclusion hashes are recorded during assembly. Edit authored prose under `authoring/`, code under `source/`, then regenerate rather than editing a printed listing and forgetting its executable counterpart. Preserve the supplied release logs before rerunning tools that write logs with the same names.

Practice without confusing failure with corruption

Use a separate experimental copy when deliberately breaking an invariant. Many mistakes can be demonstrated safely by returning a wrong value, rejecting a valid input, or omitting a necessary update. Do not invoke undefined behavior merely to create a dramatic example. A dangling reference does not become a legitimate experiment because it happened to print a familiar value once.

The fixtures use bounded local inputs. They do not open public network ports, modify system configuration, or require a GPU. The optional Chapter 20 benchmark consumes CPU and memory within its documented teaching size; do not treat it as a stress test or a production latency guarantee.

Interpret the deliverables correctly

The EPUB is reflowable and has no fixed page count. The PDF is a seven-by-ten-inch reading edition with linked contents and page numbers, not a printer-specific approval of trim, bleed, binding, or cover dimensions. The publishing scripts, styles, editable cover, complete manuscript, tests, and checksum manifest are included in the ZIP. No font files or hidden manuscript-generation service are required or supplied.

After completing a lab, write a brief engineering note explaining what the result establishes and what it does not. This habit turns a collection of programs into a reusable body of evidence. It is also the bridge from learning algorithms to making defensible engineering decisions about them.

Chapter 1 — Developing Algorithmic Problem-Solving Skills

The requirement says “return the largest reading,” but the caller actually needs the position of its first occurrence. A function returning the last occurrence is numerically plausible and still wrong. Algorithm design begins by making the question precise enough that correctness can be judged.

This volume builds data structures and algorithms as complete, testable C++ programs. The opening example finds the first maximum, handles empty input explicitly, and connects a loop invariant to both a proof and a reference-based test.

HarborWorks has retained a month of synthetic equipment readings. One colleague wants the largest value for a dashboard; another wants the earliest record that produced it; a third wants every record tied for that value. They can all say “find the maximum” while asking for different programs. The cost of implementing the wrong question is not repaired by a faster loop.

Keep three artifacts beside the source throughout this volume: a statement of the problem, a representation invariant, and an evidence record. The problem describes what a caller is entitled to receive. The invariant explains which internal states are legal. The evidence record distinguishes a proof argument, an executed test, and a performance observation. None of the three can stand in for the other two.

1.1 Translating Problems into Algorithms

An algorithm is an ordered procedure for transforming valid inputs into an intended result. Before selecting a technique, identify the input domain, output meaning, failure states, and constraints on size, values, and available resources.

A precise contract often simplifies the implementation more than an early optimization does. It distinguishes a legitimate edge case from an unspecified assumption.

1.1.1 Identifying Inputs, Outputs, and Constraints

The input is a finite vector of integers. The output is an optional index: absent for an empty vector, otherwise the earliest index whose value equals the maximum. The function does not mutate the input and does not return a borrowed pointer into it.

The word “earliest” determines the tie rule. The optional distinguishes absence from index zero, which is a valid position. The index type is `std::size_t`, matching the container’s size domain. These decisions remove ambiguity before the first loop is written and suggest immediate tests: empty, singleton, duplicates, and negative values.

Start by describing a small example in the language of the caller. For readings `[-7, -2, -2, -8]`, the first maximum is at index one, not index two. For an empty recording there is no answer, rather than

an imaginary reading of zero. For a single reading the answer is index zero regardless of its value. These examples expose decisions that a type signature alone does not explain.

The input domain includes more than an element type. Does the caller retain exclusive control of the vector while the function reads it? May the function allocate? Is the result needed before the next reading arrives? Is there a bound on retained observations? Here the algorithm assumes an already constructed, stable, finite vector and performs a synchronous read. It does not accept a concurrently changing stream. That boundary prevents a later claim that the same function is automatically safe for an acquisition thread.

An index is a description of a position in a particular sequence state. It does not own that sequence. Returning an integer avoids a dangling pointer from the function itself, but the integer can become semantically stale if the caller inserts at the front before using it. A result type solves one problem; it does not solve every lifetime and identity problem. When a result must survive reordering, return a stable record identifier or an owned copy under a different contract.

Distinguish a bound imposed for correctness from one imposed for practicality. An index must fit the container's indexing type; that is part of representability. A limit of ten thousand readings might instead protect response time or a service's memory budget. Document the reason and test the boundary. An unexplained small limit can become a permanent mystery in an otherwise correct algorithm.

Finally, write what must not change. The original values, order, and vector size remain unchanged. "Does not mutate input" is an observable requirement and can be checked by comparing a saved copy after the call. Such a check complements, rather than replaces, the `const` interface.

1.1.2 Expressing Solutions as Ordered Procedures

For nonempty input, initialize the best index to zero. Visit each later element. Replace the best index only when the new value is strictly greater than the current best. Return the final index.

This procedure separates initialization from the repeated step and avoids inventing a numeric sentinel such as zero or the minimum integer. A sentinel can fail when all values are negative or when a legitimate value equals the sentinel. Starting from an actual input element makes the candidate valid from the beginning.

A useful first procedure can be written without C++ syntax:

```
If there are no readings, report absence.  
Remember the position of the first reading.  
For each later position:  
    replace the remembered position only for a larger reading.  
Return the remembered position.
```

Every sentence has a corresponding reason. The first prevents access before a candidate exists. Remembering a position preserves both value access and the required output. Visiting later positions avoids comparing the first element with itself. Strict replacement enforces the earliest-tie rule. The last sentence returns the requested evidence, not just a related value.

Now trace an actual execution. With `[4, 1, 9, 9, 3]`, the remembered positions after processing prefixes of length one through five are `0, 0, 2, 2, 2`. The fourth reading equals the current

maximum, so it must not displace the earlier occurrence. A table of processed prefix, current input, previous candidate, and next candidate is often a better debugging instrument than printing the entire vector at every iteration.

Avoid implementing a more complicated procedure merely because its name sounds sophisticated. Sorting the readings and taking the last value spends additional work, can change the input, and loses the original position unless extra identity information is carried along. It solves a broader ordering problem when the caller requested only one selection. The simplest correct algorithm becomes the baseline against which later alternatives must justify their complexity.

Translate the procedure into code only after deciding what happens at the boundaries. Half-open ranges provide a consistent notation: $[0, i)$ contains exactly the elements already examined. The next element is at i , when i is smaller than the size. This same notation will appear in vector growth, binary search, sorting, and graph work queues.

An ordered procedure also needs an end condition. “Repeat until no better answer appears” is not a termination argument for arbitrary data. “Advance the index through a fixed finite sequence” is. Make the shrinking or advancing quantity visible before introducing recursion or a search tree.

Turning “Find the Largest Value” into a Linear Scan

Purpose. Algorithm design translates a verbal requirement into state, operations, invariants, and termination behavior.

```
#include <iostream>
#include <vector>
int maximum(const std::vector<int>& v){ int best=v.at(0); for(std::size_t
i=1;i<v.size();++i) if(v[i]>best) best=v[i]; return best; }
int main(){ std::cout<<maximum({4,9,2,7})<<'\n'; }
```

How it works.

- The problem requirement becomes a state variable `best` plus one comparison per remaining element.
- The precondition “nonempty input” is made visible by `at(0)`.
- The loop examines every candidate exactly once.

Expected behavior/output. The program prints `9`.

Experiment. Modify the requirement to return both the maximum value and its index.

1.2 Establishing Algorithm Correctness

Correctness means every input satisfying the preconditions produces the specified postcondition, with termination and resource behavior considered as part of the intended domain. Tests sample behavior; a proof explains why the procedure works throughout its stated domain.

Use both. A proof can contain a mistaken assumption, and an implementation can differ from the proved procedure.

1.2.1 Defining Preconditions and Postconditions

The empty-input branch returns no index before any element access. In the nonempty branch, the postcondition requires a valid index, a value no smaller than every input element, and no earlier equal maximum.

These are separate obligations. Returning an in-range index is not enough, and returning any maximum is not enough under the tie rule. When writing tests, check the complete result rather than only the maximum value. Otherwise two algorithms with different tie behavior can appear equivalent.

A contract can be read in two directions. The caller asks whether its input satisfies the preconditions. The implementer asks what must be true after any valid call returns. Mixing those questions encourages defensive checks in the wrong place and undocumented assumptions elsewhere.

For the nonempty case, let p be the returned index and n the size. The obligations are: $p < n$; no element has a value greater than `values[p]`; and no smaller index contains an equal maximum. The function must also preserve the input. A checker that only verifies `values[p]` equals the maximum ignores the first-occurrence requirement. A checker that dereferences p before checking its range can itself introduce undefined behavior and hide the original defect.

Write checks in dependency order. First establish presence or absence. Then establish an in-range position. Only then inspect the selected element and tie behavior. This resembles production diagnosis in the Linux book: verify the host and path before interpreting a command's output. Evidence is useful only when the steps that made it are valid.

A precondition is not permission to accept arbitrary external data without validation. A parser or public application boundary may need to reject malformed counts, impossible sizes, or missing records before constructing the algorithm's input. Once a well-formed vector is passed to this pure computation, repeatedly reparsing or revalidating unrelated external facts adds no value. Place validation where the representation changes.

Also separate ordinary results from exceptional failures. Empty input is a supported case represented by `std::nullopt`; it is not an allocation exception. This implementation needs no additional dynamic allocation, but its caller may have failed while acquiring the vector. A library's domain result should not erase the distinction between an absent answer and an infrastructure failure.

When requirements change, revise the contract before the test. A last-maximum request is a legitimate new behavior, not a justification for accepting both answers. Tests that tolerate incompatible behaviors cease to tell you which program you shipped.

1.2.2 Reasoning with Invariants

Before iteration i , the invariant is: `best` is the earliest maximum position in the prefix $[0, i)$. It holds initially at $i = 1$ because the one-element prefix contains only index zero. If the new value is larger, replacing `best` extends the invariant; if it is equal or smaller, retaining the old index preserves both maximality and earliest occurrence.

The loop advances i by one until it equals the vector size, so termination is immediate for finite input. At that point the processed prefix is the entire vector and the invariant is the desired postcondition. This proof explains why the comparison must be strict: replacing on equality would select the last maximum instead.

An invariant is a statement about a specific program point, not a reassuring comment that says “the answer is correct.” State exactly when it applies. Here it applies immediately before each loop body, after the prefix ending just before i has been processed. At that point the best position refers to a real element of that prefix.

Use three proof obligations. **Initialization:** before the first iteration the prefix has one element, so its sole position is the earliest maximum. **Preservation:** the next value is either larger, equal, or smaller. A larger value becomes the unique best among the expanded prefix; an equal or smaller value leaves the earlier best valid. **Exit:** when $i == n$, the prefix is the whole sequence. These arguments describe a relationship between state and input, rather than a list of observed test outputs.

Termination is a separate obligation. During the loop, $n - i$ is a nonnegative count of unprocessed elements, and each iteration reduces it by one. The subtraction is justified only while $i \leq n$; that fact belongs to the loop’s range reasoning. With a stable finite vector, the count reaches zero. Changing the vector’s size while iterating would invalidate this particular proof even before considering C++ iterator rules.

Consider an apparently harmless mutation from $<$ to $\leq$. Bounds and termination still hold, and the selected value remains maximal. Only the earliest-occurrence part fails. This is why a proof should preserve every clause of the postcondition. “Finds some maximum” is too weak for this contract.

The proof is about this algorithm under its assumptions, not an endorsement of every implementation with the same name. An accidental $i \leq \text{values.size}()$ or a wrong initial candidate can invalidate the translation to C++. Keep the mathematical interval and the actual comparison close enough that a reviewer can map one to the other. Later chapters use the same discipline for rotations, partitions, and dynamic-programming tables.

Loop Invariant for Summing a Prefix

Purpose. Correctness can be argued through initialization, preservation, and termination rather than only by trying a few inputs.

```
#include <cassert>
#include <vector>
int sum(const std::vector<int>& v){ int total=0; for(std::size_t i=0;i<v.size();
++i){ total+=v[i]; /* invariant: total equals sum of v[0..i] */ } return
total; }
int main(){ assert(sum({})==0); assert(sum({1,2,3})==6); }
```

How it works.

- The initial state is correct for an empty processed prefix.
- Each iteration preserves the invariant by adding exactly the next element.
- When the loop terminates, the processed prefix is the entire vector.

Expected behavior/output. All assertions pass with no output.

Experiment. Change the loop start to index 1 and identify which invariant condition is broken.

Worked example — Find the first maximum and compare against an independent library operation

Requirement and design

Return no position for empty input and otherwise the earliest maximum position. Verify representative cases and every three-element vector whose values come from minus two through two.

The exhaustive small domain contains 125 vectors. It is a finite implementation check, not exhaustive testing of every possible vector length or integer value.

Trace the requirement into the test oracle

The worked project deliberately keeps the algorithm small enough to inspect in one sitting. Its reference is `std::max_element`, whose selected first occurrence agrees with the contract. This agreement matters more than the reference's prestige. An implementation returning the last maximum would be an unsuitable oracle even if it had been tested for years.

Before running the program, predict the optional result for `{}`, `{7}`, `{3,9,9,2}`, and `{-4,-1,-3}`. Record indices, not just values. Then inspect the three nested loops: five choices for each of three positions produce $5 * 5 * 5 = 125$ nonempty vectors. They contain negative readings, ties, ascending patterns, descending patterns, and mixed order without requiring a random generator.

The finite exhaustive test and the fixed boundary tests do different jobs. Three-element enumeration never constructs an empty or singleton sequence, so the first two fixed cases are not redundant.

Conversely, four hand-picked cases exercise only a fraction of the tie and ordering patterns enumerated by the loops.

The `CHECK` helper remains active in optimized builds. Unlike a diagnostic expression compiled away under `NDEBUG`, it reports a failed contract and terminates the test. Read the helper in the companion once; a test framework is part of the evidence-producing program, not an invisible source of certainty.

Complete implementation

The complete, buildable chapter project is `source/book3/ch01/`. The following files are reproduced from that project; the publishing script inserts the source directly so the listing and companion remain synchronized.

main.cpp

```
#include "course_test.hpp"
#include <algorithm>
#include <cstdint>
#include <optional>
#include <vector>

std::optional<std::size_t> first_maximum(const std::vector<int>& values) {
    if (values.empty()) return std::nullopt;
    std::size_t best{};
    // Before each iteration, best is the first maximum in [0, i).
    for (std::size_t i = 1; i < values.size(); ++i) {
        if (values[best] < values[i]) best = i;
    }
    return best;
}

int main() {
    CHECK(!first_maximum({}));
    CHECK(first_maximum({7}) == 0);
    CHECK(first_maximum({3, 9, 9, 2}) == 1);
    CHECK(first_maximum({-4, -1, -3}) == 1);
    for (int a = -2; a <= 2; ++a) {
        for (int b = -2; b <= 2; ++b) {
            for (int c = -2; c <= 2; ++c) {
                const std::vector<int> data{a, b, c};
                const auto expected = static_cast<std::size_t>(
                    std::max_element(data.begin(), data.end()) - data.begin());
                CHECK(first_maximum(data) == expected);
            }
        }
    }
    course::report();
}
```

Build, run, and inspect

Run this block from the top-level extracted package. The fixture runner supplies input where needed and checks exit status and output in a temporary working directory.

```
cmake -S source/book3/ch01 -B build/b3-ch01 \
  -DCMAKE_BUILD_TYPE=Debug
```

```
cmake --build build/b3-ch01 --parallel 2
ctest --test-dir build/b3-ch01 --output-on-failure
```

For a multi-configuration generator on Windows, append `--config Debug` to the build command and `-C Debug` to CTest. The executable target is `b3_ch01`.

Evidence and interpretation

The fixed cases cover an empty vector, one element, repeated maxima, and all-negative values. The nested loops then generate all 125 three-element combinations and compare the custom result with the position returned by `std::max_element`, whose first-maximum semantics match the requirement.

The implementation performs one comparison for each element after the first, so nonempty input of length `n` uses `n` minus one value comparisons and constant auxiliary state. It stores only the best index and traversal index, not a second collection.

The reference comparison does not replace the invariant proof. It catches implementation mistakes such as updating on equality or initializing the candidate incorrectly. The proof, fixed edge cases, and finite exhaustive domain provide complementary evidence with clearly different scopes.

Failure clinic — Correct value, wrong record

Suppose a report highlights record two instead of record one in the duplicate-maximum fixture. Begin with the smallest input distinguishing the intended behavior: two equal elements. If the result is the second position, inspect the update comparison rather than the optional wrapper or allocator. A minimal counterexample directs the investigation to the relevant rule.

Now suppose all-negative input selects a zero-valued result that was never present. That symptom suggests a fabricated numeric initial candidate. The repair is not to choose a more negative magic number; it is to initialize from an existing element after handling emptiness. The domain can include the smallest representable integer, so a sentinel that is also a legitimate reading remains ambiguous.

A crash on empty input points to the order of checks. Do not dereference a candidate and then ask whether the vector was empty. Validate the branch boundary before element access. Sanitizers can help detect an invalid access on exercised inputs, but the explicit empty-state reasoning is the preventive control.

After a correction, rerun the entire fixture set, not only the original counterexample. A change that fixes ties can accidentally change ordinary comparisons. Preserve the failing case as a permanent regression and record the contract clause it protects. This makes a future “simplification” less likely to reintroduce the defect.

Practice and implementation lab

Change the requirement to return the last maximum and identify the smallest code change together with the corresponding invariant change. Add a test where every element is equal.

Design a function returning both minimum and maximum positions. Specify tie rules independently and define the empty result. Write the invariant before choosing whether one pass or two passes gives the clearest implementation.

Review answers and solution notes

Replacing the best index on equality as well as greater values selects the last maximum. The invariant must then state last rather than first occurrence. An all-equal vector distinguishes the two policies immediately.

An optional index avoids confusing absence with zero. Initializing from the first real element avoids a numeric sentinel. A reference algorithm is useful only when its semantics match the custom contract, including ties and empty input. Tests and proofs should agree on the same precise question.

Engineering takeaway. Define the output and tie policy first, then use an invariant and targeted tests to establish the algorithm you actually implemented.

Guided lab — Specify, mutate, and explain

Create a separate exercise file rather than editing the supplied reference. Implement `last_maximum` with the same empty result and input preservation requirement. For `[5, 5, 2]`, the expected result is one; for `[2, 5, 5]`, it is two. State the revised invariant before changing the operator. The implementation change is one comparison, but the semantic change is an observable policy.

Next return an optional pair of first minimum and first maximum positions. A simple approach maintains two candidates in one pass. With `[4, 1, 9, 1, 9]`, expect positions one and two. Explain why finding both values without their positions would not satisfy the contract. A two-pass implementation using independent selectors is also valid; compare clarity and comparison counts without declaring one universally superior.

Add three independent properties. The original vector must remain equal to a saved copy. Appending a value smaller than the existing maximum must preserve the first-maximum index. Adding the same safely representable constant to every element must preserve all ordering decisions. For the last property, restrict generated values so the addition cannot overflow; a mathematically valid transformation is not a valid C++ test when its arithmetic has undefined behavior.

Deliberately introduce each of these defects in the exercise copy: update on equality, start at index one without checking size, and return zero for empty input. Identify the smallest test that detects each defect. Do not execute an out-of-bounds version outside a controlled diagnostic build. You can often reject it by inspection without invoking undefined behavior at all.

Acceptance evidence. Retain the contract, a hand trace, the invariant, the failing counterexample, and the repaired test output. “It prints the expected number” is insufficient when the number could refer to the wrong record.

Review reasoning. Why is the result optional rather than a special index? Because every in-range index, including zero, is meaningful. Why not sort? Sorting spends work on order and changes the identity problem. What does exhaustive testing mean here? Every input in a deliberately bounded three-position

domain, not every possible vector. What does the invariant add? A reason the update rule preserves the specified answer for an arbitrary finite length.

Primary reference notes

Reference behavior and result types: [minimum/maximum algorithms](#), [optional](#), and [vector access](#). The invariant and operation count are derived from the original worked implementation.

Chapter 2 — Measuring Time and Space Complexity

Two search routines both return the right answer on a thousand records, but one performs a thousand comparisons for a missing key while the other repeatedly halves its candidate interval. Timing one run on a busy machine can hide that structural difference.

Complexity analysis describes how work and storage grow with input size. This chapter counts operations directly, distinguishes case analysis from amortized analysis, and explains which costs a simplified model leaves out.

A timing report says that an algorithm took one millisecond. Without the input size, distribution, machine, build mode, and operation being timed, that number is almost uninterpretable. Complexity analysis supplies a different kind of evidence: how a chosen measure of work grows as the problem grows. It does not require pretending that every instruction costs the same on real hardware.

In this chapter, keep an operation ledger before using a clock. Count comparisons, moved elements, and additional storage separately. A search that performs fewer comparisons may still lose for a tiny dataset; a structure with excellent average throughput may still pause during reallocation. The point is to make those tradeoffs explicit rather than collapse them into one adjective such as “fast.”

2.1 Understanding Asymptotic Complexity

Asymptotic analysis compares growth as input size increases. Big-O provides an upper-bound growth class under a stated model; Big-Theta describes a matching-order bound when both upper and lower growth are established. Neither notation is a stopwatch reading.

State the input-size variable and counted operation. Comparing element comparisons with allocated bytes as though they were the same quantity produces meaningless conclusions.

2.1.1 Interpreting Big-O Notation

A linear scan can examine every one of n elements, giving $O(n)$ comparisons in its worst case. A lower-bound search on an indexed sorted sequence halves an interval, giving $O(\log n)$ comparisons. A nested all-pairs loop often produces $O(n^2)$, while a constant amount of work independent of n is $O(1)$.

Constants and lower-order terms are suppressed in these growth descriptions, not declared irrelevant to real systems. An $O(n)$ method can be faster for a small dataset than an $O(\log n)$ method with expensive setup. The analysis answers scaling questions; measurement answers a particular implementation question over a particular workload.

Big-O expresses an eventual upper bound. Saying that a work count $T(n)$ is $O(g(n))$ means there are fixed positive constants and a starting size after which $T(n)$ does not exceed a constant multiple of $g(n)$. It does not say that the two functions are equal, that every input takes that much work, or that the hidden constant is small.

For an example derived directly from a loop, suppose visiting each element performs three counted operations and initialization performs two. The count is $3n + 2$. For every $n \geq 1$, this is at most $5n$, so a linear upper bound follows. The constants disappear from the classification, not from the running program. When comparing two implementations at realistic sizes, those constants can matter substantially.

Use **Theta** when you have matching upper and lower growth bounds for the stated case. A complete scan of n elements performs a linear number of element visits, not merely “at most quadratic.” The loose statement $O(n^2)$ could be mathematically true while concealing the useful information. Precise notation is a communication tool; the goal is not to collect the largest possible upper bound.

Always name what n counts. For a string-search problem it could mean bytes, code units, characters under a particular encoding model, or records containing strings. For a graph there are at least two natural sizes: vertices V and stored edges E . Calling a graph traversal $O(n)$ without defining n can hide the cost of a dense representation.

An operation-count model also needs a cost assumption. Comparing two machine integers can be treated as constant work in this chapter’s model. Comparing two long strings can inspect many characters. A sorting algorithm performing $O(n \log n)$ comparisons does not thereby promise $O(n \log n)$ character inspections. Write the element-operation assumption alongside the bound so it can be revisited when the domain type changes.

2.1.2 Comparing Common Growth Rates

Common growth rates separate dramatically as n increases: constant, logarithmic, linear, $n \log n$, quadratic, and exponential. For example, doubling n roughly doubles linear work, quadruples quadratic work, and adds about one halving step to a binary search.

A sorting prerequisite must be counted when comparing searches over originally unsorted data. Sorting once and performing many queries is a different workload from sorting before each single query. Likewise, an index can trade memory and update cost for lookup speed. The complete operation sequence determines which complexity comparison is relevant.

Common growth classes describe different responses to scale. Constant work does not grow with the number of stored elements. Logarithmic work grows by roughly one step when an ordered search space doubles. Linear work roughly doubles with the input. Linearithmic work adds a logarithmic factor, while quadratic work roughly quadruples when the input doubles. These are statements about models, not exact wall-clock ratios.

Derive a logarithmic count by repeated halving. Starting from 1024 candidates, the possible interval sizes can shrink through 512, 256, and so on toward one and zero. The number of halvings grows with the base-two logarithm. An extra comparison at a boundary explains why a particular implementation may perform eleven comparisons rather than the memorable number ten. The asymptotic statement tolerates a constant difference; a fixture checking exact output cannot ignore it.

For nested loops, inspect the bounds instead of multiplying mechanically. Two independent full scans nested together perform n^2 visits. But a loop whose inner limit is the outer index visits $0 + 1 + \dots + (n-1)$ pairs, or $n(n-1)/2$. Both are quadratic, yet the exact formulas differ. Two consecutive loops add costs and remain linear; their indentation does not create a product.

Exponential growth is a warning about the size of a search space, not evidence that every recursive function is exponential. A recursion making one call on half the input behaves very differently from a recursion branching into two overlapping subproblems of almost the same size. Chapters 17 and 19 separate a search tree from a graph of repeated states.

Check storage as well as time. A dense V by V matrix has a quadratic number of cells even if one query is constant time. At one thousand vertices there are one million cells; at one hundred thousand there are ten billion. Before multiplying by bytes per cell, validate that the multiplication itself is representable. A complexity estimate becomes an engineering decision only when connected to an actual resource budget.

Counting Work in Linear and Quadratic Loops

Purpose. Asymptotic analysis focuses on how operation counts grow with input size rather than on one measured runtime.

```
#include <iostream>
long linear(int n){long steps=0;for(int i=0;i<n;++i)++steps;return steps;}
long quadratic(int n){long steps=0;for(int i=0;i<n;++i)for(int j=0;j<n;++j)+
+steps;return steps;}
int main(){for(int n:{10,20,40})std::cout<<n<<' '<<linear(n)<<'
'<<quadratic(n)<<'\n';}
```

How it works.

- The linear loop performs work proportional to n .
- The nested loop performs n^2 inner operations.
- Doubling input approximately doubles linear work but quadruples quadratic work.

Expected behavior/output. Rows show 10/10/100, 20/20/400, and 40/40/1600.

Experiment. Add a logarithmic loop `for(int i=1;i<n;i*=2)` and compare growth.

2.2 Analyzing Practical Complexity

Case analysis asks which input of a given size is being considered. Amortized analysis asks how expensive operations behave over a sequence, even when an individual operation is costly. Average-case analysis additionally requires a probability model over inputs.

These concepts are often confused because all can produce a smaller bound than a naive worst-case statement about one operation.

2.2.1 Distinguishing Best, Average, and Worst Cases

A linear scan finds the first element with one comparison in its best case and uses n comparisons for an absent key in its worst case. An average-case claim requires assumptions about key presence and location; it is not justified by choosing the middle element in one test.

The binary comparison counter in the example operates on sorted values and counts each midpoint comparison. It does not count sorting, cache misses, branch prediction, or iterator movement for a linked representation. Labeling the model makes its conclusion useful without overstating what was measured.

Best, average, and worst cases quantify different sets of inputs of the same size. In linear search, a first-element match needs one equality test. A missing value requires inspecting the whole sequence. An average claim requires a distribution: how likely is each position, and how often is the key absent? Without that information, “average” is an unstated guess.

Under the illustrative assumption that a present key is equally likely to be at each of n distinct positions, successful linear search uses an average of $(n+1)/2$ comparisons. That result follows from averaging one through n . It is not the average for a cache of frequently requested items deliberately kept at the front, nor for workloads dominated by missing keys. Change the workload and the average changes.

Worst-case analysis is not necessarily pessimism. A service accepting externally chosen keys may need a response-time bound even when common requests are cheap. A hash table’s expected behavior relies on distribution and load assumptions; a degenerate collision pattern can change its cost. A balanced search tree offers a different set of guarantees. Chapter 7 makes the collision mechanism visible before promising a constant-time lookup.

Distinguish a worst-case input from a worst-case machine event. Complexity can bound the number of comparisons while the operating system preempts the process or the allocator waits for memory. These environmental effects matter for latency but are not captured by the simple comparison model. Book VI will address timing requirements more directly; here the goal is to avoid claiming that an asymptotic bound is a deadline guarantee.

Tests should identify which case they exercise. A benchmark that repeatedly searches for the first element is not representative evidence for unsuccessful lookups. Likewise, a sort measured only on ascending data says little about a pivot rule vulnerable to another arrangement. Label datasets and preserve their construction so another engineer can reproduce the case rather than infer it from a graph title.

2.2.2 Understanding Amortized Complexity

For a dynamic array that doubles capacity when full, some appends copy all existing elements, but most do not. Across growth capacities one, two, four, and so on, the total moved elements before n appends form a geometric sum less than $2n$. Adding the n ordinary insertions still gives linear total work and amortized constant append cost under the element-cost model.

This is not an average over lucky input values. The bound applies to the modeled append sequence regardless of the stored values when element movement has the assumed cost. It also does not promise that every individual append meets a constant latency deadline. A reallocation spike remains important for real-time requirements.

Amortized analysis concerns a sequence of operations, without requiring a probability distribution. A dynamic array may occasionally move every existing element when capacity is exhausted. Nevertheless, doubling capacity makes the total number of relocated elements across many appends grow linearly with the number of appends.

Consider capacities one, two, four, eight, and sixteen. The relocations needed to reach sixteen stored elements sum to $1 + 2 + 4 + 8 = 15$; initial allocation moves no old elements. The geometric series is smaller than twice the final number of elements. Charging a small constant amount of accounting work to every append can pay for the occasional expensive move. That is the idea behind an amortized constant bound.

The bound does not make an individual reallocation cheap. The append that exceeds a million-element capacity may still move a million values. Applications concerned with a particular request's latency can reserve capacity earlier, choose bounded storage, or use another representation. Throughput over a long sequence and maximum pause per operation are different requirements.

The growth rule is essential. Expanding capacity by exactly one for every append moves $0 + 1 + \dots + (n-1)$ elements, producing quadratic total movement. Repeatedly calling `reserve(size()+1)` can therefore defeat an intended geometric strategy when the implementation honors those capacities closely. Reserve a justified batch size or let the container manage ordinary growth; do not turn a hint into a mandatory per-element reallocation schedule.

Deletion requires its own analysis. Shrinking immediately after every removal and growing immediately after every insertion near one threshold can cause expensive oscillation. A gap between growth and shrink thresholds, often called hysteresis, can prevent this pattern. The chapter's simulation does not implement shrinking, so its proof must not be silently extended to a different policy.

Finally, movement cost depends on the element. This ledger treats relocating an integer as one unit. A generic container moving objects with expensive or throwing operations needs an additional semantic and cost analysis. The amortized count of relocations is only one layer of that design.

Same Big-O, Different Constant Factors

Purpose. Practical analysis supplements asymptotic complexity with constants, locality, compiler behavior, and workload characteristics.

```
#include <chrono>
#include <iostream>
#include <vector>
long sum_indexed(const std::vector<int>&v){long s=0;for(std::size_t
i=0;i<v.size();++i)s+=v[i];return s;}
long sum_ranged(const std::vector<int>&v){long s=0;for(int x:v)s+=x;return s;}
int main(){std::vector<int>v(100000,1);auto
a=sum_indexed(v),b=sum_ranged(v);std::cout<<a<<' '<<b<<'\n';}
```

How it works.

- Both functions are $O(n)$ because they visit each element once.
- The code keeps a correctness check beside any later timing experiment.
- Actual performance also depends on compiler optimization, memory access, and generated machine code.

Expected behavior/output. Both sums print 100000.

Experiment. Benchmark both functions across repeated trials with optimization enabled and verify results before comparing times.

Worked example — Count search comparisons and simulate geometric growth

Requirement and design

Build a sorted sequence of 1024 integers. Count linear comparisons for a missing key and a first-element match, count binary lower-bound comparisons for the missing key, and simulate one thousand append operations with doubling capacity.

The simulation counts moved elements; it does not allocate a real custom array or measure wall-clock time.

Establish the counting boundary

The program compares two search mechanisms on the same sorted vector. It does not time them. The linear routine counts equality comparisons until a match or the end. The binary routine counts comparisons with the middle element while maintaining a candidate interval. The chosen missing key is below all stored values, making the behavior easy to trace by hand.

Inspect the names carefully: these functions return a comparison count, not a found position. A test of a count does not prove a full search-result interface. Chapter 15 supplies that interface and its boundary

tests. Keeping this chapter's output narrow prevents a demonstration of one property from being mistaken for a complete implementation of another.

The capacity loop is a model. It updates three integers: logical size, capacity, and cumulative relocation count. No element buffer is allocated. That separation makes the mathematical growth policy easier to inspect, but it excludes allocator overhead, failures, and initialization of unused capacity. Chapter 3 implements real storage and brings those concerns back into scope.

Before execution, predict `linear_missing` and the approximate binary count. Then derive the exact binary path using half-open intervals. A mismatch between a predicted and observed count is a reason to inspect the implementation, not to change an expected value until the test turns green.

Complete implementation

The complete, buildable chapter project is `source/book3/ch02/`. The following files are reproduced from that project; the publishing script inserts the source directly so the listing and companion remain synchronized.

main.cpp

```
#include "course_test.hpp"
#include <bit>
#include <cstdint>
#include <numeric>
#include <vector>

std::size_t linear_comparisons(const std::vector<int>& values, int wanted) {
    std::size_t comparisons{};
    for (int value : values) {
        ++comparisons;
        if (value == wanted) break;
    }
    return comparisons;
}

std::size_t binary_comparisons(const std::vector<int>& values, int wanted) {
    std::size_t low{}, high = values.size(), comparisons{};
    while (low < high) {
        const auto middle = low + (high - low) / 2;
        ++comparisons;
        if (values[middle] < wanted) low = middle + 1;
        else high = middle;
    }
    return comparisons;
}

int main() {
    std::vector<int> data(1024);
    std::iota(data.begin(), data.end(), 0);
    CHECK(linear_comparisons(data, -1) == 1024);
    CHECK(linear_comparisons(data, 0) == 1);
    CHECK(binary_comparisons(data, -1) <=
static_cast<std::size_t>(std::bit_width(data.size())));
    std::size_t capacity{}, size{}, moved{};
    for (std::size_t n = 1; n <= 1000; ++n) {
        if (size == capacity) {
            moved += size;
            capacity = capacity ? capacity * 2 : 1;
        }
        ++size;
    }
}
```

```

        CHECK(moved < 2 * n);
    }
    std::cout << "linear_missing=" << linear_comparisons(data, -1)
               << " binary_missing=" << binary_comparisons(data, -1) << '\n';
    course::report();
}

```

Build, run, and inspect

Run this block from the top-level extracted package. The fixture runner supplies input where needed and checks exit status and output in a temporary working directory.

```

cmake -S source/book3/ch02 -B build/b3-ch02 \
      -DCMAKE_BUILD_TYPE=Debug
cmake --build build/b3-ch02 --parallel 2
ctest --test-dir build/b3-ch02 --output-on-failure

```

For a multi-configuration generator on Windows, append `--config Debug` to the build command and `-C Debug` to CTest. The executable target is `b3_ch02`.

Evidence and interpretation

The missing linear search performs 1024 comparisons, while the first-element match performs one. Binary lower-bound search for the chosen missing key performs eleven comparisons under this implementation, bounded by the bit width of the positive size value.

The growth simulation tracks size, capacity, and cumulative moved elements. At each reallocation, it adds the current size to the movement count and doubles capacity. After every append, the test checks the geometric upper bound, not just the final total.

This makes a complexity argument executable for a finite range of n while preserving the general geometric proof. The model excludes allocation metadata, page faults, and nonconstant element operations. Those omitted costs matter in later performance chapters but do not invalidate the clearly stated element-count analysis.

Failure clinic — A valid measurement answering the wrong question

A colleague claims binary search is always better because its comparison count is lower. Ask who paid to sort the input and how many searches will reuse that order. Sorting a fresh collection for a single query can cost more total work than one scan. For a static collection queried repeatedly, preprocessing may be justified. The correct comparison is between complete workflows with the same contract.

Another report divides a single elapsed time by n and labels the quotient the cost per element. If the timed region contains allocation, startup, parsing, and output, the quotient is not solely an element-processing cost. It may still be a useful end-to-end measure, but it needs the correct name. Do not remove inconvenient overhead from one implementation while retaining it for its competitor.

A third result calls one thousand simulated appends “proof that vector never reallocates slowly.” The simulation shows a bound on cumulative movement under a stated policy. It does not measure a real

vector, determine its growth factor, or bound a single pause. Repair the claim rather than adding more iterations to the same incomplete experiment.

When an analysis and a benchmark disagree, preserve both and investigate their assumptions. Small inputs, a different element cost, optimized-away work, allocation policy, and memory hierarchy can explain the difference. Complexity remains useful precisely because it can be examined separately from those other mechanisms.

Practice and implementation lab

Count comparisons for binary search sizes around powers of two, including zero, one, 1023, 1024, and 1025. Compare a present first key, a last key, and a missing key without changing the sorted-input precondition.

Simulate growth by one element each time instead of doubling and derive the total movement sum. Then simulate a different geometric factor using safe integer rounding and compare memory slack with movement frequency. Separate per-operation worst-case latency from amortized throughput.

Review answers and solution notes

Growing capacity by one moves zero plus one through n minus one elements, producing quadratic total movement. Geometric growth keeps the sum proportional to n but can still require a linear-time individual reallocation.

Average case needs a probability model; amortized analysis does not rely on random inputs. A binary search over pre-sorted indexed data is not the same workload as sorting an unsorted dataset for one lookup. Operation counts explain growth, while timing must include the actual environment and full workload.

Engineering takeaway. State the cost model, distinguish cases from sequences, and never confuse amortized throughput with a per-operation latency guarantee.

Guided lab — Build an operation ledger

Extend the counting program in a separate project to print rows for sizes zero, one, two, four, eight, sixteen, and 1024. For linear search, include first match, last match, and absence. For binary lower-bound counting, include keys below the range, inside it, and above it. Do not dereference a search result merely to count comparisons.

Add a second growth simulation that expands by one. For ten appends, the relocation count should be the sum zero through nine, which is 45. For doubling growth, compute the actual count from capacity boundaries. Compare cumulative movement, maximum movement in one append, and unused capacity. These three measures expose different tradeoffs.

Now change the hypothetical workload. Suppose constructing a sorted index costs a one-time amount S , a scan query costs L , and an indexed query costs B , with $L > B$. Reusing the index becomes favorable in this simplified model when $S + qB < qL$, or $q > S / (L - B)$. Do not insert arbitrary machine timings

and present the result as a universal crossover. The algebra tells you which measurements would be needed.

Investigate space terminology. The input vector occupies storage proportional to n . The search loop uses constant auxiliary storage. A result containing every matching position can itself contain n entries. State whether your reported space bound includes input, output, or only auxiliary work memory. An algorithm cannot emit a length- n owned output while using constant total result storage.

Acceptance evidence. Save the formulas and counted rows, identify the cost unit, and state whether each claim is worst-case, average under an explicit distribution, or amortized over an operation sequence. Include a counterexample to at least one misleading claim.

Review reasoning. Is $O(n)$ an exact elapsed-time promise? No. Does amortized constant append mean every append is constant work? No. Can two consecutive linear loops be quadratic merely because there are two? No; their counts add. Can a logarithmic number of comparisons coexist with linear traversal steps? Yes, when the representation lacks random access. This last distinction will matter when searching linked structures.

Primary reference notes

Library complexity context: [binary-search algorithms](#), [vector capacity](#), and [vector modifiers](#). The geometric growth bound is derived in this chapter.

Chapter 3 — Understanding Arrays and Contiguous Dynamic Storage

A dynamic array has room for eight values and receives a ninth. The program must obtain larger storage, preserve the old values, append the new value, and leave a valid object if allocation fails. The algorithm is simple only when its ownership and commit point are explicit.

This chapter implements an integer-only dynamic array to expose contiguous layout and geometric growth. It is not a replacement for the standard vector’s generic allocator, object-lifetime, and exception-safety machinery.

HarborWorks collects readings in a vector because processing usually means scanning every value in order. The useful property is not simply that a vector is “an array.” It is that logical positions correspond to a contiguous sequence of objects while a separate capacity policy controls when that sequence must move.

This chapter opens the abstraction without pretending to recreate the entire standard library. The worked `IntVector` is deliberately integer-only and move-only. Its smaller contract lets you trace allocation, copying, commit, and cleanup. A generic production container would need allocator integration, precise object construction and destruction, iterator rules, and exception guarantees for many different element types. Keep the teaching model’s boundaries visible while learning from its mechanism.

3.1 Understanding Contiguous Memory Layout

Contiguous storage places elements next to one another in the array’s element order. An index maps to an element offset, supporting constant-time random access under the ordinary address-arithmetic model.

The layout is valuable not only for indexing but also for traversing data with predictable nearby accesses. Hardware effects still depend on actual cache and workload conditions.

3.1.1 Mapping Indices to Addresses

For an array base and valid index `i`, the conceptual address offset is `i * sizeof(T)` bytes within the appropriate object representation. C++ pointer arithmetic performs element stepping directly, so code uses `data[i]` or `data + i` rather than manually inventing byte offsets for typed access.

A dynamic owner must preserve the invariant `size <= capacity` and ensure the active sequence’s indices are below `size`. Capacity describes available storage, while `size` describes the logical sequence. In this integer-only teaching implementation, the allocated array actually constructs all capacity integers; that differs from a fully generic vector managing raw capacity without live elements in every slot.

For a contiguous sequence of `T` objects, moving from one element to the next advances by `sizeof(T)` bytes in the object representation. C++ pointer arithmetic already accounts for that scale: `data + i` identifies the `i`th element, not the byte at offset `i`. Multiplying by `sizeof(T)` again in a typed pointer expression is a common low-level mistake.

An index can be valid for storage but invalid for the logical sequence. If size is three and capacity is eight, positions zero through two are live sequence entries according to the container interface. The remaining capacity is not permission to access positions three through seven with `operator[]` on a standard vector. The worked integer model happens to allocate initialized backing integers across its capacity, but its checked `at` still rejects indices outside logical size. Representation details do not override the public contract.

Use a concrete diagram to distinguish the quantities:

```
logical size = 3, allocated capacity = 8
[ 12 ][ 15 ][ 18 ][ spare ][ spare ][ spare ][ spare ][ spare ]
  valid sequence      capacity available for later appends
```

Here “spare” describes the logical role, not a promise about raw bytes or object lifetime for every container. Standard `vector<T>` manages construction of its elements; an allocation of raw storage alone does not construct arbitrary `T` objects. This distinction connects directly to Book II’s lifetime and RAII material.

The one-past position is useful for range boundaries but is not an element. `[begin, end)` can be empty when the two boundaries are equal. Dereferencing `end` is not justified by the fact that it participates in a valid range expression. Likewise, an index equal to `size()` can describe an insertion boundary while remaining invalid for element access.

When flattening a two-dimensional layout, derive the offset from the declared shape. In row-major storage with `COLUMNS` entries per row, the offset is `row * columns + column`. Check both dimensions and representability before using the product. A correct formula applied to unchecked dimensions can still create an invalid address.

3.1.2 Exploiting Cache Locality

Sequential traversal often benefits from spatial locality because nearby values can be fetched together through the memory hierarchy. A linked traversal may require discovering each next address before proceeding. These are architectural tendencies, not universal timing guarantees.

Contiguity does not make an algorithm with quadratic work automatically superior to a well-chosen logarithmic or linear alternative. Combine algorithmic complexity with layout considerations. Book III’s final chapter measures traversal layouts after verifying that both implementations compute the same result.

Contiguity helps a sequential scan because nearby logical elements occupy nearby storage. A processor commonly transfers and caches memory in blocks, so consuming adjacent entries can reuse data brought in for neighboring entries. The exact cache-line size, prefetch behavior, and latency depend on the machine; the layout argument does not require declaring one particular device geometry universal.

Consider summing the readings in order versus following an index chain that jumps among them. Both can perform one addition per reading, but the chain introduces a dependency: the next location is not known until the previous location is read. Contiguous iteration often gives the implementation more opportunity to fetch ahead. Chapter 20 measures a related traversal difference while preserving an identical numerical result.

Contiguity is not a guarantee that every object contains its payload inline. A vector of strings stores string objects contiguously, while some character data may live in separate allocations. A vector of owning pointers stores pointers contiguously, not necessarily the pointed-to nodes. Ask which bytes the hot loop actually touches before concluding that choosing **vector** removed all indirection.

Layout also affects mutation cost. Inserting at the beginning of a contiguous sequence can require moving later values so that positions remain packed. A node-based structure avoids that particular shift when the insertion location is already known, but may pay for allocation and traversal. Comparing only the final pointer update with a full vector insertion omits the work needed to locate the node.

Use workload descriptions rather than container slogans. “Append batches, scan all samples, rarely insert in the middle” is a stronger basis for choosing a vector than “vectors are fast.” If a caller needs stable addresses across growth, the representation or interface may need to change. Reserving a justified capacity can postpone relocation, but it does not create an unlimited address-stability guarantee.

Finally, measure the same operation on the same data. An array loop that sums integers cannot fairly be compared with a linked structure that parses strings during traversal. Locality is one variable in an experiment, not an excuse to change the problem being solved.

Inspecting Adjacent Vector Element Addresses

Purpose. Contiguity explains constant-time indexing and favorable cache behavior for sequential traversal.

```
#include <cstdint>
#include <iostream>
#include <vector>
int main(){std::vector<int>v{10,20,30};for(std::size_t i=0;i<v.size();+
+i)std::cout<<(reinterpret_cast<std::uintptr_t>(&v[i])-
reinterpret_cast<std::uintptr_t>(&v[0]))<<' ';;}
```

How it works.

- Vector elements occupy contiguous storage.
- Subtracting their integerized addresses from the first element shows byte offsets.
- Offsets should advance by ``sizeof(int)`` for each next element.

Expected behavior/output. Typical offsets are ``0 4 8`` when ``sizeof(int)==4``.

Experiment. Print ``sizeof(int)`` and verify the offset relationship rather than assuming a specific byte size.

3.2 Implementing Dynamic Array Growth

Growth requires a replacement allocation when size reaches capacity. The old allocation must remain owned until the new state is ready to commit. This keeps allocation failure from erasing the previously accepted sequence.

Separate the mechanics into capacity validation, allocation, element transfer, commit, and old-storage release.

3.2.1 Allocating Expanded Storage

`reserve` returns immediately when the requested capacity is already sufficient. Otherwise it checks a size limit, allocates a replacement integer array, and copies the current logical elements. Only after that succeeds does it swap storage and update capacity.

The size check prevents an obvious multiplication overflow in the teaching representation; it does not guarantee allocation success or account for every implementation allocation limit. The standard allocation mechanism can still fail. Because the original storage remains owned until the swap, that failure leaves the previous sequence available.

Growth should be treated as a small transaction. First determine a legal requested capacity. Then allocate replacement storage. Then copy the existing values. Only after preparation succeeds should the object publish the replacement and its new capacity. The old state remains the recovery state until the commit point.

In the worked `reserve`, `std::make_unique<int[]>(requested)` owns the candidate allocation immediately. If allocation fails, the original members have not changed. Copying integers cannot throw a user-defined copy exception. Swapping the owning pointers commits the new buffer, and destruction of the temporary releases the old buffer. The guarantee follows from this sequence, not from the mere presence of a smart pointer.

The arithmetic check occurs before asking for storage. A requested element count larger than the maximum representable byte count divided by `sizeof(int)` cannot describe a representable allocation size in this model. Passing that check does not mean the machine has enough memory. `std::bad_alloc` and implementation allocation limits remain possible. Integer validity and physical availability are distinct questions.

The growth policy doubles capacity except at the initial transition from zero to one. Near the arithmetic limit, the example rejects a doubling request rather than attempting a more elaborate final partial growth. That is a conservative teaching policy, not an assertion that all standard vectors use doubling or the same maximum capacity. A generic container would consult its allocator and define additional limit behavior.

Watch publication order. Setting `capacity_` before allocation would make the object claim storage it does not own if allocation throws. Setting `size_` before writing the appended value could expose an entry that has not been established. Invariants are strongest when member updates follow the resource state rather than anticipate it.

This pattern generalizes beyond arrays. A hash-table rehash, a replacement graph, or a sorted index can be prepared off to the side and committed after validation. The cost is temporary coexistence of old and new storage, which must be included in peak-memory planning.

3.2.2 Moving Elements During Reallocation

Appending doubles capacity when full, with an overflow guard before doubling. The new value is passed by value, so a call such as `push_back(at(0))` copies the input scalar before a possible reallocation invalidates its old element address.

The example copies integers during reallocation, for which assignment is nonthrowing. A generic `T` requires explicit construction into uninitialized storage, destruction of partially constructed ranges, allocator propagation rules, and decisions about throwing moves versus copies. Those are substantial responsibilities. The educational implementation exposes one growth algorithm without pretending to satisfy the full standard-vector contract.

Relocation creates an aliasing question. Suppose the caller appends the value already stored at position zero while growth is required. A function accepting a reference to that element must avoid destroying the referenced storage before reading it. The worked `push_back(int value)` captures an independent integer before entering the function body, so subsequent reallocation cannot invalidate its parameter.

This is one reason not to convert every value parameter into a reference in the name of efficiency. For a small scalar, copying can make the ownership and failure boundary simpler. A generic `emplace_back` must consider argument evaluation, construction, and internal aliases much more carefully. The chapter's integer example is not a substitute for that broader library design.

Moving the container also moves a relationship among members. Transferring only `data_` while leaving nonzero size and capacity in the source would create a source that claims elements without storage. The explicit move operations transfer the pointer and use `std::exchange` to reset the source counters. The example then reuses the source, demonstrating its chosen empty-after-move contract rather than assuming every moved-from type behaves that way.

For arbitrary `T`, moving existing elements can fail. If earlier elements have already been moved from when a later move throws, restoring the old values may not be possible. Copying into replacement storage can preserve the old sequence when copying is available and cleanup is safe, but that too has costs. Book II's distinction between valid state and unchanged state becomes central in a generic dynamic array.

Removal exposes another simplification. This integer model reduces logical size without ending each backing integer's lifetime. For a general resource-owning element, a real `pop_back` must destroy the removed logical element at the appropriate time. A vector of file wrappers cannot keep a supposedly removed file open indefinitely merely because capacity remains allocated.

When evaluating custom storage code, inspect empty, full, moved-from, and post-failure states separately. A happy-path append to unused capacity exercises none of the operations that make growth and ownership difficult.

Manual Capacity Doubling in a Small Dynamic Array

Purpose. This is the core mechanism underlying dynamic-array growth strategies such as `std::vector`.

```
#include <algorithm>
#include <cstdint>
#include <iostream>
#include <memory>
class Dyn{std::unique_ptr<int[]>d_;std::size_t size_{},cap_{};public:void
push(int x){if(size_==cap_){std::size_t nc=cap_?cap_*2:1;auto
n=std::make_unique<int[]>(nc);std::copy_n(d_.get(),size_,n.get());d_=std::move(n);cap_=nc;}
d_[size_++]=x;}std::size_t size()const{return size_;}std::size_t
capacity()const{return cap_;}};
int main(){Dyn d;for(int i=0;i<5;++i)
{d.push(i);std::cout<<d.size()<<'/'<<d.capacity()<<' ';;}}
```

How it works.

- Capacity grows geometrically rather than by one element each insertion.
- Existing elements are copied only when the current allocation is full.
- Geometric growth gives amortized constant-time append despite occasional $O(n)$ reallocations.

Expected behavior/output. The size/capacity sequence grows through capacities 1, 2, 4, and 8.

Experiment. Change growth from doubling to `cap_+1` and reason about total copies over n insertions.

Worked example — Implement and differentially test a growing integer sequence

Requirement and design

Implement checked access, geometric append growth, pop, reserve, and move-only ownership. Compare one thousand appended values with a standard vector, test an aliased append input, move the collection, and reuse the moved-from source.

The implementation is deliberately specialized to integers and deletes copying. Its capacity array contains initialized integers even beyond the logical size.

Inspect the representation before the loop

The class has three state members: one exclusive owner, one logical size, and one capacity. The central relation is `size_ <= capacity_`. A zero capacity accompanies an empty owner in the default and moved-from states. An allocated buffer has enough backing integer objects for the represented capacity, while only the first `size_` positions belong to the public sequence.

Trace `reserve` with size three, capacity four, and a request of eight. The candidate initially contains eight initialized integers. The first three are replaced by the old logical values. The pointer swap changes which allocation the object owns; the old allocation then belongs to the temporary and is released. Logical size never changes during `reserve`.

Trace `append` separately. With spare capacity it writes one value and increments size. With no spare capacity it first invokes the growth transaction. This separation lets tests distinguish a storage operation from a membership operation. Reserving eight slots must not produce eight logical readings.

The test compares observable values with `std::vector<int>`, but it does not require matching capacities. The standard container is a semantic reference for the sequence, not an oracle for this example's doubling policy. Tests that insist on identical implementation details would reject valid alternative strategies.

Complete implementation

The complete, buildable chapter project is `source/book3/ch03/`. The following files are reproduced from that project; the publishing script inserts the source directly so the listing and companion remain synchronized.

main.cpp

```
#include "course_test.hpp"
#include <cstdint>
#include <limits>
#include <memory>
#include <utility>
#include <vector>

class IntVector {
    std::unique_ptr<int[]> data_;
    std::size_t size_{}, capacity_{};
public:
    IntVector() = default;
    IntVector(const IntVector&) = delete;
    IntVector& operator=(const IntVector&) = delete;
    IntVector(IntVector&& other) noexcept
        : data_(std::move(other.data_)), size_(std::exchange(other.size_, 0)),
          capacity_(std::exchange(other.capacity_, 0)) {}
    IntVector& operator=(IntVector&& other) noexcept {
        if (this != &other) {
            data_ = std::move(other.data_);
            size_ = std::exchange(other.size_, 0);
            capacity_ = std::exchange(other.capacity_, 0);
        }
        return *this;
    }
    std::size_t size() const noexcept { return size_; }
    std::size_t capacity() const noexcept { return capacity_; }
    void reserve(std::size_t requested) {
        if (requested <= capacity_) return;
        if (requested > std::numeric_limits<std::size_t>::max() / sizeof(int)) {
            throw std::length_error("capacity overflow");
        }
        auto replacement = std::make_unique<int[]>(requested);
        for (std::size_t i = 0; i < size_; ++i) replacement[i] = data_[i];
        data_.swap(replacement);
        capacity_ = requested;
    }
};
```

```

    }
    void push_back(int value) {
        if (size_ == capacity_) {
            const auto maximum = std::numeric_limits<std::size_t>::max() /
sizeof(int);
            if (capacity_ > maximum / 2) throw std::length_error("growth overflow");
            reserve(capacity_ ? capacity_ * 2 : 1);
        }
        data_[size_++] = value;
    }
    void pop_back() {
        if (size_ == 0) throw std::out_of_range("empty vector");
        --size_;
    }
    int& at(std::size_t index) {
        if (index >= size_) throw std::out_of_range("index");
        return data_[index];
    }
};

int main() {
    IntVector actual;
    std::vector<int> expected;
    CHECK(course::throws<std::out_of_range>([&] { actual.pop_back(); }));
    for (int i = 0; i < 1000; ++i) {
        actual.push_back(i); expected.push_back(i);
        CHECK(actual.size() == expected.size() && actual.capacity() >=
actual.size());
    }
    for (std::size_t i = 0; i < expected.size(); ++i) CHECK(actual.at(i) ==
expected[i]);
    actual.push_back(actual.at(0)); // Parameter copied before any reallocation.
    CHECK(actual.at(1000) == 0);
    actual.pop_back();
    IntVector moved{std::move(actual)};
    CHECK(actual.size() == 0 && moved.size() == 1000);
    actual.push_back(42);
    CHECK(actual.at(0) == 42);
    CHECK(course::throws<std::out_of_range>([&] { (void)moved.at(1000); }));
    course::report();
}

```

Build, run, and inspect

Run this block from the top-level extracted package. The fixture runner supplies input where needed and checks exit status and output in a temporary working directory.

```

cmake -S source/book3/ch03 -B build/b3-ch03 \
  -DCMAKE_BUILD_TYPE=Debug
cmake --build build/b3-ch03 --parallel 2
ctest --test-dir build/b3-ch03 --output-on-failure

```

For a multi-configuration generator on Windows, append `--config Debug` to the build command and `-C Debug` to CTest. The executable target is `b3_ch03`.

Evidence and interpretation

After each append, the test checks size agreement and the invariant that capacity covers size. It then compares every stored value against the standard vector. Appending `actual.at(0)` verifies that the scalar parameter is captured before growth can relocate the source element.

Moving the collection transfers storage, size, and capacity together and resets the source to an empty usable state. The moved-from source accepts a new value 42. Checked access at index 1000 in the thousand-element destination fails, as does popping an initially empty collection.

The custom `pop_back` only decreases logical size because all backing objects are integers and remain part of the allocated capacity array. This is a deliberate simplification, not generic container semantics for resources requiring immediate element destruction. A production implementation should ordinarily use `std::vector` rather than extend this teaching type casually.

Failure clinic — A count survived after its storage moved

A moved-from object reports size one thousand but has a null owning pointer. The likely defect is a defaulted memberwise move that transferred the smart pointer but copied scalar metadata. The smart pointer did its own job correctly; the class failed to preserve its combined invariant. Repair the move operation or redesign the representation so the invariant follows automatically.

A second failure appears only when appending an existing element at a capacity boundary. Examine whether an input reference points into the old buffer and whether it is read after reallocation. Testing `push_back(42)` repeatedly will not expose that alias. The supplied scalar-by-value interface intentionally avoids it.

A third failure changes `size()` after `reserve`. That confuses allocation capacity with logical membership. Add a test that saves every logical value, reserves more storage, and verifies the same size and sequence afterward. Do not accept zero-initialized spare storage as newly inserted user data.

Finally, an out-of-range request must fail before returning a reference. A test that merely checks for an exception after an invalid access may already have triggered undefined behavior. Bounds checks are preventive decisions, not error labels added after memory has been touched.

Practice and implementation lab

Add tests for `reserve` smaller than current capacity, `reserve` larger than capacity without changing size, and repeated moves including self-move assignment. Verify that every old value survives an accepted `reserve`.

Instrument allocation requests through a small injected allocator abstraction and simulate a failed growth request. Confirm that size, capacity, and values remain unchanged. Then list the extra lifetime work required to support a nontrivial element type with a throwing constructor.

Review answers and solution notes

Reserve changes capacity, not logical size. Candidate allocation before swap preserves accepted state if allocation fails. Passing the appended scalar by value protects the alias case across reallocation.

Supporting arbitrary T requires more than replacing `int` with a template parameter. Object construction, destruction, throwing transfer operations, and allocator semantics all become part of the contract. The standard vector already addresses those concerns. This implementation exists to make the growth mechanism and its invariant inspectable.

Engineering takeaway. Keep size, capacity, and ownership consistent, and perform fallible growth work before committing replacement storage.

Guided lab — Make state transitions observable

Extend the exercise copy with a `const` checked-access overload. Verify that callers holding a `const` container can read elements without obtaining a mutable reference. Keep both overloads' bounds behavior identical. This addition improves the interface without changing ownership or capacity policy.

Record a table after each append: old size, old capacity, whether growth occurred, new size, and new capacity. Use at least the first seventeen appends so several boundaries are crossed. Compare the counted relocation total with Chapter 2's model. Explain the additional work caused by allocating initialized integer capacity in this implementation; the movement ledger is not a complete instruction count.

Add move-assignment tests for a populated destination, an empty destination, and self-move. The supplied self-check preserves the original object on self-move. Do not generalize that particular behavior to every type. Verify the destination's values and the source's documented empty state after ordinary moves.

For a failure experiment, reject a deliberately impossible capacity through the arithmetic guard rather than trying to exhaust machine memory. Save the logical values beforehand and verify that they remain afterward. This tests one deterministic failure path. It does not claim that every allocator failure has been injected.

Design, but do not rush to implement, a generic version holding a type whose copy constructor may throw. Write the lifetime ledger for old elements, successfully constructed replacement elements, and unconstructed replacement storage. Identify which objects need destruction if construction number four fails. The exercise is complete only when the rollback state is described as clearly as the success state.

Review reasoning. Why can an index equal to capacity be invalid even when memory was allocated? Because the public sequence is bounded by size. Why does reserve not insert values? It changes storage availability, not membership. Why is integer-by-value useful for aliased append? It captures the value before growth. Why is temporary peak memory larger than the final buffer? The old and replacement allocations coexist until commit. Why not advertise this as a drop-in vector? It intentionally omits general element lifetime, allocator, copy, and iterator semantics.

Primary reference notes

Container and lifetime context: [vectors](#), [vector capacity](#), [unique pointers](#), and [object lifetime](#).

Chapter 4 — Building Linked Data Structures

Removing a middle list node seems to require only one pointer update. The first implementation updates the forward link but forgets the backward link and tail observer, so reverse traversal reaches a destroyed object. Linked structures trade contiguous movement for more explicit relationship maintenance.

This chapter implements singly and doubly linked lists with one owning direction. It uses unique pointers for ownership and raw pointers only for clearly identified observers.

A linked structure turns a sequence into a network of ownership and observation relationships. Its correctness depends on more than the values printed by a forward traversal. Every node must remain reachable through the intended owner, every observer must point to a live node or no node, and removal must not leave a second path that silently reaches released storage.

HarborWorks uses this chapter's lists as a laboratory for those relationships, not as an automatic replacement for vectors. First learn to draw the links and repair them. Then ask whether a node-based representation actually fits the workload. An operation advertised as constant-time may still require a preceding linear search to find its location.

4.1 Implementing Singly Linked Lists

A singly linked list stores a value and a link to the next node. The head identifies the beginning, and a null next link ends the chain. Traversal follows links rather than computing an indexed address.

A node's ownership link should have one unambiguous owner. The example lets the head own the first node and each node's unique next pointer own its successor.

4.1.1 Creating and Connecting Nodes

Creating a new node before changing the existing list keeps allocation failure from disturbing accepted state. To insert at the front, the new node takes ownership of the old head and the head then takes ownership of the new node.

There is no shared ownership cycle because every owning edge points forward. A raw pointer used during traversal borrows a node while the list remains valid. It must not delete the node or survive an erasure that removes it. The ownership graph and traversal graph happen to share a direction, but their roles remain conceptually distinct.

Separate a node's payload from its structural role. In the example a node contains an integer and an owning link to the next node. The list owns the first node. Following ownership from the head reaches every stored node exactly once. A traversal pointer temporarily observes a node; it does not create another obligation to delete it.

For the sequence one, two, three, the relationship is:

```
list.head -> [1 | owner] -> [2 | owner] -> [3 | null]
           ^ temporary observers may visit these nodes
```

The arrows represent exclusive ownership, not just arbitrary addresses. Replacing them all with shared ownership would obscure the simple lifetime model and add reference-management work. A shared pointer is appropriate when the application really needs shared lifetime, not because drawing ownership first seems inconvenient.

Creation should finish before publication. `push_front` allocates a candidate node, assigns its payload, connects its next owner to the previous head, and finally publishes the candidate as the new head. If allocation fails before the old head is moved, the existing list remains intact. The integer payload avoids a throwing assignment complication; a generic payload would need a more careful construction design.

Cycles are forbidden in this representation. A cycle would prevent an ordinary forward traversal from terminating and contradict the one-way exclusive-ownership structure. Even when smart pointers make some invalid states harder to construct, algorithms should still express the intended acyclic model. Types constrain possibilities; they do not automatically prove every sequence of pointer operations correct.

Node allocation also has a cost beyond the payload. Each node stores a link and may incur allocator overhead and alignment padding. This can be a large fraction of storage for a small integer. A list's asymptotic $O(n)$ memory statement hides those constants. Compare the complete representation, including allocation count, with contiguous storage before choosing it for millions of tiny records.

4.1.2 Inserting and Removing Elements

Erasing the first matching value walks a pointer to the owning link: initially the head owner, later a node's next owner. This unifies head and interior removal. On a match, the victim is temporarily owned locally and its successor is moved into the previous owning link.

The search is $O(n)$ in the worst case. Relinking after the position is known is constant work for the integer-node model. State the full cost when advertising an erase-by-value API; it is not constant-time merely because the final pointer operations are few.

Deleting a node is easiest to understand when you identify the object that owns it. For the first node, that owner is the list's head member. For an interior node, it is the previous node's next member. The worked `erase_first` stores a pointer to that owning `unique_ptr`, allowing one update pattern for both cases.

Suppose the list is one, two, three and the key is two. The search stops with `owner` pointing to node one's next member. Moving `*owner` into `victim` temporarily gives the function ownership of node two and its suffix. Moving `victim->next` back into `*owner` reconnects node one directly to node three. When `victim` is destroyed, it owns only node two, not the retained suffix.

The order matters. Destroying node two before extracting its next owner would also destroy the rest of the chain under this ownership model. Saving a raw pointer to node three would not preserve its lifetime. You must transfer the owning link, not merely remember an address.

The method removes the first equal value. With duplicates, the tie policy is part of the interface just as it was in Chapter 1. A function called `erase` might instead remove all equal values or require a known

iterator; none of those choices can be inferred from the data structure's name. Test duplicate sequences explicitly when extending the example.

Search cost and splice cost should be reported separately. Finding a value by traversing the list is linear in the number of visited nodes. Once the owning link is known, reconnecting it requires a constant number of pointer operations. Advertising only the second cost for `erase_first(value)` would misdescribe the complete operation.

Empty and absent cases need no special resource trick. An empty owner ends the search, and absence returns false without mutation. The same invariant that makes successful deletion safe should make unsuccessful deletion uneventful.

Singly Linked List with Head Insertion

Purpose. A singly linked list trades contiguous storage for cheap link-local insertion and pointer-based traversal.

```
#include <iostream>
#include <memory>
struct Node{int value;std::unique_ptr<Node>next;};
int main(){std::unique_ptr<Node>head;for(int x:{1,2,3}){auto
n=std::make_unique<Node>();n->value=x;n-
>next=std::move(head);head=std::move(n);}for(Node*p=head.get();p;p=p-
>next.get())std::cout<<p->value<<' ' ;}
```

How it works.

- Each node owns exactly one successor.
- Head insertion changes only the new node and head pointer, so it is $O(1)$.
- Traversal follows links until a null pointer terminates the chain.

Expected behavior/output. The output is `3 2 1`.

Experiment. Implement `push\_back` and compare its cost with and without storing a tail pointer.

4.2 Implementing Doubly Linked Lists

A doubly linked list adds backward navigation. The extra links enable reverse traversal but also add invariants: the first node has no predecessor, each next node points back to its predecessor, and the tail identifies the last node or is null for an empty list.

Make one direction owning and the reverse direction observing. Making both directions unique owners is impossible, while making both shared owners can create retention cycles.

4.2.1 Maintaining Previous and Next Links

Each node's next pointer owns its successor, while `previous` and the list's tail are raw observers. Appending prepares a new node, connects it through the old tail or head, and updates the tail observer only after ownership is established.

Erasure must repair both directions. The successor's previous pointer becomes the removed node's previous observer. If no successor exists, the tail becomes the previous node instead. Removing the final node must leave both head and tail empty. These cases deserve explicit tests because a correct forward traversal can hide a stale backward link.

A doubly linked list adds a reverse navigation path, not necessarily a second ownership path. In the example, forward links own nodes; `previous` and `tail_` are raw observers. This asymmetry avoids cycles of ownership while permitting traversal in both directions.

For every adjacent live pair `a` and `b`, if `a.next` owns `b`, then `b.previous` must observe `a`. The head's previous observer is null. The tail observes the final node, whose next owner is empty. In the empty state both head and tail are empty. These relationships are the representation invariant; merely checking that size looks plausible would miss most pointer defects.

Appending allocates and initializes a new node before changing the existing chain. Its previous observer is set to the old tail. If the list was nonempty, the old tail's next member receives ownership. Otherwise the head receives it. Only then is the new tail observer published. The last step must point to the node actually owned by the structure, not to a temporary object that will disappear.

Removing a node requires repairing both paths. After extracting its successor owner, set the successor's previous observer to the victim's predecessor. If there is no successor, the predecessor becomes the new tail. Removing the sole node therefore clears the head and sets tail to null. A subsequent append is an important regression because a stale tail may not be noticed until the next mutation.

A raw observer is not inherently unsafe; an observer with an unclear validity boundary is. The list controls when these observers are created, updated, and no longer used. Exposing a node pointer to arbitrary callers would require a documented invalidation contract. The example keeps nodes private and returns owned vectors for inspection, avoiding a public iterator design in this chapter.

Adding a cached size member would add another invariant to every mutation. Do not introduce it casually: every successful insertion and removal would need to update the count, while failed searches must leave it unchanged.

4.2.2 Traversing in Both Directions

Forward traversal begins at the head and follows owned next links through borrowed raw pointers. Reverse traversal begins at the tail and follows previous observers. Comparing the two sequences in reverse order is a useful consistency check.

Destruction uses an iterative loop that detaches the next owner before deleting the current head. This avoids recursively destroying a very deep unique-pointer chain through nested node destructors. The teaching lists delete copying and do not provide a custom move interface; adding either operation requires revisiting all observer links and ownership invariants.

Forward and reverse traversal are independent observations of one intended sequence. For a correct list, collecting values backward should equal reversing the collected forward values. That property is stronger than checking a single expected output because it can be applied after every generated operation.

However, two traversals can agree while both omit a leaked node disconnected from the structure. Behavioral checks should be combined with ownership reasoning and memory diagnostics. No single observation proves the entire representation. The node ownership chain makes release responsibilities visible; sanitizer runs add evidence for the pointer paths actually exercised.

Traversal must also terminate. A broken previous link can create a reverse cycle even though forward ownership remains acyclic. When building diagnostic validators for an unsafe experimental structure, bound the number of visited nodes or track visited addresses rather than allowing a test to hang indefinitely. Such a validator is diagnostic machinery, not part of the desired steady-state traversal cost.

Destruction has its own traversal problem. Letting a very long chain of nested `unique_ptr` members recursively destroy itself can consume one stack frame per node. The worked destructor repeatedly extracts the next owner and releases the current node after its next link has been emptied. This makes cleanup iterative for the chain. The algorithmic cost remains linear, but the call-stack depth no longer grows with the list length.

The doubly linked destructor does not need to repair every previous observer on the way out because those observers are no longer used during destruction. That is a narrow lifetime boundary, not permission to leave them stale during ordinary operations. An invariant required between public calls can be temporarily relaxed during a private cleanup phase when no observer can inspect it.

Think carefully before adding callbacks to traversal or destruction. A callback that mutates the same structure can invalidate the currently visited node or its next link. The supplied routines do not support reentrant mutation. A more flexible interface would need a new contract and new tests, not just a function parameter.

Bidirectional Links with Explicit Previous and Next Pointers

Purpose. Doubly linked lists spend extra pointer storage to support efficient traversal and removal from either direction.

```
#include <iostream>
struct Node{int value;Node*prev{};Node*next{};};
int main(){Node
a{1},b{2},c{3};a.next=&b;b.prev=&a;b.next=&c;c.prev=&b;for(Node*p=&c;p=p-
>prev)std::cout<<p->value<<' ' ;}
```

How it works.

- Every interior node has links in both directions.
- Updating a relation requires keeping the forward and backward links consistent.
- Reverse traversal begins at the tail and follows `prev`.

Expected behavior/output. The output is `3 2 1`.

Experiment. Insert a new node between `b` and `c`; list every pointer that must be updated.

Worked example — Insert and erase nodes while preserving both traversal directions

Requirement and design

Build a singly linked sequence one, two, three, erase middle and endpoint values, and verify the empty state. Build a doubly linked sequence, verify reverse traversal, erase the middle and endpoints, and append again after becoming empty.

The structures own nodes exclusively in the forward direction. Backward links and tail are observers only.

Draw every affected link

Before reading the complete source, draw removal of the middle node from one, two, three. Mark the owner of the victim, the victim's next owner, its previous observer, and the tail observer. Then draw the two endpoint removals. The middle case exercises reconnection, the last-node case exercises tail repair, and the sole-node case establishes the empty state.

The code uses temporary owners named `node` and `victim` deliberately. Their lifetimes explain who cleans up if the operation exits. A raw pointer named `new_tail` is used only to remember the address of a node whose ownership is about to move into the list. Moving a `unique_ptr` changes ownership of the node, not the node's address.

Neither class implements copying or moving. Copy operations are explicitly deleted, and user-declared destruction affects automatically generated special members. Do not describe these classes as fully general movable containers merely because they contain movable smart pointers. Book II's special-member rules still apply to the surrounding class.

The inspection functions allocate result vectors. They are useful for tests but are not constant-memory traversals. When reporting the cost of `forward()` or `values()`, include the output allocation and copied values rather than counting only link following.

Complete implementation

The complete, buildable chapter project is `source/book3/ch04/`. The following files are reproduced from that project; the publishing script inserts the source directly so the listing and companion remain synchronized.

main.cpp

```
#include "course_test.hpp"
#include <memory>
#include <optional>
#include <vector>
#include <utility>

class SinglyList {
    struct Node { int value; std::unique_ptr<Node> next; };
    std::unique_ptr<Node> head_;
public:
    SinglyList() = default;
    SinglyList(const SinglyList&) = delete;
    SinglyList& operator=(const SinglyList&) = delete;
    ~SinglyList() { while (head_) { auto next = std::move(head_>next); head_ =
std::move(next); } }
    void push_front(int value) {
        auto node = std::make_unique<Node>();
        node->value = value;
        node->next = std::move(head_);
        head_ = std::move(node);
    }
    bool erase_first(int value) {
        auto* owner = &head_;
        while (*owner && (*owner->value != value) owner = &(*owner->next);
        if (!*owner) return false;
        auto victim = std::move(*owner);
        *owner = std::move(victim->next);
        return true;
    }
    std::vector<int> values() const {
        std::vector<int> result;
        for (auto* node = head_.get(); node; node = node->next.get())
            result.push_back(node->value);
        return result;
    }
};

class DoublyList {
    struct Node { int value; Node* previous{}; std::unique_ptr<Node> next; };
    std::unique_ptr<Node> head_;
    Node* tail_{}; // Observer, not an additional owner.
public:
    DoublyList() = default;
    DoublyList(const DoublyList&) = delete;
```

```

DoublyList& operator=(const DoublyList&) = delete;
~DoublyList() { while (head_) { auto next = std::move(head_>next); head_ =
std::move(next); } }
void push_back(int value) {
    auto node = std::make_unique<Node>();
    node->value = value; node->previous = tail_;
    Node* new_tail = node.get();
    if (tail_) tail_>next = std::move(node);
    else head_ = std::move(node);
    tail_ = new_tail;
}
bool erase_first(int value) {
    auto* owner = &head_;
    while (*owner && (*owner)->value != value) owner = &(*owner)->next;
    if (!*owner) return false;
    auto victim = std::move(*owner);
    Node* previous = victim->previous;
    *owner = std::move(victim->next);
    if (*owner) (*owner)->previous = previous;
    else tail_ = previous;
    return true;
}
std::vector<int> forward() const {
    std::vector<int> result;
    for (auto* p = head_.get(); p; p = p->next.get()) result.push_back(p->value);
    return result;
}
std::vector<int> backward() const {
    std::vector<int> result;
    for (auto* p = tail_; p; p = p->previous) result.push_back(p->value);
    return result;
}
};

int main() {
    SinglyList single;
    for (int value : {3, 2, 1}) single.push_front(value);
    CHECK(single.values() == std::vector<int>({1, 2, 3}));
    CHECK(single.erase_first(2));
    CHECK(!single.erase_first(9));
    CHECK(single.erase_first(1) && single.erase_first(3));
    CHECK(single.values().empty());
    DoublyList dual;
    for (int value : {1, 2, 3}) dual.push_back(value);
    CHECK(dual.backward() == std::vector<int>({3, 2, 1}));
    CHECK(dual.erase_first(2));
    CHECK(dual.forward() == std::vector<int>({1, 3}));
    CHECK(dual.backward() == std::vector<int>({3, 1}));
    CHECK(dual.erase_first(3) && dual.erase_first(1));
    CHECK(dual.forward().empty() && dual.backward().empty());
    dual.push_back(4);
    CHECK(dual.backward() == std::vector<int>({4}));
    course::report();
}

```

Build, run, and inspect

Run this block from the top-level extracted package. The fixture runner supplies input where needed and checks exit status and output in a temporary working directory.

```

cmake -S source/book3/ch04 -B build/b3-ch04 \
  -DCMAKE_BUILD_TYPE=Debug

```

```
cmake --build build/b3-ch04 --parallel 2
ctest --test-dir build/b3-ch04 --output-on-failure
```

For a multi-configuration generator on Windows, append `--config Debug` to the build command and `-C Debug` to CTest. The executable target is `b3_ch04`.

Evidence and interpretation

The singly linked erase routine handles head and interior removal through the same owning-link mechanism. Searching for a missing value returns false without mutation. Removing the remaining nodes yields an empty sequence.

For the doubly linked list, removing two leaves forward one, three and backward three, one. Removing three repairs the tail, and removing one clears the empty-state observers. Appending four afterward verifies that empty-state cleanup did not leave a stale tail influencing future insertion.

The tests inspect observable traversals, while the implementation’s ownership types constrain release behavior. Sanitizer execution adds evidence for the exercised pointer paths. None of those checks alone proves every future mutation is correct; the forward/backward invariants remain the review model.

Failure clinic — Forward output hid a broken reverse chain

A list prints one, three after removing two, so the first test passes. Reverse traversal then crashes or prints an old value. The forward owner was repaired, but the successor’s previous observer still points to the removed node. The right repair is a structural update, not suppressing the reverse test.

If appending after deleting the last element fails, inspect `tail_`. A stale observer may survive an otherwise empty head. Add the exact sequence “append, erase sole element, append again” as a regression. Testing only construction and destruction never crosses that reusable-empty-state boundary.

If deleting the head unexpectedly deletes every node, inspect ownership transfer order. Saving `head_ ->next.get()` does not keep the suffix alive. The next `unique_ptr` must be moved out before the victim is released. The raw address and the owner are different kinds of evidence.

If cleanup fails only on a very long sequence, distinguish a memory leak from recursive stack exhaustion. Both can look like a resource failure, but they require different fixes. The iterative destructor is a structural solution to depth, not a reason to increase process stack limits and declare the algorithm repaired.

Practice and implementation lab

Add tests for duplicate values, removal from an empty list, a singleton, and removing the first of several equal values. After every doubly linked mutation, compare forward traversal with the reverse of backward traversal.

Add an invariant-checking helper that confirms predecessor relationships and the tail endpoint without mutating the list. Then design a move constructor and explain which observer pointers can remain valid after ownership transfer and which enclosing-object references would require repair.

Review answers and solution notes

Erase-by-value includes linear search even when relinking is constant work. A pointer to the owning link makes head removal and interior removal follow the same ownership operation. The tail must be updated when the erased node has no successor.

Forward ownership with backward observation avoids ownership cycles. Iterative destruction prevents a long chain from recursively nesting destruction calls. Copy and move semantics are separate features requiring explicit design; raw observers cannot be ignored merely because unique pointers handle deallocation.

Engineering takeaway. Treat every link mutation as an invariant update, with one owning direction and explicitly repaired observer relationships.

Guided lab — Validate operation sequences, not isolated calls

Add an exercise method that removes every occurrence of a requested integer. Start with `[2, 1, 2, 2, 3, 2]`. After removing two, both traversals of the doubly linked version should describe `[1, 3]`, in opposite directions. After removing a missing value, the sequence should be unchanged. Explain when the owning-link cursor should remain at the same link rather than advance after deletion.

Build a small model using `std::vector<int>` or `std::list<int>`. Generate a deterministic sequence of push and erase operations, apply each to both representations, and compare after every operation. Restrict the value domain so duplicates are common. A final-state-only comparison can miss a transient corruption that happens to be overwritten later.

For a doubly linked diagnostic validator, check the predecessor relation at every forward step, remember the final visited node, and compare it with tail. Also verify that backward values reverse the forward sequence. Keep the validator outside a performance measurement: its additional full scans intentionally cost linear work per mutation.

Design an iterator interface on paper. State which insertions preserve an iterator, which erasures invalidate it, and whether an iterator to the erased node can be compared or incremented afterward. Do not assume that rules from `std::list` automatically apply to a custom type. An interface must promise only what its representation and implementation maintain.

Finally, compare two workloads. The first appends ten thousand readings and scans all of them repeatedly. The second repeatedly splices at already known positions while preserving unrelated node addresses. Explain why the same container need not be appropriate for both. Include lookup, allocation, output construction, and cleanup in the comparison.

Review reasoning. Why use one owning direction? It gives each node one release responsibility and avoids ownership cycles. Why is a tail pointer not automatically an owner? It only observes a node already owned by the head chain. Why test append after emptying? It exposes stale endpoint state. Why is deletion by value not generally constant time? The value must first be found. Why return vectors in the example's inspection API? They provide independent observable snapshots without exposing node lifetime to callers.

Primary reference notes

Ownership mechanisms: [unique pointers](#), [object lifetime](#), and [list container contracts](#). The custom link algorithms and invariants are developed in this chapter.

Chapter 5 — Implementing Last-In, First-Out Structures

A bracket validator needs to match each closing bracket with the most recently opened unmatched bracket. A postfix evaluator similarly needs the most recent operands first. Both problems share a last-in, first-out access policy even though their domain meanings differ.

A stack captures that policy through a narrow interface. This chapter implements it over a vector, then uses it in two complete algorithms with explicit malformed-input behavior.

A stack is a rule about which pending item is used next. It is not necessarily a special region of process memory, and it is not defined by whether its implementation uses an array or linked nodes. The last item accepted becomes the first item removed. That rule makes a stack a natural record of unfinished nested work.

This chapter connects the abstraction to two visible tasks: checking delimiters and evaluating postfix arithmetic. Both examples require a precise grammar and explicit failure behavior. A correct stack can support an incorrect parser if the surrounding algorithm removes operands in the wrong order or silently accepts an unfinished expression. Follow the contract all the way from input tokens to the final result.

5.1 Understanding Stack Behavior

A stack permits insertion and removal at one logical end, called the top. The last pushed element is the first popped element. Hiding arbitrary indexing prevents clients from depending on operations the abstraction does not promise.

The underlying container is an implementation choice. A vector provides convenient owned storage and amortized append behavior for this teaching stack.

5.1.1 Performing Push and Pop Operations

Push appends a value. Pop returns the top value and removes it, or returns no value when the stack is empty under this interface. The optional result makes an empty pop distinguishable from a valid stored zero or character.

For the integer and character instantiations used here, moving the top value is nonthrowing. The unconstrained template's strong exception behavior is not promised for arbitrary types with throwing moves: an exception while extracting or returning a value can leave a moved-from element or a changed stack. A production generic stack must document or constrain those semantics rather than inherit an accidental guarantee from scalar tests.

The minimal stack state is an ordered sequence of stored values. Push adds at the end designated as the top; pop removes from that same end. The observable order is last-in, first-out. If values one, two, and three are pushed in that order, successful pops must produce three, two, and one. An empty pop is a separate outcome, not an invented zero.

The worked wrapper uses a vector as its storage. This makes the ownership model familiar from Book II and the preceding chapter. Appending may occasionally reallocate, while removing the final logical element avoids shifting an entire prefix. A linked implementation could provide the same stack behavior with different allocation and locality tradeoffs. Clients should depend on the interface contract, not reach through to the underlying storage.

Separate removal from observation. `pop()` returns an optional value here, combining “is there an item?” with transfer of that item. The standard stack adaptor instead exposes `top()` and a removing `pop()` with different signatures. Neither interface should be assumed from the word stack. When translating textbook pseudocode, check the actual library operation before writing C++ around it.

Generic value transfer adds an exception question. The example first moves the last value into a local object, removes the stored element, and then returns it. For the exercised integer and character types, those moves do not throw. A type with a throwing move could leave a moved-from stored element or throw while constructing the return value after removal. The template spelling alone does not establish an unchanged-on-failure guarantee for every T.

Describe the promised state after failure rather than using “safe” without qualification. A bounded integer evaluator can be completely appropriate with this wrapper, while a generic transactional work queue may need a stronger extraction design. Reusing an abstraction responsibly means checking its semantic requirements, not just whether it compiles for a new type.

5.1.2 Examining the Top Element

Top observes the current last element without removing it and throws when the stack is empty. Returning a const reference avoids a copy but creates a borrow invalidated by removal, destruction, or relevant vector reallocation.

A caller that needs the value after the stack changes should take an independent value. Do not store a top reference, push enough elements to reallocate, and then use the old reference. The stack abstraction narrows operations but still inherits lifetime consequences from its chosen storage implementation.

The top element is the next value that would be removed. Looking at it should not change stack membership. In the example, `top()` returns a const reference and throws when the stack is empty. The caller must either establish nonemptiness or handle that documented error. It cannot infer validity because a previous unrelated operation succeeded.

A reference returned by top is a borrow. If a later push reallocates the underlying vector, the reference can become invalid. Popping the observed element ends its lifetime in the sequence. Keeping a top reference across arbitrary mutations therefore requires care even when the interface does not expose iterators. Abstraction hides representation choices; it does not eliminate lifetime dependencies.

For delimiter matching, a temporary copy of a character is sufficient. There is no benefit in retaining a long-lived borrow to the opening delimiter after popping it. For large objects, copying may be more expensive, but the ownership question remains. Decide whether a consumer needs an observation, a transferred value, or a stable independently owned snapshot.

Testing top requires more than checking the value after one push. Verify that two consecutive observations return the same value without reducing size. Then pop and verify that the previous item

becomes top. Finally empty the structure and test the documented empty behavior. The relationship between observation and removal is part of the abstraction.

Avoid the check-then-act assumption in a concurrent setting. Two separate operations, `empty()` followed by `top()`, are not automatically atomic with respect to another thread removing an item. This volume's wrapper is sequential and provides no synchronization. A thread-safe stack would need an interface and implementation designed around concurrent state changes, not a mutex added only to one of its methods.

The phrase “top of the stack” is therefore a logical position under a particular access policy. It should not be confused with a CPU stack pointer or a promise that values are stored in a process's call-stack region.

Last-In, First-Out Operations with `std::stack`

Purpose. A stack's LIFO discipline is defined by its restricted interface rather than by random positional access.

```
#include <iostream>
#include <stack>
int main(){std::stack<int>s;s.push(1);s.push(2);s.push(3);while(!s.empty())
{std::cout<<s.top()<<' ';s.pop();}}
```

How it works.

- `push` adds to the top.
- `top` observes the most recently added element.
- `pop` removes that element, exposing the previous one.

Expected behavior/output. The output is `3 2 1`.

Experiment. Push the same values in a different order and predict the pop sequence.

5.2 Applying Stacks to Real Problems

Stacks are useful when the next decision depends on the most recent unfinished work. Their correctness often follows a statement about what each stored item represents: unmatched openings, partial expression values, or deferred traversal tasks.

State that interpretation explicitly. A stack of integers without a domain invariant is just another container.

5.2.1 Evaluating Expressions

In postfix notation, operands precede their operator. On a binary operator, pop the right operand first and the left operand second, then push the result. Reversing that order changes subtraction even though addition and multiplication might hide the defect.

The evaluator accepts plus, minus, and multiplication over values bounded to plus or minus one million. It computes intermediates in `long long`, checks the result domain, and rejects missing operands, malformed numeric tokens, extra final operands, and out-of-range results. A valid expression leaves exactly one value. The bounded operands make multiplication representable in the chosen wide intermediate on the documented baseline.

Postfix notation places an operator after its operands. For `2 3 + 4 *`, the first two tokens create pending values, plus replaces them with five, four adds another pending value, and multiply produces twenty. The stack holds the results of subexpressions that are complete but not yet combined into a larger expression.

Operand order matters for noncommutative operations. On reading subtraction, the first popped value is the right operand and the second is the left. Thus `9 2 -` means nine minus two. A test suite containing only addition and multiplication can miss a reversed-pop defect because those operations are commutative over the small exact integer domain.

Tokenization and evaluation are different stages. The worked evaluator accepts an already separated vector of strings. It does not implement an infix grammar, precedence rules, quotes, or a general command language. Strict `from_chars` conversion requires consuming the complete numeric token; `12x` must not silently become twelve. Decide whether a leading plus sign or whitespace inside a token is allowed instead of relying on another parser's conventions.

The numeric domain is deliberately bounded to one million in magnitude. Each accepted operand and intermediate result remains within that range. Multiplication is evaluated in a wider `long long`, and the product of two accepted magnitudes is at most one trillion, within the selected wide type on the supported environment. Only after range checking is the result narrowed. Checking after an overflowing narrow multiplication would be too late.

The final stack must contain exactly one value. An operator with too few operands is malformed, but so is an input that leaves several unrelated values. Empty input produces no expression result. These are grammar failures even when every individual numeric token is valid.

A richer evaluator should return structured errors such as token position and error category. The optional result here keeps the first example focused on stack behavior. It is sufficient for acceptance or rejection, but not for a user-facing diagnostic explaining why evaluation failed.

5.2.2 Managing Traversal State

For bracket matching, the stack holds unmatched opening brackets in encounter order. A closing bracket must match the most recent opening; otherwise nesting is invalid. The string `( [ ] )` contains equal counts of each bracket kind but is not properly nested, demonstrating why counts alone are insufficient.

An explicit stack can also replace recursive traversal state. Store the work that remains to be processed and choose push order to obtain the required traversal order. This can avoid call-stack depth limits, but it does not remove memory complexity: deferred work still occupies storage in the explicit stack.

A delimiter checker records unfinished nesting. Each opening delimiter is pushed. Each closing delimiter must match the most recent unmatched opening. At end-of-input, no unmatched openings may remain. The stack is a compact representation of the obligations that the input has created but not yet discharged.

Consider ([]). Counting opening and closing delimiters separately would find matching totals, but the nesting is invalid. After reading the first two characters, the top opening is a square bracket. The next character is a closing parenthesis, so rejection is immediate. The order of pending obligations, not merely their number, is what the stack preserves.

The supplied grammar recognizes parentheses and square brackets and ignores other characters. It is not a C++ source parser. For example, it does not know that a delimiter inside a quoted string literal might be ordinary text in another language. A diagnostic tool using this function must describe its simple grammar rather than advertise syntax validation for an entire programming language.

Explicit stacks also support nonrecursive tree and graph traversals. Each stored item can represent a node still to visit, or a richer frame containing the node and which child comes next. Pushing only node identifiers is enough for some preorder traversals, but reproducing recursive postorder may require remembering whether child work has completed. The data stored in a frame follows from the unfinished work it must represent.

Space usage depends on pending work. A delimiter string of n opening parentheses needs n stored openings before failure at the end. A shallow alternating sequence may keep only one pending opening. Worst-case auxiliary space is therefore linear even though many ordinary inputs use little storage.

An explicit stack does not make an unbounded search free. It relocates control state from recursive calls into a container where you can inspect and potentially bound it. Chapter 17 adds a supported problem-size limit and reversible state to a backtracking search rather than treating recursion depth as an invisible resource.

Balanced-Delimiter Check Using a Stack

Purpose. Nested syntax naturally requires LIFO matching, which is why a stack models the problem directly.

```
#include <iostream>
#include <stack>
#include <string>
bool balanced(const std::string&s){std::stack<char>st;for(char c:s){if(c=='('||
c=='['||c=='{' )st.push(c);else if(c==')'||c==']'||c=='}'){if(st.empty())return
false;char o=st.top();st.pop();if((o=='('&&c!=')')||(o=='['&&c!=']')||(
o=='{'&&c!='}') )return false;}}return st.empty();}
int main(){std::cout<<std::boolalpha<<balanced("{[()]}" )<<'\n';}
```

How it works.

- Opening delimiters are pushed in encounter order.
- Each closing delimiter must match the most recent unmatched opener.
- The final stack must be empty so no opener remains unmatched.

Expected behavior/output. The output is `true false`.

Experiment. Add quoted text handling and decide whether delimiters inside strings should count.

Worked example — Validate nested brackets and evaluate bounded postfix expressions

Requirement and design

Implement a stack with explicit empty behavior. Accept balanced parentheses and square brackets, ignoring other characters under the validator's simple grammar. Evaluate tokenized postfix expressions with strict integer conversion and a bounded arithmetic domain.

Reject incomplete expressions and expressions leaving more than one result. The evaluator receives already separated tokens; it does not parse infix precedence or arbitrary whitespace text.

Trace the pending state

Write the stack after each token of `8 3 - 2 *:` `[8]`, `[8,3]`, `[5]`, `[5,2]`, `[10]`. The notation lists bottom to top from left to right. This convention should remain fixed in your notes; reversing the drawing halfway through a trace creates the same confusion as reversing operand order in code.

For bracket validation, trace `( [ ] [ ]` and `( [ ) ]` side by side. Both begin with the same two pending openings, but the third character discharges a different obligation. The earliest point at which the contract becomes impossible is a useful failure location. Continuing after that point is unnecessary for a simple yes/no validator.

Inspect the conversion path before arithmetic. A token can fail because it is empty, not a complete integer, outside the parser's representable range, or outside the evaluator's smaller teaching domain. These conditions are intentionally checked before pushing a value. Once a value enters the arithmetic stack, the domain bound becomes an invariant that justifies the wider intermediate calculation.

The complete implementation shares the stack wrapper between both algorithms, but their acceptance conditions remain separate. Reusing a container does not mean reusing a grammar or error contract.

Complete implementation

The complete, buildable chapter project is `source/book3/ch05/`. The following files are reproduced from that project; the publishing script inserts the source directly so the listing and companion remain synchronized.

main.cpp

```
#include "course_test.hpp"
#include <charconv>
#include <optional>
#include <string>
#include <string_view>
#include <vector>
#include <utility>

template<class T> class Stack {
    std::vector<T> data_;
public:
```

```

bool empty() const noexcept { return data_.empty(); }
std::size_t size() const noexcept { return data_.size(); }
void push(T value) { data_.push_back(std::move(value)); }
std::optional<T> pop() {
    if (data_.empty()) return std::nullopt;
    T result = std::move(data_.back());
    data_.pop_back();
    return result;
}
const T& top() const {
    if (data_.empty()) throw std::out_of_range("empty stack");
    return data_.back();
}
};

bool balanced(std::string_view text) {
    Stack<char> openings;
    for (char c : text) {
        if (c == '(' || c == '[') openings.push(c);
        else if (c == ')' || c == ']') {
            const auto open = openings.pop();
            if (!open || (*open == '(' && c != ')') || (*open == '[' && c != ']'))
                return false;
        }
    }
    return openings.empty();
}

std::optional<int> postfix(const std::vector<std::string>& tokens) {
    Stack<int> values;
    for (const std::string& token : tokens) {
        if (token == "+" || token == "-" || token == "*") {
            const auto right = values.pop(), left = values.pop();
            if (!left || !right) return std::nullopt;
            long long value{};
            if (token == "+") value = static_cast<long long>(*left) + *right;
            if (token == "-") value = static_cast<long long>(*left) - *right;
            if (token == "*") value = static_cast<long long>(*left) * *right;
            if (value < -1000000 || value > 1000000) return std::nullopt;
            values.push(static_cast<int>(value));
        } else {
            if (token.empty()) return std::nullopt;
            int value{};
            const auto p = std::from_chars(token.data(), token.data() + token.size(),
value);
            if (p.ec != std::errc{} || p.ptr != token.data() + token.size() ||
                value < -1000000 || value > 1000000) return std::nullopt;
            values.push(value);
        }
    }
    if (values.size() != 1) return std::nullopt;
    return values.pop();
}

int main() {
    CHECK(balanced("([])[]") && balanced(""));
    CHECK(!balanced("([])") && !balanced("("));
    CHECK(postfix({"2", "3", "+", "4", "*"}) == 20);
    CHECK(postfix({"9", "2", "-"}) == 7);
    CHECK(!postfix({"+"}) && !postfix({"2", "3"}));
    CHECK(!postfix({"1000000", "1000000", "*"}));
    Stack<int> stack;
    CHECK(!stack.pop());
    CHECK(course::throws<std::out_of_range>([&] { (void)stack.top(); }));
    course::report();
}

```

Build, run, and inspect

Run this block from the top-level extracted package. The fixture runner supplies input where needed and checks exit status and output in a temporary working directory.

```
cmake -S source/book3/ch05 -B build/b3-ch05 \
  -DCMAKE_BUILD_TYPE=Debug
cmake --build build/b3-ch05 --parallel 2
ctest --test-dir build/b3-ch05 --output-on-failure
```

For a multi-configuration generator on Windows, append `--config Debug` to the build command and `-C Debug` to CTest. The executable target is `b3_ch05`.

Evidence and interpretation

Balanced `( [ ] ) [ ]` and empty text succeed, while crossed nesting and an unmatched opening fail. The postfix sequence two, three, plus, four, multiply produces twenty. Nine, two, minus produces seven, directly checking operand order.

A lone operator fails for missing operands, two numbers without an operator fail because two values remain, and multiplying one million by one million is rejected by the domain limit after a safe wide intermediate calculation. Empty pop returns no value, while empty top throws according to the two distinct interface choices.

Each algorithm has a linear token or character scan. Auxiliary storage can grow linearly with unmatched openings or pending operands. The stack operations are amortized constant under the vector/scalar cost model, so total processing is linear apart from token parsing costs proportional to token lengths.

Failure clinic — The arithmetic was right for the wrong operands

An evaluator passes every addition example but returns negative seven for `9 2 -`. Inspect the pop order before investigating integer representation. The first removed item is the right operand. This is a semantic defect that ordinary memory sanitizers will not detect because every access can be valid.

A validator accepts `( [ ] )` after checking only counts. The missing information is nesting order. Replace the count-only state with pending opening types and compare against the most recent one. This is a representation repair, not merely another conditional at end-of-input.

A numeric token `4extra` is accepted as four. Inspect whether the conversion checked the returned end pointer. Successful conversion of a prefix is not successful parsing of the complete token. Tests should contain valid prefixes followed by invalid suffixes, not just obviously nonnumeric words.

Finally, a generic stack extraction throws after an element has been removed. Do not claim rollback merely because vector owns its memory correctly. Cleanup and state restoration are different guarantees. For the chapter's scalar domain the issue does not arise, but a reuse with throwing element transfers must be reviewed at the new boundary.

Practice and implementation lab

Add tests for a closing bracket with no opening, nested depth, ignored ordinary text, negative numeric tokens, invalid suffixes, and subtraction whose operand order matters. Define whether braces should be supported and add them consistently to both opening and matching logic.

Extend the evaluator with division. Specify division by zero, integer truncation, and the signed-minimum divided by minus-one hazard for a broader domain before coding. Keep the current bounded arithmetic invariant visible in every new operation.

Review answers and solution notes

The first popped operand is the right operand, which is essential for noncommutative operations. A valid postfix expression ends with exactly one stack value. Bracket counts alone cannot verify nesting order.

Optional pop avoids a sentinel collision, while a top reference remains a borrow. The scalar implementation does not establish strong guarantees for arbitrary throwing element types. Replacing recursion with an explicit stack changes where traversal state is stored, not whether that state consumes memory.

Engineering takeaway. Use a stack when the most recent unfinished item must be resolved first, and define empty, malformed, and arithmetic-failure behavior explicitly.

Guided lab — Build a diagnostic evaluator

Create a result type that distinguishes a numeric value from an evaluation error containing the token index and a category. Suggested categories are malformed integer, unsupported token, insufficient operands, result outside the supported domain, and unfinished expression. The categories are part of your exercise design; keep them stable enough that tests can assert them without matching an entire prose message.

Add division under a precise integer contract. Reject division by zero before evaluating it. State how negative division truncates in the selected C++ integer model and include examples with positive and negative operands. Keep the domain bounded so the exceptional minimum-integer divided by minus-one case is excluded, or explicitly handle it before arithmetic. Do not write a test that invokes undefined behavior and then asks whether an exception appeared.

For delimiter checking, return the position of the first impossible closing delimiter. At end-of-input, report that an opening remains unmatched. Decide which opening position to report when several remain; there is no single unavoidable policy. Store positions alongside opening types so the information exists when needed.

Generate short strings over the alphabet `()[]` and compare your checker with an independent recursive parser for the same deliberately small grammar. A useful independent model parses a sequence of correctly nested groups, rather than duplicating the exact same stack algorithm with different names. Keep the domain small enough that enumeration terminates quickly.

Acceptance evidence. Include subtraction, empty input, extra operands, incomplete numeric tokens, nested mismatch, unmatched opening, unmatched closing, and both arithmetic boundaries. Preserve the input tokens on failure because the evaluator's job is to compute, not rewrite the caller's data.

Review reasoning. Why can counts not prove delimiter nesting? They discard order. Why is a stack template not automatically strongly exception-safe for all types? Element moves and return construction can throw at different stages. Why check full token consumption? Prefix conversion is a weaker contract. Why is one remaining value required? A complete expression produces one combined result. Why does sequential correctness not imply thread safety? No operation establishes synchronization or an atomic multi-step observation.

Primary reference notes

Stack implementation context: [container adaptors](#), [vectors](#), [optional](#), and [character conversion](#).

Chapter 6 — Implementing Queues and Double-Ended Queues

An acquisition stage produces samples faster than a consumer can process them for a short burst. A queue preserves arrival order, but an unbounded queue can turn a temporary imbalance into exhausted memory. A bounded ring queue makes the full-state policy part of the interface.

This chapter implements first-in, first-out behavior over reusable storage and compares it with a deque. It is a single-threaded data-structure lesson; adding producers on other threads requires synchronization beyond correct indexing.

A queue preserves arrival order, but arrival order alone is not a capacity policy. When HarborWorks receives a burst of synthetic readings, an unbounded queue can keep accepting data while memory usage grows. A bounded queue forces the application to decide what “full” means: reject, wait, drop, overwrite, or redirect. That choice affects data integrity and latency more than the spelling of the container type.

The worked ring queue rejects an insertion when every slot is occupied. It never silently overwrites an unread value and never blocks a thread. Those are deliberate sequential semantics. Later systems work can build backpressure around them, but adding a producer and consumer thread does not automatically make this implementation concurrent.

6.1 Understanding First-In, First-Out Behavior

A queue removes values in the same order they were inserted. Its abstraction exposes a front for removal and a back for insertion. A bounded queue also has a capacity and a policy for an insertion attempted while full.

The example rejects a full insertion without changing existing values. Dropping old data, blocking, and overwriting are different policies that would need different contracts.

6.1.1 Enqueuing and Dequeuing Values

Enqueue adds a value after all currently queued values. Dequeue returns the oldest remaining value or reports an empty queue. The optional result distinguishes an empty queue from a legitimate stored zero.

For the ring queue, the invariant is $0 \leq \text{size} \leq \text{capacity}$, with head identifying the oldest occupied logical position when size is positive. A full insertion returns false before any slot is modified. This lets the caller decide whether to retry, apply backpressure, or record a dropped sample; the container does not silently make that operational decision.

First-in, first-out means the earliest accepted item is removed before later accepted items. Rejected items do not enter the sequence. If pushes of one, two, and three succeed, then pops return one, two, and three. A fourth push rejected because of capacity must not appear in a later pop, even if its payload happened to be written into some temporary location.

Define the transition in terms of logical state. A successful push appends one value and increases size by one. A failed full push leaves both size and sequence unchanged. A successful pop returns the front value, removes exactly that value, and decreases size by one. An empty pop reports absence without changing state. These statements are simple enough to test after every generated operation.

A queue does not require moving all remaining elements after each pop. A vector with repeated `erase(begin())` would preserve FIFO behavior but shift its remaining suffix. A ring buffer instead advances a front index, while a deque uses another representation supporting efficient endpoint operations. The abstract behavior and the physical mechanism should be chosen independently.

Application semantics determine whether retrying a rejected push is safe. If a producer retries the same event, its stable identifier can help prevent accidental duplication at a higher layer. The queue itself only knows whether it accepted this call's integer. It does not implement exactly-once delivery, durable acknowledgement, or recovery after a process crash.

Likewise, returning an item does not mean its downstream processing succeeded. Once an item is popped, the queue no longer owns the pending obligation unless the application explicitly requeues or records it. A production work system may need acknowledgement and retry states beyond a simple FIFO. Recognizing that gap prevents a teaching container from being mistaken for a complete messaging service.

For this chapter, keep the boundary narrow: one thread, integer values, fixed capacity, explicit acceptance and absence. The clarity of that contract is what makes a model-based test meaningful.

6.1.2 Implementing Circular Buffers

A circular buffer maps logical position k to $(\text{head} + k) \% \text{capacity}$. The next insertion position is $(\text{head} + \text{size}) \% \text{capacity}$; removing advances head by one modulo capacity and decreases size. Keeping an explicit size distinguishes full and empty states even when index positions coincide.

Capacity is validated before allocating storage, including rejection of zero to prevent modulo-by-zero. The teaching upper bound also keeps index addition comfortably representable on the supported platform. Each slot is optional, making occupied and released positions inspectable. Integer storage and fixed capacity keep push and pop free of further allocation after construction in this implementation.

A ring maps a logical sequence onto a fixed physical array. The head index identifies the next item to remove, and size records how many slots belong to the queue. The next insertion position is $(\text{head} + \text{size}) \% \text{capacity}$. Modulo wraps a position at the end of the array back to zero.

With capacity four, push ten, twenty, and thirty, then pop ten and twenty. The head is now at physical slot two and size is one. Pushing forty, fifty, and sixty fills slots three, zero, and one. The logical sequence is thirty, forty, fifty, sixty even though its physical representation crosses the end of the array. Logical order is determined by head and size, not by printing the backing array from index zero.

Using an explicit size avoids an ambiguous `head == tail` state. Designs storing only two indices often reserve one slot or add a separate full flag to distinguish empty from full. These are valid alternatives with different invariants. Do not combine formulas from one representation with the empty/full test of another.

The worked slots contain optional integers. A popped slot is reset, making the physical empty state visible and releasing any payload ownership that a richer element might carry. Head advances after the value is captured, and size decreases only on a successful pop. A generic version with throwing value moves would need additional exception analysis; the scalar example keeps those transfers nonthrowing.

Capacity must be positive because the index expression divides by it. The constructor validates that requirement and a deliberate upper teaching limit. Arithmetic also needs to remain representable for the chosen index type. A more general unbounded-capacity implementation can compute wrapped indices without potentially overflowing `head + size`; the fixed supported domain here keeps the arithmetic manageable.

The ring's storage stays allocated after it becomes empty. That is intentional capacity reuse, not a memory leak. Logical emptiness and releasing the backing allocation are different operations, just as vector size and capacity are different quantities.

Processing Jobs in Arrival Order with `std::queue`

Purpose. A queue models work where fairness or protocol order requires FIFO behavior.

```
#include <iostream>
#include <queue>
int main(){std::queue<int>q;for(int x:{10,20,30})q.push(x);while(!q.empty())
{std::cout<<q.front()<<' ';q.pop();}}
```

How it works.

- New jobs enter at the back.
- Processing reads from the front.
- The earliest inserted item leaves first.

Expected behavior/output. The output is ``10 20 30``.

Experiment. Interleave one push after one pop and trace the queue contents after each operation.

6.2 Using Deques for Flexible Access

A deque permits efficient insertion and removal at both ends under its standard contract. It is useful when algorithms need front and back operations rather than a strict FIFO interface alone.

More flexibility also means a broader behavioral surface. Prefer a queue adapter when callers should not rearrange work from both ends.

6.2.1 Adding Elements at Both Ends

`push_front` and `push_back` establish new elements at opposite ends. The example begins with two, inserts one at the front and three at the back, and later removes those endpoints. This is different from a stack or queue that exposes only one particular removal policy.

A standard deque grows dynamically and has its own reference and iterator invalidation rules. It is not the same as the fixed-capacity ring queue despite supporting some similar operations. Do not assume that a deque has one contiguous buffer or that an arbitrary raw pointer can traverse across all its elements.

A deque permits insertion at both ends. This makes it useful when the next item is sometimes urgent enough to go at the front while ordinary work joins the back, or when an algorithm must maintain a sliding window. The data structure provides the operations; the application must still define whether reordering priorities is allowed.

Do not call a deque a contiguous vector with extra methods. Standard deque supports random access but does not promise one contiguous allocation containing every element. An index can be efficient without implying that a pointer to the first element can be advanced through the entire logical sequence as a raw array. Representation assumptions must come from the container contract, not from similar-looking syntax.

For a sliding window, indices are often more useful than copied values. A monotonic deque for a window maximum stores candidate positions in decreasing-value order. New values remove weaker candidates from the back, while expired positions leave from the front. Each position can be inserted and removed at most once, producing a useful amortized argument. This is an application of deque operations, not a guarantee that every algorithm using a deque is linear.

The ordinary deque does not itself impose a fixed logical capacity. A queue adaptor backed by deque can therefore grow unless the application adds a bound. When comparing it with the worked ring, include the same capacity rejection rule in the reference model. Otherwise the two structures implement different contracts and a failed push cannot be compared fairly.

Insertion also raises reference and iterator questions. The standard library distinguishes element-reference validity from iterator validity for different deque operations. Do not retain an iterator across mutation merely because an element address appears unchanged in one run. In the chapter's reference model, tests inspect values immediately and avoid relying on those implementation-sensitive observations.

Choose a deque when its endpoint operations match the problem. Choose a ring when fixed capacity and predictable storage reuse are part of the contract. Neither choice removes the need to validate inputs, define full behavior, or measure the actual workload.

6.2.2 Removing Elements at Both Ends

Removing from the front or back requires a nonempty deque. Endpoint references and iterators must respect the operation's invalidation rules. A sliding-window algorithm can exploit both ends, but its proof should explain what the stored candidates mean and why a removed candidate will never be needed again.

The ring queue's tests use a deque as an independent FIFO reference: push at the back, inspect and pop at the front. Restricting the reference to the same abstract operations makes the differential comparison meaningful even though the internal representations differ.

Removing from either end changes which items remain in the logical sequence. A front removal discards the oldest item under FIFO use; a back removal discards the newest. They are not interchangeable just

because both reduce size. Tests should label the expected order after each operation rather than only checking the count.

Consider a deque initialized with two. Push one at the front and three at the back to obtain one, two, three. Pop front and then back, leaving two. This small fixture verifies the endpoints but not a long mixed-operation history. Generated tests can explore alternating endpoint operations, repeated empty states, and duplicate payloads while a simple vector model defines the expected sequence.

A reference or iterator to a removed element must not be used afterward. The fact that storage may be retained for reuse does not preserve the element's logical lifetime. This is the same distinction encountered with vector spare capacity. A container can keep memory while the removed object is no longer a valid observation target.

Backpressure is an application-level response to capacity pressure. A producer might pause, reduce its sampling rate, or return a temporary-unavailable response. Silent overwriting is instead a data-loss policy and may be appropriate only when explicitly chosen, such as retaining the most recent observations. The worked queue's false return allows the caller to choose; it does not choose on the caller's behalf.

A bounded sequential queue is not a lock-free queue. The head, size, and optional slots are ordinary mutable objects. Simultaneous unsynchronized producer and consumer access would introduce data races and break the relationship among them. Do not attempt to make it concurrent by marking counters `volatile`; volatile is not a substitute for synchronization.

The useful engineering lesson is separation: container order, storage capacity, rejection behavior, and concurrency are four different concerns. A robust design states each one, then verifies the interactions actually implemented rather than borrowing guarantees from another category.

Sliding Window Maintained with a Deque

Purpose. A deque is useful when both ends of a sequence are active, as in sliding windows and work queues.

```
#include <deque>
#include <iostream>
int main(){std::deque<int>w;for(int x:{1,2,3,4,5})
{w.push_back(x);if(w.size()>3)w.pop_front();std::cout<<'[';for(int
v:w)std::cout<<v;std::cout<<"] ";}}
```

How it works.

- New elements enter at the back.
- Once the fixed window is exceeded, the oldest element is removed from the front.
- Deque supports both operations efficiently without shifting the remaining elements.

Expected behavior/output. The final windows progress to `[345]`.

Experiment. Change the window size to 2 and predict every printed state.

Worked example — Wrap a bounded FIFO and compare every operation with a reference queue

Requirement and design

Implement an eight-slot integer ring queue with explicit full and empty results. Run five thousand deterministic pseudo-random push/pop choices and compare every result and size with a deque-based reference.

The test's random sequence explores many wraparound states but is not a concurrency stress test. Both queues are used sequentially by one thread.

Build a model with the same acceptance rule

The reference deque is allowed to contain at most eight entries, matching the ring's capacity. Before a generated push, the test checks whether the reference has room. Both models accept or reject the same value, and the reference is updated only after acceptance. This prevents an otherwise unbounded deque from becoming an unfair oracle.

On pop, compare presence first. Only when a value exists should it be compared with the reference front. The test then removes that reference value. Size agreement is checked after every operation, so a rejected push that accidentally increments the ring's count is exposed immediately rather than only after a later wraparound.

The pseudo-random generator is seeded with a fixed value. That makes the sequence reproducible for this implementation and generator, but it is not an exhaustive exploration of every possible operation history. The explicit zero-capacity rejection and endpoint deque fixture supplement the generated sequence with known boundary checks.

For a hand trace, use capacity three and label both physical slots and logical order. Perform enough operations to wrap more than once. If you cannot reconstruct the next pop from the stored head and size, the representation invariant is not yet clear enough to implement confidently.

Complete implementation

The complete, buildable chapter project is `source/book3/ch06/`. The following files are reproduced from that project; the publishing script inserts the source directly so the listing and companion remain synchronized.

main.cpp

```
#include "course_test.hpp"
#include <deque>
#include <optional>
#include <random>
#include <vector>

class RingQueue {
    std::vector<std::optional<int>> slots_;
    std::size_t head_{}, size_{};
```

```

public:
    static std::size_t valid_capacity(std::size_t capacity) {
        if (capacity == 0 || capacity > 1000000) throw
std::invalid_argument("capacity");
        return capacity;
    }
    explicit RingQueue(std::size_t capacity) : slots_(valid_capacity(capacity)) {}
    std::size_t size() const noexcept { return size_; }
    bool push(int value) {
        if (size_ == slots_.size()) return false;
        slots_[head_ + size_ % slots_.size()] = value;
        ++size_;
        return true;
    }
    std::optional<int> pop() {
        if (size_ == 0) return std::nullopt;
        const auto result = slots_[head_];
        slots_[head_].reset();
        head_ = (head_ + 1) % slots_.size();
        --size_;
        return result;
    }
};

int main() {
    CHECK(course::throws<std::invalid_argument>([] { RingQueue invalid{0}; }));
    RingQueue ring{8};
    std::deque<int> reference;
    std::mt19937 random{42};
    for (int i = 0; i < 5000; ++i) {
        if (random() % 2 == 0) {
            const bool available = reference.size() < 8;
            CHECK(ring.push(i) == available);
            if (available) reference.push_back(i);
        } else {
            const auto value = ring.pop();
            CHECK(value.has_value() == !reference.empty());
            if (value) { CHECK(*value == reference.front()); reference.pop_front(); }
        }
        CHECK(ring.size() == reference.size());
    }
    std::deque<int> flexible{2};
    flexible.push_front(1); flexible.push_back(3);
    flexible.pop_front(); flexible.pop_back();
    CHECK(flexible.front() == 2);
    course::report();
}

```

Build, run, and inspect

Run this block from the top-level extracted package. The fixture runner supplies input where needed and checks exit status and output in a temporary working directory.

```

cmake -S source/book3/ch06 -B build/b3-ch06 \
    -DCMAKE_BUILD_TYPE=Debug
cmake --build build/b3-ch06 --parallel 2
ctest --test-dir build/b3-ch06 --output-on-failure

```

For a multi-configuration generator on Windows, append `--config Debug` to the build command and `-C Debug` to CTest. The executable target is `b3_ch06`.

Evidence and interpretation

Before each push, the reference size determines whether space exists. The ring's boolean result must agree, and the reference accepts the value only on success. For a pop, optional presence must agree with reference emptiness and any returned value must equal the reference front.

Checking size after every operation catches drift that a final drain alone might miss. The deterministic seed makes the operation sequence repeatable in this code path, while the explicit zero-capacity test verifies construction's boundary rule.

The fixed storage avoids per-operation growth but does not by itself provide real-time guarantees. Scheduler delays, synchronization, and element-type behavior matter in a concurrent pipeline. This queue deliberately stores integers and contains no atomic or locking protocol; do not share it between threads without redesigning those requirements.

Failure clinic — Full and empty looked identical

A ring reports empty immediately after its final slot is filled. The likely defect is an ambiguous head/tail representation that uses equality for both empty and full without another distinguishing state. Adding special cases to individual calls will not repair an inconsistent representation. Choose a count, a reserved slot, or a full flag and derive all transitions from that choice.

Another queue returns values in physical-array order after wrapping. Its storage is correct, but the traversal ignored head. Printing every slot from zero can be useful for diagnostics only when clearly labeled physical storage. Logical iteration must apply the wrap mapping.

A full push returns false but changes the next value popped. That violates the unchanged-on-rejection contract. Place the capacity test before writing a slot. A test should save and drain the queue after rejection, not merely check the boolean result.

A program works with one thread and fails when acquisition and processing run concurrently. That is not evidence of a rare wraparound bug alone. The original class offers no synchronization. Return to the concurrency contract before trying to fix individual arithmetic expressions; races can invalidate otherwise correct sequential reasoning.

Practice and implementation lab

Add deterministic tests for fill-to-capacity, rejected extra push, drain-to-empty, wraparound after partial drain, and repeated singleton cycles. Verify that rejected pushes preserve every queued value.

Design an overwrite-oldest policy as a separate interface and state how callers learn that data was lost. Then outline a producer/consumer synchronization contract without claiming that replacing indices with atomics automatically makes the current implementation thread-safe.

Review answers and solution notes

Head plus logical offset determines the physical slot, while size disambiguates full from empty. A rejected insertion must not change existing data. Fixed capacity bounds storage growth but does not determine the application's backpressure or loss policy.

A deque can serve as a FIFO reference through back insertion and front removal. Its dynamic storage and invalidation rules differ from a ring buffer. Thread safety is an additional correctness problem involving both data access and ordering, not a property obtained from circular indexing.

Engineering takeaway. Make queue capacity, full-state behavior, and ordering explicit, and test wraparound separately from concurrency.

Guided lab — Exhaust small histories

Construct queues of capacities one, two, and three. Enumerate short histories containing push-zero, push-one, and pop operations, and replay each history from a fresh state. Compare acceptance, returned values, and size with a bounded deque model after every step. A tiny capacity forces wraparound and full transitions much more frequently than a large benchmark buffer.

Add an explicit rejection property. Fill the queue, attempt another push, and then drain it completely. The drained sequence must equal the originally accepted values in order. The rejected value must not appear, and the queue must accept a new value after becoming empty.

Implement a separate overwrite-oldest variant only after writing its new contract. A full push in that variant discards one accepted old value and appends the new one. Return enough information to make the loss observable, such as the displaced value. Do not change the original queue's meaning while retaining the same tests and documentation.

Explore a sliding-window maximum with a deque of indices. For window size three and values $[2, 1, 3, 2, 5]$, the maxima are three, three, and five. Explain why expired positions are removed from the front and weaker new candidates from the back. Check equal-value policy: retaining the earlier or later equal candidate can both support correct maxima, but they produce different internal histories.

Acceptance evidence. Include capacity-one behavior, repeated wraparound, full rejection without mutation, empty pop without mutation, and reuse after draining. Describe the operation-history length and alphabet when calling an enumeration exhaustive.

Review reasoning. Why is a full queue not necessarily an error? It can be a normal bounded-state outcome. Why is modulo by capacity safe only after validation? Zero capacity makes the expression invalid. Why not compare against an unrestricted deque? It would accept operations the bounded contract rejects. Why does fixed storage not imply thread safety? The state updates still need synchronization. Why can a deque support a linear sliding-window algorithm? Each candidate enters and leaves a bounded number of times, an amortized property of that algorithm.

Primary reference notes

Queue context: [queue adapters](#), [deque](#), and [optional](#). The ring indexing and invariant are derived from the worked implementation.

Chapter 7 — Implementing Hash Tables

Two different keys map to the same bucket. A naive table overwrites the first value, while another searches forever after every slot has been used. Hashing reduces the candidate search space only when collision resolution, deletion, and termination are designed together.

This chapter implements separate chaining and bounded linear probing. The open-addressed version uses tombstones so deletion does not break a later key's search path.

A hash table is not a machine that makes collisions disappear. It is a system for selecting a small candidate region and then resolving ambiguity correctly. HarborWorks wants identifier lookups, updates, and removals to remain correct even when several keys choose the same bucket or when deleted entries leave holes in a probe sequence.

This chapter implements both chaining and open addressing with deliberately small fixed storage counts. Small tables make collision paths easier to see. The purpose is not to promise production-grade hashing, automatic resizing, or resistance to hostile keys. It is to establish the invariants that any larger design must preserve before discussing average performance.

7.1 Understanding Hash Functions

A hash function maps a key into a hash value, and the table maps that value into its current bucket or slot range. Different keys can map to the same location; collisions are expected and must not be treated as impossible errors.

Hashing does not replace equality. The table still compares keys to decide whether a candidate is the requested entry.

7.1.1 Mapping Keys to Buckets

The examples use `std::hash<int>` and reduce its result modulo the storage count. The storage count must be nonzero. A key's table location can change when the table capacity changes, so resizing requires reinserting entries according to the new range rather than copying slots blindly.

The examples use fixed bucket counts to isolate collision handling. They do not implement automatic rehashing or promise a particular integer hash representation. Standard hash results are not a portable serialized identifier or a cryptographic fingerprint. A hash's role is candidate organization within the container's contract.

The hash value and the table index are different objects. A hash function produces a value in its result type; the table reduces that value to a bucket or slot index using the current storage count. Changing the storage count changes this second mapping even if the hash function itself is unchanged.

For a hand example, invent a teaching hash that returns the integer key as a nonnegative value and use four buckets. Keys one, five, and nine all reduce to bucket one. This arithmetic example explains collisions, but it must not be presented as the portable behavior of `std::hash<int>`. Standard-library implementations need not expose the same integer mapping across platforms or releases.

Equal keys under the table's equality relation must have equal hashes. The converse is not required: unequal keys may collide. If a custom equality policy treats two identifiers as equivalent while the hash policy sends them to different candidate regions, a lookup can miss an existing equivalent entry. Design equality and hashing as one coherent policy, not as unrelated utility functions.

Stored keys must retain the properties used to organize them. A string view pointing to caller-owned text that changes after insertion can alter apparent key content without a corresponding table update. The hash table's own links may remain intact while its lookup invariant has been broken by an external mutation. Owned immutable keys or a disciplined lifetime and mutation contract prevent that class of defect.

A hash is not a durable record identifier. Even a stable hash algorithm can map different inputs to the same value, and standard hashes are not promised as a portable serialization format. Store the actual key when equality matters. Likewise, the noncryptographic organization used here is not a replacement for a cryptographic integrity mechanism.

Begin every lookup by asking what equality means and where equivalent keys can be located. The hash calculation narrows the search; the equality comparison decides the answer.

7.1.2 Evaluating Hash Distribution

Good distribution spreads expected keys so candidate sets and probe sequences remain short. Load factor compares stored entries with available buckets or slots, but identical load factors can behave differently under clustering or adversarial keys.

Average constant-time claims require assumptions about distribution and operation costs. Worst-case lookup can be linear when many keys collide or a probing table becomes congested. Measure probe lengths or bucket lengths in addition to wall time when investigating performance. A hash table that happens to pass a small random test is not automatically resistant to malicious collision patterns.

Load factor summarizes occupancy, but it does not describe the complete distribution. Two tables with eight entries and sixteen slots can have very different lookup behavior if one spreads entries evenly and the other forms a long cluster. Measure maximum and typical bucket lengths or probe counts when diagnosing a table rather than reporting only total size.

For chaining, a simple expected-cost argument assumes keys distribute reasonably across m buckets. With n entries, the average bucket population is n/m . That ratio helps explain why growth can maintain short candidate scans. It does not prove that an individual bucket is short or that an externally chosen workload satisfies the assumption.

For linear probing, occupied runs matter because each collision pushes a key into a later slot, potentially joining neighboring clusters. Deleted slots marked as tombstones also affect future searches even though they do not count as live entries. A table can have low live occupancy but long probe paths after many deletions. Tracking live count and used-or-deleted count separately can inform a rebuild policy.

The worked tables deliberately do not rehash. Their tests use a bounded key domain and enough open-addressed slots to accommodate that domain. This isolates lookup, update, and deletion semantics. A benchmark of this fixed fixture cannot establish general scaling behavior as the number of unique keys grows indefinitely.

When a table is slow, ask whether the cause is collision distribution, excessive load, expensive key comparison, allocation, or cache behavior. A long string key may cost substantial work even with few probes. Replacing the table without measuring those components can move the bottleneck rather than solve it.

For untrusted keys, choose a design according to the actual threat and latency requirements. Randomized or keyed hashing and alternative structures can change collision risk, but those protections require their own specification and review. This educational integer table makes no such security promise; its bounded probe loop guarantees termination of a call, not uniformly short service time.

Combining Two Fields into a Custom Hash

Purpose. Hash functions map a potentially large key space into a finite integer space used to choose buckets.

```
#include <cstdint>
#include <functional>
#include <iostream>
#include <string>
struct Key{int id;std::string region;};
std::size_t hash_key(const Key&k){auto h1=std::hash<int>{}(k.id);auto
h2=std::hash<std::string>{}(k.region);return h1^(h2+0x9e3779b9+(h1<6)+
(h1>2));}
int main(){std::cout<<hash_key({7,"east"})<<'\n';}
```

How it works.

- Each field contributes its own hash.
- The combine step mixes both contributions into one `size\_t`.
- Equal keys must always produce equal hash values; different keys may still collide.

Expected behavior/output. The program prints a deterministic hash for that implementation run.

Experiment. Change only `region` and compare the resulting hash; do not assume distinct keys can never collide.

7.2 Resolving Hash Collisions

Collision resolution determines where displaced entries live and how search follows them. Chaining stores a collection per bucket. Open addressing stores entries directly in a slot array and follows a probe sequence.

Deletion must preserve the search invariant of the selected technique, and every probe loop needs a termination bound.

7.2.1 Implementing Separate Chaining

In separate chaining, each bucket owns a vector of key/value pairs. Put scans that bucket to update an existing key or appends a new pair. Get scans for equality, and erase removes the matching pair. The outer table owns all buckets and their contained values.

Lookup cost depends on the bucket length; insertion can also allocate as a bucket grows. This representation uses vectors rather than linked nodes, demonstrating that chaining is an abstract collision strategy, not a requirement for one node layout. A missing key returns an empty optional, while storing a value zero remains unambiguous.

Chaining gives every bucket its own collection. The bucket can be a list, a vector, or another structure; the defining feature is that colliding entries remain grouped under the same outer index. In the worked version, vectors of key/value pairs make ownership and inspection straightforward.

An update must first search for an equal key. Replacing the mapped integer preserves the number of entries. Appending a second pair with the same key would change a map into a duplicate-bearing structure and make later lookup depend on incidental search order. The distinction between insert and update is observable even if the caller only asks for the latest value.

Deletion scans the selected bucket and removes the matching entry. Because this example enforces unique keys, at most one logical entry should match. The erase operation on a vector bucket can shift later pairs. Chaining therefore does not guarantee constant pointer work for every deletion in this particular representation. Its cost includes the bucket scan and any movement in that bucket.

A missing lookup returns an empty optional. A stored value of zero remains a present value, and a negative value remains valid. Do not use a mapped value as an absence sentinel unless the domain explicitly excludes it. Key presence and payload content are separate properties.

Allocation failure during bucket growth is another boundary. The integer pairs have simple copy and move behavior, and vector manages their storage. A generic key or mapped value could introduce throwing operations and different guarantees. Retaining the old table until a rehash is fully prepared is one way to structure a stronger replacement operation, but this chapter does not implement that extension.

For inspection, print the bucket index and every contained key rather than only the values. If two unequal keys collide, they must remain independently retrievable. A test suite with only one key per bucket never exercises the mechanism the chapter is intended to teach.

7.2.2 Implementing Open Addressing

Linear probing checks successive slots modulo capacity. Each slot is empty, occupied, or a tombstone. Search can stop at a never-used empty slot, but must continue past a tombstone because another key may have been displaced beyond it.

Insertion remembers the first reusable slot but continues far enough to find an already existing key. Stopping immediately at the first tombstone can create a duplicate key instead of updating the later entry. Every scan is bounded by the slot count, so a full or all-tombstone table terminates. A full insertion with no existing key and no reusable slot returns false rather than looping indefinitely.

A never-used empty slot is evidence that a search can stop. Under the insertion rule, a key following the same probe sequence would have occupied an earlier available position rather than passing that untouched empty slot. A tombstone is different: it says an entry used to occupy the position, so a displaced key may still exist farther along.

Imagine three colliding keys placed in consecutive slots. Remove the first. If its slot is changed to never-used empty, a lookup for the third key stops too early. Marking the slot as a tombstone preserves the path information. The payload can be discarded, but the search history encoded by the slot state must remain until a rebuild repairs the cluster.

Insertion has a subtler obligation. Remember the first reusable tombstone, but continue looking for an equal existing key. Otherwise updating a key beyond the tombstone creates a second copy in the earlier slot. The table then contains conflicting mapped values, and deleting one copy may reveal the other. This defect often survives simple insert-then-get tests and appears only after a delete-update sequence.

Every probe loop is bounded by capacity. An all-occupied table with a missing key must return failure, and an all-tombstone table must eventually conclude that the key is absent. Waiting for a never-used empty slot is not sufficient because such a slot may no longer exist. A finite scan bound is part of termination correctness.

The revised constructor validates the supported capacity before allocating its slot array. Rejecting an excessive requested count after constructing a huge vector would not enforce a meaningful pre-allocation limit. The arithmetic and memory-policy check belongs before the resource request.

Automatic growth would require reinserting live entries using the new capacity, not copying old slot positions. Tombstones need not be carried into the rebuilt table. That rebuild can improve probe paths, but it also requires temporary storage and a commit policy. The fixed-capacity example leaves that design as an explicit extension rather than concealing it behind an unexplained resize.

Separate-Chaining Bucket with Two Colliding Keys

Purpose. Collision resolution is necessary because a hash table has fewer buckets than possible keys and collisions are always possible.

```
#include <iostream>
#include <list>
#include <vector>
int main(){std::vector<std::list<int>>>b(3);for(int x:
{1,4,7})b[x%3].push_back(x);for(std::size_t i=0;i<b.size();++i)
{std::cout<<i<<": ";for(int x:b[i])std::cout<<x<<', ';std::cout<<'\n';}}
```

How it works.

- The toy hash `x % 3` sends 1, 4, and 7 to the same bucket.
- A list stores multiple entries in that bucket rather than overwriting one another.
- Lookup must compare candidate keys inside the selected bucket.

Expected behavior/output. Bucket 1 contains all three values.

Experiment. Increase bucket count to 5 and inspect how the distribution changes.

Worked example — Compare two collision strategies through updates and deletions

Requirement and design

Maintain the same integer map in a three-bucket chained table, a seventeen-slot open table, and a standard unordered map. Apply two thousand deterministic updates and deletions over sixteen keys, comparing all possible keys after each operation.

A separate one-slot table verifies full insertion failure, deletion, tombstone reuse, and successful retrieval after reuse. The custom tables do not resize.

Follow a delete-update chain

The program tests both representations against one `unordered_map<int,int>` model. It repeatedly inserts or updates a key, sometimes erases it, and then checks the complete small key domain. Comparing every key after each operation makes corruption visible near its cause rather than only at the end of two thousand operations.

For an additional hand trace, use an explicitly controlled teaching hash in an exercise copy so that all chosen keys collide. Do not assume a particular `std::hash<int>` mapping. Insert three distinct keys, remove the first, update the third, and verify that exactly one logical entry exists for the third with its new value. Then erase it and verify absence.

The one-slot fixture establishes full-table behavior independently of the larger generated test. A second distinct key cannot be inserted until the existing key is removed. After deletion, the tombstone must be reusable. This tiny table is useful precisely because there is no spare untouched slot to hide an unbounded-loop defect.

The capacity policy is intentionally fixed. Neither representation is judged by whether its layout matches the standard reference. Only key/value semantics, removal results, and supported-capacity behavior are compared.

Complete implementation

The complete, buildable chapter project is `source/book3/ch07/`. The following files are reproduced from that project; the publishing script inserts the source directly so the listing and companion remain synchronized.

main.cpp

```
#include "course_test.hpp"
#include <algorithm>
#include <functional>
#include <optional>
#include <random>
#include <unordered_map>
#include <utility>
#include <vector>

class ChainedTable {
    std::vector<std::vector<std::pair<int, int>>> buckets_;
public:
    explicit ChainedTable(std::size_t capacity) : buckets_(capacity) {
        if (capacity == 0) throw std::invalid_argument("zero buckets");
    }
    void put(int key, int value) {
        auto& bucket = buckets_[std::hash<int>{}(key) % buckets_.size()];
        for (auto& item : bucket) if (item.first == key) { item.second = value;
return; }
        bucket.emplace_back(key, value);
    }
    std::optional<int> get(int key) const {
        const auto& bucket = buckets_[std::hash<int>{}(key) % buckets_.size()];
        for (const auto& item : bucket) if (item.first == key) return item.second;
        return std::nullopt;
    }
    bool erase(int key) {
        auto& bucket = buckets_[std::hash<int>{}(key) % buckets_.size()];
        return std::erase_if(bucket, [key](const auto& item) { return item.first ==
key; }) != 0;
    }
};

class OpenTable {
    enum class State { empty, occupied, tombstone };
    struct Slot { State state{State::empty}; int key{}; int value{}; };
    std::vector<Slot> slots_;
public:
    static std::size_t valid_capacity(std::size_t capacity) {
        if (capacity == 0 || capacity > 1000000) throw
std::invalid_argument("capacity");
        return capacity;
    }
    explicit OpenTable(std::size_t capacity) : slots_(valid_capacity(capacity)) {}
```

```

bool put(int key, int value) {
    const auto first = std::hash<int>{}(key) % slots_.size();
    std::optional<std::size_t> available;
    for (std::size_t step = 0; step < slots_.size(); ++step) {
        const auto i = (first + step) % slots_.size();
        Slot& slot = slots_[i];
        if (slot.state == State::occupied && slot.key == key) {
            slot.value = value; return true;
        }
        if (slot.state != State::occupied && !available) available = i;
        if (slot.state == State::empty) break;
    }
    if (!available) return false;
    slots_[*available] = Slot{State::occupied, key, value};
    return true;
}

std::optional<int> get(int key) const {
    const auto first = std::hash<int>{}(key) % slots_.size();
    for (std::size_t step = 0; step < slots_.size(); ++step) {
        const Slot& slot = slots_[(first + step) % slots_.size()];
        if (slot.state == State::empty) return std::nullopt;
        if (slot.state == State::occupied && slot.key == key) return slot.value;
    }
    return std::nullopt;
}

bool erase(int key) {
    const auto first = std::hash<int>{}(key) % slots_.size();
    for (std::size_t step = 0; step < slots_.size(); ++step) {
        Slot& slot = slots_[(first + step) % slots_.size()];
        if (slot.state == State::empty) return false;
        if (slot.state == State::occupied && slot.key == key) {
            slot.state = State::tombstone; return true;
        }
    }
    return false;
}

};

int main() {
    CHECK(course::throws<std::invalid_argument>([] { OpenTable invalid{1000001}; }));
    CHECK(course::throws<std::invalid_argument>([] { OpenTable invalid{0}; }));
    ChainedTable chained{3};
    OpenTable open{17};
    std::unordered_map<int, int> expected;
    std::mt19937 random{42};
    for (int step = 0; step < 2000; ++step) {
        const int key = static_cast<int>(random() % 16);
        if (random() % 3 == 0) {
            const bool existed = expected.erase(key) != 0;
            CHECK(chained.erase(key) == existed);
            CHECK(open.erase(key) == existed);
        } else {
            expected[key] = step; chained.put(key, step); CHECK(open.put(key, step));
        }
        for (int k = 0; k < 16; ++k) {
            const auto it = expected.find(k);
            const auto want = it == expected.end() ? std::optional<int>{} : it->second;
            CHECK(chained.get(k) == want && open.get(k) == want);
        }
    }
    OpenTable one{1};
    CHECK(one.put(1, 10)); CHECK(!one.put(2, 20));
    CHECK(one.erase(1)); CHECK(one.put(2, 20)); CHECK(one.get(2) == 20);
}

```

```
    course::report();  
}
```

Build, run, and inspect

Run this block from the top-level extracted package. The fixture runner supplies input where needed and checks exit status and output in a temporary working directory.

```
cmake -S source/book3/ch07 -B build/b3-ch07 \  
  -DCMAKE_BUILD_TYPE=Debug  
cmake --build build/b3-ch07 --parallel 2  
ctest --test-dir build/b3-ch07 --output-on-failure
```

For a multi-configuration generator on Windows, append `--config Debug` to the build command and `-C Debug` to CTest. The executable target is `b3_ch07`.

Evidence and interpretation

The reference map defines the intended key/value relation. For each deletion, both custom tables must agree on whether the key existed. For each update, later retrieval must return the most recent value. Checking all sixteen keys after every operation catches damage to neighboring collision paths.

The one-slot test makes the full-state boundary deterministic rather than relying on random coverage. After deleting the first key, the tombstone becomes reusable for a new key. Searches and insertions still terminate because their probe loops cannot exceed capacity.

The tests do not establish a universal constant-time cost. The chained table intentionally has few buckets to exercise collisions, and the open table operates near its maximum distinct-key domain. This is correctness evidence under a specified workload, not a performance endorsement or a hardened hash-table implementation.

Failure clinic — Deletion made an unrelated key disappear

A lookup works until another colliding key is erased. Inspect the deleted slot's state. Replacing occupied with empty instead of tombstone breaks the proof that search may stop at an empty slot. The repair is to preserve probe continuity, not to scan every table slot unconditionally and abandon the intended mechanism.

An update appears correct, but a subsequent erase reveals an older value for the same key. That symptom suggests duplicate insertion at a tombstone before checking for an existing later key. Preserve the first reusable slot while continuing the equality search. The new value must replace the existing entry, not create a competing one.

A missing lookup hangs only after many insertions and deletions. Inspect whether the loop waits for an empty slot without a step limit. Tombstones can occupy every slot even when live size is small. Capacity-bounded probing provides a termination argument for all supported slot-state combinations.

Finally, a capacity rejection consumes excessive memory before throwing. Move validation into the initializer's argument path so it happens before vector construction. A policy check performed after resource acquisition can be logically correct as a condition and operationally wrong as a protection.

Practice and implementation lab

Add a deterministic test that inserts colliding keys, deletes the first one, and then updates a later key through the tombstone. Confirm that the table does not create two copies of that later key.

Instrument maximum bucket length and probe count. Compare low and high load factors while preserving the same correctness checks. Design rehashing as a candidate-table rebuild with a commit point, and explain what must happen if replacement allocation fails.

Review answers and solution notes

A tombstone preserves the fact that a probe path may continue beyond a deleted entry. An empty never-used slot can terminate a search, but a tombstone cannot. Insertion must distinguish finding reusable space from proving that the key is absent.

Rehashing recomputes locations under the new capacity and should preserve the old table until the new state is ready. Average-case performance depends on distribution and load, while the explicit capacity bound guarantees termination in these fixed tables. Equality remains necessary after hashing.

Engineering takeaway. Design collision handling, deletion markers, duplicate updates, and bounded termination as one coherent search invariant.

Guided lab — Force collisions and test the model

Build an exercise table with a configurable hash policy and supply a constant hash for tests. This is not a performance recommendation. It is a deliberate way to force every key through the same collision path regardless of standard-library implementation details.

Enumerate short operation histories over three keys and two values using insert/update, get, and erase. Compare with a simple vector of unique key/value pairs maintained by linear search. The slow reference is valuable because its mechanism differs from the hash implementation. Use capacities one through four so full and tombstone states occur frequently.

Instrument probes without changing lookup semantics. Report the count for successful lookup, unsuccessful lookup, update, and rejected insertion. Run a sequence that fills the table, deletes most entries, and performs missing lookups. Explain why low live size need not imply short searches when many tombstones remain.

Design a rehash transaction. Allocate a new table, reinsert only live entries, verify the same mapping, and publish the new storage after success. State whether failure preserves the original table. Include peak temporary storage in the design. Do not copy slot indices or assume that a tombstone is a live key.

Acceptance evidence. Retain a delete-before-displaced-key case, an update-past-tombstone case, all-tombstone absence, full-table rejection, zero-capacity rejection, and pre-allocation rejection of an unsupported capacity. Check that a mapped zero remains distinguishable from absence.

Review reasoning. Why is equality still needed after hashing? Different keys can collide. Why must equal keys hash equally? Otherwise lookup can search the wrong candidate region. Why continue after finding a tombstone during insertion? An equal key may already exist later. Why is a probe bound necessary? Empty slots may not exist. Why is a hash value not a portable identifier? Collisions and implementation choices remain outside that promise.

Primary reference notes

Hash-table context: [unordered-container requirements](#), [hash support](#), and [MIT algorithm lecture directory](#) for further study. The custom probing implementation is original teaching code.

Chapter 8 — Organizing Unique and Keyed Data

A permissions report needs unique account IDs, overlap between two groups, and a count of each operation name. Treating all three as arbitrary vectors forces callers to repeatedly reinvent uniqueness and key lookup.

Sets and maps name two useful abstractions: membership and key-to-value association. This chapter connects their semantics to sorted set operations and compares ordered and hash-based map behavior without confusing output order with stored content.

HarborWorks compares two inventories: machines that should receive an update and machines that actually reported completion. A set difference can identify missing completions, but only if both collections use the same definition of machine identity. Separately, a report counts events by category, where repeated occurrences matter even though each category appears once as a map key.

This chapter distinguishes mathematical intent from container syntax. Uniqueness, equivalence, multiplicity, ordering, and lookup side effects are all design choices. Selecting `set` or `map` is the implementation step that follows those decisions, not a substitute for making them.

8.1 Understanding Set Abstractions

A set represents membership without duplicate-equivalent keys. The definition of equivalence depends on the chosen ordering or equality policy. A multiset is a different abstraction when repeated equivalent keys must remain distinct entries.

Before choosing a container, decide whether duplicates are invalid, ignored, counted, or meaningful as separate records.

8.1.1 Maintaining Unique Values

Constructing a standard ordered set from one, two, two, and four leaves the unique keys one, two, and four under ordinary integer ordering. This is useful when the application cares about presence rather than arrival multiplicity.

For custom keys, comparator equivalence means neither key orders before the other, which need not be identical to an unrelated `operator==`. The comparison relation must remain consistent while keys are stored. Mutating a key in place so that its ordering position becomes wrong would violate the container's organization; use the supported erase/reinsert or node-handling mechanisms when changing keys.

A set answers a membership question. It intentionally forgets how many times an equivalent value arrived unless that information is stored separately. Constructing a set from identifiers one, two, two, and four yields three members. That is correct for “which machines appeared?” and wrong for “how many reports arrived?” if the duplicate count is discarded before it can be used.

Equivalence deserves explicit attention for domain keys. An ordered set treats two keys as equivalent when neither compares before the other under its comparator. A case-insensitive identifier policy might therefore consider two differently cased strings the same key. The stored spelling may still be one of the original representations, so output policy must be decided separately from identity policy.

The ordering relation must behave consistently. It should not change because a mutable global flag switches modes during a lookup. Nor should a comparator use an external table that can change while keys remain stored. The tree or other ordered structure was organized under earlier comparisons, and a changed relation can invalidate that organization without moving a single node.

Canonicalization is one way to make identity explicit. For an ASCII-only exercise, validate the permitted characters and convert letters according to a fixed rule before inserting. Do not casually call that general Unicode normalization; text identity across languages has additional rules. A precise narrow grammar is more reliable than a broad unsupported claim.

The container also protects its ordering by restricting ordinary mutation of stored keys. Changing a key's comparison-relevant value in place could make it belong elsewhere. A supported extraction and reinsertion mechanism or an erase-and-insert operation must reestablish the organization. Updating a mapped value in a map is a different action because that value need not determine key position.

When testing a custom policy, include equivalent-but-not-identical inputs and unrelated inputs that compare in both directions. The goal is to test the domain's identity rule, not merely demonstrate that integers happen to sort normally.

8.1.2 Performing Set Operations

Intersection contains keys present in both sets; union contains keys present in either; difference contains keys from the left set absent from the right. Difference is directional, so left-minus-right is generally not the same as right-minus-left.

The standard set-operation algorithms operate on sorted ranges under a compatible comparator, not on arbitrary unsorted containers. Their merge-like traversal advances through the ranges without searching each key from scratch. For input lengths n and m , the work is linear in their combined traversal size under the comparison model. Multiset inputs have multiplicity semantics that differ from simply deduplicating everything first.

Set operations are useful descriptions of real reconciliation tasks. Let **required** contain machines scheduled for maintenance and **completed** contain machines reporting success. **required** - **completed** identifies outstanding machines. **completed** - **required** identifies unexpected reports. Their intersection identifies scheduled machines that completed. Their union identifies every machine mentioned by either source.

The direction of difference matters. Swapping the inputs does not merely change presentation; it asks a different operational question. Name variables by role rather than calling them only **a** and **b** when the distinction matters to a report. A mathematically correct result can still be misleading when labeled as the wrong side of a reconciliation.

The standard set algorithms work by coordinated traversal of sorted ranges. If the left current value is smaller, it cannot match a later smaller value on the right because the right range is ordered. This allows

one side to advance without restarting a search. The linear combined-traversal bound comes from each iterator moving forward at most through its input length.

Both ranges must use a compatible ordering relation. Two separately sorted vectors are not enough if one was ordered ascending and the other descending while the operation assumes ascending order. Likewise, sorting by display label and comparing by stable identifier does not establish the required partition. State the comparator at the point where the ranges are prepared and consumed.

Multiplicity changes the interpretation. On sorted ranges containing duplicates, standard set operations have specified multiset-style counts rather than an automatic “deduplicate everything” rule. A union can preserve the greater multiplicity of equivalent elements; an intersection can preserve the smaller. When the application truly wants membership, build unique inputs or explicitly deduplicate first.

For tests, compare the resulting values and their order under the selected policy. Also test empty inputs and identical inputs. Empty reconciliation is a legitimate boundary: no required and no completed machines should produce no outstanding machines, not an error fabricated by the implementation.

Deduplicating Values with a Set

Purpose. A set models membership and uniqueness rather than positional duplicates.

```
#include <iostream>
#include <set>
int main(){std::set<int>s;for(int x:{4,2,4,1,2})s.insert(x);for(int x:s)std::cout<<x<<' ';std::cout<<"size="<<s.size()<<'\n';}
```

How it works.

- Insertion preserves at most one copy of each key.
- Iteration over `std::set` follows sorted key order.
- The size reports the number of unique values.

Expected behavior/output. The output contains `1 2 4 size=3`.

Experiment. Replace `std::set` with `std::unordered_set` and distinguish uniqueness from iteration order.

8.2 Understanding Map Abstractions

A map associates a key with a mapped value. The key identifies the entry; the mapped value stores the associated information. This is a natural model for counting words, indexing records, or looking up configuration by name.

Define whether a missing lookup should report absence, create an entry, or return a default without mutation.

8.2.1 Associating Keys with Values

The word-count loop uses `++counts[word]` intentionally: a missing key is inserted with an initialized integer count, then incremented. Here insertion during lookup is the desired counting operation, not an accidental side effect.

A read-only report should instead use `find`, `contains`, or `at` according to its error policy. Conflating those two use cases can inflate a map with keys that were only queried. The mapped count also needs a numeric bound for unbounded real input; the short fixture’s four words are safely representable.

A map separates identity from associated information. A machine identifier can map to its last observed software version; an event category can map to a count. The key determines which entry is addressed, while the mapped value describes that entry. Two records with the same key require an update or conflict policy, not an accidental duplicate.

The counting idiom `++counts[word]` intentionally creates a missing entry with a value-initialized integer and then increments it. This is concise because insertion on absence is the desired behavior. In a read-only report, however, `counts[word]` can silently add a category that never appeared. Use `find`, `contains`, or `at` according to whether absence is expected or exceptional.

Consider a report checking for the category `error`. If no error occurred, using a mutating lookup can change the map’s size and later make the output include a zero-valued category. That may be acceptable under a deliberate “show all categories” policy, but it is not a neutral observation. Document whether querying is allowed to create state.

Counts need a numeric contract for large input. The small four-token example cannot overflow its integer count, but an unbounded event stream can outgrow a chosen type. Select a suitable representation, validate increments before overflow, and decide whether saturation, rejection, or a larger type is appropriate. Merely changing `int` to an unsigned type replaces one failure mode with modular arithmetic; it does not make unlimited counts possible.

When loading a complete replacement mapping, a temporary map can separate validation from publication. Parse and validate the candidate records, resolve duplicate-key policy, then swap the accepted candidate into place. If parsing fails, the prior mapping can remain intact. This is the same prepare-and-commit pattern used for vector growth, applied to domain state rather than a raw buffer.

Tests should distinguish absent, present-zero, and present-nonzero entries. A map whose values include optional data has another layer of absence; do not collapse “no entry” and “entry with no measurement” unless the application explicitly treats them as equivalent.

8.2.2 Comparing Ordered and Hash-Based Maps

An ordered map provides traversal by its comparator, which directly supports lexical reporting. An unordered map organizes by hash/equality and offers no sorted traversal promise. Both can represent the same mathematical mapping while presenting entries in different orders.

Compare contents by key rather than comparing incidental iteration sequences. If deterministic sorted output is required from an unordered map, materialize and sort a report or choose an ordered map. The

right choice depends on lookup frequency, updates, ordering needs, memory, and key-operation costs—not solely on a headline average lookup bound.

An ordered map directly supports traversal in comparator order. An unordered map directly supports hash-based candidate lookup without promising sorted iteration. Both can represent the same mapping, but their natural output sequences differ. A report comparing raw iteration order can therefore declare equivalent contents unequal.

Deterministic output should be an explicit layer. Either choose an ordered map whose comparator matches the report order, or copy keys or records from an unordered map and sort the report snapshot. The second approach pays a materialization and sorting cost at reporting time while keeping another lookup representation during updates. The workload determines whether that tradeoff is worthwhile.

“Deterministic” also needs a tie and formatting policy. Sorting by a displayed count alone leaves equal-count categories unordered unless a secondary key is specified or stable prior order is meaningful. A reproducible report might sort by descending count and then ascending category name. That rule should be tested with equal counts, not only distinct ones.

Key cost changes the comparison. An ordered lookup can perform several comparisons, each potentially inspecting a string prefix. A hashed lookup computes a hash over the key and then may compare colliding candidates. Long shared prefixes, expensive normalization, and repeated hashing can dominate container overhead. Complexity labels about the number of key operations do not remove the cost of those operations.

Memory and invalidation also matter. Node-based ordered storage and bucket-organized unordered storage have different overhead and iterator behavior. A caller retaining iterators across insertions must follow the selected container’s documented rules. Choosing by a headline lookup bound while ignoring a required lifetime guarantee can make the overall design incorrect.

For HarborWorks, separate ingestion from reporting in the design notes. Explain which operations dominate, how often a full ordered report is required, whether range queries exist, and how many distinct keys are expected. Those questions produce a defensible choice without declaring one map category universally superior.

Frequency Table with a Map

Purpose. A map represents an association from unique keys to related values.

```
#include <iostream>
#include <map>
#include <string>
#include <vector>
int main(){std::map<std::string,int>count;for(const
auto&w:std::vector<std::string>{"red","blue","red"})++count[w];for(const
auto&[k,v]:count)std::cout<<k<< '=' <<v<< ' ' ;}
```

How it works.

- The key identifies the word and the mapped value stores its count.
- `operator[]` creates a zero-initialized mapped integer for a missing key before incrementing it.
- Structured binding exposes each key/value pair during iteration.

Expected behavior/output. The output reports `blue=1 red=2` in sorted-key order.

Experiment. Use `find` instead of `operator[]` when querying a key that should not be inserted if missing.

Worked example — Compute set relationships and verify equivalent word-count maps

Requirement and design

Compute intersection, union, and left difference for two integer sets. Count a short word sequence in both an ordered map and an unordered map, then verify matching values by key and the ordered map's first key.

The sorted set inputs satisfy the algorithms' ordering requirements. The word-count insertion side effect is intentional.

Read the example as two different applications

The first half reconciles membership with ordered sets. Duplicate input two is intentionally collapsed. The set-operation outputs are vectors because the algorithms emit a sequence into an output iterator. Their values are already ordered by the coordinated traversal; the example does not need a second sorting pass.

The second half counts words in both ordered and unordered maps. It compares each mapped count by key rather than comparing iteration order. The ordered map's first key is checked separately to

demonstrate its lexical traversal policy. This is an order assertion about that representation, not a requirement imposed on the unordered model.

Before running, compute the intersection, union, and left difference by hand. Then count `read`, `test`, `read`, and `build`. Predict both the number of distinct keys and the count associated with `read`. A test that checks only the total number of tokens would not verify the mapping.

The fixture is deliberately small. It introduces set and map semantics, not a throughput claim for word counting. A real text ingestion system would also need tokenization, encoding, resource bounds, and error reporting, all of which are outside this four-word example.

Complete implementation

The complete, buildable chapter project is `source/book3/ch08/`. The following files are reproduced from that project; the publishing script inserts the source directly so the listing and companion remain synchronized.

main.cpp

```
#include "course_test.hpp"
#include <algorithm>
#include <iterator>
#include <map>
#include <set>
#include <string>
#include <unordered_map>
#include <vector>

int main() {
    const std::set<int> left{1, 2, 2, 4}, right{2, 3, 4};
    std::vector<int> intersection, united, difference;
    std::set_intersection(left.begin(), left.end(), right.begin(), right.end(),
                          std::back_inserter(intersection));
    std::set_union(left.begin(), left.end(), right.begin(), right.end(),
                  std::back_inserter(united));
    std::set_difference(left.begin(), left.end(), right.begin(), right.end(),
                       std::back_inserter(difference));
    CHECK(intersection == std::vector<int>({2, 4}));
    CHECK(united == std::vector<int>({1, 2, 3, 4}));
    CHECK(difference == std::vector<int>({1}));
    std::map<std::string, int> ordered;
    std::unordered_map<std::string, int> hashed;
    for (const std::string word : {"read", "test", "read", "build"}) {
        ++ordered[word]; ++hashed[word]; // Insertion on absence is intended here.
    }
    CHECK(ordered.size() == 3 && ordered.at("read") == 2);
    for (const auto& [key, value] : ordered) CHECK(hashed.at(key) == value);
    CHECK(ordered.begin()->first == "build");
    course::report();
}
```

Build, run, and inspect

Run this block from the top-level extracted package. The fixture runner supplies input where needed and checks exit status and output in a temporary working directory.

```
cmake -S source/book3/ch08 -B build/b3-ch08 \
  -DCMAKE_BUILD_TYPE=Debug
cmake --build build/b3-ch08 --parallel 2
ctest --test-dir build/b3-ch08 --output-on-failure
```

For a multi-configuration generator on Windows, append `--config Debug` to the build command and `-C Debug` to CTest. The executable target is `b3_ch08`.

Evidence and interpretation

The intersection is two and four, the union is one through four, and the left difference is one. These exact result vectors verify both content and the sorted output order produced from the chosen ordered inputs.

The word `read` appears twice, while `test` and `build` appear once. Both maps therefore contain three keys with identical mapped counts. The ordered map begins with `build`; no analogous ordering assertion is made about the unordered map.

The test deliberately distinguishes abstract equality of mappings from representation or traversal equality. That distinction is useful when replacing an implementation: the external contract may require only content, or it may additionally require a specific report order. Know which before calling two implementations interchangeable.

Failure clinic — A report changed the data it inspected

The number of map entries grows after a dashboard refresh even though no new events arrived. Inspect read-only code for `operator[ ]`. A missing-key query can insert a zero-valued entry. Replace it with a nonmutating lookup unless creating default categories is an intentional requirement.

A reconciliation lists completed machines as outstanding. Check which set is on the left of difference. The algorithm may be perfectly correct for the reversed question. Preserve a fixture where the two directional differences are distinct so that the report label and operation remain aligned.

A union contains surprising duplicates. Determine whether the inputs represent sets or sorted multisets. The algorithm does not infer that repeated values should disappear merely because its name contains “set.” Deduplication is a separate representation decision.

A report changes ordering between runs while counts remain correct. Inspect whether it serializes unordered-map iteration directly. Sort a report snapshot under an explicit tie rule or use an ordered representation. Do not treat one observed hash iteration order as a portable output contract.

Practice and implementation lab

Add empty-set, disjoint-set, equal-set, and reversed-difference tests. Repeat an operation with sorted vectors containing duplicates and document the multiplicity semantics instead of assuming ordinary set uniqueness.

Add a read-only missing-word query and verify that map size remains unchanged. Then design a frequency report sorted by decreasing count and lexical tie-break, which is not the same as the ordered map's key traversal.

Review answers and solution notes

Difference is directional, while union and intersection are symmetric as set contents. Sorted-range algorithms require compatible ordering and preserve defined multiplicity behavior for repeated inputs.

`operator[]` is appropriate for intentional counting insertion but not for a nonmutating missing-key query. Ordered and unordered maps can contain the same relation with different iteration order. A frequency ranking requires its own explicit sort keys rather than relying on either container's incidental traversal.

Engineering takeaway. Choose set or map semantics deliberately, and separate abstract content from ordering and insertion side effects.

Guided lab — Reconcile an inventory and produce a stable report

Create two small identifier collections called `planned` and `observed`. Define an ASCII identifier grammar and a duplicate policy. Produce outstanding, unexpected, and matched identifiers in lexical order. Include an empty source, repeated identifiers, identical collections, and partially overlapping collections.

Implement the same mathematical reconciliation in two ways: ordered set algorithms and a simple membership loop over a reference vector. The reference can be slower because the fixture domain is small. Compare values rather than relying on both implementations sharing the same library operation.

Next write a category counter with checked increments and a nonmutating query function. Verify that asking for a missing category leaves the number of entries unchanged. Decide whether the query returns optional count or zero, and explain why either can be appropriate only under its documented meaning.

Build a report sorted by descending count and then ascending category. Use a fixture with at least two equal counts. Generate several permutations of the same input tokens and verify identical reports. This tests that ingestion order does not accidentally determine presentation order when the contract says it should not.

For a design exercise, add a range query such as identifiers between two lexical boundaries. Explain how this changes the relative convenience of ordered and hash-based representations. Do not conclude from a single extra operation that all hash tables should be replaced; consider its frequency and the full workload.

Acceptance evidence. Record the identity policy, duplicate policy, missing-query behavior, report order, and numeric limit. Retain a regression showing that a read-only query does not mutate state.

Review reasoning. Why does a set lose frequency information? It stores membership under an equivalence rule. Why is difference directional? It asks which left-side members are absent on the right. Why can two maps be equal while their iteration orders differ? Mapping semantics and traversal policy are separate. Why test equal-count reports? They expose an unspecified tie rule. Why must equality and

ordering remain stable while keys are stored? The representation's organization depends on those decisions.

Primary reference notes

Set and map contracts: [associative containers](#), [unordered containers](#), and [sorted set operations](#).

Chapter 9 — Representing Hierarchical Data with Trees

A project hierarchy contains a root, several departments, and nested teams. A report can list a parent before its descendants, after its descendants, or level by level. All are valid traversals, but they answer different operational questions.

This chapter models a general rooted tree, fixes a height convention, and implements preorder, postorder, and breadth-first traversal. It also distinguishes the depth of recursion from the width of a pending-work queue.

HarborWorks describes a site as a hierarchy: a root organization contains locations, locations contain racks, and racks contain machines. That ownership-shaped model is a tree only while each contained object has one parent and the relationships contain no cycles. A machine shared by two locations or a dependency pointing back to an ancestor requires a different model.

Trees are valuable because their structural restrictions support simpler reasoning. A traversal does not need to discover whether a child was already reached through another parent when unique ownership makes that impossible. The simplicity is conditional on the representation. This chapter teaches the hierarchy first; Chapter 13 deliberately relaxes its restrictions when introducing graphs.

9.1 Understanding Tree Terminology

A rooted tree organizes nodes through parent/child relationships with one root and no cycles in the intended structure. Each non-root node has one parent under this model. A leaf has no children.

The example's unique ownership naturally expresses a hierarchy: each node owns its child nodes. It is not a general graph with shared descendants or back edges.

9.1.1 Identifying Roots, Parents, Children, and Leaves

The root begins the hierarchy. A parent owns an ordered child collection, and sibling order follows that collection's order. A traversal's output therefore depends on both the traversal rule and the order in which children were inserted.

The `add` function returns a reference to the newly allocated child object. Growing the vector of unique pointers can move pointer owners without moving the separately allocated child itself, so that child reference remains valid while its node is retained. Erasing or destroying the owning subtree would end the reference's validity. Distinguish owner-container relocation from pointee relocation.

A root is the entry point of a rooted hierarchy. Every nonroot node has one parent in this model, and a leaf has no children. Siblings share a parent. These are structural relationships, not assumptions about a node's stored value. Two distinct nodes can contain the same integer while occupying different positions in the tree.

The worked representation stores each node's children in an ordered vector of exclusive owners. That child order determines the traversal order among siblings. It does not imply numeric sorting. If children are added with values three and then two, a parent-first traversal visits them in that insertion order unless another rule is introduced.

A reference returned by `add` observes the separately allocated child. Reallocating the parent's vector moves `unique_ptr` objects, but does not move their heap-allocated pointees. That reference can therefore remain valid across additions of siblings. It cannot remain valid after the child's owning subtree is erased or destroyed. Distinguish relocation of an owner from relocation or destruction of the owned object.

A parent pointer is not needed for the supplied downward traversals. Adding one could make upward navigation convenient, but it would introduce another observer invariant to maintain during reparenting. Design from required operations rather than storing every relationship imaginable. Redundant metadata increases the number of facts every mutation must keep consistent.

A tree can model containment but not every real-world association. A service depending on two databases has multiple outgoing dependencies, and two services may share one database. Duplicating the database into a tree can falsely imply independent identity and lifetime. A graph or a tree plus separate reference edges may better represent that domain.

Before building a hierarchy from external records, validate parent existence, uniqueness, and cycle absence under the intended model. The in-memory example constructs its tree directly and relies on exclusive child ownership; it does not implement a general malformed-hierarchy importer. State that boundary before applying the code to arbitrary configuration data.

9.1.2 Measuring Height and Depth

Depth is the number of edges from the root to a node. This chapter measures height as the maximum number of edges from a node to a descendant leaf. A leaf therefore has height zero, and an empty tree is represented by height minus one for the recursive helper.

Some books count nodes rather than edges when defining height. Either convention can work if used consistently; mixing them causes off-by-one errors in base cases and balance formulas. State the convention next to the implementation rather than assuming every reader uses the same one. The empty sentinel also requires a result type that can represent minus one.

Depth measures distance from the root along edges. Height measures the longest downward distance to a leaf under this chapter's convention. In the example, the root has depth zero and height two; its child containing two has depth one and height one; the grandchildren have depth two and height zero.

The empty-tree helper returns minus one so that a leaf's height becomes `1 + max(empty-child heights)` in an equivalent binary formulation. This is a convention that simplifies a recurrence, not a physical negative distance. Chapter 11 deliberately uses node-count height with empty zero and leaf one for AVL metadata. The two conventions are consistent within their own formulas but differ by one for nonempty trees.

State a convention before writing a validator. An assertion expecting leaf height one will falsely reject a correctly implemented edge-count height zero. Such an off-by-one is a specification mismatch, not

necessarily a traversal bug. Printed diagrams should label whether numbers represent depth from root or height to leaves; they are not interchangeable on an unbalanced tree.

The height recurrence follows the dependency structure. A node cannot know its own height until the heights of its children have been computed. Each child contributes a candidate downward path; the longest is chosen, then the edge from the node to that child is counted. This is an example of bottom-up aggregation and prepares the reader for postorder processing.

Shape affects resources. A chain of n nodes has height $n - 1$ and can create linear recursion depth. A broad shallow tree can have small height but a large breadth-first queue. The number of nodes alone does not determine which auxiliary resource becomes large. Draw a chain and a star to expose the two extremes.

External size limits should therefore consider both node count and depth when recursive processing is used. A file containing only a few thousand nodes can still form a deeply nested chain. “The input is small” is not a substitute for checking the dimension that controls the call stack.

Computing Leaf Status and Height in a Small Binary Tree

Purpose. Tree terminology becomes concrete when parent/child, leaf, and height properties are calculated from an actual structure.

```
#include <algorithm>
#include <iostream>
#include <memory>
struct Node{int value;std::unique_ptr<Node>left,right;};
int height(const Node*n){return n?1+std::max(height(n->left.get()),height(n->right.get())):0;}
bool leaf(const Node*n){return n&&!n->left&&!n->right;}
int main(){Node
root{1,std::make_unique<Node>(Node{2}),std::make_unique<Node>(Node{3})};std::cout<<height(&root)
'<<std::boolalpha<<leaf(root.left.get())<<'\n';}
```

How it works.

- `root` has depth 0 and two child nodes.
- A leaf has no children.
- Height is computed recursively from the longest path to an empty child.

Expected behavior/output. The output is `2 true` under the node-count height convention used here.

Experiment. Add a grandchild under the left node and recompute the height.

9.2 Traversing Trees

Depth-first traversal follows a branch before moving to later siblings. Breadth-first traversal processes all nodes at one depth before the next. Their data structures differ: recursion or an explicit stack for depth-first work, a queue for breadth-first work.

Choose the order that matches the operation's dependencies. Children must often be processed before a parent can aggregate their results.

9.2.1 Performing Depth-First Traversals

Preorder visits the node before its children, which is useful for parent-first listing or propagating inherited context. Postorder visits children before the node, which is useful for bottom-up aggregation or releasing dependent work.

The recursive implementations process each node once, giving linear traversal work in the number of nodes. Auxiliary call-stack depth is proportional to tree height, while the returned output vector itself contains one value per node. A very deep untrusted hierarchy can exceed call-stack limits; an explicit stack or bounded-depth ingestion policy is needed for that broader domain.

Depth-first traversal completes a branch before moving to the next sibling branch. Preorder records the current node before descending, while postorder records it after all children. The same child order can therefore produce different valid outputs depending on when the node is visited.

For a root one with children two and three, and children four and five beneath two, preorder is one, two, four, five, three. Postorder is four, five, two, three, one. Both contain the same five node visits. Their usefulness differs because their dependency ordering differs.

Preorder suits a parent-first outline or propagation of inherited context. Postorder suits computing a total from child totals, such as a subtree's record count. A deletion or cleanup procedure often needs children handled before the parent's owning state disappears, but the exact lifetime behavior of C++ members must still be considered rather than inferred from a traversal name.

A recursive call stores unfinished work: after returning from one child, the function must continue with the next child and eventually finish the parent. An explicit stack can model that work. For preorder, pushing children in reverse order preserves the original left-to-right visit order when they are later popped. For postorder, a frame can remember the node and next child index, or a second state can mark that children have been scheduled.

Do not duplicate visits accidentally while converting recursion. Pushing every child and then recursively visiting those same children mixes two control strategies. Compare the complete output against a hand trace and check that each node identity appears exactly once. Equal stored values make identity-based validation more informative than value counts alone.

The returned vector is part of memory usage. Even with a shallow recursion stack, collecting every visit requires linear output storage. A callback-based traversal can avoid that particular result vector, but then callback failure, mutation, and lifetime rules become part of the interface. The simplest complete example makes one clear tradeoff at a time.

9.2.2 Performing Breadth-First Traversals

Breadth-first traversal enqueues the root, repeatedly removes the front node, records it, and enqueues its children. FIFO order ensures that all already-discovered shallower nodes are processed before newly discovered deeper nodes.

Its auxiliary queue can grow with the tree's width, potentially linearly in the total node count. The queue stores borrowed const pointers while the tree remains alive and unchanged. It does not become a second owner. A cyclic graph would require a visited-state policy; the tree algorithm relies on the acyclic ownership structure and should not be reused for arbitrary graphs without adaptation.

Breadth-first traversal uses FIFO pending work. The root is initially the only queued node. When a node is removed, its children join the back. All nodes already queued at the current depth remain ahead of newly discovered deeper nodes, which establishes level order.

The queue contains borrowed const pointers. It does not own the tree and cannot prolong a node's lifetime after the tree is destroyed. The traversal therefore requires the hierarchy to remain alive and unchanged until processing completes. Moving the owning tree or erasing a queued subtree during traversal would require a separate invalidation analysis.

For the worked hierarchy, queue states begin with [1], then [2 , 3], then [3 , 4 , 5], then [4 , 5]. Notice that a queue can temporarily contain nodes from two neighboring levels. The invariant is not "every queued node has the same depth." It is that pending nodes are ordered so shallower discovered work precedes deeper discovered work under the traversal policy.

A star-shaped tree illustrates the space boundary. Visiting the root enqueues nearly every other node at once. The auxiliary queue can therefore be linear in total node count even though height is only one. A chain has the opposite profile: the breadth-first queue stays tiny while a recursive depth-first implementation can grow a deep call stack.

A general graph cannot simply use this tree routine. Shared descendants would be visited more than once, and cycles could make the traversal continue indefinitely. Marking graph vertices when first discovered is the adaptation developed later. The absence of a visited set here is justified by the tree model, not by a belief that duplicate discovery never matters.

Use level order when the question is about layers or shortest unweighted distance from the root. Use depth-first order when branch completion or subtree aggregation is the more natural dependency. Neither traversal is intrinsically more correct; correctness means matching the requested order and resource constraints.

Preorder, Inorder, and Postorder Traversal

Purpose. Traversal order determines when each node is visited relative to its descendants.

```
#include <iostream>
struct Node{int v;Node*l{};Node*r{}};
void pre(Node*n){if(!n)return;std::cout<<n->v;pre(n->l);pre(n->r);}void
in(Node*n){if(!n)return;in(n->l);std::cout<<n->v;in(n->r);}void post(Node*n)
{if(!n)return;post(n->l);post(n->r);std::cout<<n->v;}
int main(){Node a{2},b{1},c{3};a.l=&b;a.r=&c;pre(&a);std::cout<<'
';in(&a);std::cout<<' ';post(&a);std::cout<<'\n';}
```

How it works.

- Preorder processes the node before its subtrees.
- Inorder processes left subtree, node, then right subtree.
- Postorder processes the node after both subtrees.

Expected behavior/output. The output is `213 123 132`.

Experiment. Use the same functions on a one-sided tree and trace the recursive call order.

Worked example — Traverse one hierarchy in three meaningful orders

Requirement and design

Build a root one with children two and three, then give node two children four and five. Compute preorder, postorder, breadth-first order, and edge-based height.

The hierarchy is a small owned tree. Recursive traversal and recursive unique-owned subtree destruction are suitable for this bounded fixture, not a guarantee for arbitrarily deep input.

Make node identity and child order visible

The sample constructs one rooted hierarchy using `add`, then collects preorder, postorder, and breadth-first results. Read the construction sequence before the expected outputs. The children of node two are added in the order four then five, which is why those values appear in that order within both depth-first variants.

The local reference named `left` remains valid after adding the root's other child because the referenced child object lives in its own allocation. The parent's vector can move its owners without relocating the child. This is a useful concrete application of Book II's distinction between an owning pointer and its pointee.

The height check uses two examples: the actual tree and a null root. The null case verifies the selected empty convention directly. A leaf-only case would be a useful additional exercise because it bridges empty minus one to a leaf's zero-edge height.

No mutation occurs during any traversal. The source does not claim iterator stability under reparenting, thread safety, or resilience to an externally supplied cyclic structure. These omitted behaviors are new design problems, not missing flags to add at the call site.

Complete implementation

The complete, buildable chapter project is `source/book3/ch09/`. The following files are reproduced from that project; the publishing script inserts the source directly so the listing and companion remain synchronized.

main.cpp

```
#include "course_test.hpp"
#include <algorithm>
#include <memory>
#include <queue>
#include <vector>

struct Node {
    int value;
    std::vector<std::unique_ptr<Node>> children;
    explicit Node(int v) : value(v) {}
    Node& add(int v) { children.push_back(std::make_unique<Node>(v)); return
*children.back(); }
};

int height(const Node* node) {
    if (!node) return -1; // Height measured in edges; a leaf has height zero.
    int longest{-1};
    for (const auto& child : node->children) longest = std::max(longest,
height(child.get()));
    return longest + 1;
}

void preorder(const Node& node, std::vector<int>& result) {
    result.push_back(node.value);
    for (const auto& child : node.children) preorder(*child, result);
}

void postorder(const Node& node, std::vector<int>& result) {
    for (const auto& child : node.children) postorder(*child, result);
    result.push_back(node.value);
}

std::vector<int> breadth_first(const Node& root) {
    std::vector<int> result;
    std::queue<const Node*> pending;
    pending.push(&root);
    while (!pending.empty()) {
        const Node* node = pending.front(); pending.pop();
        result.push_back(node->value);
        for (const auto& child : node->children) pending.push(child.get());
    }
    return result;
}

int main() {
    Node root{1};
    Node& left = root.add(2);
    root.add(3);
}
```

```

    left.add(4); left.add(5);
    std::vector<int> pre, post;
    preorder(root, pre); postorder(root, post);
    CHECK(pre == std::vector<int>({1, 2, 4, 5, 3}));
    CHECK(post == std::vector<int>({4, 5, 2, 3, 1}));
    CHECK(breadth_first(root) == std::vector<int>({1, 2, 3, 4, 5}));
    CHECK(height(&root) == 2 && height(nullptr) == -1);
    course::report();
}

```

Build, run, and inspect

Run this block from the top-level extracted package. The fixture runner supplies input where needed and checks exit status and output in a temporary working directory.

```

cmake -S source/book3/ch09 -B build/b3-ch09 \
  -DCMAKE_BUILD_TYPE=Debug
cmake --build build/b3-ch09 --parallel 2
ctest --test-dir build/b3-ch09 --output-on-failure

```

For a multi-configuration generator on Windows, append `--config Debug` to the build command and `-C Debug` to CTest. The executable target is `b3_ch09`.

Evidence and interpretation

Preorder is one, two, four, five, three. Postorder is four, five, two, three, one. Breadth-first order is one, two, three, four, five. The root's height is two edges, while the empty helper result is minus one.

These outputs make sibling order and visit timing visible. The height helper computes each child's height, keeps the longest, and adds one. With no children, the initial longest value minus one yields leaf height zero.

The reference to node two remains valid when the root adds node three because the child object is separately allocated and its unique owner moving within a vector does not relocate the pointee. That observation is specific to this representation; a vector storing child objects directly would have different reference invalidation behavior.

Failure clinic — The output was complete but in the wrong order

A preorder conversion using an explicit stack returns one, three, two, five, four. Every node appears once, but sibling order has been reversed by last-in, first-out behavior. Push children in reverse order when the required visit order is the original vector order. The stack's policy must be accounted for when scheduling work.

A subtree height is consistently one larger than expected. Check the height convention before changing recursion. An implementation counting nodes can be correct under one definition while a test counts edges. Align definitions in the interface, explanation, and fixture.

A breadth-first traversal hangs after reusing it for a dependency graph. The tree assumption no longer holds. Add a graph representation and visited-state policy rather than inserting an arbitrary iteration limit and returning a truncated result as though it were complete.

A child reference is valid after sibling insertion but invalid after subtree deletion. Those observations are not contradictory. Owner relocation preserved the child object; deletion ended its lifetime. Draw the actual ownership event instead of assigning one blanket “stable pointer” label to every operation.

Practice and implementation lab

Add a singleton tree, a chain, and a wide root. Predict each traversal and height before running. Instrument maximum recursion depth and maximum queue length to compare tall and wide shapes.

Implement iterative preorder using an explicit stack. Push children in reverse order when necessary to preserve the original left-to-right traversal. Add an input-depth limit before adapting the recursive representation to externally supplied hierarchies.

Review answers and solution notes

Preorder processes parents first, postorder processes descendants first, and breadth-first traversal processes depth layers. A leaf has height zero under the edge convention used here. Recursive depth and BFS queue width are different auxiliary-space concerns.

Moving a unique pointer does not move its separately allocated pointee, but destroying the owner still destroys that pointee. A tree traversal without visited state is not automatically safe for a general cyclic graph. The representation’s acyclic ownership is part of the algorithm’s precondition.

Engineering takeaway. Choose traversal order from dependency needs, and state height, ownership, and depth limits explicitly.

Guided lab — Compare shapes and traversal strategies

Construct three hierarchies with the same node count: a chain, a star, and a roughly balanced tree. Record height, maximum recursive depth in an instrumented depth-first traversal, and maximum queue size in breadth-first traversal. These are deterministic structural counts, not wall-clock benchmark claims.

Implement iterative preorder using an explicit vector stack of borrowed pointers. Preserve the recursive routine’s child order by scheduling children appropriately. Then implement iterative postorder with frames containing a node pointer and the next child index. Explain which unfinished action each frame represents.

Add a subtree-size aggregation. A leaf contributes one, and an internal node contributes one plus all child sizes. Compute it postorder and compare the root’s total with the number of visits collected by another traversal. This is a useful cross-check because the two procedures expose different state.

For repeated values, assign separate diagnostic node identifiers in the exercise. A value-only result such as `[1, 1, 1]` cannot reveal which equal-valued node was visited twice or omitted. Identity-aware checks should be used only for validation; they need not change the public payload semantics.

Discuss a malformed import boundary. Given records containing identifier and parent identifier, list the checks needed before publishing a tree: unique identifiers, one designated root, existing parents, no cycles, and permitted depth and node count. Keep the original live tree unchanged if validation fails. The

exercise is a design, not a claim that the chapter's direct-construction example already performs those checks.

Acceptance evidence. Include empty, leaf, chain, star, repeated-value, and mixed-depth fixtures. Preserve the selected height convention and child-order policy in the test names or notes.

Review reasoning. Why does BFS not need visited state here? The model is a tree with one ownership path per node. Why can its queue be large despite shallow height? A broad level contains many pending nodes. Why can a child reference survive vector growth? The separately allocated child does not move with its owner. Why is postorder useful for aggregation? Child results exist before the parent combines them. Why is an explicit stack not free? It still stores pending control state, just in a visible container.

Primary reference notes

Tree implementation context: [unique pointers](#), [queue adapters](#), and [MIT algorithm lecture directory](#) for further reading on tree algorithms.

Chapter 10 — Building Binary Search Trees

A sorted vector supports binary search but requires shifting elements for many insertions. A binary search tree organizes keys through left/right relationships instead. It can offer short search paths when balanced, but sorted insertion can turn an unbalanced tree into a chain.

This chapter implements the basic search invariant and all three deletion shapes before adding balancing in the next chapter. Correct ownership during node replacement is as important as the comparison logic.

An ordinary binary search tree adds an ordering rule to a hierarchy. That rule lets each comparison discard an entire subtree when searching for a key. It does not, by itself, guarantee a short path. A sequence of already sorted insertions can create a structure that behaves like a linked chain.

HarborWorks uses this chapter to learn search-tree invariants and deletion before introducing balance repair. The worked tree contains unique integer keys and exposes value-based operations. It does not promise stable record identity when a node's key is replaced during deletion, nor does it bound recursion depth for arbitrary external input. Those limits help keep the first implementation explainable.

10.1 Maintaining the Binary Search Property

For this unique-key binary search tree, every key in a node's left subtree is smaller and every key in its right subtree is larger. The rule applies to whole subtrees, not only immediate children.

Search and insertion follow one branch at each comparison because the invariant excludes the other subtree from containing the requested key.

10.1.1 Searching for Keys

To search, compare the key with the current node. Equality succeeds, a smaller key moves left, and a larger key moves right. Reaching a null child proves absence under the invariant.

The work is $O(h)$, where h is tree height. Calling it $O(\log n)$ without a balancing assumption is incorrect. A chain of n nodes has linear height, while a well-balanced shape has logarithmic height. Keep the structural parameter h visible until the implementation actually enforces a height bound.

The ordering rule applies to every descendant, not only immediate children. At a node with key ten, every key in its left subtree must be less than ten, including a right descendant several levels below the left child. Checking only parent-child comparisons can accept a tree that is locally plausible and globally wrong.

Searching uses this global invariant. If the target is smaller than the current key, no key in the right subtree can match. If it is larger, the left subtree is excluded. Equality returns success. Reaching an empty child link returns absence. Each step removes the current node and one entire excluded region from consideration.

Trace searching for six in a tree rooted at eight with left child four and four's right child six. The comparisons choose left, then right, then equality. Searching for five follows the same first two decisions and then reaches an empty child. Absence is established by the maintained ordering and exhausted path, not by comparing with every stored key.

A validator should carry lower and upper bounds inherited from ancestors. The left recursive call tightens the upper bound; the right call tightens the lower bound. Optional bounds avoid inventing numeric sentinels that fail at the smallest or largest integer. This is a direct application of the boundary discipline from Chapter 1.

The number of comparisons is proportional to path length. A tree with n nodes can have a path of length proportional to n , so a plain binary search tree has linear worst-case lookup. Calling it “binary” does not guarantee the repeated halving achieved by binary search on an ordered array. Shape, not merely two child pointers, determines how much remains after each comparison.

The comparator also defines equivalence. This integer example uses ordinary less-than and equality consistently. A generic tree should derive duplicate-equivalence from its ordering policy and require that policy to remain stable while keys are stored.

10.1.2 Inserting New Nodes

Insertion follows the same path and creates a node only at a null owning link. An equal existing key returns false without adding a duplicate. Allocation happens at the empty position, so a failed allocation does not remove an existing subtree.

The recursive implementation receives the owning unique-pointer link by reference. This lets it replace a child or the root uniformly without inventing a separate raw owner. The example stores integers and supports bounded test domains; a general key/value container needs comparator, allocation, iterator, and exception contracts beyond this teaching implementation.

Insertion follows the same comparisons as search until it reaches an empty owning link. A new node is attached there. Every ancestor decision constrains the new key to the interval represented by that position, so inserting at the discovered empty link preserves global ordering.

Duplicates are rejected in this unique-key tree. The boolean return distinguishes a newly inserted key from one already present. A multiset tree would need a different policy, such as storing a count or defining where duplicate nodes belong. Allowing equal keys on either side without a consistent rule complicates search, deletion, and validation.

The source allocates only when it reaches the empty link. If allocation throws, existing ancestors retain their owning links and no new node is published. For the integer payload, comparisons and construction are straightforward. A generic comparator with side effects or exceptions would need additional reasoning about whether the path and state remain valid.

Insertion order controls shape. Insert one, two, three, four into an unbalanced tree and each new key becomes a right descendant of the previous maximum. Insert three, one, four, two and the shape is different even though the final key set is identical. An in-order snapshot can prove key order while concealing a poor shape.

This distinction matters for testing. Comparing sorted output with `std::set` checks membership and ordering, but it does not show that the custom tree has logarithmic height. Add a shape measurement only when the interface or representation promises one. Chapter 11 introduces such a promise with AVL balance rather than inferring it from a successful content comparison.

For repeated bulk ingestion, consider whether an already sorted vector with binary search would satisfy the workload, whether a balanced tree is needed for ongoing updates, or whether a batch tree construction is appropriate. The existence of a custom insertion method is not a requirement to call it repeatedly on every possible input representation.

BST Insertion and Search

Purpose. The binary search property is the invariant that makes ordered navigation possible.

```
#include <iostream>
#include <memory>
struct Node{int v;std::unique_ptr<Node>l,r;};
void insert(std::unique_ptr<Node>&n,int x){if(!n)
{n=std::make_unique<Node>(Node{x});return;}if(x<n->v)insert(n->l,x);else if(x>n-
>v)insert(n->r,x);}
bool contains(const Node*n,int x){while(n){if(x==n->v)return true;n=x<n->v?n-
>l.get():n->r.get();}return false;}
int main(){std::unique_ptr<Node>r;for(int x:
{5,2,8,3})insert(r,x);std::cout<<std::boolalpha<<contains(r.get(),3)<<'
'<<contains(r.get(),7)<<'\n';}
```

How it works.

- Insertion sends smaller keys left and larger keys right.
- Search uses the same ordering rule to discard one subtree at each node.
- Duplicates are ignored by this policy.

Expected behavior/output. The output is ``true false``.

Experiment. Insert values in sorted order and observe how the tree shape affects search path length.

10.2 Removing Nodes Safely

Deletion must preserve both the ordering invariant and ownership of all remaining nodes. The required replacement depends on whether the matched node has zero, one, or two children.

Keep a local owner for the removed node while transferring its surviving subtree, so replacing an owning link does not accidentally destroy the resource you still need to move.

10.2.1 Removing Nodes with Zero or One Child

A leaf can be replaced with an empty link. A node with one child can be replaced by that child's subtree because all its keys already satisfy the ancestor bounds. The code moves the old node into a local unique owner, then moves its surviving child into the original owning link.

This ordering is more robust than reaching through an object while assigning over the unique pointer that owns it without carefully sequencing access. After the child is detached, destroying the old node no longer destroys the surviving subtree. Root deletion uses the same operation as child deletion because both are owning links.

A node with no children can be removed by clearing the owning link that reaches it. A node with one child can be removed by replacing that owning link with the child's owner. In both cases the subtree's remaining keys continue to satisfy the interval constraints inherited from the removed node's ancestors.

The temporary owner named `old` is important. Moving the current node into it keeps the node alive while its child owner is extracted. Replacing the original link with the child then leaves `old` owning only the removed node. Cleanup does not accidentally release the retained subtree.

Root removal should use the same owning-link abstraction as interior removal. Treating the root as a special raw pointer often creates a separate code path that forgets to update the public entry point. Passing a reference to the owning `unique_ptr` lets the recursive helper replace either the root or a child link through the same operation.

The returned boolean describes whether a key was present and removed. Searching for a missing key returns false without mutation. Removing the sole root returns true and leaves an empty tree that can later accept another insertion. Reusable empty states deserve explicit tests because stale metadata or observers often survive endpoint mutations.

Removing a node invalidates any observation of that node's lifetime. The example does not expose node pointers, so callers use keys and owned sorted snapshots instead. A public iterator design would need to specify which observations survive deletion and whether replacement semantics affect identity. That additional interface is not implied by private pointer stability.

Cleanup depth remains a limitation. Although search is iterative, insertion, erase, in-order traversal, and nested node destruction can follow a degenerate tree recursively. Avoid presenting an iterative `contains` method as proof that the entire class is safe for arbitrarily deep input. Shape-related resource constraints apply to the whole object lifecycle.

10.2.2 Removing Nodes with Two Children

For a node with two children, choose the in-order successor: the smallest key in its right subtree. Replace the node's integer key with that successor key, then erase the successor from the right subtree. The successor has no left child, so its removal reduces to an easier case.

The integer key assignment is nonthrowing. For a generic payload whose copy can throw or whose identity must remain stable, replacement needs a more careful contract or a node-relinking strategy. The temporary duplicate key during this internal operation is resolved before returning; callers observe the restored unique-key invariant. Do not expose concurrent unsynchronized traversal during such mutation.

A two-child node cannot simply be replaced by either child without accounting for the other subtree. The example chooses the in-order successor: the smallest key in the right subtree. That key lies after every left-subtree key and before or equal to every other right-subtree key, with uniqueness making the relevant inequalities strict after removal.

Find the successor by following left links from the right child. The successor has no left child; otherwise a smaller key would exist in that subtree. Its later removal therefore reduces to the zero- or one-child case already understood. This is a useful design pattern: transform a difficult structural case into a simpler one with a proven update rule.

The implementation copies the successor's integer key into the target node, then removes the successor key from the right subtree. During the private operation, the same key briefly appears twice. The unique-key invariant is restored before the method returns. No unsynchronized observer is allowed to inspect the intermediate state.

For integers, key assignment does not throw and node identity is not part of the public contract. A generic record could be different. Replacing a payload might throw, move expensive resources, or make a retained node handle suddenly identify another record. A node-relinking deletion can preserve different identity properties, but it is more involved. Choose the algorithm according to the promised interface, not only the diagram.

Test a successor that is the immediate right child and one several levels down. Also test a successor with its own right child, which must remain attached when the successor is removed. A three-node root deletion covers only the simplest two-child arrangement.

The sorted snapshot after deletion must equal the prior key set minus exactly the requested key. Checking only that the requested key is absent can miss accidentally discarded subtrees. Differential testing against an independent standard set provides a convenient complete membership oracle for the bounded integer domain.

BST Deletion Covering Leaf, One-Child, and Two-Child Cases

Purpose. Safe deletion must preserve both ownership and the BST ordering invariant across all structural cases.

```
#include <memory>
#include <iostream>
struct Node{int v;std::unique_ptr<Node>l,r;};
void insert(std::unique_ptr<Node>&n,int x){if(!
n)=std::make_unique<Node>(Node{x});else if(x<n->v)insert(n->l,x);else if(x>n-
>v)insert(n->r,x);}
void erase(std::unique_ptr<Node>&n,int x){if(!n)return;if(x<n->v)return erase(n-
>l,x);if(x>n->v)return erase(n->r,x);if(!n->l){n=std::move(n->r);return;}if(!n-
>r){n=std::move(n->l);return;}Node*s=n->r.get();while(s->l)s=s->l.get();n->v=s-
>v;erase(n->r,s->v);}
void in(const Node*n){if(!n)return;in(n->l.get());std::cout<<n->v<<' ';in(n-
>r.get());}
int main(){std::unique_ptr<Node>r;for(int x:
{5,2,8,1,3,7,9})insert(r,x);erase(r,5);in(r.get());}
```

How it works.

- Deleting a zero/one-child node can replace the owning pointer directly.
- For two children, the inorder successor supplies a replacement key before that successor is removed.
- `unique_ptr` ownership moves prevent leaks and dangling ownership during structural replacement.`

Expected behavior/output. The inorder output remains sorted and omits 5.

Experiment. Delete a leaf (1), a one-child node after modifying the tree, and the root; verify sorted order each time.

Worked example — Differentially test insertion, lookup, and every deletion shape

Requirement and design

Apply three thousand deterministic insert/erase operations over one hundred possible integer keys and compare the tree's sorted traversal with a standard set after every operation. Check return values and membership, and explicitly delete a root with two children.

The custom tree is unbalanced. The small bounded key domain limits recursion in the fixture; arbitrary deep production trees require a stronger strategy.

Track the owning link through deletion

The public methods delegate mutation to helpers receiving an owning-link reference. This makes replacement of a subtree explicit. Draw the tree before and after each branch in `erase`: missing link, recurse left, recurse right, replace by right child, replace by left child, and replace the key using a successor.

The generated test uses keys from a deliberately small range. Repeated insertions and removals therefore exercise duplicates, absence, and many reuse patterns. After every operation it compares the entire in-order result with a standard set, then checks membership for the current key. These observations complement each other: the full snapshot detects lost unrelated keys, while the return value detects an incorrect operation outcome.

The fixed tree containing two, one, three explicitly exercises two-child root removal. It is useful even though generated tests may also encounter that shape because its purpose is obvious and reproducible without inspecting a random history.

The standard set is not an oracle for custom-tree height or node layout. It establishes the same mathematical unique-key collection. Performance and recursion limits remain separate questions, developed in the next chapter.

Complete implementation

The complete, buildable chapter project is `source/book3/ch10/`. The following files are reproduced from that project; the publishing script inserts the source directly so the listing and companion remain synchronized.

main.cpp

```
#include "course_test.hpp"
#include <memory>
#include <optional>
#include <random>
#include <set>
#include <vector>
#include <utility>

class SearchTree {
    struct Node {
        int key;
        std::unique_ptr<Node> left, right;
        explicit Node(int k) : key(k) {}
    };
    std::unique_ptr<Node> root_;
    static bool insert(std::unique_ptr<Node>& node, int key) {
        if (!node) { node = std::make_unique<Node>(key); return true; }
        if (key == node->key) return false;
        return insert(key < node->key ? node->left : node->right, key);
    }
    static bool erase(std::unique_ptr<Node>& node, int key) {
        if (!node) return false;
        if (key < node->key) return erase(node->left, key);
        if (key > node->key) return erase(node->right, key);
        if (!node->left) { auto old = std::move(node); node = std::move(old->right); }
        else if (!node->right) { auto old = std::move(node); node = std::move(old-
```

```

>left); }
    else {
        Node* successor = node->right.get();
        while (successor->left) successor = successor->left.get();
        node->key = successor->key;
        return erase(node->right, successor->key);
    }
    return true;
}
static void inorder(const Node* node, std::vector<int>& result) {
    if (!node) return;
    inorder(node->left.get(), result); result.push_back(node->key);
    inorder(node->right.get(), result);
}
public:
    bool insert(int key) { return insert(root_, key); }
    bool erase(int key) { return erase(root_, key); }
    bool contains(int key) const {
        const Node* node = root_.get();
        while (node) {
            if (key == node->key) return true;
            node = key < node->key ? node->left.get() : node->right.get();
        }
        return false;
    }
    std::vector<int> sorted() const {
        std::vector<int> result; inorder(root_.get(), result); return result;
    }
};

int main() {
    SearchTree actual;
    std::set<int> expected;
    std::mt19937 random{7};
    for (int step = 0; step < 3000; ++step) {
        const int key = static_cast<int>(random() % 100) - 50;
        if (random() % 2 == 0) CHECK(actual.insert(key) ==
expected.insert(key).second);
        else CHECK(actual.erase(key) == (expected.erase(key) != 0));
        CHECK(actual.sorted() == std::vector<int>(expected.begin(), expected.end()));
        CHECK(actual.contains(key) == expected.contains(key));
    }
    SearchTree three;
    for (int k : {2, 1, 3}) three.insert(k);
    CHECK(three.erase(2)); CHECK(three.sorted() == std::vector<int>({1, 3}));
    course::report();
}

```

Build, run, and inspect

Run this block from the top-level extracted package. The fixture runner supplies input where needed and checks exit status and output in a temporary working directory.

```

cmake -S source/book3/ch10 -B build/b3-ch10 \
  -DCMAKE_BUILD_TYPE=Debug
cmake --build build/b3-ch10 --parallel 2
ctest --test-dir build/b3-ch10 --output-on-failure

```

For a multi-configuration generator on Windows, append `--config Debug` to the build command and `-C Debug` to CTest. The executable target is `b3_ch10`.

Evidence and interpretation

Each insertion result agrees with whether the reference set added a new key. Each erase result agrees with whether a key existed. In-order traversal must exactly equal the reference's sorted unique sequence after every update, checking the global ordering and content rather than only the recently touched key.

The explicit three-node tree with root two and children one and three exercises the two-child replacement case. Erasing two leaves one and three in order. Randomized operations supplement this targeted shape rather than replacing it.

The tree's recursive insertion, deletion, traversal, and default subtree destruction can all become deep in a degenerate shape. Correctness on the bounded fixture does not make that representation safe for an adversarial million-key insertion sequence. The next chapter introduces balancing, while iterative destruction remains a separate engineering concern.

Failure clinic — A local ordering check missed an ancestor violation

A validator accepts a left child smaller than its parent, but a deeper right descendant is larger than the root. Local comparisons are insufficient. Carry ancestor bounds through recursion so every node is checked against its full permitted interval. A tree can satisfy every immediate comparison used by a weak validator and still misdirect search.

Deleting a two-child node removes several unrelated keys. Inspect the successor removal and ownership transfers. A raw successor pointer is only an observer; clearing its parent's owner before extracting a retained child can destroy too much. Compare the entire sorted snapshot with the reference set after the failing operation.

A repeated deletion returns true twice. The method is confusing "traversal completed" with "a key was removed." Preserve the boolean result through recursive calls and return false at an empty search link. Test success followed immediately by absence for the same key.

A sorted input becomes progressively slower despite correct outputs. That is the expected weakness of an unbalanced tree, not necessarily a defect in comparison code. Measure height and operation counts, then select a balancing strategy or another representation. A correctness regression should not be rewritten to hide the performance limitation.

Practice and implementation lab

Add targeted deletion tests for an empty tree, a leaf, a root with one child, a two-child node whose successor has a right child, and a missing key. Compare full sorted content after each change.

Insert ascending keys and measure height using a helper. Compare with a deliberately balanced insertion order without claiming the unbalanced implementation guarantees either shape. Define how a future map-like tree would preserve key/value association during successor replacement.

Review answers and solution notes

The search invariant constrains entire subtrees, so in-order traversal produces sorted keys when the tree is correct. Operation cost is proportional to height, not automatically logarithmic in size.

One-child deletion transfers ownership of the surviving subtree. Two-child deletion uses the right subtree's minimum successor and then removes its old occurrence. Integer assignment makes this teaching replacement simple; generic payload exceptions and stable-identity requirements need additional design. Targeted shape tests are essential alongside differential random sequences.

Engineering takeaway. Preserve subtree ordering and ownership on every update, and express performance in terms of the height you actually maintain.

Guided lab — Validate whole-subtree ordering

Add a diagnostic validator using optional lower and upper integer bounds. Do not reuse `contains` as the validator: a search can miss exactly the keys misplaced by a broken invariant. Recompute the constraints from the root and inspect every node.

Create fixed deletion fixtures for a leaf, a root with one left child, a root with one right child, an immediate successor, a deep successor, and a successor with a right child. For each fixture, compare the complete remaining key set and verify that deleting the same key again returns false.

Add duplicate-heavy histories and a repeated empty-reuse cycle. Keep a vector of operations so a generated failure can be replayed exactly. When a failure occurs, shorten the history while preserving it; a minimal sequence often exposes one incorrect structural transition more clearly than thousands of preceding operations.

Instrument comparisons for sorted and shuffled insertion orders containing the same distinct keys. Report total insertion comparisons and final height. Do not use wall-clock time yet. Explain why identical in-order outputs can coexist with different search costs and recursion depths.

For an interface-design exercise, imagine nodes store records with stable external identifiers and mutable metadata. Decide whether copying a successor payload into another node is allowed under your identity promise. If callers retain node handles, document what deletion changes. The correct answer may require a different deletion mechanism rather than a more elaborate test for the current one.

Acceptance evidence. Include global-bound validation, complete membership comparison, accurate boolean results, and every structural deletion case. Record that the implementation is unbalanced and that recursive paths remain bounded only by the chosen test domain.

Review reasoning. Why can binary search in an array halve candidates while a binary tree search does not? Tree shape may be unbalanced. Why does the successor have no left child? It is the minimum of the right subtree. Why retain a temporary owner during replacement? It keeps the victim alive while retained child ownership is transferred. Why compare all keys after deletion? Absence of the target alone does not prove preservation of unrelated data. Why does a sorted snapshot not prove good performance? It says nothing about height.

Primary reference notes

Search-tree context: [ordered-container requirements](#), [unique pointers](#), and [MIT tree-algorithm lecture directory](#). The custom deletion cases are explained directly from the implementation.

Chapter 11 — Keeping Search Trees Balanced

A search tree can preserve sorted order and still be a poor index. Insert increasing identifiers into the ordinary tree from Chapter 10 and each new key extends the same branch. Searching becomes a walk through nearly every element. HarborWorks needs the ordering property without allowing an input sequence to dictate an arbitrarily long search path.

Balancing is a structural invariant maintained during updates, not a periodic wish that the data will arrive randomly. This chapter implements AVL insertion and its four rotation cases, then examines the different color invariants used by red-black trees. The complete program includes a red-black color validator, not a second, fully implemented red-black container.

The previous chapter's tree can represent the right keys and still have the wrong shape for the workload. HarborWorks now wants ordered lookup whose path length is controlled as new identifiers arrive. A balancing invariant adds a structural guarantee beyond ordinary binary-search ordering.

The chapter's complete implementation performs AVL insertion with rotations and independent validation. It also includes a small red-black color-property validator for comparison. These are different deliverables: the color checker is not a complete red-black map, and the AVL example does not implement deletion. Understanding the precise scope of each program is part of learning to evaluate data-structure claims.

11.1 Understanding Tree Imbalance

A binary search tree's cost is governed by height. A tree with one thousand nodes can have a short branching structure or resemble a linked list. Both satisfy the ordering rule. A balance requirement adds information about shape that ordering alone cannot supply.

11.1.1 Recognizing Degenerate Trees

For a chain of n nodes, a search can compare against all n keys. Building that chain by repeated ordinary insertion takes a quadratic number of comparisons in the worst case. A sorted source file is therefore a useful adversarial input, not an exotic failure case. Randomized test data alone can conceal this defect.

A degenerate shape also affects recursive traversal, validation, and destruction. Replacing recursive search with a loop avoids one stack problem but does not restore logarithmic lookup. The remedy must address the height, not merely the syntax of one operation. A balanced representation bounds the number of ancestor decisions required to locate a key.

Sorted insertion is a simple adversarial input for an ordinary search tree. Insert zero, one, two, and so on. Every comparison selects the right subtree, so the structure becomes a chain. The next insertion visits all existing keys before finding an empty link. Summing those visits across the build gives a quadratic total in the number of inserted keys.

This input is not artificial in an operational sense. Identifiers can arrive in timestamp order, sequence-number order, or sorted file order. A structure that behaves acceptably only when input happens to be shuffled has an unstated workload assumption. Random testing can help discover defects, but it cannot replace a structural guarantee when order is controlled by external data.

A tall tree also affects more than lookup. Recursive traversal, validation, mutation, and destruction may follow the same long chain. Making one method iterative removes that method's recursion depth but does not restore short search paths or repair the rest of the lifecycle. Measure the property causing the cost: height.

A balanced tree need not have perfectly equal subtree sizes. Requiring equal sizes after every insertion could be unnecessarily expensive. AVL trees instead bound the difference in child heights at every node. That local condition is strong enough to constrain global height while permitting many valid shapes.

Do not mistake a visually symmetric root for a balanced tree. One side can contain a deep imbalance below a superficially even top level. A validator must inspect every node and independently derive child heights. This is similar to global-bound checking in the previous chapter: a local-looking property has to hold throughout the structure.

When choosing a representation, separate the requirement from the mechanism. "Ordered lookup with a worst-case logarithmic path" is a requirement. "AVL insertion" is one mechanism that can satisfy part of it under its maintained invariant. A particular library implementation may choose another mechanism while exposing comparable observable guarantees.

11.1.2 Measuring Balance

This implementation defines an empty subtree's height as zero and a leaf's height as one. Chapter 9 also discussed edge-count height; the two conventions differ by one for nonempty trees. State the convention before checking a formula. Here, `height(node) = 1 + max(height(left), height(right))`.

The AVL balance factor is left height minus right height. Every node must have a factor in $\{-1, 0, 1\}$. It is not enough for the root alone to look balanced. The validator independently recomputes child heights, checks each stored height, checks the balance condition, and carries lower and upper key bounds through the whole tree.

An update must repair both topology and cached metadata. Stale height fields can lead a later insertion to choose the wrong repair even when the current picture looks acceptable. Validation should compare stored metadata with independently derived values rather than trusting the same update function it is intended to test.

Cached height is metadata derived from topology. In this chapter, an empty subtree has height zero and a leaf has height one. A node's height is one plus the larger child height. The balance factor is left height minus right height, and the permitted values are minus one, zero, and one.

The height field must be updated after a child link changes. A stale height can remain unnoticed while searches still return correct keys, then misdirect a later rotation decision. Testing only sorted output therefore misses a class of structural defects. The validator recomputes heights from children and compares them with stored values rather than trusting the update routine under test.

A small recurrence explains the logarithmic consequence. Let $N(h)$ be the minimum nodes needed for an AVL tree of node-count height h . To make height h with as few nodes as possible, one child can have height $h-1$ and the other $h-2$; a larger difference is forbidden. Thus $N(h) = 1 + N(h-1) + N(h-2)$, with empty and leaf base cases. The resulting sequence grows rapidly enough that height grows only logarithmically with node count.

This is a bound derived from the invariant, not from the fact that one thousand test insertions produced a short tree. The test verifies that this implementation maintains the invariant on its exercised inputs. The recurrence explains why a correctly maintained invariant has the desired asymptotic consequence.

Use diagnostic assertions carefully in performance experiments. Calling a full recursive validator after each insertion deliberately adds linear work per validation. The resulting overall test runtime can look quadratic even when each insertion's balancing path is logarithmic. That does not invalidate the data structure; it means the timed workload includes expensive verification. Keep correctness instrumentation outside measurements intended to isolate insertion cost.

The validator also carries ordering bounds, because balance without ordering is not a search tree. Both independent properties must hold. A short arbitrary binary hierarchy cannot answer ordered lookup correctly merely because its cached heights are consistent.

Sorted Insertion Producing a Degenerate BST

Purpose. A BST's asymptotic advantage depends on shape; an ordering invariant alone does not guarantee balance.

```
#include <iostream>
#include <memory>
struct Node{int v;std::unique_ptr<Node>r;};
int main(){auto root=std::make_unique<Node>(Node{1});Node*p=root.get();for(int
x=2;x<=6;++x){p->r=std::make_unique<Node>(Node{x});p=p->r.get();}int
depth=0;for(Node*n=root.get();n;n=n->r.get())++depth;std::cout<<depth<<'\n';}
```

How it works.

- Each larger key becomes the right child of the previous node.
- The resulting tree height equals the number of elements rather than logarithmic height.
- Search then behaves like a linked-list scan.

Expected behavior/output. The program prints depth `6`.

Experiment. Compare with a manually balanced insertion order such as 4,2,6,1,3,5.

11.2 Understanding Self-Balancing Trees

Self-balancing trees preserve a height bound while changing only a small local part of the structure at each repair. Rotations change parent-child relationships without changing the in-order sequence. They are ownership transfers, not copies of all the keys in a subtree.

11.2.1 Understanding AVL Rotations

For a right rotation, call the old root y , its left child x , and x 's right subtree B . Before the rotation, every key in B is greater than x and less than y . Promote x , make B the new left subtree of y , and attach y as x 's right child. The ordering of A, x, B, y, C is unchanged. Update the demoted node's height before the promoted node's height because the latter depends on the former.

A left-left insertion is repaired by one right rotation. A right-right insertion is symmetric. For left-right, rotate the left child left and then rotate the root right; right-left uses the opposite pair. The tests use the three-key insertion orders 3,2,1; 1,2,3; 3,1,2; and 1,3,2 so each branch is deliberately exercised.

AVL height grows logarithmically: a minimum-size AVL tree of height h needs a root and subtrees of heights $h-1$ and $h-2$. The resulting Fibonacci-like lower bound on node count grows exponentially with h . Inverting that relationship gives a logarithmic height bound. The conclusion follows from the maintained invariant, not from a favorable benchmark.

The implementation allocates the prospective node before moving ownership of the root into recursive insertion. If allocation fails, the original tree is still owned and unchanged. The remaining insertion path moves `unique_ptr` objects and performs integer comparisons. A more general comparator or payload could throw; its exception contract would need fresh analysis rather than copying this `noexcept` declaration unchanged.

A rotation changes which node is the local root while preserving the in-order sequence. For a right rotation, let y be the old root and x its left child. The subtree between their keys, conventionally called B , moves from the right of x to the left of y . The other subtrees remain on their corresponding outer sides.



Before and after, the key order is A, x, B, y, C . This ordering argument is the reason the structural change is legal. A rotation is not a swap of two key values, and it does not copy every node in B . It transfers owning links while preserving the nodes' identities.

Update the demoted node first because its children have changed. Then update the promoted node, whose new height depends on the demoted node's repaired height. Reversing that order can cache a height based on stale information and cause later balancing failures.

The insertion direction identifies four familiar cases. Left-left and right-right need one rotation. Left-right and right-left need a child rotation followed by a root rotation. Trace the three-key sequences in the test rather than memorizing labels without a picture. Inserting three, one, two produces a different local shape from three, two, one even though both become balanced around two.

The implementation allocates the prospective node before transferring root ownership into its recursive helper. That ordering protects the existing tree from allocation failure. The remaining helper uses nonthrowing integer operations and owning-pointer transfers under this specialized contract. A generic comparator or key type could throw; copying the `noexcept` declaration into a generic version without reanalyzing that possibility would be incorrect.

Rotations repair shape, not every possible invariant. A structure with subtree counts, sums, or parent observers must update those fields too. Add derived metadata only with a complete rotation update plan and an independent validator.

11.2.2 Understanding Red-Black Tree Properties

A conventional red-black tree labels nodes red or black, treats null leaves as black, requires a black root, forbids a red node from having a red child, and requires equal black height along paths from a node to descendant null leaves. These rules allow some paths to be longer than others while still bounding the longest path to at most twice the shortest black-height path.

The included validator checks these color properties. A null leaf contributes one black count under this program's convention. An invalid child returns -1; unequal child black heights or a red-red edge also return -1. The wrapper separately checks the root color. Its nodes do not even store keys, so this validator cannot verify binary-search ordering. A complete red-black container needs both ordering and color checks, as well as insertion and deletion repairs.

AVL and red-black trees make different balancing tradeoffs. Do not select one solely from a slogan about faster searches or cheaper writes: payload size, allocator behavior, locality, and the workload matter. `std::map` specifies observable behavior and complexity, not a mandate that your implementation use this particular teaching tree.

Red-black trees use color rules to constrain path lengths more loosely than AVL height differences. Under the conventional model, the root is black, null leaves are black, red nodes do not have red children, and every path from a node to descendant null leaves contains the same number of black nodes under a fixed counting convention.

The no-adjacent-red rule limits how many extra red nodes can appear between black nodes on a path. Equal black height then prevents one root-to-leaf path from becoming arbitrarily longer than another. This yields a logarithmic height bound while allowing shapes that an AVL balance check would reject.

The included `RedBlackNode` stores only color and child observers. Its validator can reject a red root, a red-red parent-child edge, or unequal black heights. It cannot check key ordering because there are no keys to inspect. It also does not perform insertion or deletion repairs. Treating that demonstration as a complete red-black container would overstate both code and evidence.

A full validator for a real red-black search tree would combine ownership or structural checks, global ordering bounds, root color, red-child rules, and black-height consistency. Cached size or other augmentations would add further obligations. The checks should be independent of the mutation code so the same mistaken update does not validate itself.

The choice between AVL and red-black mechanisms is not settled by a slogan about one being for reads and the other for writes. Path lengths, rotations, allocation, cache behavior, payload size, and operation mix all contribute. Standard containers specify requirements and complexity rather than requiring the exact teaching representation shown here.

Deletion is especially important when assessing completeness. A balanced insertion implementation that never removes nodes can satisfy an append-only experiment while remaining unsuitable for a mutable

index requiring erasure. Before adopting a custom tree, inventory every required operation and its failure behavior. A correct fragment is useful instructional material; it is not automatically a complete library.

Single AVL-Style Right Rotation

Purpose. Self-balancing trees restore bounded height through local rotations that preserve inorder key order.

```
#include <iostream>
#include <memory>
struct Node{int v;std::unique_ptr<Node>l,r;};
void rotate_right(std::unique_ptr<Node>&root){auto old=std::move(root);auto
pivot=std::move(old->l);old->l=std::move(pivot->r);pivot-
>r=std::move(old);root=std::move(pivot);}
int main(){auto r=std::make_unique<Node>(Node{3});r-
>l=std::make_unique<Node>(Node{2});r->l-
>l=std::make_unique<Node>(Node{1});rotate_right(r);std::cout<<r->v<<' '<<r->l-
>v<<' '<<r->r->v<<'\n';}
```

How it works.

- The left child becomes the new subtree root.
- The old root becomes the pivot's right child.
- The transferred middle subtree preserves key ordering during rotation.

Expected behavior/output. The output is `2 1 3`.

Experiment. Draw the pointers before and after rotation, then implement the symmetric left rotation.

Worked example — Keep an ordered insertion workload shallow

Requirement and design

Implement a duplicate-ignoring integer AVL tree with unique node ownership. Validate every mutation against key ordering, cached height, and local balance. Demonstrate all four rotation shapes and a run of one thousand ascending keys. Separately validate a small red-black color structure without claiming to implement red-black insertion or deletion.

Make each rotation branch unavoidable

The test constructs four separate three-key trees. The insertion orders isolate left-left, right-right, left-right, and right-left repair. Validation runs after every insertion, so a faulty intermediate rotation cannot be hidden by a later one. Each completed tree should have node-count height two.

The ordered sequence of one thousand keys then exercises repeated rebalancing under an input that degenerates an ordinary tree. The full validator checks ordering, local balance, and cached height throughout. The final height assertion is a coarse additional check, not the mathematical source of the logarithmic claim.

Notice that duplicate insertion still prepares a prospective node before the recursive search discovers equality. The unused candidate is then released. This simplifies the exception boundary at the cost of an unnecessary allocation for duplicates. It is a deliberate tradeoff in the teaching implementation, not an optimal policy for every workload.

The red-black fixtures are separate const node graphs used only for color validation. Their borrowed child pointers remain valid because the local objects outlive the checks. They are not owned by the AVL tree and are not part of its mutation algorithm.

Complete implementation

The complete, buildable chapter project is `source/book3/ch11/`. The following files are reproduced from that project; the publishing script inserts the source directly so the listing and companion remain synchronized.

main.cpp

```
#include "course_test.hpp"
#include <algorithm>
#include <cmath>
#include <memory>
#include <optional>
#include <vector>
#include <utility>

class AvlTree {
    struct Node {
        int key, height{1};
        std::unique_ptr<Node> left, right;
        explicit Node(int k) : key(k) {}
    };
    std::unique_ptr<Node> root_;
    static int height(const std::unique_ptr<Node>& p) { return p ? p->height : 0; }
    static void update(Node& p) { p.height = 1 + std::max(height(p.left),
height(p.right)); }
    static int balance(const Node& p) { return height(p.left) - height(p.right); }
    static std::unique_ptr<Node> rotate_right(std::unique_ptr<Node> top) {
        auto promoted = std::move(top->left);
        top->left = std::move(promoted->right);
        update(*top);
        promoted->right = std::move(top);
        update(*promoted);
        return promoted;
    }
    static std::unique_ptr<Node> rotate_left(std::unique_ptr<Node> top) {
        auto promoted = std::move(top->right);
        top->right = std::move(promoted->left);
        update(*top);
        promoted->left = std::move(top);
        update(*promoted);
        return promoted;
    }
    static std::unique_ptr<Node> insert(std::unique_ptr<Node> node, int key,
```

```

        std::unique_ptr<Node>& fresh) noexcept {
    if (!node) return std::move(fresh);
    if (key < node->key) node->left = insert(std::move(node->left), key, fresh);
    else if (key > node->key) node->right = insert(std::move(node->right), key,
fresh);
    else return node;
    update(*node);
    if (balance(*node) > 1) {
        if (key > node->left->key) node->left = rotate_left(std::move(node-
>left));
        return rotate_right(std::move(node));
    }
    if (balance(*node) < -1) {
        if (key < node->right->key) node->right = rotate_right(std::move(node-
>right));
        return rotate_left(std::move(node));
    }
    return node;
}

static int validate(const Node* node, std::optional<int> low, std::optional<int>
high) {
    if (!node) return 0;
    CHECK(!low || *low < node->key);
    CHECK(!high || node->key < *high);
    const int left = validate(node->left.get(), low, node->key);
    const int right = validate(node->right.get(), node->key, high);
    CHECK(std::abs(left - right) <= 1);
    CHECK(node->height == 1 + std::max(left, right));
    return node->height;
}

public:
    void insert(int key) {
        auto fresh = std::make_unique<Node>(key);
        root_ = insert(std::move(root_), key, fresh);
    }

    int validate() const { return validate(root_.get(), {}, {}); }
};

struct RedBlackNode {
    bool red;
    const RedBlackNode* left{};
    const RedBlackNode* right{};
};

int black_height(const RedBlackNode* node) {
    if (!node) return 1; // Null leaves are black in this representation.
    if (node->red && ((node->left && node->left->red) ||
        (node->right && node->right->red))) return -1;
    const int left = black_height(node->left), right = black_height(node->right);
    if (left < 0 || left != right) return -1;
    return left + (node->red ? 0 : 1);
}

bool valid_colors(const RedBlackNode* root) {
    return (!root || !root->red) && black_height(root) >= 0;
}

int main() {
    for (const auto& order : {std::vector<int>{3, 2, 1}, {1, 2, 3}, {3, 1, 2}, {1, 3,
2}}) {
        AvlTree tree;
        for (int key : order) { tree.insert(key); tree.validate(); }
        CHECK(tree.validate() == 2);
    }
    AvlTree ordered;
    for (int key = 0; key < 1000; ++key) { ordered.insert(key); ordered.validate(); }
    CHECK(ordered.validate() < 20);
    const RedBlackNode left{true}, right{true};
    const RedBlackNode valid{false, &left, &right};
}

```

```
const RedBlackNode invalid_root{true, &left, &right};
CHECK(valid_colors(&valid)); CHECK(!valid_colors(&invalid_root));
course::report();
}
```

Build, run, and inspect

Run this block from the top-level extracted package. The fixture runner supplies input where needed and checks exit status and output in a temporary working directory.

```
cmake -S source/book3/ch11 -B build/b3-ch11 \
  -DCMAKE_BUILD_TYPE=Debug
cmake --build build/b3-ch11 --parallel 2
ctest --test-dir build/b3-ch11 --output-on-failure
```

For a multi-configuration generator on Windows, append `--config Debug` to the build command and `-C Debug` to CTest. The executable target is `b3_ch11`.

Evidence and interpretation

After each of the four three-key sequences, the node-count height is two. During the ascending workload, the validator runs after every insertion; its traversal cost is intentional test overhead and is not the complexity of the public insertion algorithm. The final height check is a loose regression condition, while the recursive invariant checks provide the more informative evidence.

Duplicate insertion returns the existing subtree. Because the public method allocates first, a duplicate attempt still allocates and then destroys an unused node. That is a documented simplicity cost, not an ownership leak. A production design could search first or integrate a safer allocation strategy, but must preserve the exception boundary.

The color test accepts a black root with red leaf children and rejects a red root. Additional invalid fixtures belong in the lab. A passing small fixture does not prove every color violation is already covered.

Failure clinic — The keys were right but the next rotation was wrong

A tree produces the expected in-order keys immediately after a rotation, then fails on a later insertion. Inspect cached heights and update order. Correct topology with stale metadata can misclassify the next imbalance. The repair must update every derived field affected by the rotation.

A rotation loses the middle subtree **B**. The code promoted a child but failed to transfer its inner subtree to the demoted node before overwriting the link. Draw all five ordered regions, not just the two named nodes. A three-node fixture with empty inner subtrees can miss this defect, so add a case where **B** is nonempty.

A benchmark reports quadratic “AVL insertion” while validation runs after every call. Separate the cost of the full-tree checker from mutation. Keep the checker in correctness tests, then measure a build without per-operation validation and perform a final validation outside the timed region.

A red-black checker accepts a tree with unordered keys. That is not surprising when the checker only examines colors. Name each validator by the properties it actually inspects, and combine them when the

required invariant is broader. A passed color check is evidence about colors, not complete search-tree correctness.

Practice and implementation lab

Add fixtures with an unequal black height and a red node whose child is red. Extend the AVL tests with repeated duplicate keys and descending insertion. Record the invariant that each fixture is intended to violate or preserve. Then add a non-owning lookup method and compare its answers with a `std::set<int>` over a fixed-seed randomized insertion sequence.

Review answers and solution notes

A black child on only one side creates unequal black height. A red child with a red grandchild creates a red-red edge even when the root is black. Duplicate keys must not add nodes or change membership. Lookup should follow the ordering comparison until it finds a match or a null child; it must not take ownership of the traversed nodes. Rotations preserve membership because they preserve the complete in-order sequence.

Engineering takeaway. Balance is an invariant over every subtree; verify ordering, topology, ownership, and cached metadata together.

Guided lab — Validate topology, metadata, and order independently

Extend the AVL exercise with an in-order snapshot and compare it with a standard set after deterministic generated insertions. Retain validation after each mutation in this correctness run. Include duplicates and both ascending and descending sequences.

Add rotation fixtures whose inner transferred subtree is nonempty. Label every subtree's permitted key interval before the operation and verify the same sorted sequence afterward. This tests preservation of more than the promoted and demoted nodes.

Introduce a cached subtree size in the exercise. Update it wherever height is updated, then independently recompute it in validation. Explain why rotation repair must update the demoted node before the promoted node for both fields. Do not use the cached size to validate itself.

For the red-black demonstration, create one fixture for each rejected property: red root, red parent with red child, and unequal black heights. Include a valid all-black small tree and a valid tree containing red children. Then explain which malformed key arrangements would remain undetected because the model has no keys.

Derive the first several $N(h)$ values from the AVL minimum-node recurrence. Compare the allowed height implied by the recurrence with an observed tree height, clearly separating mathematical bound and measured implementation state. Avoid claiming that a single fixture proves the recurrence.

Acceptance evidence. Retain all four insertion cases, a nonempty inner-subtree transfer, global ordering validation, independent height and size recomputation, and explicit statements that deletion and a full red-black container are not implemented here.

Review reasoning. Why does balance need checking at every node? A root can look balanced while a descendant is not. Why does a rotation preserve search order? It preserves the in-order sequence of the affected regions. Why allocate before moving the root owner? A failed allocation should not abandon the existing tree. Why is validation excluded from timing? It measures additional full-tree work. Why is color validity insufficient? Search ordering, ownership, and required operations are separate obligations.

Primary reference notes

Library ordering and container requirements: [associative containers](#). Ownership transfers: [unique ownership](#) and [utility operations](#). Further algorithm reading: [MIT algorithm lecture directory](#). The AVL implementation, proofs, and fixtures here are original teaching material.

Chapter 12 — Prioritizing Data with Heaps

HarborWorks schedules work by urgency. It does not need the complete queue sorted after every arrival; it needs the next most urgent item quickly. Maintaining more order than the operation requires can waste work. A heap preserves exactly enough structure to expose one extreme element while supporting efficient updates.

This chapter builds a vector-backed minimum heap and checks it against the standard priority-queue adaptor. The example uses integer priorities. Real task scheduling additionally needs a tie policy, ownership of task payloads, and a decision about changing priorities after insertion.

HarborWorks now wants the next most urgent job, not a fully sorted list after every insertion. A heap keeps just enough order to expose an extreme element efficiently while leaving unrelated positions unordered. That weaker invariant can be exactly what a priority queue needs.

The chapter's minimum heap stores integers in a vector and supports insertion and optional removal of the minimum. The implementation is small, but its correctness depends on complete-tree indexing, selecting the correct child during repair, and preserving shape before restoring order. Treat those as separate steps rather than memorizing a sift routine as an opaque block.

12.1 Building Binary Heaps

A binary heap combines a complete-tree shape with a parent-child ordering property. The tree is filled level by level, left to right. Its shape can therefore be represented by vector indices without allocating a separate node for each item.

12.1.1 Maintaining Heap Properties

In a zero-based array, the children of index p are $2p+1$ and $2p+2$ when those indices exist. The parent of a nonroot index i is $(i-1)/2$. In a minimum heap, every parent is no greater than either child. This makes the root a minimum of the entire heap because every downward path is nondecreasing.

The rule does not order siblings or unrelated subtrees. Printing the backing vector need not produce sorted output. The complete-tree shape gives logarithmic height independently of the values and their arrival order; there is no pointer-tree degeneration from inserting sorted keys.

Index arithmetic still has preconditions. The implementation only computes a child while the parent lies before $\text{size}()/2$, the range of internal nodes. It only computes $(\text{child}-1)/2$ after confirming child is nonzero. These guards avoid unsigned underflow and keep the derived indices inside the represented tree.

A binary heap has two independent properties. Its shape is a complete binary tree, filled level by level from left to right. Its order is a parent-child relation: in a minimum heap, no child is smaller than its parent. Shape makes array indexing possible; order makes the root a minimum.

The root-minimum conclusion follows along every path. A descendant cannot be smaller than its parent, whose value cannot be smaller than its parent, and so on to the root. Transitivity connects the local comparisons to the global minimum property. The same reasoning does not sort siblings or unrelated branches, so an array dump is generally not sorted.

For zero-based storage, children of index p are $2*p+1$ and $2*p+2$ when they exist. The parent of a nonroot index i is $(i-1)/2$. Derive these formulas from level-order positions once, then keep their preconditions visible. Computing $i-1$ for unsigned zero is a bug even when the eventual result is never used.

The worked sift-down loop enters only for an internal-node index before $size()/2$. That guard ensures a left child exists before its index is formed. It separately checks whether the right child exists. A heap with an even number of elements can have a last internal node with only a left child, an important boundary for tests.

The vector's logical size determines tree membership. Unused vector capacity is not a set of additional heap nodes. As with Chapter 3, storage availability does not make an index part of the data structure. All validation and repair loops must use `size`, not `capacity`.

Equal priorities are permitted. The parent relation need not be strict, and duplicate integers should remain separate entries if they were separately inserted. A set would discard duplicates under its uniqueness policy; a heap normally preserves multiplicity. Choose the abstraction according to whether repeated jobs are meaningful.

12.1.2 Performing Heapify Operations

Sifting down assumes both child subtrees are already heaps and only the current root may violate their order. Choose the smaller existing child. If that child is not smaller than the parent, stop; otherwise exchange them and repeat at the child's former index. Choosing an arbitrary child can leave a smaller sibling above an incorrectly placed value.

Bottom-up heap construction starts at the last internal node and sifts each internal node down toward the root. Leaves need no work. This construction takes $O(n)$, not $O(n \log n)$: most nodes are close to leaves and can move only a short distance. Summing work over levels gives a bound proportional to n times a convergent series of $h/2^h$.

Repeated insertion is a different construction method with $O(n \log n)$ worst-case work. Both can be correct; they need not have the same setup cost. The constructor's countdown loop uses $i > 0$ and passes $i-1$, avoiding the common unsigned loop that wraps after processing index zero.

Sift-down begins with a precise assumption: both child subtrees are already heaps, and only the current root may violate their order. Compare the existing children and select the smaller one. If that child is not smaller than the parent, the whole local subtree is a heap. Otherwise swap and continue at the child position.

Choosing the smaller child is necessary. Suppose a parent has value nine and children four and two. Swapping with four leaves two smaller than the new parent four, so the parent-child invariant is still broken. The repair must satisfy both child relationships, not merely improve one comparison.

Bottom-up construction starts from the final internal node and moves toward the root. Leaves already satisfy heap order. When an internal node is processed, its children have been processed or are leaves, so sift-down's precondition holds. This is a dependency-ordered algorithm much like postorder aggregation in a tree.

The construction's linear bound is subtler than multiplying n by a maximum height. Many nodes are leaves and need no sift; many others can move only one level. Only a small number near the root can travel far. Summing the work by possible height yields a linear total because the number of nodes decreases geometrically as possible travel distance increases.

Repeatedly pushing all entries is another correct construction strategy, but its worst-case work can be $O(n \log n)$. The constructor and repeated insertion therefore have different setup costs even though both produce valid heaps. Test them for equivalent removal sequences, not identical backing arrays.

The countdown loop uses a positive counter and passes $i - 1$ to sift-down. A loop written as `unsigned i >= 0` never becomes false after wrapping. This is a recurring C++ boundary lesson: choose loop form from the index domain rather than copying a signed pseudocode countdown mechanically.

A comparator must remain stable while the heap is stored. Changing an external priority policy without rebuilding can make an old arrangement invalid even when no vector element has moved.

Maintaining a Max-Heap with Standard Heap Algorithms

Purpose. A binary heap maintains a partial order sufficient for fast access to the extreme element without fully sorting all elements.

```
#include <algorithm>
#include <iostream>
#include <vector>
int main()
{std::vector<int>h{4,1,7,3};std::make_heap(h.begin(),h.end());h.push_back(9);std::push_heap(h.begin(),h.end());std::pop_heap(h.begin(),h.end());std::cout<<h.back()<<'\n';}
```

How it works.

- `make_heap` rearranges the range so the maximum is at the front.
- `push_heap` restores the heap property after appending one element.
- `pop_heap` moves the maximum to the end while preserving a heap in the shortened prefix.

Expected behavior/output. The first printed maximum is 9 and the popped element is also 9.

Experiment. Repeat with `std::greater<>` to build a min-heap.

12.2 Implementing Priority Queues

The priority-queue abstraction offers insertion and access to or removal of an extreme element. It need not expose its internal tree. Separating the abstraction from the representation allows callers to ask for scheduling behavior without depending on a vector layout.

12.2.1 Inserting Prioritized Values

Insertion appends the new value at the next complete-tree position. Every old parent-child relationship remains valid except possibly the new value's relationship with its parent. Sift upward while the child is smaller, exchanging with its parent until the rule holds or the root is reached. Each exchange moves up one level, giving $O(\log n)$ comparisons in the worst case.

Vector growth can occasionally relocate the backing array. The sift work is logarithmic, but a particular append may also incur $O(n)$ relocation. With geometric vector growth, the allocation component is amortized; reserve storage ahead of time when a capacity plan justifies it. Do not confuse heap-order complexity with all storage-management costs.

The teaching class contains integers, for which comparisons and swaps do not throw. A generic heap must address exceptions from comparison, movement, and allocation. It must also define whether an exception leaves heap order preserved, merely leaves objects valid, or requires rebuilding.

Insertion first preserves complete-tree shape by appending at the next array position. Every preexisting parent-child relationship remains unchanged. Only the new value's relationship with its parent may violate order, so repair follows that one ancestor chain upward.

At each step, compare the child with its parent. If the child is not smaller, stop. Otherwise exchange them and continue from the parent position. The number of possible exchanges is bounded by the complete tree's height. The proof depends on the appended position and unchanged sibling subtrees, not on a global sort after every insertion.

Storage management remains a separate cost. Vector append may allocate and relocate existing elements when capacity is exhausted. The sift itself has logarithmic comparison work, while a particular storage expansion can cost linear movement. An amortized storage argument explains long-run behavior but does not eliminate a single pause. Reserve when a justified capacity plan is part of the application contract.

A scheduler needs more than an integer priority. If equal priorities must preserve arrival order, compare a pair containing priority and a sequence number under a documented direction. The sequence number must have an overflow policy. A default heap does not promise stable equal-priority ordering merely because one small run happened to preserve it.

Mutable priorities require an update mechanism. Changing a value through a retained pointer bypasses sift repair and can invalidate the heap. A design can provide indexed updates, remove and reinsert entries, or use immutable queued records with stale-entry filtering. The routing capstone later uses the last strategy for tentative distances.

The integer example avoids throwing comparisons and swaps. A generic heap must consider whether an exception during sift leaves the array merely containing valid objects or still satisfying heap order. Owning storage safely is not the same as preserving the data structure's semantic invariant after every possible element operation.

12.2.2 Removing Highest-Priority Values

To remove the minimum, save the root's value, move the final array value into the root slot, and remove the final slot. This preserves the complete-tree shape. Sift the replacement down to restore order. The

one-element case needs no sift after removal. Empty removal returns `std::nullopt`, which distinguishes absence from a legitimate priority such as zero.

The standard `std::priority_queue` is a maximum-priority adaptor under its default comparator. The reference uses `std::greater<int>` to expose the minimum instead. Its `top()` and `pop()` are separate operations, so the test reads top before popping and never calls either on an empty reference queue.

Equal priorities are permitted, but this representation is not stable. A scheduler that promises first-arrived-first-served among equal priorities can compare `(priority, sequence_number)` with a documented direction and overflow policy. Updating a stored priority through an external pointer would bypass heap repair; prefer an explicit update mechanism or remove-and-reinsert design when requirements include reprioritization.

Removing the minimum begins by saving the root value. The final array value is moved into the root position, then the final slot is removed. This preserves the complete-tree shape without shifting an entire level-order sequence. Only the root may now violate heap order, so sift-down repairs it.

The one-element case needs special attention. Saving the root and removing the last element leaves an empty heap; no child index should be computed. An empty removal returns absence. A minimum value of zero, a negative value, or a duplicate remains a legitimate present result rather than an empty-state sentinel.

Draining a minimum heap produces a nondecreasing sequence containing every inserted value with its original multiplicity. That is a strong end-to-end property for tests. Checking only the first removed value verifies the root minimum once but may miss a broken repair that affects later removals.

The standard priority queue defaults to exposing the maximum under its usual comparator. The reference in this chapter uses `std::greater<int>` so its top is the minimum. Comparator direction is easy to reverse when moving between a sorting predicate and a priority-queue adaptor. Verify a tiny two-value fixture before trusting a large generated comparison.

A priority queue is useful for selecting a small extreme subset. To retain the largest *k* values from a stream, one design keeps a minimum heap of at most *k* candidates and discards values that cannot improve that set. This solves a different problem from sorting all input and has its own tie, multiplicity, and output-order policy. Do not present the final internal heap array as a sorted top-*k* report.

For an application, removal is a transfer of pending work, not proof that the work completed. A job scheduler may need cancellation, retry, and acknowledgement states outside the heap. The data structure establishes which accepted entry is selected next under its comparison rule; operational completion belongs to another layer.

Scheduling Highest-Priority Work First

Purpose. A priority queue provides heap semantics through a restricted interface tailored to repeated best-element extraction.

```
#include <iostream>
#include <queue>
#include <string>
#include <vector>
struct Job{int priority;std::string name;};struct Compare{bool operator()(const
Job&a,const Job&b)const{return a.priority<b.priority;}};
int main()
{std::priority_queue<Job,std::vector<Job>,Compare>q;q.push({2,"normal"});q.push({5,"urgent"});q.
q.empty()){std::cout<<q.top().name<<' ';q.pop();}}
```

How it works.

- The comparator defines which job has highest priority.
- Insertion does not require maintaining full sorted order.
- `top` always exposes the highest-priority remaining job.

Expected behavior/output. The output begins `urgent normal low`.

Experiment. Reverse the comparator and verify that the lowest numeric priority is processed first.

Worked example — Implement and differentially test a minimum heap

Requirement and design

Provide construction from a vector, insertion, optional removal, and a structural validator. First drain a known heap to verify duplicate preservation and sorted removal order. Then interleave two thousand operations while comparing every removal with a standard minimum-priority queue.

Separate construction, mutation, and validation

The constructor receives a vector containing nine, three, seven, one, and one. It applies bottom-up repair, then the test drains the heap expecting one, one, three, seven, and nine. This verifies both multiplicity and successive minimum extraction rather than only a single root check.

The generated phase compares push and pop behavior with a standard minimum-priority queue. It checks the custom representation's parent-child invariant after each operation. The reference top is read before its pop and only when the reference is nonempty, so the oracle does not invoke an invalid empty operation.

The `valid()` method scans all parent-child relations. That is intentionally linear diagnostic work. It belongs in correctness tests and must be excluded from a measurement claiming to isolate logarithmic

heap mutation. The distinction between algorithm work and verification work appeared in AVL testing and applies equally here.

Neither the constructor nor the reference comparison requires the same backing-array layout. Many arrays satisfy heap order for the same multiset. The contract is the priority behavior and preserved values, not one canonical internal arrangement.

Complete implementation

The complete, buildable chapter project is `source/book3/ch12/`. The following files are reproduced from that project; the publishing script inserts the source directly so the listing and companion remain synchronized.

main.cpp

```
#include "course_test.hpp"
#include <algorithm>
#include <functional>
#include <optional>
#include <queue>
#include <random>
#include <vector>
#include <utility>

class MinHeap {
    std::vector<int> values_;
    void sift_down(std::size_t parent) {
        while (parent < values_.size() / 2) {
            std::size_t child = 2 * parent + 1;
            if (child + 1 < values_.size() && values_[child + 1] < values_[child]) +
+child;
            if (!(values_[child] < values_[parent])) break;
            std::swap(values_[child], values_[parent]);
            parent = child;
        }
    }
public:
    MinHeap() = default;
    explicit MinHeap(std::vector<int> values) : values_(std::move(values)) {
        for (std::size_t i = values_.size() / 2; i > 0; --i) sift_down(i - 1);
    }
    void push(int value) {
        values_.push_back(value);
        std::size_t child = values_.size() - 1;
        while (child > 0) {
            const std::size_t parent = (child - 1) / 2;
            if (!(values_[child] < values_[parent])) break;
            std::swap(values_[child], values_[parent]); child = parent;
        }
    }
    std::optional<int> pop() {
        if (values_.empty()) return std::nullopt;
        const int result = values_.front();
        values_.front() = values_.back(); values_.pop_back();
        if (!values_.empty()) sift_down(0);
        return result;
    }
    bool valid() const {
        for (std::size_t i = 1; i < values_.size(); ++i) {
            if (values_[i] < values_[(i - 1) / 2]) return false;
        }
    }
};
```

```

        return true;
    }
};

int main() {
    MinHeap heap{{9, 3, 7, 1, 1}};
    CHECK(heap.valid());
    for (int value : {1, 1, 3, 7, 9}) CHECK(heap.pop() == value);
    CHECK(!heap.pop());
    std::priority_queue<int, std::vector<int>, std::greater<int>> reference;
    std::mt19937 random{42};
    for (int i = 0; i < 2000; ++i) {
        if (reference.empty() || random() % 2 == 0) {
            const int value = static_cast<int>(random() % 100);
            heap.push(value); reference.push(value);
        } else {
            CHECK(heap.pop() == reference.top()); reference.pop();
        }
        CHECK(heap.valid());
    }
    course::report();
}

```

Build, run, and inspect

Run this block from the top-level extracted package. The fixture runner supplies input where needed and checks exit status and output in a temporary working directory.

```

cmake -S source/book3/ch12 -B build/b3-ch12 \
      -DCMAKE_BUILD_TYPE=Debug
cmake --build build/b3-ch12 --parallel 2
ctest --test-dir build/b3-ch12 --output-on-failure

```

For a multi-configuration generator on Windows, append `--config Debug` to the build command and `-C Debug` to CTest. The executable target is `b3_ch12`.

Evidence and interpretation

The fixed input 9,3,7,1,1 must drain as 1,1,3,7,9 and then report absence. The randomized operation sequence inserts when the reference is empty or a deterministic engine decision requests insertion; otherwise it compares the custom result with the reference minimum.

`valid()` checks every parent-child relation after each operation. This is $O(n)$ diagnostic work and is deliberately outside a performance measurement. Differential checks catch observable ordering mistakes, while the invariant check can identify corruption before a later removal exposes it.

The reference queue shares the same abstract behavior, not the same implementation. This makes it useful evidence against mistakes in the hand-written sift code. Tests still do not cover every allocation failure or a generic throwing comparator because those features are outside this integer example.

Failure clinic — The smaller sibling stayed below a larger parent

After removing the minimum, the next result is wrong even though the root was swapped with one child. Inspect child selection in sift-down. Choosing the first available child instead of the smaller child can leave the other child violating order immediately.

A heap works for odd sizes but reads beyond the end for an even size. The final internal node may have only a left child. Check the right-child existence condition separately from the left-child guarantee. A three-element fixture cannot expose every incomplete-final-level case.

An unsigned construction loop hangs after processing the root. Replace a `>= 0` countdown with a positive counter and a derived `i - 1` index. The wraparound is a type-domain defect, not evidence that heap construction is inherently difficult to terminate.

A scheduler reorders equal priorities unexpectedly. The heap may be correct under its weak priority contract. Add a sequence tie-breaker only when arrival order is actually required, then test that richer comparison rule. Do not “fix” a valid heap by assuming stability it never promised.

Practice and implementation lab

Add an optional `minimum()` observation and a `size()` query without exposing mutable backing storage. Test repeated equal priorities, a one-element heap, and draining the remaining randomized contents. Extend the value to a task record with priority and insertion sequence, then define and test stable tie handling.

Review answers and solution notes

A minimum observation returns absence on an empty vector and otherwise returns the first value without mutation. The size query delegates to the vector. Draining both queues to empty checks elements that were still pending at the end of the mixed workload. For stable ties, encode the arrival sequence in the comparator and verify the direction explicitly; a minimum heap should select the earlier sequence when priorities are equal.

Engineering takeaway. Maintain only the order the interface needs, and test heap structure separately from scheduling policy.

Guided lab — Validate a heap as a multiset and an order rule

Enumerate small integer arrays with values from a duplicate-heavy domain. Construct a heap from each, check its invariant, drain it, and compare the resulting sequence with a sorted copy of the original. Include empty arrays and sizes one through several incomplete final levels.

Compare bottom-up construction with repeated insertion using counters for comparisons and swaps. Keep timing out of this first experiment. Explain why the two valid heaps may have different internal layouts and setup counts while producing identical drained sequences.

Implement a stable scheduling record containing priority, arrival sequence, and job identifier. Define the preferred priority direction and arrival tie rule. Verify equal-priority ordering with at least three jobs, and reject or explicitly handle exhaustion of the sequence-number domain rather than silently wrapping.

Add a bounded top-k exercise. State what happens for $k == 0$, k larger than the input length, and duplicates at the selection boundary. Compare with sorting a complete reference copy. Sort the selected output only if the report contract requires it, and include that reporting cost separately.

For a failure-analysis exercise, imagine a comparator that throws during sift. Describe which values may already have swapped and whether heap order is still guaranteed. The answer should not rely on memory ownership alone. A production generic design needs a specified guarantee or a restricted nonthrowing comparator contract.

Acceptance evidence. Retain complete draining comparisons, invariant checks after mutations, one-child sift cases, duplicate priorities, and explicit comparator direction. Keep timing thresholds out of correctness tests.

Review reasoning. Why is heap storage not sorted? Only ancestor-child order is maintained. Why choose the smaller child in a minimum heap? The repaired parent must be no greater than both children. Why is bottom-up construction linear? Most nodes have little possible downward travel. Why can one push still relocate many values? The vector's storage policy is separate from sift complexity. Why does stable priority need extra state? Equal priority alone does not encode arrival order.

Primary reference notes

Standard adaptor behavior: [priority queues and queues](#). Backing-storage costs: [vector capacity](#). Comparisons and ordering: [sorting operations](#). Further reading: [MIT algorithm lectures](#).

Chapter 13 — Representing Networks with Graphs

A dependency between two services is not the same relationship as a bidirectional cable between two machines. If the data model erases that distinction, later graph algorithms answer the wrong question perfectly. HarborWorks therefore begins graph work by specifying what a vertex, edge, direction, and weight mean.

This chapter implements an adjacency-list graph and derives an optional-weight matrix from it. It deliberately tests a zero-weight edge, an absent edge, parallel edges, and invalid vertex identifiers. Those representation choices determine what information survives a conversion.

A dependency network is not necessarily a hierarchy. HarborWorks can have several services depending on one shared resource, two locations connected by multiple links, or a directed relationship that does not imply a return path. A graph represents those possibilities without forcing them into a tree's single-parent model.

The representation must preserve the distinctions the application cares about. A zero-cost edge is not the same as no edge. Two parallel links are not necessarily one link. An undirected self-loop may be stored once even though a mathematical degree convention counts it twice. Decide these semantics before writing traversal or optimization code; otherwise a later algorithm can be correct for a graph different from the one you intended to model.

13.1 Understanding Graph Models

A graph consists of vertices and relationships between vertices. Identifiers in this example are dense indices from zero through V-1. A domain label such as a server name would be mapped to an index separately, with validation of duplicate names and missing endpoints.

13.1.1 Distinguishing Directed and Undirected Graphs

A directed edge from u to v permits movement or dependency in one direction. The reverse edge is a separate fact. An undirected edge represents a symmetric relationship, so an adjacency-list representation usually stores it in both endpoint lists. The example avoids storing the same self-loop twice when both endpoints are identical.

Direction changes the question you can ask. “Can service A reach service B?” is not equivalent to “Are A and B mutually reachable?” In a directed graph, the latter involves stronger conditions. A topological dependency ordering also makes sense only with an appropriate directed acyclic relation, not with every graph-shaped dataset.

Parallel edges can represent distinct links between the same endpoints. This graph permits them. A simple-graph design could reject duplicates or update an existing edge, but that would be a different

contract. Decide the policy before choosing a matrix representation that has only one cell per endpoint pair.

A directed edge from *u* to *v* permits movement or expresses a relationship in that direction only. An undirected edge represents a symmetric connection. The same pair of identifiers therefore has different meaning depending on the graph type. A firewall reachability model, for example, should not infer the reverse path merely because an outbound relation exists.

The worked class stores a boolean direction policy for the whole graph. Adding an undirected edge normally stores one adjacency entry at each endpoint. Adding a directed edge stores only the outgoing entry. The policy is fixed at construction so existing entries are not reinterpreted halfway through the graph's lifetime.

Vertices are present independently of edges. An isolated vertex with no neighbors is still a valid member of the graph. A graph with five vertices and no edges differs from an empty graph with zero vertices. Traversal, connected-component counting, and source validation must preserve that distinction.

Self-loops require a convention. The example stores an undirected self-loop once in its adjacency list rather than adding two identical entries to the same vector. That storage choice is convenient, but an algorithm computing undirected degree under the usual convention must account for a loop contributing two incidences. Stored adjacency length and mathematical degree are not automatically identical under every representation.

Parallel edges are retained. Two links from the same source to the same destination can carry different weights or represent independent physical connections. A simple reachability query may not care about multiplicity, while capacity planning or link failure analysis might. Collapsing duplicates is a domain transformation, not an innocuous cleanup.

A general graph also needs stable vertex identity. The chapter uses dense indices from zero through *V* - 1, which make vector indexing straightforward. A user-facing application can map names to those indices at an ingestion boundary. Renumbering vertices after deletion would then require updating every affected edge or using another identity strategy. The example avoids vertex deletion so that additional problem remains explicit.

13.1.2 Distinguishing Weighted and Unweighted Graphs

An unweighted graph treats each traversed edge as one step. A weighted graph attaches a value such as distance, cost, delay, or capacity. Those meanings are not interchangeable. A path's total delay might be a sum; its available bandwidth may depend on the minimum link capacity instead.

The example stores integer weights and permits zero and negative values. That is a representation capability, not permission to use every shortest-path algorithm unchanged. Breadth-first search computes minimum hop counts in an unweighted graph. An algorithm requiring nonnegative weights must validate that stronger precondition at its boundary.

Absence must not be encoded by an ordinary valid weight. A zero-cost connection is still a connection. The matrix therefore uses `std::optional<int>: nullopt` means no edge, and a present zero means an edge with weight zero. This avoids a sentinel convention that silently discards legitimate data.

An unweighted graph treats every edge as one step for a hop-count question. A weighted graph attaches a value such as distance, latency, cost, or capacity. The meaning of that number determines which algorithms are valid. A minimum-cost path and a maximum-capacity path are different problems even when both use an integer edge field.

The representation permits zero and negative integer weights. That flexibility does not make every later optimization algorithm applicable. Breadth-first search ignores weights and finds minimum edge count. Dijkstra's shortest-path method requires nonnegative weights in the routing capstone. A minimum spanning tree can legitimately contain negative-weight edges. Algorithm preconditions belong alongside the algorithm, not hidden inside a generic graph name.

Absence should be represented independently from weight. Using zero to mean "no edge" makes a legitimate zero-weight connection disappear. The matrix view therefore stores optional integers. A present optional containing zero is an edge; an empty optional is no edge. This is the same present-zero distinction already used for map values and optional indices.

Units matter. Adding milliseconds to kilometers yields a number but not a meaningful route cost. A production graph model can use strong domain types or validated metadata to keep units consistent. The teaching example uses plain integers so the representation mechanism remains visible, while the application notes should still state what a weight means.

For path computations, accumulated values need a wider or checked arithmetic domain. Individually valid edge weights can overflow when added along a long path. Bounding vertex count and edge magnitude can justify a safe sum in a selected type; alternatively, perform checked addition. Never infer that a valid `int` field makes every aggregate `int` result valid.

Finally, an edge weight can change over time. A shortest-path result refers to a graph state. Retaining a path while updating weights can make its reported cost stale without invalidating any pointer. Snapshot identity and algorithm-result validity are semantic concerns that remain even when memory ownership is correct.

Modeling Directed Relationships as Vertices and Edges

Purpose. A graph model begins by deciding what entities are vertices and what relationships are edges, including whether direction matters.

```
#include <iostream>
#include <vector>
int main(){std::vector<std::vector<int>>g(4);g[0]={1,2};g[1]={3};for(std::size_t
u=0;u<g.size();++u)for(int v:g[u])std::cout<<u<<"->"<<v<<'\\n';}
```

How it works.

- Vector indices represent vertices.
- Each nested vector stores outgoing neighbors, making edges directional.
- Vertices with no outgoing edges still exist as empty adjacency lists.

Expected behavior/output. Edges 0->1, 0->2, and 1->3 print.

Experiment. Add the reverse edge for each relationship and explain how that changes the model to an undirected graph.

13.2 Choosing Graph Representations

Representation determines the cost of enumerating neighbors, testing one endpoint pair, and storing sparse or dense relationships. Choose against the operations, not merely a familiar diagram. Both representations in this chapter use the same dense vertex-index convention.

13.2.1 Building Adjacency Matrices

An adjacency matrix has one cell for each ordered pair of vertices, requiring $O(V^2)$ cells even when there are few edges. Endpoint-pair lookup is direct indexing once bounds are checked. Enumerating a vertex's neighbors scans its row and therefore takes $O(V)$, including absent entries.

An undirected matrix is symmetric when its edge semantics are symmetric. The diagonal can contain self-loops; do not force it to zero or absent without a domain rule. A boolean matrix is enough only when weights and multiplicity are irrelevant.

The conversion here collapses parallel edges by selecting the smallest weight. That policy is useful for some cost problems but loses multiplicity and all larger parallel weights. It is not a reversible serialization of the adjacency lists. The conversion allocates V^2 optional cells, initializes them as absent, then examines each stored edge: $O(V^2+E)$ time and $O(V^2)$ additional space, counting stored adjacency entries.

An adjacency matrix assigns a cell to every ordered pair of vertices. For V vertices, it contains $V \times V$ cells. A direct edge query can index one cell, but storage grows quadratically even when only a few edges exist. This is why representation choice must consider both query shape and graph density.

The worked conversion creates a matrix of optional weights. It then visits every stored adjacency entry and updates the corresponding cell. Parallel edges are collapsed by retaining the minimum weight. That rule makes sense for some minimum-cost edge queries, but it loses multiplicity and information about more expensive parallel links. It is not a lossless serialization of the adjacency list.

A matrix diagonal is not automatically a zero-length path. The conversion records explicit self-loop edges only. Shortest-path algorithms sometimes initialize a distance matrix diagonal to zero because a vertex can reach itself without traversing an edge. That is an algorithmic distance convention, different from an adjacency representation of actual edges. Keep the two matrices conceptually separate.

Validate scale before conversion in a real application. A sparse graph with one hundred thousand vertices might have a manageable adjacency list and an infeasible dense matrix. Checking only that the original graph fits in memory says nothing about the derived $V \times V$ representation. The complete capstone avoids a dense production-sized conversion and uses only tiny dense reference graphs in tests.

Nested vectors add another layout consideration. A vector of row vectors gives each row its own allocation; it is not necessarily one contiguous $V \times V$ block. A flattened vector can improve layout predictability but requires checked product and offset arithmetic. Either representation must preserve optional absence and the chosen parallel-edge policy.

When testing conversion, include a zero-weight edge, an absent edge, parallel edges with different weights, and directionality. A matrix containing only positive distinct weights can conceal several of the semantic losses this design is meant to prevent.

13.2.2 Building Adjacency Lists

An adjacency list stores a collection for each vertex and an entry for each outgoing edge. Space is $O(V+E)$; an undirected non-self edge contributes two stored entries. Enumerating neighbors costs $O(\text{degree}(v))$, which is advantageous for sparse graphs. Testing whether one edge exists generally scans the relevant list unless another index is added.

The `add` method validates both endpoints before indexing. For an undirected edge, it reserves capacity on both sides before inserting either logical edge. If the second reservation fails, the graph may have a changed capacity on the first side but has not acquired a one-way logical edge. With this trivial `Edge` type, the subsequent reserved insertions do not need another allocation.

The method's per-add `reserve(size()+1)` is deliberately explicit for the transaction demonstration; it can defeat geometric growth and make repeated high-degree insertion costly. A production bulk builder should reserve known degrees once or use a transactional strategy that preserves amortized growth. Correct logical rollback and efficient allocation are separate requirements.

`neighbors()` returns a const reference to the list. Its lifetime is bounded by the graph, and subsequent mutations can invalidate iterators into that list. A read-only access path does not make concurrent mutation safe. Establish a construction phase followed by a read-only algorithm phase unless the design supplies synchronization and stronger handle guarantees.

An adjacency list stores outgoing entries for each vertex. Its storage grows with vertices plus stored edge entries rather than every possible pair. For an undirected nonloop edge represented in both directions, two

entries are stored. Complexity statements should specify whether **E** counts logical edges or stored adjacency entries; the constant factor is often harmless, but the definition should not remain ambiguous.

Traversal naturally scans the neighbors of a selected vertex. Asking whether one specific edge exists may require scanning that neighbor list unless another index is added. A sorted list or per-vertex hash structure changes lookup and mutation costs. Choose the extra structure only when the query workload justifies maintaining it.

The revised **add** prepares capacity on both affected adjacency vectors before publishing either logical edge. With integer payloads, the subsequent appends do not throw once sufficient capacity exists. If preparing the second side fails, the first side may have a different capacity but its logical edges remain unchanged. This is a logical-content guarantee, not a promise to preserve every iterator or allocation address.

Capacity preparation uses geometric growth rather than repeatedly reserving exactly one extra slot. Reserving **size()+1** before every insertion can defeat vector's ordinary amortized growth and repeatedly relocate a high-degree vertex's adjacency list. The revision retains the two-sided preparation sequence while avoiding that unnecessary per-edge growth pattern.

Borrowed neighbor references are valid only under the documented mutation boundary. A reference to a row vector can remain valid while that row's elements reallocate, but an iterator or reference to an individual edge may not. A caller should not retain an edge borrow across mutation without a specific guarantee.

Representation invariants include endpoint bounds. Every destination index must identify an existing vertex, and public insertion validates both endpoints before changing logical contents. A later traversal should not have to discover malformed destinations through an out-of-bounds access. Validation belongs where external identifiers become internal indices.

Adjacency List Versus Adjacency Matrix for the Same Graph

Purpose. Representation choice changes memory cost and operation cost even though the abstract graph is the same.

```
#include <iostream>
#include <vector>
int main(){int
n=4;std::vector<std::vector<int>>>list(n);std::vector<std::vector<bool>>>matrix(n,std::vector<bool>
add=[&](int u,int v){list[u].push_back(v);matrix[u]
[v]=true;};add(0,1);add(0,3);std::cout<<list[0].size()<<' '<<matrix[0]
[3]<<'\n';}
```

How it works.

- The adjacency list stores only present neighbors.
- The matrix reserves an entry for every ordered vertex pair.
- Edge existence is $O(1)$ to query in the matrix, while sparse lists use less space.

Expected behavior/output. The output is `2 1`.

Experiment. Scale `n` to 1000 with only 100 edges and estimate the matrix versus list storage shape.

Worked example — Preserve edge meaning across two representations

Requirement and design

Create directed and undirected graphs with bounds-checked endpoints. Preserve zero-weight edges and parallel edges in adjacency lists. Convert to a matrix using an explicit minimum-parallel-weight rule, and verify that an absent edge remains distinct from a present zero.

Compare three different meanings of “missing”

The fixture contains three vertices. An absent edge between zero and two is different from the present zero-weight edge between zero and one, and both differ from an invalid vertex index three. The matrix optional, graph membership, and bounds exception preserve those distinctions.

The second edge between one and two has lower weight than the first. The list retains both entries, while the derived matrix chooses the minimum. Trace this conversion explicitly so the test’s expected value three is understood as a policy result, not accidental last-write behavior.

The directed graph test checks that adding zero-to-one does not create outgoing neighbors for vertex one. This is a minimal directionality fixture. A larger strongly connected graph would make it harder to see whether a reverse edge was inserted incorrectly.

The capacity helper is part of the mutation's resource plan. It validates representable growth and allocates before logical publication. It does not reserve enough storage for every future edge, impose a total graph memory budget, or claim that allocation cannot fail. Those concerns belong to an application-level ingestion policy such as the capstone's explicit bounds.

Complete implementation

The complete, buildable chapter project is `source/book3/ch13/`. The following files are reproduced from that project; the publishing script inserts the source directly so the listing and companion remain synchronized.

main.cpp

```
#include "course_test.hpp"
#include <optional>
#include <stdexcept>
#include <vector>

struct Edge { std::size_t to; int weight; };
class Graph {
    std::vector<std::vector<Edge>> adjacent_;
    bool directed_;
    static void prepare_append(std::vector<Edge>& edges) {
        if (edges.size() == edges.max_size()) throw std::length_error("edge
capacity");
        if (edges.size() < edges.capacity()) return;
        const auto maximum = edges.max_size();
        const auto capacity = edges.capacity();
        const auto next = capacity == 0 ? std::size_t{1}
            : (capacity <= maximum / 2 ? capacity * 2 : maximum);
        edges.reserve(next);
    }
public:
    Graph(std::size_t vertices, bool directed) : adjacent_(vertices),
        directed_(directed) {}
    void add(std::size_t from, std::size_t to, int weight) {
        if (from >= adjacent_.size() || to >= adjacent_.size()) throw
std::out_of_range("vertex");
        // Allocate both sides before adding either logical edge.
        prepare_append(adjacent_[from]);
        if (!directed_ && from != to) prepare_append(adjacent_[to]);
        adjacent_[from].push_back({to, weight});
        if (!directed_ && from != to) adjacent_[to].push_back({from, weight});
    }
    const std::vector<Edge>& neighbors(std::size_t vertex) const { return
adjacent_.at(vertex); }
    std::vector<std::vector<std::optional<int>>> matrix() const {
        const auto size = adjacent_.size();
        std::vector<std::vector<std::optional<int>>> result(size,
            std::vector<std::optional<int>>(size));
        for (std::size_t from = 0; from < size; ++from) {
            for (const Edge edge : adjacent_[from]) {
                auto& cell = result[from][edge.to];
                if (!cell || edge.weight < *cell) cell = edge.weight;
            }
        }
        return result;
    }
};

int main() {
```

```

    Graph undirected{3, false};
    undirected.add(0, 1, 0); undirected.add(1, 2, 5);
    auto matrix = undirected.matrix();
    CHECK(matrix[0][1] == 0 && matrix[1][0] == 0);
    CHECK(!matrix[0][2]);
    undirected.add(1, 2, 3);
    CHECK(undirected.matrix()[1][2] == 3); // Parallel edges collapsed by minimum.
    Graph directed{2, true}; directed.add(0, 1, 7);
    CHECK(directed.neighbors(1).empty());
    CHECK(course::throws<std::out_of_range>([&] { directed.add(2, 0, 1); }));
    course::report();
}

```

Build, run, and inspect

Run this block from the top-level extracted package. The fixture runner supplies input where needed and checks exit status and output in a temporary working directory.

```

cmake -S source/book3/ch13 -B build/b3-ch13 \
  -DCMAKE_BUILD_TYPE=Debug
cmake --build build/b3-ch13 --parallel 2
ctest --test-dir build/b3-ch13 --output-on-failure

```

For a multi-configuration generator on Windows, append `--config Debug` to the build command and `-C Debug` to CTest. The executable target is `b3_ch13`.

Evidence and interpretation

The undirected fixture adds 0–1 with weight zero and 1–2 with weight five. Both symmetric 0–1 matrix cells must contain zero, while 0–2 must be absent. Adding another 1–2 edge with weight three changes the derived matrix minimum to three without removing the earlier list entry.

The directed fixture adds $0 \rightarrow 1$ and verifies that vertex 1 has no outgoing neighbors. An invalid source index throws before any logical insertion. These checks test representation semantics; they do not establish connectivity, shortest paths, or a minimum spanning tree.

A graph with millions of sparse vertices may be suitable for adjacency lists but impossible to convert to a full matrix within the machine’s memory budget. Validate the intended conversion size before allocating in an application that accepts untrusted or remotely supplied graph dimensions.

Failure clinic — A representation erased a valid edge

A route disappears only when its cost is zero. Inspect whether zero was used as an absence sentinel in a matrix or lookup result. Replace the sentinel with a distinct presence representation. Adding a small positive epsilon changes the graph’s meaning and is not a legitimate repair for an integer-cost model.

Two physical links become one after export. Determine whether the export format intentionally collapses parallel edges. The worked matrix view does so by minimum weight; it cannot reconstruct all original links. Use an edge list or another multiplicity-preserving format when lossless round trips are required.

An undirected insertion succeeds on one side and fails on the other. Prepare both allocations before publishing either logical entry, or provide a rollback that is valid for the element type. Merely catching the exception after two independent appends does not restore symmetry.

A high-degree vertex becomes slow to build after adding a “safety reserve” before each edge. Inspect whether exact one-slot reserve calls force repeated relocation. Correctness preparation and amortized growth can coexist; the helper should grow geometrically while still acquiring both sides before mutation.

Practice and implementation lab

Add tests for an undirected self-loop and multiple parallel edges inserted in different orders. Introduce a named conversion policy that can select minimum weight or reject parallel edges. Write down whether the converted matrix can reconstruct the original graph exactly.

Review answers and solution notes

A self-loop appears once in the list under this contract and occupies a diagonal matrix cell. Minimum selection should not depend on insertion order. Rejection should identify the first endpoint pair with more than one stored logical candidate. A single optional weight cannot preserve every parallel edge, so exact reconstruction requires additional data or a graph restriction that forbids multiplicity.

Engineering takeaway. A graph representation must preserve the distinctions that the later algorithm relies on.

Guided lab — Specify a graph before choosing storage

Write a graph policy sheet: directed or undirected, weighted or unweighted, self-loops allowed or rejected, parallel edges retained or collapsed, vertex identity scheme, and weight domain. Use one small example where each policy choice changes an observable result.

Implement a matrix-to-list conversion for the matrix’s reduced semantics, then explain why converting the original parallel-edge list to a matrix and back is not generally lossless. Verify zero-weight presence and directed asymmetry explicitly.

Add a logical-content regression around an invalid endpoint. Save every neighbor list, attempt insertion with a nonexistent source or destination, and compare afterward. For allocation failure, use an injected test allocator or a controlled abstraction in a separate exercise rather than attempting to exhaust the host’s memory.

Compare representation budgets on paper. For a graph with V vertices and E stored entries, estimate matrix cells, list entries, row-vector overhead, and temporary conversion storage. State the assumed cell and edge sizes instead of presenting implementation-dependent byte counts as universal constants.

Create an undirected self-loop fixture and document whether adjacency length is intended to equal mathematical degree. Add a helper only when its counting convention is explicit. A useful test contains both a self-loop and an ordinary edge so the distinction is visible.

Acceptance evidence. Include absent versus zero weight, isolated vertex versus invalid vertex, directionality, parallel-edge policy, self-loop policy, and logical preservation after rejected insertion.

Review reasoning. Why can a graph be sparse yet have an enormous matrix? The matrix reserves a cell for every possible pair. Why does a zero diagonal have different meanings in adjacency and distance matrices? One records actual edges; the other may include empty paths. Why are nonnegative weights a separate algorithm precondition? Different algorithms interpret weights differently. Why does two-sided capacity preparation not preserve all borrows? Reallocation can occur before logical publication. Why is exact per-edge reserve undesirable? It can replace geometric growth with repeated linear relocation.

Primary reference notes

Storage and borrowing behavior: [vector](#), [optional values](#), and [vector modifiers](#). Further graph-model reading: [MIT algorithm lecture directory](#).

Chapter 14 — Traversing and Analyzing Graphs

An inventory graph becomes operationally useful when it can answer reachability questions. HarborWorks wants minimum hop counts from one machine, separate components of an undirected topology, and cycles in a directed dependency graph. These are related traversals, but they do not use identical state or prove identical properties.

This chapter makes discovery state explicit. Breadth-first search uses a queue and marks at discovery. Directed cycle detection distinguishes active from completed work. Component counting starts a new traversal only from a vertex that previous traversals have not reached.

HarborWorks asks three questions about a topology: how many hops separate a source from each reachable vertex, how many disconnected groups exist in an undirected network, and whether a directed dependency cycle exists. All three involve traversal, but they do not share one interchangeable definition of visited state.

This chapter keeps those questions in separate functions. Breadth-first distances use discovery layers. Directed cycle detection distinguishes active recursion from completed work. Undirected components restart traversal from each unseen vertex under a symmetry precondition. The separation prevents a common error: taking a traversal that answers one question and assuming its boolean visited array answers every other graph question too.

14.1 Performing Breadth-First Search

Breadth-first search processes vertices in nondecreasing distance from a source when every edge counts as one step. Its queue is not merely an implementation detail: FIFO order is the mechanism that preserves distance layers.

14.1.1 Traversing by Distance Layers

Initialize all distances as unseen, set the source distance to zero, and enqueue the source. When removing a vertex, inspect each outgoing neighbor. For an unseen neighbor, assign the current distance plus one and enqueue it immediately. Marking when enqueued prevents several parents from scheduling the same vertex repeatedly.

The queue contains vertices from at most two consecutive distance layers at the transition between layers. Every vertex in layer d is removed before any vertex in layer $d+1$ that it discovers can get ahead of it. This is why a newly discovered neighbor receives the smallest possible hop count rather than merely the first arbitrary path length.

The implementation first validates all endpoint indices. It limits the distance problem to at most one million vertices so the longest simple discovered path fits comfortably in the chosen `int` distance type. A

larger application could use a different representation and bound rather than silently overflowing a distance counter.

Breadth-first search processes vertices in nondecreasing number of edges from the source. The source receives distance zero and is queued first. Each newly discovered neighbor receives one more than the distance of the vertex that discovered it. FIFO scheduling preserves the layer order.

Mark a vertex when it is enqueued, not only when it is later removed. Several current-layer vertices may point to the same next-layer vertex. Marking at discovery prevents multiple queue entries from being created for that shared neighbor. This controls both work and the meaning of first discovery.

A queue can contain vertices from adjacent layers at the same time. The invariant is not that all pending vertices have equal distance. It is that no deeper discovered vertex is processed ahead of pending shallower work under FIFO order. This ordering is what supports the shortest-hop proof.

The distance vector uses minus one for unreachable vertices in a graph with a supported bounded size. Zero remains reserved for the source's zero-edge path. An isolated but valid vertex can remain minus one when searching from another source; that is different from a source index outside the graph, which is rejected.

The source is marked before traversal so a self-loop or a cycle back to it does not rediscover it. The same discovery rule handles parallel adjacency entries without repeated scheduling. A graph can contain redundant edges while the algorithm still visits each vertex once and scans every stored entry a bounded number of times.

Validation of endpoint indices occurs before traversal. That separates malformed representation from genuine unreachability. If an adjacency list contains vertex nine in a one-vertex graph, the correct outcome is rejection, not a distance of minus one for an imagined disconnected vertex.

The result describes one fixed graph state. Mutating adjacency during the traversal can invalidate both the memory access and the layer proof. The supplied functions require a stable graph for the duration of the call.

14.1.2 Finding Unweighted Shortest Paths

Suppose a vertex is first discovered with distance d . Any path with fewer than d edges would have reached it from an earlier layer, which the FIFO traversal already processed. That contradiction establishes shortest hop count. The proof depends on unit edge costs and is not a proof for arbitrary weighted distances.

Unreachable vertices retain -1 . The source has distance zero even if it has a self-loop. A predecessor array can reconstruct an actual shortest path: record the discovering parent for each newly reached vertex, then follow parents backward from the destination and reverse the sequence. Equal-length alternatives may produce different valid parent trees depending on neighbor order.

With adjacency lists, each vertex is enqueued at most once and each outgoing entry is inspected once, giving $O(V+E)$ time including validation and initialization. The queue and distance storage use $O(V)$ space. A matrix-based neighbor scan would change the traversal cost, which is why representation analysis came first.

The first-discovery distance is minimal in an unweighted graph because every shorter path would end at a predecessor in an earlier layer. That predecessor would have been processed before any longer-layer discovery. Therefore a supposedly shorter undiscovered route contradicts the FIFO layer order.

This proof depends on every edge contributing one hop. With unequal weights, the route discovered first can have fewer edges but greater total cost. A direct edge of cost one hundred and a two-edge route costing one each make the distinction obvious. Use BFS for minimum hop count; use an algorithm with appropriate weight assumptions for minimum accumulated cost.

To reconstruct an actual shortest-hop path, store the predecessor when a vertex is first discovered. Starting at the destination, follow predecessors back to the source, then reverse the collected sequence. Unreachable destinations have no path. The source-to-source path contains the source under a vertex-sequence convention and has zero edges.

Predecessor choice among equal-length paths depends on neighbor order. BFS guarantees a shortest distance, not necessarily the lexicographically smallest path under arbitrary adjacency order. A deterministic path policy can sort neighbors or specify another tie rule, but that extra promise requires additional reasoning and tests.

Path verification should check more than length. Every consecutive pair must be a valid edge in the selected direction, the endpoints must match the requested source and target, and the edge count must equal the reported distance. A fabricated sequence of the right length is not a path witness.

Space usage includes one distance value per vertex and a queue that can hold many discovered vertices. If predecessor reconstruction is added, it adds another linear array. These are auxiliary costs in addition to adjacency storage and any returned path. Reporting only queue size would understate the full algorithm's work memory.

For multiple sources, initializing several source vertices with distance zero can answer distance to the nearest source under a new contract. That is a useful extension, but it changes the meaning of zero and the predecessor roots and should not be silently substituted for a single-source result.

BFS Computing Unweighted Shortest Edge Distances

Purpose. BFS is defined by FIFO frontier expansion and is therefore suited to shortest paths in unweighted graphs.

```
#include <iostream>
#include <queue>
#include <vector>
int main(){std::vector<std::vector<int>>g{{1,2},{3},{3},
{}};std::vector<int>d(g.size(),-1);std::queue<int>q;d[0]=0;q.push(0);while(!
q.empty()){int u=q.front();q.pop();for(int v:g[u])if(d[v]==-1){d[v]=d[u]
+1;q.push(v);}}for(int x:d)std::cout<<x<<' ';
```

How it works.

- The queue processes vertices layer by layer.
- Distance is assigned the first time an unvisited vertex is discovered.
- In an unweighted graph, first discovery gives the minimum edge count from the source.

Expected behavior/output. The distances print `0 1 1 2`.

Experiment. Change the graph so vertex 3 is directly connected from 0 and verify its distance becomes 1.

14.2 Performing Depth-First Search

Depth-first search follows one branch before returning to alternatives. Its state can be stored in language call frames or an explicit stack. A visited flag is enough for some reachability problems, but cycle detection in a directed graph needs to distinguish a current ancestor from a vertex whose exploration has finished.

14.2.1 Traversing Recursive Graph Structure

The directed traversal uses three colors: unseen, active, and finished. On entry it marks a vertex active. An edge to another active vertex closes a path back into the current recursion chain and therefore identifies a cycle. An edge to a finished vertex does not by itself prove a cycle; two branches may legitimately converge on completed work.

After every outgoing neighbor has been considered without finding a cycle, the vertex becomes finished. The outer loop starts a traversal at each still-unseen vertex so disconnected regions are not ignored. An empty graph has no directed cycle, while a self-loop reaches the current active vertex immediately.

Recursive depth can grow with the longest explored path. A large adversarial dependency chain may exhaust the native stack even though the algorithm is $O(V+E)$. An explicit stack must retain both the current vertex and the next outgoing-edge position to reproduce active/finished transitions correctly; merely pushing all neighbors and marking them finished on push changes the meaning.

Depth-first search records a different kind of pending work. A recursive visit remains active until all of its outgoing neighbors have been considered. The active calls form a path through the exploration. This path is precisely the information needed to recognize a directed edge returning to an ancestor.

The color scheme uses unseen, active, and finished states. Unseen means no visit has started. Active means the vertex is on the current unfinished recursion path. Finished means all outgoing exploration for that call has completed. An edge to an active vertex creates a directed cycle; an edge to a finished vertex can simply be convergence on already explored work.

Consider a directed acyclic graph with edges zero-to-one, zero-to-two, and one-to-two. DFS can finish two while exploring one, then later encounter zero-to-two. A simple “already visited means cycle” rule would falsely reject this graph. The active/finished distinction preserves the difference between a back edge and harmless convergence.

Recursive depth can be linear in the number of vertices for a long chain. The original function is a compact teaching implementation, not a hardened importer for arbitrarily deep untrusted graphs. An explicit frame stack can preserve the same state machine without relying on one C++ call frame per vertex. The appendix’s traversal lab demonstrates that adaptation.

When translating to frames, remember which outgoing edge comes next. A stack of vertex identifiers alone can lose the distinction between entering a vertex and finishing it. Marking a vertex finished immediately after pushing its neighbors would destroy the active-path information that the cycle proof uses.

Cycle detection across the whole graph must start a new DFS from every unseen vertex. A cycle in a disconnected directed region should still be found. Restricting traversal to one source answers a narrower reachability-scoped question and must be named accordingly.

14.2.2 Detecting Cycles and Connected Components

For undirected components, start from an unseen vertex, increment the component count, and traverse every reachable neighbor. Mark on push so a vertex cannot enter the pending stack repeatedly. When that traversal ends, all vertices in its connected component are marked. Repeat until every vertex belongs to a completed component.

The component function checks endpoint bounds but requires the caller to supply symmetric adjacency. On arbitrary directed input, this routine does not compute strongly connected components and its grouping can depend on traversal direction. State the domain promise instead of presenting one generic `components` function as correct for every graph.

The directed three-color cycle rule also needs adaptation for undirected cycle detection: the immediate reverse edge to a parent is expected in a symmetric representation. A dedicated undirected algorithm tracks the parent edge and handles parallel-edge conventions deliberately. Mixing these cases is a common source of false cycle reports.

Undirected connected components group vertices linked by some undirected path. Start a traversal from an unseen vertex, mark every reachable vertex in that component, then continue scanning for another unseen start. Each new start increments the component count. An isolated vertex forms its own component.

The worked function assumes symmetric adjacency in addition to valid endpoint indices. Its validator checks bounds but not symmetry. Applying it to arbitrary directed adjacency does not compute strongly connected components and may produce a traversal-order-dependent grouping. This is an important precondition to preserve in both documentation and callers.

Directed cycles and undirected cycles also require different rules. In an undirected representation, traversing an edge usually exposes the reverse edge back to the parent. That reverse observation alone is not evidence of a cycle distinct from the edge just used. A cycle detector for undirected multigraphs may need edge identity so a second parallel edge can be distinguished from the same edge's reverse incidence.

Strongly connected components in a directed graph require mutual reachability. The simple component routine here does not implement that problem. Naming a function `components` without documenting direction can encourage an invalid substitution. Use a precise name and state whether the desired relation is weak connectivity, strong connectivity, or ordinary undirected connectivity.

The iterative component traversal marks vertices before pushing them, preventing duplicate scheduling. It uses a vector as a LIFO stack because only reachability matters, not shortest-hop order. A queue would reach the same component membership under the same graph model, with different traversal order.

Tests should include zero vertices, isolated vertices, a single connected group, several groups, self-loops, and repeated adjacency entries where allowed. The expected component count follows from vertex identity and reachability, not from how many edge records happen to be stored.

A useful operational report should distinguish malformed input from a disconnected but valid topology. One requires repairing the representation; the other may be a legitimate state of the modeled network. Returning the same error label for both removes information an administrator needs.

Recursive DFS Marking Reachable Vertices

Purpose. DFS explores depth-first because its call stack stores deferred alternatives implicitly.

```
#include <iostream>
#include <vector>
void dfs(int u, const std::vector<std::vector<int>>&g, std::vector<bool>&seen)
{ seen[u]=true; std::cout<<u<<' '; for(int v:g[u]) if(!seen[v]) dfs(v,g,seen); }
int main() { std::vector<std::vector<int>>g{{1,2},{3},{},{} };
std::vector<bool>seen(g.size()); dfs(0,g,seen); }
```

How it works.

- A vertex is marked before exploring its outgoing edges.
- Recursive calls continue down one path before returning to try alternatives.
- The visited set prevents cycles from causing infinite recursion.

Expected behavior/output. With the listed adjacency order, output is `0 1 3 2`.

Experiment. Add an edge 3->0 and verify the visited check prevents infinite recursion.

Worked example — Answer three distinct topology questions

Requirement and design

Provide bounds-validated breadth-first distances, directed cycle detection, and undirected component counting. Use an undirected graph with one isolated vertex to distinguish unreachable distance from missing input. Use a directed acyclic graph, a three-vertex cycle, and a self-loop to test the active-state rule.

Keep three result contracts separate

The five-vertex undirected fixture has one isolated vertex. Its BFS distances from zero are zero, one, one, two, and minus one. Its component count is two. These are different outputs describing the same graph; neither can be inferred merely from the other's length or success status.

The directed fixtures include an acyclic convergence, a three-vertex cycle, and a self-loop. Each protects a distinct state transition in the color algorithm. The acyclic convergence is especially valuable because it rejects the incorrect simplification that every edge to a visited vertex is a cycle.

The bounds validator runs before all three algorithms. The component function's symmetry requirement remains a caller obligation, explicitly documented in its comment and this chapter. A successful bounds check must not be described as complete validation of every graph-model property.

For the recursive cycle checker, trace colors at entry, before following an edge, and at function exit. The moment a vertex changes from active to finished is semantically important. Moving that assignment earlier can erase evidence of a genuine back edge.

Complete implementation

The complete, buildable chapter project is `source/book3/ch14/`. The following files are reproduced from that project; the publishing script inserts the source directly so the listing and companion remain synchronized.

main.cpp

```
#include "course_test.hpp"
#include <cstdint>
#include <functional>
#include <queue>
#include <vector>

using Graph = std::vector<std::vector<std::size_t>>;
void validate(const Graph& graph) {
    for (const auto& neighbors : graph) {
        for (auto v : neighbors) if (v >= graph.size()) throw
std::out_of_range("vertex");
    }
}

std::vector<int> distances(const Graph& graph, std::size_t source) {
    validate(graph);
```

```

    if (source >= graph.size() || graph.size() > 1000000) throw
std::out_of_range("source/size");
std::vector<int> distance(graph.size(), -1);
std::queue<std::size_t> pending;
distance[source] = 0; pending.push(source);
while (!pending.empty()) {
    const auto from = pending.front(); pending.pop();
    for (const auto to : graph[from]) {
        if (distance[to] != -1) continue;
        distance[to] = distance[from] + 1; pending.push(to);
    }
}
return distance;
}

bool directed_cycle(const Graph& graph) {
    validate(graph);
    std::vector<int> color(graph.size()); // 0 unseen, 1 active, 2 finished.
    std::function<bool(std::size_t)> visit = [&](std::size_t vertex) {
        color[vertex] = 1;
        for (auto next : graph[vertex]) {
            if (color[next] == 1) return true;
            if (color[next] == 0 && visit(next)) return true;
        }
        color[vertex] = 2; return false;
    };
    for (std::size_t v = 0; v < graph.size(); ++v) if (!color[v] && visit(v)) return
true;
    return false;
}

std::size_t undirected_components(const Graph& graph) {
    validate(graph); // Caller additionally promises symmetric adjacency.
    std::vector<bool> seen(graph.size());
    std::size_t count{};
    for (std::size_t start = 0; start < graph.size(); ++start) {
        if (seen[start]) continue;
        ++count;
        std::vector<std::size_t> stack{start}; seen[start] = true;
        while (!stack.empty()) {
            auto from = stack.back(); stack.pop_back();
            for (auto to : graph[from]) if (!seen[to]) {
                seen[to] = true; stack.push_back(to);
            }
        }
    }
    return count;
}

int main() {
    const Graph undirected{{1, 2}, {0, 3}, {0}, {1}, {}};
    CHECK(distances(undirected, 0) == std::vector<int>({0, 1, 1, 2, -1}));
    CHECK(undirected_components(undirected) == 2);
    CHECK(!directed_cycle(Graph{{1, 2}, {2}, {}}));
    CHECK(directed_cycle(Graph{{1}, {2}, {0}}));
    CHECK(directed_cycle(Graph{{0}}));
    CHECK(!directed_cycle(Graph{}});
    CHECK(course::throws<std::out_of_range>([] { (void)distances(Graph{{9}}, 0); }));
    course::report();
}

```

Build, run, and inspect

Run this block from the top-level extracted package. The fixture runner supplies input where needed and checks exit status and output in a temporary working directory.

```
cmake -S source/book3/ch14 -B build/b3-ch14 \
  -DCMAKE_BUILD_TYPE=Debug
cmake --build build/b3-ch14 --parallel 2
ctest --test-dir build/b3-ch14 --output-on-failure
```

For a multi-configuration generator on Windows, append `--config Debug` to the build command and `-C Debug` to CTest. The executable target is `b3_ch14`.

Evidence and interpretation

The five-vertex undirected fixture produces distances 0,1,1,2,-1 from vertex zero and contains two components. The isolated final vertex is present in the model but unreachable from that source. Directed fixtures separately establish that convergence in an acyclic graph is not a cycle and that both a multi-edge loop and a self-loop are cycles.

The structural validation rejects an adjacency entry pointing outside the graph. That check happens before traversal so a malformed endpoint cannot be mistaken for a disconnected vertex. It does not validate symmetry; the component routine's additional precondition remains visible in the code.

The recursive directed checker is a bounded teaching implementation. Its tests exercise correctness on small graphs, not stack resilience on a million-vertex chain. An iterative version is the appropriate extension when accepting large externally supplied topologies.

Failure clinic — Visited did not mean finished

A directed acyclic convergence is reported as cyclic. Inspect whether the implementation uses one boolean for both active and completed vertices. Replace that collapsed state with the three-state model and preserve the active recursion path until outgoing work is complete.

A BFS queue grows with many duplicate copies of the same vertex. Check whether discovery is marked only when dequeued. Marking before enqueue prevents another predecessor from scheduling the same vertex again. This change repairs the work invariant as well as reducing redundant storage.

A weighted route with fewer hops is reported as the cheapest route. The traversal can be correct for hop count while the application asked for total weight. Change the algorithm and its preconditions rather than relabeling the BFS distance as cost.

A component count changes when directed vertex order is permuted. Determine whether an undirected-only routine was applied to asymmetric adjacency. Endpoint validity is not symmetry, and reachability from successive starts is not strong connectivity. Restore the correct model at the boundary.

Practice and implementation lab

Add predecessor tracking to BFS and reconstruct a path to vertex three. Return absence for the isolated vertex. Add a disconnected directed cycle to verify that the outer DFS loop visits every region. Finally, replace recursive directed DFS with explicit frames while preserving active and finished transitions.

Review answers and solution notes

A shortest reconstructed path is $0 \rightarrow 1 \rightarrow 3$ under the fixture's neighbor order. Reconstruction must stop at the source and must not follow a predecessor for an unreachable vertex. A cycle outside the first traversal must still be detected. An iterative frame can contain a vertex and next-neighbor index; mark active on initial entry and finished only after that frame has exhausted its adjacency list.

Engineering takeaway. Traversal state must encode the property being proved: discovery distance, active ancestry, and component membership are different facts.

Guided lab — Produce and verify path witnesses

Extend BFS to return predecessor information and a reconstruction function. Test a reachable target, an unreachable target, and the source itself. Verify every consecutive edge in the reconstructed path and compare its edge count with the distance. Include a graph with two equally short routes and document how neighbor order affects the chosen witness.

Write an independent tiny-graph distance oracle using repeated relaxation or exhaustive simple-path enumeration under a small vertex bound. Compare all source-target pairs in generated unweighted graphs. Keep the oracle deliberately small; its purpose is semantic independence, not production performance.

Implement iterative directed cycle detection with frames containing vertex and next-neighbor position. Compare it with the recursive routine on small graphs, then use the iterative version on a long chain that would create deep recursion. Do not deliberately overflow the process stack to prove the recursive limitation.

Add a symmetry checker for undirected adjacency under a declared multiedge convention. Decide whether matching reverse multiplicities are required. Reject malformed input before component counting, and keep this validation cost separate when reporting traversal complexity.

For conceptual contrast, draw a directed chain, a directed cycle, and two vertices connected only in one direction. Explain their reachability, weak connectivity, and strong connectivity without using the undirected component function as an oracle for all three.

Acceptance evidence. Include discovery-time marking, valid path witnesses, acyclic convergence, self-loop cycle, disconnected cycle, isolated components, and explicit graph-model validation boundaries.

Review reasoning. Why does BFS find minimum hops? FIFO processing preserves distance layers. Why not minimum arbitrary weight? Edge costs need not be equal. Why distinguish active and finished in directed DFS? Only an active ancestor establishes the relevant back-edge cycle. Why can the undirected component routine reject no asymmetric input itself? Its current validator checks only indices; symmetry is a separate precondition. Why do iterative frames store a next-edge position? They must preserve unfinished recursive work and the correct finish event.

Primary reference notes

FIFO and stack storage: [queue adaptors](#) and [vector](#). Further BFS/DFS reading: [MIT algorithm lectures](#). The traversal proofs and fixtures are developed in this chapter.

Chapter 15 — Designing Efficient Search Algorithms

Finding an identifier is easy until duplicates, empty inputs, and insertion boundaries enter the contract. HarborWorks needs the first matching occurrence in a sorted collection, and it also needs the insertion position when a key is absent. A boolean search result would discard useful information.

This chapter treats search as a shrinking interval with a proof attached. It implements linear search, a lower-bound index, and exact binary search built on that index. The same data is checked against a standard-library lower bound and an independently structured linear reference.

HarborWorks searches a sorted identifier list for the first matching occurrence and also needs the position where a missing identifier would belong. These are related questions, but they are not identical. An insertion boundary equal to the vector size is valid as a boundary and invalid as an element to dereference.

This chapter builds exact search from a lower-bound primitive. The primitive maintains a shrinking half-open interval, and the exact search performs a final presence check. Separating the two responsibilities produces cleaner reasoning about emptiness, duplicates, and absent keys than a collection of special cases around an ambiguous “found” index.

15.1 Performing Linear Search

Linear search asks elements in sequence whether they satisfy the match condition. It requires no sorted-order promise. This simplicity is valuable when the input is short, arrives only once, or is changing too often to justify maintaining an index.

15.1.1 Searching Unordered Data

Scan from index zero upward and return as soon as the key is equal to an element. Return an optional index rather than using a valid index value as an absence sentinel. Index zero is a legitimate first match. If the loop finishes, return `nullopt`.

The increasing scan naturally implements first occurrence. A reverse scan would implement last occurrence, and a loop that continues after every match needs a separate result policy. Express this distinction in tests with duplicate values, not only in prose.

Linear search is also a useful oracle for testing a faster search because it has different control flow and fewer boundary decisions. However, the input domain must be shared: compare first occurrence with first occurrence, not with an arbitrary matching element returned by some other interface.

Linear search imposes no ordering requirement. It visits elements in sequence and returns the first equal position, or absence after the sequence is exhausted. This makes it a useful baseline and a useful independent oracle for a sorted exact-search implementation.

The first-occurrence policy is visible when duplicates exist. For `[4, 7, 7, 9]`, searching for seven returns one. A function that returns two has found a matching value but violates the selected position contract. Tests should compare positions, not merely check that the returned element equals the target.

A linear scan can be appropriate even when binary search has a better asymptotic lookup bound. A tiny collection, a one-time query over unsorted input, or a workload already scanning every record for another reason may not justify sorting or maintaining an index. Include preprocessing and mutation costs when comparing complete workflows.

The equality relation must match the domain. Searching records by identifier differs from comparing their entire payload. A predicate-based scan can express that distinction, but the caller must still preserve the input's lifetime and define any side effects of the predicate. A search predicate that mutates the sequence it is scanning invalidates simple reasoning about stable positions.

Returning an optional index separates absence from zero. The index refers to the sequence state used during the call. Inserting an earlier element afterward can change what that position identifies, so long-lived results may need stable record identifiers instead. Memory-safe integer output can still become semantically stale.

For validation, check result presence before indexing. An oracle that dereferences an absent result or an out-of-range position can introduce its own defect. The test harness should establish the same access preconditions it expects the implementation to respect.

A useful first property is input preservation. Save a small copy, run the search, and verify equality afterward. Search should not quietly sort the caller's vector unless mutation is explicitly part of a different interface.

15.1.2 Understanding Linear Complexity

For n values, an absent key requires n comparisons. A match at position i requires $i+1$ comparisons, giving $O(n)$ worst-case work and $O(1)$ additional storage. The best case is constant work when the first value matches. An average-case claim requires assumptions about the distribution of keys and hit locations.

Contiguous scanning can be efficient on real hardware even when its asymptotic count is larger. Binary search jumps around the data and may incur different memory-access costs. For a tiny vector or a single query, paying to sort beforehand can cost more than the search saves.

Analyze the complete workload: preprocessing, updates, query count, and the required result semantics. For many queries against a stable sorted vector, logarithmic search is attractive. For a one-pass stream, it may be unavailable because the necessary random access and retained ordered data do not exist.

For a missing key in a sequence of length n , linear search performs n equality comparisons. A first-position match needs one; an empty input needs none. These exact counts are easy to derive because every visited element corresponds to one comparison and the algorithm stops at the first match.

The worst-case linear bound does not require every input to take the full scan. An early match can be much cheaper, and an average depends on where matches occur and how often queries miss. Keep the distinction from Chapter 2 rather than describing one favorable fixture as the algorithm's general cost.

If a predicate itself is expensive, multiply or otherwise account for its cost. A scan over long strings may inspect many characters per equality check. A scan over records that triggers disk access or decoding is not described fully by counting record comparisons alone. Choose the model that answers the actual performance question.

Two scans can sometimes be fused, but fusion changes code structure and potentially error behavior. If a program already computes a maximum and also needs a first matching identifier, one pass can maintain both results. The clearer design may still use separate algorithms unless measurement or a resource constraint justifies combination. Avoid premature complexity in pursuit of saving one pass over a small vector.

Linear search is also valuable as a test reference because its control flow differs from binary partitioning. Agreement between two identical binary-search implementations with renamed variables gives weaker independent evidence. A straightforward scan can expose off-by-one and duplicate-boundary mistakes even though it is slower on large input.

For repeated membership queries, estimate the entire workload. q scans of n values require up to proportional $q \cdot n$ comparison work, while a reusable ordered index changes the tradeoff. The input's update rate and required output identity determine whether that index remains valid between queries.

Do not use a benchmark of successful first-element queries to justify a system dominated by misses. Preserve query distributions alongside measured results so the experiment remains interpretable.

Linear Search Returning the Matching Index

Purpose. Linear search trades simplicity and no preconditions for $O(n)$ worst-case work.

```
#include <iostream>
#include <optional>
#include <vector>
std::optional<std::size_t>find(const std::vector<int>&v,int target)
{for(std::size_t i=0;i<v.size();++i)if(v[i]==target)return i;return
std::nullopt;}
int main(){auto i=find({4,8,2,8},2);std::cout<<(i?
std::to_string(*i):"none")<<'\\n';}
```

How it works.

- Elements are checked from the beginning until a match is found.
- No ordering precondition is required.
- The optional result distinguishes a valid index from “not found.”

Expected behavior/output. The program prints index `2`.

Experiment. Search for 8 and note that this implementation returns the first match.

15.2 Performing Binary Search

Binary search eliminates portions of an ordered search space by comparing one middle value with the key. The search must preserve a specific partition invariant. “Move left or right until found” is not a sufficient design for duplicates and absent keys.

15.2.1 Halving Ordered Search Spaces

For lower bound, maintain an unknown half-open interval $[low, high)$. Every index below `low` contains a value smaller than the key. Every index at or above `high` is known not to contain a smaller value. Initially `low` is zero and `high` is the collection size, so both known regions are empty.

Compute `mid = low + (high - low) / 2`. If the middle value is smaller than the key, it and all earlier values can be excluded; set `low` to `mid + 1`. Otherwise keep `mid` as a possible answer and set `high` to `mid`. When `low` equals `high`, the unknown region is empty and that index is the first position not smaller than the key.

For random-access indexing, the unknown interval shrinks by about half per comparison, producing $O(\log n)$ comparisons. The standard lower-bound algorithm can also operate on weaker iterator categories, but logarithmic comparisons do not necessarily mean logarithmic iterator movement. This chapter’s vector indexing supplies the required constant-time midpoint access.

Lower bound identifies the first position whose value is not less than the key. The answer may be zero, an interior position, or the end boundary. On `[1, 3, 3, 8]`, lower bounds for zero, three, four, and nine are zero, one, three, and four respectively.

The implementation maintains `low <= high <= size`. Everything before `low` is known to be less than the key, and everything from `high` onward is known not to be less under the input’s partition property. The unknown region is $[low, high)$. At termination the unknown region is empty and the two boundaries identify the transition.

When the middle value is less than the key, the middle cannot be the answer and neither can any earlier position in the unknown range. Set `low = middle + 1`. Otherwise the middle may be the first qualifying position, so retain it as a possible boundary by setting `high = middle`. The asymmetry is essential: one branch excludes the middle, the other preserves its candidacy.

Sorted input under the matching comparator is sufficient for the needed partition. More generally, a lower-bound operation needs the predicate “element is less than this key” to be true for a prefix and false afterward. The book’s public example uses sorted vectors because that stronger precondition is easier to maintain across many different keys.

Random access lets each midpoint be reached directly. A generic library lower bound can use non-random-access iterators while keeping logarithmic comparisons, but traversal steps can be linear. Do not transfer the vector implementation’s whole cost model to a linked list merely because the function name is the same.

Exact search then checks whether the lower-bound position is inside the vector and equal to the key. A lower bound by itself is not a membership result. This final distinction prevents absent keys between values or above the range from being incorrectly reported as present.

15.2.2 Avoiding Boundary Errors

The half-open convention handles an empty vector without dereferencing anything: `low` and `high` both begin at zero. It also permits the answer to equal size when every value is smaller than the key. That is a valid insertion position, not a valid element index.

To turn the lower bound into an exact search, first check that the result is less than `size`, then compare the element with the key. Reversing those checks would read beyond the end for an absent high key. With short-circuit `&&`, the boundary check protects the access.

Using `low + (high - low) / 2` avoids forming a potentially overflowing `low + high` sum. The update `high = mid` retains a qualifying midpoint and avoids subtracting one from an unsigned zero. Each branch strictly shrinks a nonempty interval, which establishes termination and prevents a two-position infinite loop.

Sorted input is a precondition. The implementation does not perform a hidden $O(n)$ sortedness check on every query, since that would defeat the advertised search cost. Validate order at construction or update boundaries, test it in debug-only infrastructure when appropriate, and document the requirement at the public interface.

Half-open intervals simplify emptiness. An empty vector starts with `low == high == 0`, so the loop performs no access and returns boundary zero. There is no need to compute `size() - 1`, which would underflow for an unsigned empty size.

The midpoint is `low + (high - low) / 2`. Given the interval invariant, the subtraction is nonnegative and the result remains below `high` while the interval is nonempty. This avoids the unnecessary overflow risk of `(low + high) / 2`. The formula is justified by established bounds, not by treating arithmetic expressions as universally safe in every context.

Progress must occur on every iteration. If the middle is known too small, assigning `low = middle` can leave a one-element interval unchanged forever. The `+1` records that the middle has been excluded. In the other branch, `high = middle` reduces the interval because `middle` is strictly below the old `high`. The unknown interval's length is the termination measure.

Duplicates explain why equality does not end lower-bound search immediately. Finding one equal value does not establish that no earlier equal value exists. Keeping the qualifying `middle` in the retained left interval lets the algorithm continue toward the first occurrence. A conventional “return on equality” binary search can satisfy a different any-occurrence contract but not this one.

Upper bound uses a related partition with a different comparison: it finds the first element greater than the key. The half-open range between lower and upper bounds describes all equivalent occurrences under the ordering policy. When extending to custom comparators, derive the directions carefully rather than replacing symbols mechanically.

Input validation has a cost. Calling `is_sorted` before every lookup adds a linear scan, changing the end-to-end complexity. A public boundary can validate a batch once and encapsulate the ordered state so later searches rely on it. A fast algorithm with an unchecked precondition is not appropriate for arbitrary external input, but repeatedly rechecking an already maintained invariant is also a design choice that should be named.

Manual Lower/Upper Bound Narrowing in a Sorted Vector

Purpose. Binary search achieves logarithmic comparisons by relying on sorted input and a carefully maintained interval invariant.

```
#include <iostream>
#include <vector>
bool binary(const std::vector<int>&v,int x){std::size_t
lo=0,hi=v.size();while(lo<hi){auto mid=lo+(hi-lo)/2;if(v[mid]<x)lo=mid+1;else
hi=mid;}return lo<v.size()&&v[lo]==x;}
int main(){std::cout<<std::boolalpha<<binary({1,3,5,7,9},7)<<'
'<<binary({1,3,5,7,9},6)<<'\n';}
```

How it works.

- The active search interval is the half-open range `[lo, hi)`.
- Each comparison discards roughly half the remaining positions.
- The final candidate is checked for exact equality.

Expected behavior/output. The output is `true false`.

Experiment. Trace `lo`, `hi`, and `mid` for target 9 by hand before instrumenting the loop.

Worked example — Find first occurrences and insertion positions without boundary defects

Requirement and design

Implement a linear optional-index search, a lower-bound index for sorted integer vectors, and exact binary search using that lower bound. Compare results over hundreds of deterministic datasets and a key range extending below and above their values. Include empty and all-equal inputs.

Use two independent answers for two different questions

The program compares the custom insertion boundary with `std::lower_bound` and compares exact binary search with the simple first-occurrence linear scan. The two references match different contracts. A standard lower-bound iterator is converted to an index only within the same vector, where the difference is meaningful.

Generated datasets are sorted before either binary operation. This setup is part of the fixture, not hidden repair inside the algorithm. Keys extend below and above the generated value range, ensuring that both outer boundaries are exercised. Duplicate-heavy values make first-occurrence behavior common rather than rare.

The all-one fixed case requires exact search to return zero. The empty lower-bound case requires boundary zero without a match. Trace both before running the test so that a boundary and an optional result are not conflated.

The comparison suite is deterministic but finite. Its strength comes from varied boundaries, independent mechanisms, and the interval proof together. Increasing the number of generated trials cannot compensate for an oracle that asks the wrong question.

Complete implementation

The complete, buildable chapter project is `source/book3/ch15/`. The following files are reproduced from that project; the publishing script inserts the source directly so the listing and companion remain synchronized.

main.cpp

```
#include "course_test.hpp"
#include <algorithm>
#include <optional>
#include <random>
#include <vector>

std::optional<std::size_t> linear_find(const std::vector<int>& values, int key) {
    for (std::size_t i = 0; i < values.size(); ++i) if (values[i] == key) return i;
    return std::nullopt;
}

std::size_t lower_bound_index(const std::vector<int>& values, int key) {
    std::size_t low{}, high = values.size();
    while (low < high) {
        const auto middle = low + (high - low) / 2;
        if (values[middle] < key) low = middle + 1;
        else high = middle;
    }
    return low;
}

std::optional<std::size_t> binary_find(const std::vector<int>& values, int key) {
    const auto index = lower_bound_index(values, key);
    if (index == values.size() || values[index] != key) return std::nullopt;
    return index;
}

int main() {
    std::mt19937 random{42};
    for (int trial = 0; trial < 300; ++trial) {
        std::vector<int> data(static_cast<std::size_t>(trial) % 129);
        for (int& value : data) value = static_cast<int>(random() % 41) - 20;
        std::sort(data.begin(), data.end());
        for (int key = -25; key <= 25; ++key) {
            const auto expected = static_cast<std::size_t>(
                std::lower_bound(data.begin(), data.end(), key) - data.begin());
            CHECK(lower_bound_index(data, key) == expected);
            CHECK(binary_find(data, key) == linear_find(data, key));
        }
    }
    CHECK(lower_bound_index({}, 1) == 0);
    CHECK(binary_find({1, 1, 1}, 1) == 0);
    course::report();
}
```

Build, run, and inspect

Run this block from the top-level extracted package. The fixture runner supplies input where needed and checks exit status and output in a temporary working directory.

```
cmake -S source/book3/ch15 -B build/b3-ch15 \
  -DCMAKE_BUILD_TYPE=Debug
cmake --build build/b3-ch15 --parallel 2
ctest --test-dir build/b3-ch15 --output-on-failure
```

For a multi-configuration generator on Windows, append `--config Debug` to the build command and `-C Debug` to CTest. The executable target is `b3_ch15`.

Evidence and interpretation

The randomized test sorts each generated dataset before calling the binary routines. For every key, it compares the custom insertion index with `std::lower_bound` and compares exact binary search with linear first occurrence. This tests both absent-key boundaries and duplicate handling.

An all-one collection searched for one must return index zero. An empty vector has lower-bound index zero but no exact match. A key above every element has insertion index size, which must not be dereferenced.

Passing these checks is evidence for the implementation over the fixture domain. The interval invariant explains why the method generalizes; the fixtures alone are not a mathematical proof. Conversely, a correct proof does not exempt an implementation from tests that catch an accidentally reversed comparison or incorrect assignment.

Failure clinic — A valid boundary was treated as an element

A search for a key above every stored value crashes after the loop finishes. The lower-bound result is size, which is correct as an insertion position. The exact-search wrapper dereferenced it before checking the end boundary. Repair the wrapper's access order rather than changing lower bound to return a misleading last index.

A two-element search hangs. Inspect whether a branch assigns `low = middle` after ruling the middle out. A shrinking-interval proof immediately exposes the lack of progress. The smallest nonterminating fixture is more useful than waiting for a timeout on a large input.

A duplicate lookup returns an arbitrary middle occurrence. The implementation probably returns on equality. That can be valid for any-match search but violates a first-match contract. Keep searching the left qualifying interval or explicitly change the interface and tests to the weaker promise.

A benchmark shows linear scaling despite a logarithmic loop. Check for per-query sorting validation, data copying, or a non-random-access representation around the loop. The core algorithm and the complete workflow can have different costs without contradiction.

Practice and implementation lab

Implement upper bound, the first position strictly greater than the key. Use lower and upper bounds to count duplicates without scanning them. Add fixtures for a missing key between two values, a key below the minimum, and a key above the maximum. Explain how the partition predicates differ between the two boundary searches.

Review answers and solution notes

Upper bound moves low past a midpoint when the midpoint is less than or equal to the key; otherwise it moves high to mid. The duplicate count is `upper_index` minus `lower_index` because the matching region is half-open. Both results may equal size. For a key absent between values, the two bounds coincide. Keep the interval proof and comparator ordering consistent when adapting the algorithm to domain records.

Engineering takeaway. Choose a boundary contract first; then make every branch preserve the interval that proves it.

Guided lab — Search for a boundary, then derive membership

Implement upper-bound indexing beside lower bound. On `[1, 3, 3, 8]`, verify that key three has lower one and upper three. Use their difference to count occurrences and the resulting half-open subrange to report matching positions. Test absent keys and the end boundary without dereferencing them.

Enumerate all nondecreasing vectors of small length over a three-value domain. Compare lower and upper bounds with the standard algorithms for keys inside and outside that domain. This controlled enumeration targets partitions and duplicates directly rather than relying only on randomly sorted inputs.

Write a generic `first_true` exercise over an index interval and a monotone predicate. Derive which branch retains the midpoint. Use it to find the first capacity meeting a simple checked requirement. State that arbitrary nonmonotone predicates invalidate the result, even if the function always terminates.

Create an ordered-index wrapper that validates and owns its input once, then exposes read-only searches. Explain how the wrapper prevents callers from mutating the stored order without rebuilding. This turns a precondition into a maintained representation invariant and avoids paying a linear validation cost on every query.

For mutation testing, remove the `+1`, return immediately on equality, and reverse the comparison in separate exercise copies. Identify a minimal counterexample for each. Do not keep a hanging variant in an unbounded automated run; apply a timeout or reject it by the interval proof.

Acceptance evidence. Include empty input, singleton, all equal, below range, above range, between values, first and last matches, and input preservation. State whether the API returns a boundary, any match, first match, or all matches.

Review reasoning. Why can lower bound return size? It describes a valid insertion boundary. Why is equality not enough to stop? Earlier equal values may exist. Why is `low + (high - low) / 2` preferred? Its safety follows from the maintained interval. Why is sortedness checking not free? It scans

the input. Why can linear search be a good oracle? Its simpler mechanism independently checks the exact-result semantics.

Primary reference notes

Standard contracts: [binary-search algorithms](#), [nonmodifying sequence operations](#), and [iterator operations](#).
Further search analysis: [MIT algorithm lecture directory](#).

Chapter 16 — Designing and Comparing Sorting Algorithms

Sorting is not one operation with one universal implementation. HarborWorks may sort a handful of already-nearly-ordered readings, merge large ordered batches, or reorder a duplicate-heavy event stream. Input distribution, memory budget, stability, and worst-case requirements all affect the choice.

This chapter implements four integer sorting algorithms rather than timing unrelated library calls and calling that an explanation. Each algorithm is checked against the same sorted reference. The goal is to connect loop invariants and partition rules to both correctness and cost.

HarborWorks needs a sorted report, but several implementations can satisfy that requirement with different costs, storage needs, and tie behavior. An insertion sort can be attractive for a small nearly ordered batch, a merge process can preserve equal-key order, and a partitioning sort can avoid a full auxiliary value buffer. None of those advantages is a universal verdict.

This chapter compares four complete integer sorting implementations under one shared contract: preserve every value and produce nondecreasing order. Stability is discussed separately because equal integers alone cannot demonstrate whether equal-key records retained their original identity order. The supplemental stability lab makes that distinction executable with tagged records.

16.1 Understanding Elementary Sorting Algorithms

Elementary sorts expose the decisions hidden inside a sorting interface. They are useful learning tools and can be practical for small ranges. Their simplicity does not make their costs interchangeable: insertion sort responds to existing order, while selection sort keeps scanning for a minimum.

16.1.1 Implementing Insertion and Selection Sort

Insertion sort maintains a sorted prefix $[0, i)$. Save the value at i , shift larger prefix values one position right, and place the saved value in the resulting gap. During the shift, the saved local value prevents it from being overwritten. The condition checks $j > 0$ before reading index $j-1$.

The comparison uses strict less-than, so a value is not shifted past an equal predecessor. This makes the algorithm stable when generalized to records under a consistent comparator: equal-key records retain their input order. Stability is about distinguishable equal-key records, not about identical integers alone.

Selection sort maintains a prefix of final smallest elements. At each position i , scan the remaining suffix for its smallest value and exchange it with the value at i . An exchange can move a record across another equal-key record, so this implementation is not stable. Selection uses relatively few swaps but still performs the full suffix scans.

Insertion sort maintains a sorted prefix. At iteration i , save the next value, shift larger prefix values one position right, and place the saved value in the resulting gap. The prefix grows by one while the suffix

remains unprocessed. This is a direct loop-invariant design rather than repeated global searching for order.

The strict comparison in the shifting loop matters for stability. Equal values are not shifted past the saved value, so an equal-key record inserted later remains after earlier equal-key records. To observe that property, records need a separate identity tag; two equal integers are indistinguishable in the output.

Selection sort instead finds the smallest value in the remaining suffix and swaps it into the next output position. The prefix contains the correct smallest values after each iteration. This can use few swaps, but the distant swap can reorder equal-key records. For tagged keys $(2, A)$, $(2, B)$, $(1, C)$, swapping the minimum into the first position can leave $(1, C)$, $(2, B)$, $(2, A)$.

Both methods are easy to trace on a handful of values. For insertion sort, record the saved value and gap position. For selection sort, record the current suffix minimum index. A vague statement that the array “gets more sorted” is not enough to justify every read and write; name the exact prefix property.

The worked implementations use integers, so assignment and swapping are nonthrowing under the selected model. Generic records can make movement expensive or throwing. Fewer swaps may then matter, but a correctness contract must address partial mutation if an operation fails. A comparison of integer timings does not automatically predict behavior for large resource-owning objects.

Avoid calling an educational algorithm useless merely because its asymptotic worst case is quadratic. Its small-input behavior and role inside hybrid implementations can be valuable. Equally, do not deploy it on large arbitrary batches without considering the growth in comparisons and moves.

16.1.2 Understanding Quadratic Complexity

Selection sort performs roughly $n(n-1)/2$ comparisons regardless of existing order, giving quadratic time. Insertion sort has the same worst-case growth when values arrive in reverse order, but an already sorted input requires only one failed shift comparison per insertion and therefore linear work.

Both examples use constant extra element storage. That advantage can matter under a strict memory budget, but it does not justify quadratic work on every large dataset. The right question is whether the actual range size and input order make the cost acceptable.

Counting comparisons and movements can be more illuminating than wall-clock timing at this stage. For records with expensive movement, two algorithms with similar comparison counts may behave very differently. Use a counted record type in the lab to separate those costs instead of attributing every timing difference to asymptotic notation.

Selection sort scans suffixes of lengths n , $n-1$, and so on to find successive minima. Ignoring a constant convention about the current candidate, the number of comparisons follows a triangular sum and is quadratic. Already sorted input does not eliminate those scans in this implementation.

Insertion sort’s work depends more strongly on disorder. On an already sorted sequence, each next value needs only the failed comparison that confirms its position. On reverse order, each inserted value can shift the entire prefix, producing a quadratic total. The number of inversions helps explain movement: each shift removes one out-of-order pair for this scalar algorithm.

An inversion is a pair of positions $i < j$ whose values are out of the requested order. A nearly ordered sequence can have few inversions even when its length is moderately large. That observation provides a more precise workload description than the vague phrase “almost sorted.” It also suggests generated fixtures with controlled local swaps rather than only fully sorted and fully reversed extremes.

Complexity counts should distinguish comparisons from moves. Selection sort can perform quadratic comparisons while using only a linear number of final swaps. Insertion sort can perform many shifts. If comparison is expensive and movement cheap, the tradeoff differs from a workload with cheap integer comparison and large objects. One scalar wall-clock number hides those mechanisms.

Quadratic growth can still be acceptable under a deliberate small bound. If a component sorts at most sixteen entries, a simple implementation may be easier to verify than a more complex one. The bound must be enforced or maintained, not merely assumed from today’s input. A future change from sixteen to sixteen thousand alters the engineering decision.

For a benchmark, do not compare one algorithm on fresh unsorted data with another on the previous algorithm’s sorted result. Copy the same original dataset for each candidate and keep setup outside or inside all timed regions consistently. The test suite follows this rule for correctness comparisons as well.

Insertion Sort Showing the Growing Sorted Prefix

Purpose. Insertion sort illustrates an $O(n^2)$ elementary algorithm whose simplicity and good behavior on small/nearly sorted inputs can still be useful.

```
#include <iostream>
#include <vector>
void insertion(std::vector<int>&a){for(std::size_t i=1;i<a.size();++i){int
key=a[i];std::size_t j=i;while(j>0&&key<a[j-1]){a[j]=a[j-1];--j;}a[j]=key;}}
int main(){std::vector<int>a{5,2,4,1};insertion(a);for(int x:a)std::cout<<x<<'
';}
```

How it works.

- Before each outer iteration, the prefix ``[0,i)`` is sorted.
- Larger prefix elements shift right until the insertion position is found.
- The key is placed once, preserving the sorted-prefix invariant.

Expected behavior/output. The output is ``1 2 4 5``.

Experiment. Count element shifts for already sorted input and reverse-sorted input.

16.2 Understanding Efficient Sorting Algorithms

Efficient comparison sorts reduce the amount of repeated searching. Merge sort combines already-sorted subranges. Quick sort partitions around a pivot so later work can focus on independent sides. Their extra-memory, stability, and worst-case properties differ.

16.2.1 Implementing Merge Sort

Merge sort divides a half-open range into two smaller ranges until each has at most one element. After recursively sorting both halves, merge them by repeatedly selecting the smaller remaining front value. Every selected value is the smallest still available across both sorted halves, which proves the output prefix remains sorted.

When keys are equal, the code selects from the left half. That tie rule preserves stability across the split. A reusable temporary vector is allocated once outside recursion; each merge writes its range into the temporary storage and copies it back. The implementation uses $O(n)$ auxiliary element storage and $O(\log n)$ call depth.

At each recursion level the total merge work is $O(n)$, and there are $O(\log n)$ levels, giving $O(n \log n)$ time. Empty and one-element ranges return immediately. Allocation failure occurs before any sorting mutation in the wrapper because the temporary vector is constructed first. This integer implementation has simpler movement behavior than a fully generic merge sort over throwing element types.

Merge sort divides a range into two smaller ranges, sorts both, then combines their ordered outputs. The merge step repeatedly selects the next smaller front value from the two remaining subranges. Each output position is filled once, and each input position advances at most once during that merge.

The dependency structure is important. Merging assumes both halves are already sorted under the same comparator. Calling the merge routine on arbitrary halves does not establish a sorted result. The recursive split reaches empty or one-element ranges, which are sorted without work, and then builds larger sorted ranges upward.

The example allocates one temporary vector for the entire sort and reuses its indexed regions. It does not allocate a new buffer at every recursive call. This keeps auxiliary value storage linear, while recursive call depth is logarithmic for the balanced splits. Output is copied back into the original range after each merge.

On equivalent values, the comparison chooses the left side first. For tagged records, that policy preserves relative order across the two halves when each half is itself stably sorted. Stability follows inductively through the merge tree. Choosing the right on equality would still sort keys but could reorder equal-key records.

The midpoint uses the same safe interval style as binary search. The base condition checks range length before recursion. The merge loop checks whether either half is exhausted before reading it. Exhaustion branches are not incidental edge handling; they are part of the proof that every output position receives a valid remaining value.

The $O(n \log n)$ comparison/work model comes from logarithmically many balanced merge levels with linear combined work per level. That model does not include arbitrary costs hidden in a custom comparator or element move. The example's integer contract keeps those operations simple enough to make the structural argument visible.

Allocation failure before mutation leaves the original vector untouched in this implementation. A generic merge whose element assignments can throw during later stages needs a separate partial-mutation guarantee; one temporary allocation does not automatically make the entire operation transactional.

16.2.2 Implementing Quick Sort

The quick-sort implementation uses a three-way partition: values smaller than the pivot, equal to the pivot, and greater than the pivot. During partitioning, `[low, less)` is smaller, `[less, current)` is equal, `[current, greater)` is unknown, and `[greater, high)` is greater. The pivot is copied into a local integer so swapping elements cannot change the comparison value.

A smaller value is exchanged into the less region and advances both less and current. A larger value is exchanged with the final unknown position after decrementing greater; current does not advance because the arriving value has not yet been classified. An equal value advances current. When current reaches greater, the unknown region is empty.

Only the smaller and greater regions need further sorting. A large equal region is already finished, making this approach effective at avoiding pointless recursion on duplicate-heavy input. The middle-position pivot is not a guarantee of balanced partitions. Adversarial inputs can still produce $O(n^2)$ time.

The implementation recurses on the smaller side and loops over the larger side. Each nested call therefore receives at most about half the preceding non-equal range, bounding call depth by $O(\log n)$ even if total partition work is quadratic. Bounding stack depth is not the same as guaranteeing $O(n \log n)$ execution time. A production introspective sorter adds a worst-case fallback and other engineering refinements; the standard library is normally preferable to a home-grown sorter.

Quick sort partitions values around a selected pivot, then sorts the regions that remain out of order internally. The worked variant uses three regions: less than pivot, equal to pivot, and greater than pivot. This handles duplicate-heavy input without repeatedly recursing on every equal value.

During partitioning, the array is divided into a known-less region, a known-equal region, an unknown region, and a known-greater region. When the current value is smaller, swap it into the less region and advance both relevant boundaries. When equal, advance the current boundary. When greater, swap with the last unknown position and shrink the unknown region without advancing current, because the newly arrived value has not yet been classified.

That last rule is a frequent defect source. Advancing after the greater-side swap skips an unexamined value. A small mixture of low, high, and equal values can expose the error more directly than a large random benchmark. The termination measure is the size of the unknown interval, which shrinks on every branch.

The pivot is copied from the middle position before partitioning. It is not a reference into a slot that may be swapped, and it is not the median of the range. Choosing the middle position does not guarantee balanced partitions. The algorithm can still have quadratic worst-case work for adversarial arrangements.

The implementation recurses on the smaller outer region and iterates on the larger one. This bounds recursive stack depth logarithmically because each nested recursive call handles at most about half the previous active range. It does not change the worst-case amount of partition work. Stack-depth control and time-complexity guarantees are separate properties.

Quick sort here is not stable. Equal-key records may move relative to one another during swaps. A report requiring stable order should select a stable mechanism or explicitly include an original-position tie-breaker under a new ordering contract. Passing integer equality tests cannot certify record stability.

Merge Sort with Linear Merge Steps

Purpose. Efficient comparison sorts reduce the number of ordering comparisons by recursively structuring the problem rather than repeatedly scanning all pairs.

```
#include <algorithm>
#include <iostream>
#include <vector>
void sort(std::vector<int>&a, std::size_t lo, std::size_t hi) { if (hi -
lo <= 1) return; auto mid = lo + (hi - lo) /
2; sort(a, lo, mid); sort(a, mid, hi); std::vector<int> tmp; tmp.reserve(hi -
lo); std::merge(a.begin() + lo, a.begin() + mid, a.begin() + mid, a.begin()
+ hi, std::back_inserter(tmp)); std::copy(tmp.begin(), tmp.end(), a.begin() + lo); }
int main() { std::vector<int> a{8, 3, 7, 1, 5}; sort(a, 0, a.size()); for (int
x : a) std::cout << x << ' '; }
```

How it works.

- The recursive calls sort two roughly equal halves.
- `std::merge` combines two already sorted ranges in linear time.
- The recurrence yields $O(n \log n)$ comparison work.

Expected behavior/output. The output is ``1 3 5 7 8``.

Experiment. Measure temporary allocations, then reuse one auxiliary buffer to reduce allocation overhead.

Worked example — Compare four sorting mechanisms against one contract

Requirement and design

Implement insertion, selection, merge, and three-way quick sort for integer vectors. Preserve every input value and produce nondecreasing order. Test empty ranges, duplicate-heavy inputs, ascending and descending inputs, and deterministic random values against `std::sort`.

Test one semantic contract across four mechanisms

Each generated input is copied separately for insertion, selection, merge, and quick sort. The standard reference is also computed from the unchanged original. This arrangement avoids accidentally giving later algorithms an already sorted result and makes equality a check of both order and retained multiplicity.

The dataset families include ascending, descending, all-equal, and generated duplicate-heavy values. These exercise different control paths. An all-equal input is particularly important for three-way partitioning, while reverse input makes insertion shifts visible. None of the families is presented as a complete worst-case performance proof.

The integer reference comparison cannot test stability because equal values carry no distinguishable identity. The appendix's tagged-record lab adds keys and original sequence tags, then checks whether equal-key tags remain ordered. That is an additional contract, not a reinterpretation of the existing integer fixtures.

Read the quick-sort recursion choice separately from its partition loop. The smaller-side recursion is a stack-resource design; the pivot choice governs partition balance. Keeping those mechanisms separate prevents an optimized call structure from being misreported as a guaranteed $O(n \log n)$ time algorithm.

Complete implementation

The complete, buildable chapter project is `source/book3/ch16/`. The following files are reproduced from that project; the publishing script inserts the source directly so the listing and companion remain synchronized.

main.cpp

```
#include "course_test.hpp"
#include <algorithm>
#include <functional>
#include <random>
#include <vector>
#include <utility>

void insertion_sort(std::vector<int>& data) {
    for (std::size_t i = 1; i < data.size(); ++i) {
        const int value = data[i];
        std::size_t j = i;
        while (j > 0 && value < data[j - 1]) { data[j] = data[j - 1]; --j; }
        data[j] = value;
    }
}

void selection_sort(std::vector<int>& data) {
    for (std::size_t i = 0; i < data.size(); ++i) {
        auto smallest = i;
        for (auto j = i + 1; j < data.size(); ++j) if (data[j] < data[smallest])
            smallest = j;
        std::swap(data[i], data[smallest]);
    }
}

void merge_range(std::vector<int>& data, std::vector<int>& temp,
                 std::size_t low, std::size_t high) {
    if (high - low <= 1) return;
    const auto middle = low + (high - low) / 2;
    merge_range(data, temp, low, middle); merge_range(data, temp, middle, high);
    auto left = low, right = middle;
    for (auto out = low; out < high; ++out) {
        if (left < middle && (right == high || !(data[right] < data[left]))) {
            temp[out] = data[left++];
        } else { temp[out] = data[right++]; }
    }
    for (auto i = low; i < high; ++i) data[i] = temp[i];
}

void merge_sort(std::vector<int>& data) {
    std::vector<int> temporary(data.size());
    merge_range(data, temporary, 0, data.size());
}

void quick_range(std::vector<int>& data, std::size_t low, std::size_t high) {
```

```

    while (high - low > 1) {
        const int pivot = data[low + (high - low) / 2];
        auto less = low, current = low, greater = high;
        while (current < greater) {
            if (data[current] < pivot) std::swap(data[less++], data[current++]);
            else if (pivot < data[current]) std::swap(data[current], data[--
greater]);
            else ++current;
        }
        // Recurse on the smaller side; iterate on the larger side.
        if (less - low < high - greater) {
            quick_range(data, low, less); low = greater;
        } else {
            quick_range(data, greater, high); high = less;
        }
    }
}
void quick_sort(std::vector<int>& data) { quick_range(data, 0, data.size()); }

int main() {
    std::mt19937 random{42};
    const std::vector<std::function<void(std::vector<int>&)>> algorithms{
        insertion_sort, selection_sort, merge_sort, quick_sort};
    for (int trial = 0; trial < 200; ++trial) {
        std::vector<int> input(static_cast<std::size_t>(trial) % 129);
        for (int& value : input) value = static_cast<int>(random() % 31) - 15;
        if (trial % 4 == 0) std::sort(input.begin(), input.end());
        if (trial % 4 == 1) std::sort(input.rbegin(), input.rend());
        if (trial % 4 == 2) std::fill(input.begin(), input.end(), 7);
        auto expected = input; std::sort(expected.begin(), expected.end());
        for (const auto& algorithm : algorithms) {
            auto actual = input; algorithm(actual); CHECK(actual == expected);
        }
    }
    course::report();
}

```

Build, run, and inspect

Run this block from the top-level extracted package. The fixture runner supplies input where needed and checks exit status and output in a temporary working directory.

```

cmake -S source/book3/ch16 -B build/b3-ch16 \
  -DCMAKE_BUILD_TYPE=Debug
cmake --build build/b3-ch16 --parallel 2
ctest --test-dir build/b3-ch16 --output-on-failure

```

For a multi-configuration generator on Windows, append `--config Debug` to the build command and `-C Debug` to CTest. The executable target is `b3_ch16`.

Evidence and interpretation

Each of two hundred datasets is copied separately for each algorithm, so one algorithm cannot receive another algorithm's already-sorted output. The reference is also built from the original input. Equality with that reference checks both sorted order and the multiset of retained integer values.

The dataset pattern deliberately alternates sorted, reverse-sorted, all-equal, and general generated inputs. This exposes different loop paths without claiming to prove worst-case performance. The tests do not establish stability because equal integers carry no distinct identity.

The three-way quick-sort branch that swaps from the greater region is particularly important: advancing current there would skip examination of the arriving value. A small fixture such as several values on both sides of the pivot helps make the invariant visible before relying on the randomized suite.

Failure clinic — Sorted output lost a value

A candidate output is nondecreasing but contains a duplicate replacing a missing value. An `is_sorted` check alone passes. Compare the full output with a sorted copy of the original or independently verify the multiset. Sorting correctness includes preservation, not just final order.

A quick-sort partition occasionally leaves one high value among the low region. Inspect whether current advanced after swapping from the greater region. The incoming value still belongs to the unknown region and must be classified before advancing.

A merge reads beyond one half near the final output positions. Check exhaustion conditions before comparing front values. Once one half is empty, only the other can supply remaining values. A comparison expression with unsafe evaluation order can dereference an exhausted side even when another condition would have selected the right branch.

A stable-sort claim fails for records with equal keys. Determine whether the algorithm actually preserves equal-key order and whether the tests used identity tags. Equal integers conceal instability. Change either the algorithm or the claimed contract; do not weaken the identity test to hide the distinction.

Practice and implementation lab

Instrument comparisons and assignments for each algorithm on sorted and reverse-sorted sequences. Generalize insertion and merge sort to records containing a key and original position, then verify stability. Add a quick-sort recursion-depth counter and distinguish its bound from the number of comparisons.

Review answers and solution notes

Insertion's shift count is near zero for sorted data and quadratic for reversed data. Selection still scans every suffix in both cases. Stability means original positions increase within each equal-key group after sorting. The smaller-side recursion strategy bounds nested call depth but does not eliminate highly unbalanced repeated partition work; a low depth reading alone is not evidence of fast sorting.

Engineering takeaway. Separate ordering correctness, stability, memory use, stack depth, and running time; no single test establishes all five.

Guided lab — Verify order, preservation, and stability separately

Create tagged records containing key and original position. Sort by key only. Check nondecreasing keys, preservation of every original tag exactly once, and increasing original positions within each equal-key group when stability is required. These are three independent properties.

Implement a generic stable merge exercise with a nonthrowing integer-key comparator. Compare it with `std::stable_sort` on small duplicate-heavy datasets. Then deliberately choose the right element on equality and retain the smallest fixture that exposes the stability change.

Instrument insertion and selection sort with comparison and movement counters. Use sorted, reverse, one-local-swap, and all-equal inputs of the same length. Explain the observed counts from their invariants instead of selecting a winner from one elapsed-time measurement.

Enumerate short arrays over a tiny value domain and compare all four algorithms with the standard sorted reference. Include lengths zero and one explicitly. Keep enumeration bounds visible so “exhaustive” refers to the chosen finite domain.

For quick sort, add a diagnostic count of maximum recursive depth and total partition comparisons. Test that smaller-side recursion limits depth on the exercised cases, but do not turn those observations into a universal bound on work. Write separate statements for stack depth, auxiliary storage, and worst-case comparisons.

Acceptance evidence. Retain order and multiset checks, tagged stability cases, exhausted-merge-side cases, duplicate-heavy partitions, and fresh-input fairness for every candidate algorithm.

Review reasoning. Why is sortedness alone insufficient? Values can be lost or duplicated. Why does stable merge select the left on equality? It preserves prior cross-half order. Why can selection sort be unstable? A distant swap can cross equal-key records. Why not advance after swapping from quick sort’s greater region? The incoming value is unclassified. Why does smaller-side recursion not guarantee good time complexity? It controls call depth, not pivot balance or total partition work.

Primary reference notes

Standard algorithm contracts: [sorting operations](#) and [algorithm requirements](#). Further analysis: [MIT algorithm lectures](#). The four implementations and partition explanations are original.

Chapter 17 — Solving Recursive and Backtracking Problems

Some problems do not reveal the correct next choice immediately. HarborWorks can model a placement problem as a sequence of tentative decisions, rejecting a branch as soon as it conflicts with earlier decisions. Recursion describes the remaining subproblem; backtracking restores the state needed to try a different choice.

The worked example counts placements of n non-attacking queens on an n -by- n board. This is a bounded algorithm exercise, not an unbounded puzzle service. The explicit limit of twelve keeps both recursion depth and the accepted search domain visible.

A recursive program can be short enough to fit on a postcard and still explore more work than a machine can finish. The useful question is not whether the function calls itself. It is what becomes smaller, what the current call assumes, what remains true while a choice is active, and how the caller's state is restored afterward. Those questions connect the correctness discipline of Chapter 1 to the structures and algorithms already built.

HarborWorks uses a small placement problem to study this process. Each row must receive one item, and occupied columns or diagonals forbid certain placements. The chapter's queens example is a mathematical teaching workload, not a scheduling optimizer for actual equipment. Its bounded input makes the search observable while leaving the difference between pruning and guaranteed efficiency visible.

17.1 Designing Recursive Solutions

A recursive function needs a well-defined smaller problem and a stopping condition. The call stack remembers work that must resume after a child call completes. Recursion does not eliminate state; it gives that state a structured lifetime.

17.1.1 Defining Base Cases

A base case handles an input that no longer needs decomposition. In the queens problem, reaching row n means every row has a valid queen placement, so count one solution and return. The empty board also has one valid empty placement. This convention makes the combinatorial result for $n=0$ explicit rather than treating it as an accidental loop outcome.

A base case must be reachable from every permitted recursive path. A function that checks only $n == 0$ but subtracts two from odd positive n can skip it. Establish a measure that decreases toward the base case and validate inputs for which that measure would not apply.

Return behavior matters as much as the test. After recognizing a completed placement, the function returns immediately instead of continuing to index another row. A base-case condition that increments a result and then falls through into the recursive body can still access invalid state.

A base case describes a state whose answer is available without making another recursive call. It should follow from the problem's meaning rather than from a desire to stop the debugger. For a sum of an empty sequence, the result is zero. For counting arrangements after every row has received a valid queen, the result contributed by that arrangement is one. These are different empty-state meanings, and choosing the wrong identity can corrupt every larger result.

In the worked program, `row == n` means that rows `[0, row)` have already been assigned without conflicts and there are no rows left to assign. Incrementing the solution count records that one complete arrangement. The function then returns immediately. Continuing to test columns would attempt to place an extra queen beyond the declared board.

The separate `n == 0` result is also one. This convention counts the single empty arrangement, not a physical queen or a move. It is useful because combinatorial construction often treats doing nothing as one valid way to satisfy zero requirements. A product of independent counts would otherwise be destroyed by an empty subproblem. State that convention in the contract and test it explicitly.

A base case must be reachable from every accepted input. Merely writing `if (n == 0)` does not make a recursion safe when the transition moves from a negative value to a more negative value. Validate the domain before entering the recursive mechanism, then show that each transition remains in that domain or reaches a designated terminal state.

It helps to distinguish success leaves from failure leaves. A completed placement contributes one solution. A partially filled board with no legal column contributes zero. The implementation reaches the second outcome by exhausting the loop without making a successful recursive call. There need not be a separate `return 0` expression for the branch to have this meaning.

17.1.2 Reducing Problems Recursively

At row `r`, the smaller problem is to place queens in rows `r+1` through `n-1` after selecting a legal column for `r`. The number of unfilled rows decreases by one on every recursive call, so depth is at most `n`. This proves termination for the finite, bounded board even though the number of branches may be large.

The partial-placement invariant is: every earlier row has exactly one queen, no used column repeats, and neither diagonal family contains a conflict. The function does not need to recheck every pair of earlier queens if it maintains occupancy arrays that encode those facts.

Recursive decomposition should match the information passed down. Here the mutable occupancy arrays represent the current path, while the row number identifies progress. A function that copied the entire board at every branch would use a different memory and movement strategy. Neither style is inherently correct merely because it uses recursion; both must preserve the same legal-placement invariant.

Termination follows from a measure that decreases in a well-founded domain. Here that measure is the number of unassigned rows, `n - row`. A recursive call advances from `row` to `row + 1`, reducing the measure by one. Because calls begin at zero and stop at `n`, a chain contains at most `n` placements. This argument bounds recursion depth; it does not bound the number of different chains cheaply.

A complete search may branch into many alternatives at each level. Even though each individual branch is short, the total tree can be large. This is why a recursion that has a convincing termination argument

can still be unsuitable for unrestricted input. The supplied $n \leq 12$ teaching limit bounds the exercise, while the mathematical argument explains its structure.

Track the meaning of a call as a sentence: “Count completions of the already valid assignments in rows before **row**.” That sentence identifies the call’s inherited state and its responsibility. It also explains why the recursive function does not need a second loop to choose a row: the row order has already been fixed by the decomposition. Each level chooses only a column for its own row.

Some recursive algorithms shrink by halving rather than subtracting one. Binary search reduces an interval, merge sort reduces segment lengths, and a tree traversal moves to proper subtrees. The relevant measure must match the representation. In a graph with cycles, moving to a neighbor does not necessarily reduce anything; a visited policy or a bounded search state is required.

Do not translate a recursive solution into an iterative one by keeping only the current index. The active call also remembers where it will resume, which choice is installed, and which alternatives remain. An explicit frame structure can represent those facts, but it must preserve the same state machine. Replacing the call stack is an implementation choice, not a substitute for defining the recurrence.

Recursive Tree Sum with a Clear Base Case

Purpose. A recursive solution is natural when the data itself is recursively defined and each call works on strictly smaller substructures.

```
#include <iostream>
struct Node{int v;Node*l{};Node*r{};};
int sum(const Node*n){if(!n)return 0;return n->v+sum(n->l)+sum(n->r);}int main()
{Node a{1},b{2},c{3};a.l=&b;a.r=&c;std::cout<<sum(&a)<<'\n';}
```

How it works.

- The null pointer is the base case and contributes zero.
- The recursive case reduces the problem to the two smaller subtrees.
- Each node contributes exactly once as recursive calls return.

Expected behavior/output. The program prints `6`.

Experiment. Add a chain of nodes and trace the call stack depth.

17.2 Exploring Search Spaces with Backtracking

Backtracking explores a decision tree with a repeated pattern: choose, record, recurse, undo. The undo step makes sibling branches independent. Without it, a failed or completed branch contaminates later choices and silently removes valid solutions.

17.2.1 Making and Reversing Choices

At row r , try each column c . A column occupancy bit rejects a reused column. One diagonal index is $r+c$; the other is $r+n-1-c$. For $n>0$, both lie between zero and $2n-2$, so arrays of length $2n-1$ are sufficient. The early $n=0$ return avoids unsigned underflow when computing that length.

For a legal choice, set its column and both diagonal bits, recurse to the next row, and clear those same three bits on return. The code relies on the legal-choice test to ensure those bits were previously false. Clearing a bit that belonged to an earlier queen would corrupt the path, which is why “record first, validate later” is not equivalent.

If a future extension performs throwing work after setting the bits, ordinary explicit cleanup after the recursive call would be skipped during unwinding. A scope guard or copied immutable state can provide rollback in that stronger model. The current bounded counter performs no dynamic allocation within the search after its arrays and callable are established, keeping the normal choose/undo mechanism easy to inspect.

Backtracking installs a tentative choice, explores the consequences, and removes that choice before considering the next alternative. The undo step is part of correctness. A board that retains a column mark from a completed sibling branch will reject valid future placements even though no real queen occupies that column in the current branch.

For the queens example, `column[c]` records whether an earlier row uses column c . The two diagonal arrays record $\text{row} + c$ and $\text{row} + n - 1 - c$. Each index identifies one family of parallel diagonals. With $0 \leq \text{row}, c < n$, both expressions are between zero and $2*n - 2$, which fits arrays of length $2*n - 1$. The early empty-board return avoids computing an inappropriate diagonal size for $n == 0$.

Before placing a queen, all three marks must be false. The algorithm sets them true together, calls the next row, then sets them false together. The invariant is that these arrays describe exactly the placements made by the active ancestor calls. They do not describe every placement ever visited during the search.

The supplied program counts solutions but does not store each board. A sequence of selected columns could be added for visualization. Push a column before descending and remove it afterward, keeping its length equal to the current depth. Storing every complete solution changes the output-size and memory contract even when the search order stays the same.

Cleanup during ordinary return is not automatically cleanup during every failure. The present recursive body performs bounded marking and counting without a user callback. Adding a callback that may throw, a cancellation exception, or another fallible operation between marking and unmarking requires deliberate restoration. A scope-bound undo object can preserve the invariant during unwinding. Alternatively, redesign the state so each recursive call receives its own independent copy and accept the copying cost. Neither choice should be hidden behind the word “recursive.”

17.2.2 Pruning Impossible Solutions

Pruning rejects partial choices that cannot be extended to a legal solution under the current constraints. A column conflict already violates the final requirement, so exploring descendants cannot repair it.

Rejecting that branch is therefore safe. A heuristic such as “this column looks unlikely” would need a different justification before it could discard solutions.

The search can still be exponential despite constant-time conflict checks. There are n choices at each row before constraints, and no repeated columns limits complete assignments to at most $n!$ candidates. Early diagonal pruning reduces actual work but does not turn the general search into a polynomial-time method.

Symmetry can reduce work further, but counts need careful weighting. Searching only half the first-row columns requires accounting for mirrored solutions, and an odd-sized board has a central case that cannot be handled by blindly doubling everything. Optimize only after the unpruned-by-symmetry reference is validated.

Pruning rejects a partial candidate when no completion of that candidate can satisfy the requirement. In the queens problem, an occupied column or diagonal proves that the proposed queen already conflicts with an earlier queen. Adding more queens cannot repair that conflict while earlier choices remain fixed. Skipping the branch is therefore sound.

A heuristic is different. Trying central columns first may change the order in which solutions are found, but it does not prove that other columns are impossible. A complete counting algorithm must still explore all legal alternatives unless a justified symmetry or equivalence rule accounts for the omitted work. Do not label an attractive guess as a correctness-preserving prune.

Separate soundness from completeness. Soundness says every counted board is valid. The marks enforce this locally, and the base case requires all rows to be assigned. Completeness says every valid board is eventually counted. Fixed row order plus iteration over every legal column supports that claim: the next column of any valid board is one of the alternatives considered at that level. Because each board has exactly one column choice per row, that construction also explains why it is counted once.

Symmetry reduction can be valuable, but its counting rule needs care. Reflecting a board can pair solutions, while some structures may map to themselves under a symmetry. A naive multiplication factor can overcount or undercount. Keep the straightforward reference search available before adding a symmetry-aware optimization, and compare exact counts for small boards where both are practical.

Pruning effectiveness depends on the instance and on the state representation. Testing all earlier placements for conflict would still be correct but would add work to each candidate. The three mark arrays summarize the same relevant information for constant-time conflict checks. That is a representation improvement separate from the combinatorial benefit of pruning entire branches.

Generating Binary Strings by Choose-Recurse-Unchoose

Purpose. Backtracking systematically explores a search tree while restoring state after each attempted branch.

```
#include <iostream>
#include <string>
void gen(std::string&cur,int n){if((int)cur.size()==n)
{std::cout<<cur<<'\n';return;}for(char c:{'0','1'})
{cur.push_back(c);gen(cur,n);cur.pop_back();}}
int main(){std::string s;gen(s,3);}
```

How it works.

- Each level chooses one candidate symbol.
- The recursive call explores every completion of that choice.
- `pop\_back` undoes the choice so the next branch starts from the previous state.

Expected behavior/output. Eight strings from `000` through `111` are printed.

Experiment. Add a rule that forbids consecutive `1` characters and prune branches as soon as the rule is violated.

Worked example — Count legal placements with reversible state

Requirement and design

Count all n -queen placements for n from zero through twelve using column and diagonal occupancy arrays. Reject larger inputs before computing or allocating the arrays. Check known small counts and the eight-queen count; preserve the full board's solution count rather than counting symmetry classes.

Read the search as a sequence of reversible states

Trace the first few calls for a four-row board on paper. Begin with no marks. Choose a legal column in row zero, record the affected column and diagonal indices, and move to row one. When no legal column remains, return to the caller and erase exactly the caller's most recent choice. Do not erase an ancestor's marks or leave a sibling's marks behind.

A compact trace can record (row, column, action) where action is place, undo, or complete. This is diagnostic data, not the result itself. Two correct implementations may visit candidates in different orders yet produce the same count. When a trace is used as a fixture, document whether traversal order is part of the public contract or merely the present implementation.

The domain limit is checked before the diagonal arrays are created. This makes an oversized request a deliberate error rather than an uncontrolled allocation or an unexpectedly long search. It also bounds the

intended counter range for the supplied workload. Raising the limit is not a cosmetic edit: review search time, recursion depth, array-index arithmetic, and the maximum representable solution count together.

Complete implementation

The complete, buildable chapter project is `source/book3/ch17/`. The following files are reproduced from that project; the publishing script inserts the source directly so the listing and companion remain synchronized.

main.cpp

```
#include "course_test.hpp"
#include <cstdint>
#include <functional>
#include <vector>

unsigned long long queens(std::size_t n) {
    if (n > 12) throw std::invalid_argument("teaching limit: n <= 12");
    if (n == 0) return 1; // One empty placement.
    std::vector<bool> column(n), descending(2 * n - 1), ascending(2 * n - 1);
    unsigned long long solutions{};
    std::function<void(std::size_t)> place = [&](std::size_t row) {
        if (row == n) { ++solutions; return; }
        for (std::size_t c = 0; c < n; ++c) {
            const auto d = row + c, a = row + n - 1 - c;
            if (column[c] || descending[d] || ascending[a]) continue;
            column[c] = descending[d] = ascending[a] = true;
            place(row + 1);
            column[c] = descending[d] = ascending[a] = false;
        }
    };
    place(0);
    return solutions;
}

int main() {
    CHECK(queens(0) == 1); CHECK(queens(1) == 1);
    CHECK(queens(2) == 0); CHECK(queens(3) == 0);
    CHECK(queens(4) == 2); CHECK(queens(8) == 92);
    CHECK(course::throws<std::invalid_argument>([] { (void)queens(13); }));
    course::report();
}
```

Build, run, and inspect

Run this block from the top-level extracted package. The fixture runner supplies input where needed and checks exit status and output in a temporary working directory.

```
cmake -S source/book3/ch17 -B build/b3-ch17 \
  -DCMAKE_BUILD_TYPE=Debug
cmake --build build/b3-ch17 --parallel 2
ctest --test-dir build/b3-ch17 --output-on-failure
```

For a multi-configuration generator on Windows, append `--config Debug` to the build command and `-C Debug` to CTest. The executable target is `b3_ch17`.

Evidence and interpretation

The checks distinguish several boundary behaviors: $n=0$ returns one empty placement; $n=1$ returns one; $n=2$ and $n=3$ return zero; $n=4$ returns two; and $n=8$ returns ninety-two. An input of thirteen throws the documented teaching-limit exception.

These counts are directly verified by the implemented bounded search in the test environment. They are not measurements of a separate optimized solver. The difference between zero solutions and an invalid input is preserved in the interface: an impossible but valid board returns zero, while an unsupported size is rejected.

Trace $n=4$ by writing the row, selected column, and occupied diagonals before and after one recursive call. The occupancy state after return must equal the state before choosing that row's queen. That local restoration condition is an effective debugging tool when total counts are unexpectedly low.

Failure clinic — A branch contaminates its sibling

Suppose the program reports too few solutions after a change that removes the unmarking statement. The first successful path can leave several columns unavailable to later alternatives. Looking only at the final count reveals the symptom but not the violated state transition.

Add a debug invariant at function entry and exit: the mark arrays must correspond to the active placement prefix, and the arrays at return must equal their values at entry. A small saved copy in a diagnostic build can check restoration. This is intentionally more expensive than the optimized mechanism; the purpose is to detect where state diverges.

Another failure appears when diagonal indices are calculated with signed arithmetic on one line and unsigned arithmetic on another. Derive the bounds from the accepted domain before selecting the expression. The supplied formula keeps the intermediate additions large enough that the final subtraction is nonnegative for valid row and column values. Moving operations around without preserving that reasoning can introduce underflow.

A search returning the right count once does not establish that it restores state correctly for repeated invocations with reused buffers. The chapter creates fresh state per call. A reusable solver should include repeated-run tests, changed-size tests, and cancellation tests if it exposes cancellation. Hidden state is a separate contract obligation.

Practice and implementation lab

Add counters for recursive calls and rejected column or diagonal checks without changing the returned solution count. Add a version that returns one placement as a vector of columns. Then design an early-exit path that correctly restores state even when a solution is found before all sibling choices are explored.

Review answers and solution notes

Instrumentation should not become part of the placement legality state. A one-solution search can return a boolean success and preserve a separate result vector. The unwind path must either retain an intentionally documented final state or undo every temporary placement; mixing those conventions is the defect to avoid. Check the returned columns for uniqueness and pairwise diagonal separation independently of the search's own occupancy arrays.

Engineering takeaway. Every recursive branch needs a progress measure, and every tentative mutation needs a defined restoration path.

Guided lab — Separate exploration from evidence

First reproduce the known counts already in the companion, including the empty board and the unsatisfiable two- and three-row boards. Add a version that stores the column chosen in each row for a four-row board. Validate each emitted arrangement with an independent pairwise checker rather than by reusing the same diagonal marks that generated it. For every pair of rows, the columns must differ and the absolute column difference must not equal the row difference.

Count emitted arrangements and compare that count with the original count-only function. Then check that no arrangement appears twice. These are three different properties: each result is valid, the number agrees, and outputs are unique. A bug shared by both generators could still escape, so retain known small counts and manually inspected examples.

Next record candidate checks, accepted placements, and completed boards. Define each counter before collecting it. Compare a conflict checker that scans earlier rows with one that uses the mark arrays. Both should enumerate the same boards under the same candidate order. Their candidate counts can match while their work per candidate differs. This is a useful bridge to Chapter 20's measurement discipline.

Finally introduce a callback that requests stopping after the first complete board. Choose whether stopping is expressed as a returned status or an exception. Specify whether the solver is reusable afterward. Restore all active choices before returning control, then call the solver again and verify the full count. This exercise makes cleanup a testable requirement rather than an assumption about normal return.

Review and solution reasoning

The recursion terminates because the number of unassigned rows decreases, not because the machine has a finite stack. Counting the empty arrangement as one follows the stated combinatorial convention. A conflicting partial placement can be pruned because retaining it cannot lead to a valid completion. Changing the order of legal alternatives alone does not change the solution set.

The crucial restoration property is local: after exploring one chosen column, the caller must see precisely the state it had before that choice. Proving and testing this property at each level makes the entire search understandable. A faster solver should preserve these contracts before receiving credit for reducing elapsed time.

Primary reference notes

Language mechanisms: [function execution](#), [object lifetime](#), and [lambda expressions](#). Further algorithm study: [MIT algorithm lecture directory](#). The bounded queens solver and its reasoning are original teaching material.

Chapter 18 — Solving Optimization Problems with Greedy Algorithms

A locally attractive choice can be either the key to a simple optimal algorithm or the beginning of a wrong answer. HarborWorks wants to select the largest number of nonoverlapping maintenance jobs and connect a set of sites at minimum total link cost. Both admit greedy solutions, but for different reasons.

This chapter gives the reason before celebrating the code. It develops an exchange argument for interval scheduling and a cut-based argument for a minimum spanning tree. It also shows why these proofs do not authorize greedy selection for every optimization problem.

A greedy algorithm commits to a locally attractive choice and does not revisit that commitment. This can be dramatically simpler than searching every combination, but simplicity is not evidence that the choice is valid. The missing ingredient is a reason that at least one globally optimal solution is compatible with the choice being made.

HarborWorks has two teaching requirements: accept as many nonoverlapping maintenance intervals as possible on one resource, and connect a set of sites using minimum total link weight. They sound similar because both ask for an optimum, yet their proofs use different structures. The first uses an exchange argument over finishing times. The second uses the cut property of spanning trees and a disjoint-set representation of connectivity.

18.1 Understanding Greedy Choice

A greedy algorithm commits to a choice without revisiting it later. That can avoid an enormous search space, but only when some optimal solution is guaranteed to contain a compatible greedy choice. Speed alone is not evidence of optimality.

18.1.1 Selecting Locally Optimal Decisions

A local rule uses the state currently visible: earliest finish time, smallest edge weight, or largest available denomination. The rule becomes an algorithm when it also specifies which choices remain feasible afterward and when it stops.

For example, picking the largest coin first for amount six with denominations 1,3,4 produces $4+1+1$, three coins. The optimal solution is $3+3$, two coins. The locally largest reduction leaves an awkward remainder. This counterexample disproves a universal greedy coin rule even though the rule succeeds for some denomination systems.

Write a small counterexample search before trusting a plausible heuristic. Exhaustively compare the heuristic with a correct bounded solver on small inputs. A found counterexample decisively rejects a universal claim; failure to find one is not a proof that none exists.

A local decision is defined relative to the current partial solution. For interval scheduling, the candidate with the earliest finishing time leaves as much of the remaining time axis available as possible. For

Kruskal's spanning-tree algorithm, the next eligible edge has the smallest weight among the edges not yet considered and joins two currently separate components.

The objective must be explicit before the word "best" has meaning. Maximizing the number of accepted intervals differs from maximizing their total payment, minimizing their total duration, or prioritizing a particular customer. The supplied scheduling function solves the unweighted maximum-cardinality problem. Adding a payment field does not make earliest finish a correct weighted scheduler.

A simple counterexample exposes the distinction. Suppose one long interval has a payment of one hundred, while two compatible short intervals each have a payment of one. Earliest finish can correctly choose two intervals for the count objective and incorrectly choose them for the payment objective. The algorithm did not suddenly become unreliable; the requested problem changed.

Greedy commitment also requires a precise feasibility test. Intervals in this chapter are half-open: `[start, finish)`. An interval beginning exactly when the previous one finishes is compatible. If a real maintenance task requires a cooldown or setup period, the compatibility rule must include it. Sorting the same records differently cannot repair a missing constraint.

The implementation uses an optional previous finish rather than inventing a numeric sentinel such as zero. Negative time coordinates are valid in the teaching model, so a zero sentinel would incorrectly reject intervals before the origin. Absence is a state, not necessarily a special value inside the data domain. This repeats the type-modeling discipline from Book II in an algorithmic setting.

18.1.2 Recognizing Greedy-Solvable Problems

A useful proof pattern is exchange: take an optimal solution and show how to replace one of its choices with the greedy choice without making the objective worse or destroying feasibility. Then solve the smaller remaining problem using the same argument.

For maximum-cardinality interval scheduling, choosing the compatible interval that finishes first leaves at least as much future time as any other first interval. Replace the first interval in an optimal schedule with that earliest-finishing one. Every later interval that followed the old first interval can still follow the replacement, so cardinality does not decrease.

The objective is essential. If jobs have different profits and the goal is maximum total profit, earliest finish need not be optimal. If jobs require setup gaps or multiple resources, feasibility changes. A proof belongs to its exact objective and constraints, not to the visual shape of the input table.

An exchange argument starts with an optimal solution and shows that its first decision can be replaced by the greedy choice without making the solution worse. Let the greedy interval finish no later than the first interval in an optimal schedule. Replacing that first interval with the greedy one cannot overlap a later interval that was compatible with the old first interval, because the replacement ends no later. The number of accepted intervals remains unchanged.

After making that replacement, the remaining problem has the same form: select the maximum number of intervals beginning at or after the chosen finish. Repeating the argument supports the full algorithm. The proof does not depend on a particular input order because the algorithm first orders candidates by finishing time.

Notice what the argument does not say. It does not say that the greedy choice appears in every optimum, nor that the optimal schedule is unique. Equal finishing times can admit several equally good choices. The companion breaks ties by start time to make its selection procedure explicit, but the cardinality proof is about feasibility and count, not about a unique identity sequence.

For another problem, search for the structural property that permits such an exchange. Some greedy algorithms rely on cut properties, others on prefix dominance, and some familiar-looking problems do not admit the proposed greedy rule at all. A few successful examples are a poor substitute for this step.

Use counterexample search as a practical companion to proof. Enumerate small interval sets, compare the greedy count with exhaustive subset selection, and inspect any disagreement. Exhaustive testing over a bounded universe does not prove the general theorem, but it can quickly reveal that a proposed rule such as “shortest duration first” or “earliest start first” lacks the required property. Treat a counterexample as a refinement of the specification and reasoning, not merely an inconvenient test.

Earliest-Finish-Time Interval Selection

Purpose. Greedy algorithms commit to locally optimal choices; correctness depends on a problem-specific exchange or dominance argument.

```
#include <algorithm>
#include <iostream>
#include <vector>
struct Job{int start,end;};
int main(){std::vector<Job>j{{0,6},{1,4},{3,5},{5,7},{5,9},
{8,9}};std::sort(j.begin(),j.end(),[](auto a,auto b){return a.end<b.end;});int
last=-1,count=0;for(auto x:j)if(x.start>=last){++count;last=x.end;}
std::cout<<count<<'\n';}
```

How it works.

- Jobs are ordered by earliest finish time.
- Each compatible job is accepted immediately and moves the availability boundary.
- The greedy choice leaves as much room as possible for later jobs.

Expected behavior/output. For this set, the algorithm selects a maximum-size compatible subset and prints its count.

Experiment. Sort by shortest duration instead and find an input where that alternative greedy rule performs worse.

18.2 Applying Greedy Techniques

The two implementations operate on copied input vectors, sort by a deterministic priority, and accept only choices preserving a feasibility invariant. They differ in that interval compatibility is summarized by one finish time, while graph connectivity requires a disjoint-set structure.

18.2.1 Solving Scheduling Problems

Represent jobs as half-open intervals $[start, finish)$ with $start < finish$. Half-open semantics permit one job to start exactly when the previous job finishes. Sort by finish time, using start as a deterministic tie breaker. Accept a job when no earlier job has been accepted or its start is at least the last accepted finish.

The implementation uses an optional last finish rather than initializing it to zero. Negative timestamps are valid under the contract, so zero would incorrectly reject some early jobs. The validation pass rejects empty or backward intervals instead of trying to infer their meaning.

Sorting dominates at $O(n \log n)$; scanning afterward is $O(n)$. The output vector stores the selected schedule. The exchange argument applies repeatedly after every accepted job because the remaining compatible jobs form the same kind of problem. The code returns one optimal-cardinality schedule, not every optimal schedule.

The supplied scheduler validates each interval before sorting. An interval must satisfy `start < finish`; zero-duration and reversed intervals are outside this contract. Another application might accept instantaneous events, but it would need a separate definition of compatibility and counting. Rejecting them here keeps the model unambiguous.

Sorting by $(finish, start)$ costs $O(n \log n)$ comparisons under the usual random-access model. The subsequent scan is linear. A statement that greedy scheduling is “linear” is therefore incomplete unless the input is already sorted under the required order or sorting is accounted for elsewhere. If a service repeatedly receives an ordered stream, maintaining that order is itself an interface requirement.

The accepted schedule is constructed incrementally. Once an interval is accepted, later candidates must start no earlier than its finish. Because candidates are processed by nondecreasing finish, replacing the current accepted interval with a later-finishing one would not improve the remaining feasible region for the unweighted objective.

The chapter uses integer coordinates as abstract time. They are not automatically dates, clock readings, or durations with units. A production scheduler must resolve time zones, daylight-saving transitions, resource identities, reservations, and simultaneous changes. Those issues do not invalidate the algorithm; they determine whether the application has actually produced the interval model the algorithm assumes.

An acceptance test should verify both the optimum count and the feasibility of the returned schedule. A result containing three overlapping intervals is not redeemed by matching the expected count three. A result with a valid but smaller count exposes a different failure. When identity matters, retain stable record identifiers through sorting so the report can refer to the original requests rather than only to their positions in a reordered vector.

18.2.2 Applying Greedy Graph Algorithms

Kruskal’s minimum-spanning-tree algorithm sorts undirected edges by weight and accepts an edge only when its endpoints belong to different current components. A disjoint-set structure answers that connectivity question. Path compression shortens future searches, while union by rank prevents needlessly tall representative trees.

The maintained forest has no cycles: accepting an edge between different components joins two trees. For correctness, consider the cut separating one current component from the others. A lightest eligible crossing edge can be exchanged into an appropriate minimum spanning tree without increasing total weight; removing an edge from the resulting cycle preserves a spanning tree. This safe-edge principle supports the greedy construction.

A connected graph with V vertices needs $V-1$ accepted edges. If the scan cannot reach that count, the function returns absence rather than reporting a spanning tree for a disconnected graph. Negative weights are permitted; avoiding cycles, not avoiding negative numbers, is the relevant condition here. Self-loops do not join different components and are ignored by union failure.

The implementation bounds vertices at one million and edge weights to plus or minus one billion. At most $V-1$ accepted weights are accumulated into `long long`, keeping the total within its guaranteed range on the teaching baseline. Edge endpoints are validated before entering the disjoint-set operations. The internal disjoint-set methods assume valid indices; they are not a general checked public service.

A spanning tree connects all vertices of an undirected graph without cycles. For V vertices, a tree has $V - 1$ edges when V is positive. A minimum spanning tree minimizes the sum of selected edge weights among all such trees. It does not minimize every pairwise route, and it is not a shortest-path tree from a chosen source.

Kruskal's algorithm processes edges in nondecreasing weight and accepts an edge when its endpoints currently belong to different components. The disjoint-set structure answers this connectivity question. Accepting an edge merges components; rejecting an edge prevents a cycle. Path compression and rank improve the repeated set operations, while sorting remains a major part of the overall cost.

The cut property explains the greedy step. Consider the cut separating one current component from the others. A minimum-weight edge crossing a suitable cut can be included in some minimum spanning tree. If an optimal tree does not contain that edge, adding it creates a cycle containing another crossing edge; exchanging the heavier or equal crossing edge preserves connectivity without increasing weight. Ties can produce multiple optimal trees.

Negative edge weights are acceptable for a minimum spanning tree. The objective sums a finite acyclic selection, so the negative-cycle concern associated with some shortest-path problems does not apply in the same way. Self-loops never join different components and are rejected by the union test. Parallel edges are candidates; a heavier one may become unnecessary after a cheaper connection is accepted.

The companion returns an optional total. A disconnected positive-size graph has no spanning tree, so absence differs from a valid total of zero. The teaching contract defines empty and single-vertex graphs to have total zero. Vertex and weight bounds keep the accumulation within `long long`; changing those bounds requires rechecking arithmetic as well as connectivity.

Dijkstra-Style Greedy Relaxation on Nonnegative Edges

Purpose. This applies greedy selection to shortest paths: repeatedly commit to the smallest frontier distance under the nonnegative-edge assumption.

```
#include <iostream>
#include <limits>
#include <queue>
#include <utility>
#include <vector>
int main(){using
P=std::pair<int,int>;std::vector<std::vector<P>>>g(4);g[0]={{1,2},
{2,5}};g[1]={{2,1},{3,5}};g[2]={{3,1}};const int
INF=std::numeric_limits<int>::max()/
4;std::vector<int>d(4,INF);std::priority_queue<P,std::vector<P>,std::greater<P>>>q;d[0]=0;q.push(
q.empty()){auto [du,u]=q.top();q.pop();if(du!=d[u])continue;for(auto
[v,w]:g[u])if(du+w<d[v]){d[v]=du+w;q.push({d[v],v});}}std::cout<<d[3]<<'\\n';}
```

How it works.

- The priority queue selects the unsettled vertex with smallest known distance.
- Relaxation improves neighboring distance estimates.
- The nonnegative-weight condition is essential to the greedy finalization logic.

Expected behavior/output. The shortest distance from 0 to 3 is `4`.

Experiment. Introduce a negative edge and explain why the algorithm's correctness argument no longer applies.

Worked example — Select a schedule and build a minimum-cost connection forest

Requirement and design

For interval scheduling, maximize the number of nonoverlapping positive-length half-open jobs. For an undirected weighted graph, return the weight of a minimum spanning tree or absence when no spanning tree exists. Make empty, singleton, negative-time, negative-weight, and disconnected cases explicit.

Make the certificate visible before optimizing

For scheduling, a useful certificate is the selected interval sequence. Check that every selected record belongs to the input and each accepted start is at least the preceding finish. For a spanning tree, a useful certificate is the selected edge list. Check endpoint validity, edge membership, acyclicity, connectivity, edge count, and total weight.

The compact worked example returns only the quantities needed for its chapter tests. That keeps the implementation focused, but it also means the caller cannot independently inspect a selected tree from the

return value alone. An extension that returns selected edge identities is a meaningful interface improvement, not just extra logging.

When comparing results, distinguish optimum value from exact solution identity. Two equally weighted spanning trees can both be correct. An exact edge-list fixture would be appropriate only if deterministic tie behavior were explicitly required and implemented. Otherwise compare the objective and validate the certificate.

The disjoint-set helper assumes the caller supplies valid indices. The public spanning-tree function validates graph endpoints first. This division avoids repeated checks in the inner mechanism, but it is safe only while the helper remains behind that validated boundary. Moving it into a public library requires documenting or enforcing its own domain.

Complete implementation

The complete, buildable chapter project is `source/book3/ch18/`. The following files are reproduced from that project; the publishing script inserts the source directly so the listing and companion remain synchronized.

main.cpp

```
#include "course_test.hpp"
#include <algorithm>
#include <numeric>
#include <optional>
#include <tuple>
#include <vector>
#include <utility>

struct Interval { int start, finish; };
std::vector<Interval> schedule(std::vector<Interval> jobs) {
    for (const auto job : jobs) if (job.start >= job.finish) throw
std::invalid_argument("interval");
    std::sort(jobs.begin(), jobs.end(), [](const auto& a, const auto& b) {
        return std::tie(a.finish, a.start) < std::tie(b.finish, b.start);
    });
    std::vector<Interval> result;
    std::optional<int> last_finish;
    for (const auto job : jobs) {
        if (!last_finish || job.start >= *last_finish) {
            result.push_back(job); last_finish = job.finish;
        }
    }
    return result;
}

class DisjointSet {
    std::vector<std::size_t> parent_, rank_;
public:
    explicit DisjointSet(std::size_t n) : parent_(n), rank_(n) {
        std::iota(parent_.begin(), parent_.end(), 0);
    }
    std::size_t find(std::size_t x) {
        if (parent_[x] != x) parent_[x] = find(parent_[x]);
        return parent_[x];
    }
    bool unite(std::size_t a, std::size_t b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (rank_[a] < rank_[b]) std::swap(a, b);
```

```

        parent_[b] = a;
        if (rank_[a] == rank_[b]) ++rank_[a];
        return true;
    }
};

struct Edge { std::size_t from, to; int weight; };
std::optional<long long> spanning_weight(std::size_t vertices, std::vector<Edge>
edges) {
    if (vertices > 1000000) throw std::invalid_argument("teaching size limit");
    for (const auto edge : edges) {
        if (edge.from >= vertices || edge.to >= vertices) throw
std::out_of_range("edge");
        if (edge.weight < -1000000000 || edge.weight > 1000000000) {
            throw std::invalid_argument("weight outside teaching range");
        }
    }
    if (vertices == 0) return 0;
    std::sort(edges.begin(), edges.end(), [](const auto& a, const auto& b) {
        return std::tie(a.weight, a.from, a.to) < std::tie(b.weight, b.from, b.to);
    });
    DisjointSet components{vertices};
    long long sum{}; std::size_t accepted{};
    for (const auto edge : edges) {
        if (components.unite(edge.from, edge.to)) {
            sum += edge.weight; ++accepted;
            if (accepted == vertices - 1) break;
        }
    }
    if (accepted != vertices - 1) return std::nullopt;
    return sum;
}

int main() {
    const auto selected = schedule({{0, 6}, {1, 2}, {2, 4}, {4, 5}});
    CHECK(selected.size() == 3);
    CHECK(schedule({{-5, -3}, {-3, -1}}).size() == 2);
    CHECK(spanning_weight(4, {{0, 1, 1}, {1, 2, 2}, {2, 3, 3}, {0, 3, 20}}) == 6);
    CHECK(!spanning_weight(3, {{0, 1, 1}}));
    CHECK(spanning_weight(1, {}) == 0);
    CHECK(spanning_weight(0, {}) == 0);
    CHECK(spanning_weight(2, {{0, 1, -3}}) == -3);
    course::report();
}

```

Build, run, and inspect

Run this block from the top-level extracted package. The fixture runner supplies input where needed and checks exit status and output in a temporary working directory.

```

cmake -S source/book3/ch18 -B build/b3-ch18 \
  -DCMAKE_BUILD_TYPE=Debug
cmake --build build/b3-ch18 --parallel 2
ctest --test-dir build/b3-ch18 --output-on-failure

```

For a multi-configuration generator on Windows, append `--config Debug` to the build command and `-C Debug` to CTest. The executable target is `b3_ch18`.

Evidence and interpretation

The schedule fixture contrasts a long interval 0–6 with three compatible shorter jobs 1–2, 2–4, and 4–5. The result contains three jobs. A negative-time fixture verifies that the optional initial state does not impose an artificial zero boundary.

The four-vertex graph accepts edges of weights one, two, and three for a total of six instead of using the weight-twenty alternative. A disconnected graph reports absence. Empty and singleton graphs return zero under the documented convention; a two-vertex graph with one negative edge returns its negative weight.

The tests demonstrate the contract on selected fixtures. The exchange and cut arguments justify the optimization rule. A production result that must display the actual links should return accepted edge identities as well as total weight, since a scalar total cannot be independently audited as a topology.

Failure clinic — Solving the wrong optimization problem

An engineer replaces earliest finish with shortest duration because short jobs “use fewer resources.” The sample still returns the expected count, so the change is accepted. A later input contains one short interval in the middle that overlaps two mutually compatible longer intervals, and the revised rule loses a job. The mistake is not in sorting syntax; it is in replacing a proved choice rule with an intuition.

A similar mismatch occurs when a minimum spanning tree is used to answer shortest-route queries. Connecting all sites cheaply may force a particular pair to travel along a long detour. The spanning-tree objective is global construction cost. A route query has a source, destination, and path-cost objective. The same graph can support both problems without making their outputs interchangeable.

Tie handling can also create misleading tests. A changed edge order may produce a different optimal tree with the same total. Before calling this a regression, determine whether the application required a specific tree or merely an optimum cost. Conversely, matching a total is not enough when the output edges contain a cycle and fail to connect an isolated vertex.

Keep the requirement, feasibility checker, and objective calculation separate in your test design. A solver should not be its own only judge.

Practice and implementation lab

Add a weighted interval counterexample where earliest finish does not maximize profit. Extend the spanning result to return its accepted edges, then independently check connectivity, acyclicity, and the sum. Add parallel edges and a self-loop to the graph fixtures. Compare tiny graph results with exhaustive spanning-tree enumeration as an additional oracle.

Review answers and solution notes

A short low-profit interval can block a longer high-profit interval, so the cardinality proof cannot establish maximum profit. For a returned tree, verify exactly $V-1$ edges for nonempty connected input, reachability from one vertex, and the declared total. Parallel edges are acceptable; the cheaper useful

connection may be selected. A self-loop cannot help connect components and must not contribute to the tree.

Engineering takeaway. Greedy decisions are justified by an exchange or safe-choice proof tied to the exact objective.

Guided lab — Challenge the greedy rule with exhaustive evidence

Build a small scheduling oracle that enumerates every subset of at most ten intervals. For each subset, sort its selected intervals by start time, reject overlaps under the half-open convention, and keep the largest valid count. Generate bounded interval sets with negative coordinates, equal finishes, nesting, touching endpoints, and duplicates. Compare the greedy count with the oracle.

Then substitute earliest start and shortest duration deliberately in your experimental copy. Keep searching until you obtain a counterexample for each proposed rule. Write the counterexample as a short explanation of which local choice prevented a better global solution. Do not alter the reference implementation in the distributed source.

For the graph problem, extend the return type to retain selected edge indices. On tiny connected graphs, enumerate subsets of exactly $V - 1$ input edges, validate each candidate independently, and compare the minimum total with Kruskal's result. Keep the universe small because the number of subsets grows rapidly. Include a disconnected graph and verify that no candidate is accepted.

Add a negative-weight edge, parallel equal-weight edges, and a self-loop. Predict which properties change and which do not. Do not require one exact edge set when ties permit alternatives. If deterministic output is needed, define stable input identities and a total tie order, then test that order separately from optimality.

Review and solution reasoning

A greedy algorithm is justified by a structural argument, not by the adjective “efficient.” Earliest finish is correct for the stated unweighted interval objective because an optimal first interval can be exchanged without reducing the number of remaining choices. Kruskal's method uses connectivity and the cut property, not the interval proof.

An absent spanning-tree result signals disconnection under the contract; zero can be a valid cost. Negative weights do not automatically invalidate spanning-tree algorithms. Finally, sorting and validation belong in the end-to-end cost even when the decision scan itself is linear. A professional report states both the solved problem and the work included in its measurement.

Primary reference notes

Library operations: [sorting](#), [tuple comparison](#), and [optional results](#). Further graph and optimization study: [MIT algorithm lecture directory](#).

Chapter 19 — Solving Repeated Subproblems with Dynamic Programming

The coin counterexample from Chapter 18 points to a different strategy: compare the best achievable result for each remaining amount instead of committing to the first attractive denomination. Many branches ask the same smaller question. Computing each answer once can replace repeated search with a table of solved states.

This chapter implements minimum coin count both top-down with memoization and bottom-up with reconstruction. The two implementations use different state conventions and are compared across the same bounded input domain. Neither relies on a denomination-specific greedy assumption.

Backtracking explores alternatives and greedy methods commit when a structural argument permits commitment. Dynamic programming takes another route: identify subproblems whose answers recur, compute each relevant answer once, and combine those answers according to a recurrence. The table is not the idea by itself. The idea is a state that contains enough information to make future decisions independent of the path used to reach it.

HarborWorks uses a bounded change-making problem to study this discipline. Given positive integer denominations and a nonnegative target, find the minimum number of coins that reaches the target exactly. Coins can be reused without limit. That last sentence is essential: a finite inventory of each coin would require a different state and transition model.

19.1 Recognizing Dynamic-Programming Structure

Dynamic programming is appropriate when subproblems repeat and an optimal larger solution can be built from optimal compatible smaller solutions. Naming the state precisely is more important than deciding whether the implementation uses recursion or loops.

19.1.1 Identifying Overlapping Subproblems

For unlimited positive denominations and target amount a , let $F(a)$ be the minimum number of coins needed to make exactly a , or impossible if no combination exists. Different first choices can lead to the same remaining amount. Without caching, a recursion tree repeatedly solves that identical state.

The state is sufficient only because coin supply is unlimited and the objective depends on the remaining amount, not the order of earlier choices. With limited inventory, $F(a)$ alone forgets how many coins remain. The state would need an inventory or denomination-position dimension, and reusing the old recurrence would solve the wrong problem.

The number of distinct states here is amount+1, from zero through the target. A recursion tree may contain many more calls than distinct states. Memoization is valuable because it changes repeated state evaluation into a lookup, not because recursion itself is slow or a table is automatically faster.

Repeated subproblems appear when different decision paths reach the same remaining requirement. With denominations one, three, and four, a search for amount six can reach amount two after choosing four, or after choosing one and three in a different order. If only the remaining amount affects future choices, solving amount two again does not add information.

Memoization records a solved state so later callers can reuse it. This is valid only when the state's answer is stable under the inputs not represented in the key. In the chapter's function, the denomination collection remains fixed for the whole computation. Caching only by amount across calls with different denomination collections would be incorrect unless the cache were cleared or the collection were part of the cache identity.

Distinguish uncomputed from computed-and-impossible. An impossible amount is still a completed answer and should not trigger the same futile search repeatedly. The compact implementation uses separate integer sentinels for these states and zero for the base amount. Those sentinels are implementation details; the public interface uses absence to represent no exact solution.

The number of distinct states is not the number of recursion paths. For target A , there are at most $A + 1$ nonnegative remaining amounts in this model. Many possible coin sequences can lead to those same states. That difference is where the reduction from repeated exponential exploration to a bounded state computation comes from.

Not every recurrence benefits equally. A recursion that creates disjoint subproblems without overlap may gain little from a memo table and pay extra storage costs. Inspect which arguments actually repeat before adding caching. Also account for key construction and lookup costs when the state is a string, a subset mask, or a composite object rather than a small integer index.

19.1.2 Identifying Optimal Substructure

The base value is $F(0)=0$. For positive a , try every denomination c not exceeding a . When $F(a-c)$ is possible, it gives a candidate $F(a-c)+1$. The minimum candidate is the answer; no candidate means impossible.

To justify optimal substructure, take an optimal nonempty combination and remove one chosen coin c . If its remainder were not optimally solved, replacing that remainder by a better solution would improve the original combination, contradicting optimality. Conversely, every candidate is feasible because it adds a valid coin to a feasible remainder.

Positive denominations ensure $a-c < a$, making the dependency graph acyclic. A zero denomination would recurse into the same unresolved amount, and a negative denomination could increase it. Validation is therefore part of termination and correctness, not only user-interface friendliness.

Optimal substructure means an optimal solution can be described using optimal solutions of appropriate smaller states. Suppose an optimal exact solution for amount a ends with denomination c . The preceding coins must form an optimal solution for $a - c$; otherwise replacing them with a smaller valid set would improve the alleged optimum for a .

This yields the recurrence: the answer for zero is zero, and the answer for a positive amount is one plus the minimum reachable answer among $a - c$ for denominations that fit. If no predecessor is reachable, the amount is impossible. The word reachable prevents a sentinel for impossibility from being treated as a valid low count.

The state must preserve every constraint affecting future choices. If only one coin of each physical item may be used, remaining amount alone is insufficient because two paths can leave different inventories. A state might then include the next available item index or a bounded inventory description. Reusing the unbounded-coin recurrence would silently allow spending an item more than once.

Optimal substructure is not permission to take the largest denomination greedily. For denominations one, three, and four, amount six is best represented by two threes. Taking four first leaves two ones and uses three coins. Dynamic programming considers each valid last choice and compares the resulting totals rather than assuming that the locally largest coin must belong to an optimum.

The proof also identifies what a reconstruction witness must satisfy. A reported count is not enough: the returned coins must all be allowed denominations, sum exactly to the target, and have length equal to the count. Optimality then needs the recurrence argument or an independent oracle on a bounded domain. These checks diagnose different defects and should remain separate in the test suite.

Memoized Fibonacci Showing Overlapping Subproblems

Purpose. Dynamic programming applies when subproblems overlap and their solutions can be reused systematically.

```
#include <iostream>
#include <vector>
long fib(int n, std::vector<long>&memo) { if (n < 2) return n; if (memo[n] != -1) return
memo[n]; return memo[n] = fib(n-1, memo) + fib(n-2, memo); } int main()
{ std::vector<long> m(41, -1); std::cout << fib(40, m) << '\n'; }
```

How it works.

- Naive recursion repeatedly asks for the same smaller Fibonacci values.
- The memo table stores each solved subproblem once.
- The recurrence has optimal substructure because a solution is composed from solutions to smaller indices.

Expected behavior/output. The program prints `102334155`.

Experiment. Count recursive calls with and without memoization for $n=30$.

19.2 Implementing Dynamic Programming

Top-down and bottom-up implementations evaluate the same recurrence in different orders. Their sentinel values must distinguish uncomputed, impossible, and a real count. Accidentally conflating those states can make an impossible subproblem look like a free solution.

19.2.1 Solving Top-Down with Memoization

The memoized function initializes entries to -2 for uncomputed, uses -1 for impossible, and stores nonnegative counts for solved states. It sets amount zero to zero. When evaluating a state, it examines smaller remainders and retains the best feasible count.

Setting a state initially to impossible before its loop is safe here because every recursive dependency has a strictly smaller amount. There is no cycle that can revisit an in-progress state under validated positive denominations. This shortcut would not generalize to an arbitrary cyclic dependency graph.

The maximum accepted amount is five hundred. This deliberately bounds recursion depth, table size, and count arithmetic for the teaching example. Top-down evaluation can avoid states that no denomination path reaches, but its call stack and callable overhead remain real implementation costs. A larger target range may favor the iterative version even when asymptotic bounds agree.

Top-down memoization preserves the recursive shape of the problem. A request for an amount first checks whether the answer is already stored. If not, it asks for smaller reachable amounts, chooses the best result, records it, and returns. Only states reached through these calls are evaluated.

The supplied function requires every denomination to be positive. This is both a domain choice and a termination condition. A zero denomination would recursively request the same amount without progress before its result had been stored. A negative denomination could increase the remaining amount and leave the allocated table. Rejecting these inputs before recursion is more reliable than hoping the memoization machinery will rescue them.

With a denomination of one, a recursive chain can have depth proportional to the target amount. The companion deliberately limits the target to five hundred. That bound keeps the teaching stack depth and table size modest; it is not an assertion that every platform has the same safe recursion limit. A larger application may favor bottom-up iteration even when the recurrence is unchanged.

Failure caching requires care with side effects. This example's result depends only on fixed denominations and an integer amount. A subproblem that performs a network request or reads changing external state is not made pure merely by putting its return value in a map. Decide which observations can be reused and for how long before applying memoization outside a mathematical kernel.

The usual time bound here is $O(A \cdot K)$ for K denominations, because each of at most $A + 1$ states considers the denomination collection. It is a bound in the numeric value of the amount, not merely in the number of bits used to encode it. Calling this pseudo-polynomial makes that distinction explicit: doubling the bit length can allow a vastly larger table even though the input representation grew modestly.

19.2.2 Solving Bottom-Up with Tabulation

The tabulated version fills amounts from one upward, so every dependency at value-coin is already solved. It uses amount+1 as a finite impossible sentinel: with positive integer coins, any feasible minimum count is at most amount. Adding one to a sentinel stays safely representable within the bounded domain and cannot improve a legitimate current answer.

Alongside the best count, store the denomination that produced an improvement. After solving the target, repeatedly append its chosen coin and subtract it from the remaining value. Each step strictly decreases

the value, and following the stored decisions reconstructs a combination whose length equals the optimum count.

If the target remains impossible, return `nullopt` before reconstruction. For target zero, return a solution with count zero and an empty coin vector. An empty denomination list therefore solves zero but no positive amount. Equal-quality candidates retain the first one encountered because the update requires strict improvement; the interface promises an optimal count, not a unique canonical coin ordering.

With k denominations and target A , both implementations use $O(kA)$ worst-case state-transition work and $O(A)$ table storage. Top-down adds up to $O(A)$ recursive stack depth; reconstruction uses $O(A)$ output in the worst case. The dependence on numeric A makes this a pseudo-polynomial bound relative to the number of bits used to encode the target.

Bottom-up tabulation chooses an order in which every dependency is available before a state is computed. Since all denominations are positive, each valid predecessor $a - c$ is smaller than a . Processing amounts from one through the target therefore respects the dependency graph. The iteration order is justified by the recurrence, not by a general rule that every dynamic program should count upward.

The companion stores both the best count and the denomination chosen for each improved state. After computing the target, reconstruction repeatedly subtracts the recorded positive denomination until reaching zero. Each subtraction makes progress, and the number of emitted coins should match the stored optimum count. Missing or invalid predecessor choices indicate a reconstruction bug even when the count table itself is correct.

When two denominations yield the same count, the code keeps the first strict improvement. This produces one optimum influenced by the denomination iteration order. The public goal is minimum count, not a unique lexicographically ordered coin sequence. Tests should not demand an exact witness across reordered inputs unless a tie-breaking policy has been added deliberately.

Storage can sometimes be reduced, but reconstruction changes the tradeoff. A rolling table sufficient for a numerical answer may discard the decisions needed to recover the actual solution. Before optimizing memory, ask whether the caller needs only an optimum value, one witness, all witnesses, or a count of optima. Those outputs can require different retained information.

The bounded integer sentinel is selected so adding one to a reachable count cannot overflow under the target limit. More general shortest-path-like recurrences must guard arithmetic around infinity values. The safe pattern is to test reachability first and add only to a valid finite cost. An arbitrary large number is not mathematical infinity and does not suspend the language's arithmetic rules.

Bottom-Up 0/1 Knapsack Table

Purpose. A bottom-up DP replaces recursive recomputation with an explicit state table whose meaning is defined by the recurrence.

```
#include <algorithm>
#include <iostream>
#include <vector>
int main(){std::vector<int>w{2,3,4},val{4,5,10};int
cap=6;std::vector<int>dp(cap+1);for(std::size_t i=0;i<w.size();++i)for(int
c=cap;c>=w[i];--c)dp[c]=std::max(dp[c],dp[c-w[i]]
+val[i]);std::cout<<dp[cap]<<'\n';}
```

How it works.

- `dp[c]` stores the best value achievable within capacity `c`.
- Capacity iterates downward so each item is used at most once.
- Each update chooses between excluding the item and including it.

Expected behavior/output. The program prints the best value for capacity 6, which is `14`.

Experiment. Iterate capacity upward and explain why that changes the problem toward unbounded reuse of an item.

Worked example — Compute an optimum and reconstruct evidence for it

Requirement and design

For positive unlimited coin denominations and amounts from zero through five hundred, compute the minimum count or impossibility. Provide independent top-down and bottom-up implementations. The bottom-up result must include an actual coin combination that sums to the target and has the reported length.

Derive a table before reading the implementation

For denominations one, three, and four, fill amounts zero through six by hand. Record zero coins for zero. For each later amount, list the fitting last denominations and the already known predecessor counts. Amount four can use one coin, while amount six compares the predecessor for three against the predecessor for four and chooses two coins through a three.

Add a second column containing the chosen last denomination. Follow it backward from six to zero. This makes the difference between computing an optimum value and recovering a witness concrete. Then repeat with denominations two and four and target three. No fitting chain reaches zero, so the result must remain absent rather than becoming a huge sentinel count in the public output.

The worked example compares memoized and tabulated answers over many small inputs. That is useful differential evidence because the evaluation orders differ. However, both implementations use the same mathematical recurrence. A mistaken model shared by both can still pass the comparison. The supplemental laboratory therefore includes a small independent breadth-first state-space oracle as an additional check.

Complete implementation

The complete, buildable chapter project is `source/book3/ch19/`. The following files are reproduced from that project; the publishing script inserts the source directly so the listing and companion remain synchronized.

main.cpp

```
#include "course_test.hpp"
#include <algorithm>
#include <functional>
#include <numeric>
#include <optional>
#include <vector>

void validate(const std::vector<int>& coins, int amount) {
    if (amount < 0 || amount > 500) throw std::invalid_argument("amount 0..500");
    for (int coin : coins) if (coin <= 0) throw std::invalid_argument("positive coins
required");
}

int memoized_count(const std::vector<int>& coins, int amount) {
    validate(coins, amount);
    std::vector<int> memo(static_cast<std::size_t>(amount) + 1, -2);
    memo[0] = 0;
    std::function<int(int)> solve = [&](int remaining) {
        int& result = memo[static_cast<std::size_t>(remaining)];
        if (result != -2) return result;
        result = -1;
        for (int coin : coins) {
            if (coin > remaining) continue;
            const int previous = solve(remaining - coin);
            if (previous != -1 && (result == -1 || previous + 1 < result)) result =
previous + 1;
        }
        return result;
    };
    return solve(amount);
}

struct Solution { int count; std::vector<int> coins; };
std::optional<Solution> tabulated(const std::vector<int>& coins, int amount) {
    validate(coins, amount);
    std::vector<int> best(static_cast<std::size_t>(amount) + 1, amount + 1);
    std::vector<int> chosen(best.size(), -1);
    best[0] = 0;
    for (int value = 1; value <= amount; ++value) {
        for (int coin : coins) {
            if (coin <= value && best[static_cast<std::size_t>(value - coin)] + 1 <
best[static_cast<std::size_t>(value)]) {
                best[static_cast<std::size_t>(value)] =
best[static_cast<std::size_t>(value - coin)] + 1;
                chosen[static_cast<std::size_t>(value)] = coin;
            }
        }
    }
}
```

```

    if (best.back() > amount) return std::nullopt;
    Solution result{best.back(), {}};
    for (int value = amount; value > 0;) {
        const int coin = chosen[static_cast<std::size_t>(value)];
        result.coins.push_back(coin); value -= coin;
    }
    return result;
}

int main() {
    for (const auto& coins : {std::vector<int>{1, 3, 4}, {2, 4}, {3, 5, 7}, {}}) {
        for (int amount = 0; amount <= 100; ++amount) {
            const int top_down = memoized_count(coins, amount);
            const auto bottom_up = tabulated(coins, amount);
            CHECK(bottom_up.has_value() == (top_down != -1));
            if (bottom_up) {
                CHECK(bottom_up->count == top_down);
                CHECK(bottom_up->coins.size() == static_cast<std::size_t>(top_down));
                CHECK(std::accumulate(bottom_up->coins.begin(), bottom_up->coins.end(), 0) == amount);
            }
        }
    }
    CHECK(tabulated({1, 3, 4}, 6)->count == 2);
    CHECK(!tabulated({2}, 3));
    CHECK(course::throws<std::invalid_argument>([] { (void)tabulated({0, 1}, 2); }));
    course::report();
}

```

Build, run, and inspect

Run this block from the top-level extracted package. The fixture runner supplies input where needed and checks exit status and output in a temporary working directory.

```

cmake -S source/book3/ch19 -B build/b3-ch19 \
  -DCMAKE_BUILD_TYPE=Debug
cmake --build build/b3-ch19 --parallel 2
ctest --test-dir build/b3-ch19 --output-on-failure

```

For a multi-configuration generator on Windows, append `--config Debug` to the build command and `-C Debug` to CTest. The executable target is `b3_ch19`.

Evidence and interpretation

The tests compare both algorithms for amounts zero through one hundred across denomination sets 1,3,4; 2,4; 3,5,7; and the empty set. Whenever a solution exists, they verify agreement on count, agreement between count and reconstructed vector length, and equality between reconstructed sum and target.

The explicit amount-six fixture with coins 1,3,4 returns two, correcting the greedy result from Chapter 18. Coins 2 cannot produce amount three. A zero denomination is rejected before table evaluation.

Agreement between two implementations is helpful but not absolute proof: both could encode the same mistaken recurrence. The recurrence proof, hand-checkable fixtures, and reconstruction checks provide different forms of evidence. A tiny exhaustive enumeration offers an additional independent oracle in the lab.

Failure clinic — A table hides an invalid state model

A programmer adds a finite stock count to each denomination but leaves the memo key as the remaining amount. Two paths now reach the same amount with different stocks left. The first cached answer is reused for a state in which its required coins are unavailable. The resulting table can be internally consistent while solving the wrong problem.

Repair the state model before changing loop syntax. An item-index dimension can express whether a particular item is still available; bounded counts may need another representation. Write two concrete paths that reach the same numeric amount with different future options. If they require different answers, they cannot share one cache entry under that key.

Another defect occurs when an impossible predecessor is incremented and compared without a reachability check. A negative sentinel plus one may look better than every valid count. A very large sentinel plus one may overflow in an unbounded implementation. Keep the semantic states visible and never perform ordinary cost arithmetic on an absent answer.

Finally, reconstruction can accidentally reuse a stale choice after a failed computation. Build results from a fresh candidate table, publish only a completed solution, and test the returned witness independently. The count and the witness are related outputs, but agreement between their lengths alone does not establish denomination membership or the target sum.

Practice and implementation lab

Reject a negative denomination in a dedicated fixture and test the maximum accepted amount and one amount above it. Verify that every reconstructed coin belongs to the supplied denomination set. Add exhaustive comparison for tiny targets, then adapt the state definition to a bounded-inventory coin problem and explain why the one-dimensional recurrence is insufficient there.

Review answers and solution notes

The existing validator rejects nonpositive denominations and out-of-range targets. Membership checking strengthens reconstruction evidence beyond merely matching count and sum. A bounded-inventory model must retain enough information to know which denominations or remaining supplies are available; otherwise it can reuse a coin that was already consumed. A common formulation adds a denomination index or remaining-count state with a precisely defined transition.

Engineering takeaway. Define the state and prove the recurrence before choosing memoization, tabulation, or a compressed table.

Guided lab — Test the recurrence from three directions

Extend the companion checks so every reconstructed coin belongs to the input denomination set, the sum equals the requested amount, and the number of coins equals the reported count. Include empty denominations, target zero, duplicate denominations, unsorted denominations, and impossible odd targets under even denominations. Duplicates should not improve the optimum, though they can add redundant work.

Build a tiny breadth-first oracle over amounts from zero through a small target bound. An edge adds one allowed denomination without exceeding the target. The shortest number of edges from zero to the target is the minimum coin count. This uses the unweighted shortest-path reasoning from Chapter 14 rather than the same recursive implementation. Compare all three methods over a bounded collection of denomination sets.

Instrument state evaluations separately from recursive calls. Memoized calls can revisit a cached state without recomputing it, so these counters answer different questions. Compare them with the number of bottom-up table entries examined. Choose an input whose denomination restrictions leave many states unreachable and explain why top-down and bottom-up work can differ while sharing the same upper bound.

For a design extension, change the objective to minimize total handling cost when each denomination has a separate nonnegative cost. Derive the revised recurrence and define the witness objective before writing code. A minimum number of coins is no longer necessarily the minimum handling cost. Retain the original problem as a separate function rather than changing its meaning silently.

Review and solution reasoning

Memoization is correct when equal state keys imply equal future subproblems under fixed inputs. Positive denominations make the remaining amount decrease. Tabulation works upward because every dependency has a smaller amount. The $O(A \cdot K)$ bound is numeric-amount dependent and therefore pseudo-polynomial in the encoded input.

A valid witness demonstrates feasibility, not optimality by itself. Agreement between two programs using one recurrence is stronger than one unchecked result but weaker than independent modeling evidence. The professional habit is to combine a clear recurrence, a dependency argument, bounded exhaustive checks, and witnesses that can be validated without trusting the solver's internal table.

Primary reference notes

Implementation mechanisms: [vector](#), [optional](#), [lambda expressions](#), and [numeric algorithms](#). Further dynamic-programming study: [MIT algorithm lectures](#).

Chapter 20 — Designing Algorithms for Real Hardware

Two loops can perform the same number of additions and still have different elapsed times. HarborWorks now has enough algorithmic vocabulary to ask why without confusing an observation with a universal rule. This final volume chapter connects data layout, access order, measurement design, and correctness evidence.

The example compares row-major and column-major traversal of the same row-major square matrix. It also contrasts array-of-structures and structure-of-arrays representations. Timing is optional and never a pass/fail requirement; the automated tests establish result agreement, not a promised speedup on every machine.

The previous chapters explain why one algorithm performs fewer abstract operations or uses less auxiliary storage than another. Real machines add another layer: the location of data, the order of accesses, the generated instructions, and interference from the surrounding system. Performance engineering connects these layers without confusing a measurement on one machine with a universal property of the language.

HarborWorks compares two traversals of the same matrix and two layouts for small records. The examples are deliberately modest. They establish a correctness-preserving experiment and a measurement vocabulary rather than promising a speedup. A faster result matters only when both candidates compute the required answer and the measured boundary corresponds to the workload that needs improvement.

20.1 Understanding Cache-Aware Performance

Asymptotic analysis describes how work grows, but real execution also depends on how that work touches memory. Cache behavior is influenced by access order, layout, working-set size, competing activity, and the processor. Treat locality as a design property to investigate, not a fixed nanosecond constant.

20.1.1 Improving Spatial Locality

Spatial locality means nearby locations are accessed close together in time. In a flat row-major matrix, the element at row r and column c is stored at index $r \cdot \text{width} + c$. Advancing c walks adjacent elements; advancing r while holding c fixed jumps by an entire row width.

Caches transfer data in implementation-specific blocks. Adjacent traversal can make good use of data already brought close to the processor, whereas a large stride may use only part of each transferred block before moving elsewhere. Prefetching, cache associativity, translation behavior, and matrix dimensions can affect the observed difference.

Both traversal functions here execute width squared additions and use $O(1)$ additional accumulation storage. Their asymptotic time class is the same. The difference being studied is the memory-access pattern, not an algorithmic improvement from quadratic to linear work.

The helper functions assume the vector contains the full width-by-width matrix. The application constructs that invariant using fixed small or benchmark dimensions. A reusable public matrix API should validate dimensions and multiplication overflow at construction rather than repeatedly accepting an unchecked arbitrary width.

Spatial locality means that accessing one location makes nearby locations useful soon afterward. A row-major matrix stored in one contiguous vector places adjacent columns of a row next to each other.

Traversing rows in order consumes those neighboring elements before moving to the next row.

Traversing columns jumps by the row width between successive elements.

The mathematical index formula is `row * width + column`, under the precondition that the flat buffer contains the required square matrix and multiplication fits the chosen index type. Both traversals visit the same logical cells. Their difference is the temporal order of addresses, which can affect cache-line use, address translation, prefetching, and memory-level parallelism on actual hardware.

A cache line commonly contains several values, but its exact size is a platform property, not a C++ guarantee. A contiguous traversal can use multiple values from a fetched line. A strided traversal may touch a small part of many lines before returning, potentially causing useful data to be evicted. The magnitude of that effect depends on dimensions, element size, cache capacity, associativity, and other activity.

Do not convert this explanation into “columns are always slow.” A small matrix may fit in cache, a compiler may transform a loop, and a different storage layout may make columns contiguous. The relevant question is how logical indices map to physical addresses for the representation actually used.

A useful first diagnostic is an address trace over a tiny matrix. Print indices rather than raw pointer values to keep the output portable and interpretable. For width four, row traversal begins 0, 1, 2, 3, 4, while column traversal begins 0, 4, 8, 12, 1. This trace proves the access-order distinction. It does not measure the target machine’s cache behavior, which requires separate observation.

20.1.2 Improving Temporal Locality

Temporal locality means recently used data is accessed again before it is displaced. Blocking a matrix computation into tiles can improve reuse when several operations consume the same region. A tile that is too large can defeat the purpose, while a tiny tile can add unnecessary loop overhead. Choose dimensions through workload-aware measurement.

For records $\{x, y, z\}$, an array of structures keeps each record’s fields together. That can suit operations that consume all fields of one record. Separate x , y , and z arrays can suit operations scanning one field across many records, reducing irrelevant field traffic and potentially simplifying vectorization.

The structure-of-arrays representation introduces an invariant: corresponding positions across arrays describe the same logical record, and their lengths must agree. A layout optimization that loses field alignment is a correctness defect regardless of speed. The sample checks a small field-total equivalence and equal lengths; it does not benchmark the two record layouts or claim one universally wins.

Temporal locality means that data is reused soon enough for a previous access to remain useful. A computation that repeatedly uses a small active block can benefit from keeping that block near the processor. A computation that streams through a dataset once has a different opportunity: its main goal may be efficient sequential movement rather than retaining old values.

Blocking or tiling reorganizes work so a bounded region is reused before moving elsewhere. Matrix multiplication is a familiar setting because each output combines many input values, but the principle also applies to graph batches, image neighborhoods, and grouped record processing. The tile size is not a magic constant. It interacts with the number of simultaneously active arrays, cache levels, vector widths, and concurrency.

The chapter's sum visits each cell once per traversal, so it is primarily an access-order demonstration rather than a rich temporal-reuse kernel. Repeating the whole sum in a benchmark can warm caches, but that repetition belongs to the measurement protocol. It should not be mistaken for useful reuse inside the original single-pass workload.

Data-oriented layout also depends on which fields are reused together. An array of structures keeps each record's fields adjacent. A structure of arrays keeps one field across many records adjacent. If a pass reads only one field, separate arrays may avoid moving unused fields. If every operation needs all fields of one record, the tradeoff changes. Neither representation wins merely because its name sounds more modern.

Separate layout from ownership and interface. An application can expose a clear record-oriented API while storing hot numeric fields in separate arrays internally. That design introduces synchronization obligations among array lengths and record identities. A performance improvement that loses correspondence between fields is a correctness defect, not an acceptable optimization tradeoff.

Row-Major Versus Column-Major Matrix Traversal

Purpose. Cache-aware performance depends on memory access order even when algorithmic Big-O and arithmetic counts are identical.

```
#include <iostream>
#include <vector>
int main(){const int rows=512,cols=512;std::vector<int>a(rows*cols,1);long
row=0,col=0;for(int r=0;r<rows;++r)for(int c=0;c<cols;++c)row+=a[r*cols+c];for(int c=0;c<cols;++c)for(int r=0;r<rows;++r)col+=a[r*cols+c];std::cout<<row<<' '<<col<<'\n';}
```

How it works.

- Both loops perform the same arithmetic and produce the same result.
- The first traversal accesses adjacent row-major elements sequentially.
- The second jumps by an entire row between accesses, creating a less cache-friendly pattern.

Expected behavior/output. Both totals are 262144; timing may differ depending on hardware and optimization.

Experiment. Increase the matrix size and benchmark repeated traversals after validating equal results.

20.2 Measuring Algorithm Performance

A benchmark is an experiment with a declared question, controlled inputs, an observable result, and known limitations. A timer around a loop is only one component. Separate correctness tests from performance experiments so a scheduling interruption cannot masquerade as an algorithmic failure.

20.2.1 Benchmarking Competing Implementations

The optional benchmark uses a 1024-by-1024 matrix, nine measurements of each traversal, and `std::chrono::steady_clock`. Each pair processes the same input, and the code alternates which traversal runs first to reduce a simple first-versus-second bias. The matrix's first value changes between pairs, and a checksum is printed so the computed values remain observable.

Measured durations exclude matrix construction and include the traversal call and clock overhead. Sorting the nine durations and selecting the middle gives a median. That is a simple descriptive statistic, not a confidence interval or a guarantee that noise has been removed.

Use an optimized build for a performance investigation and record the compiler, flags, hardware, operating system, input dimensions, and whether the machine was otherwise busy. Keep sanitizer runs separate because instrumentation changes execution. Run correctness checks before considering a timing difference meaningful.

Observable checksums discourage dead-code elimination but are not a formal guarantee against every optimizer transformation. Inspect generated code when making serious claims, vary sizes and data, repeat whole experiments, and use a dedicated benchmark framework or hardware counters when the question requires stronger controls.

A benchmark begins with a question precise enough to choose an experimental boundary. Are you measuring only the traversal, allocation plus traversal, parsing plus computation, or a complete request including output? Different boundaries can produce different conclusions. The supplied optional benchmark allocates and initializes its matrix before the timed traversal, so its durations do not include input construction.

Use a monotonic clock for elapsed intervals. `std::chrono::steady_clock` is intended for durations unaffected by wall-clock adjustments. Its resolution and overhead still matter, especially for very short operations. Repeating enough work to exceed timer noise can help, but the repetition must not allow the compiler to remove or replace the computation under the as-if rule.

The example returns the computed sum, checks that both candidates agree, and contributes results to an externally printed checksum. It also changes an input value between trials. These choices make the result observable and discourage trivial elimination. They are not a universal barrier against every optimizer transformation. Inspect generated code or use a suitable benchmark framework when a serious microbenchmark depends on precise execution assumptions.

The companion alternates which traversal is measured first across paired trials. This reduces one obvious order bias, but it does not eliminate thermal changes, scheduling interruptions, cache warming, or frequency variation. Repeated measurements should be reported as a distribution or summary with context rather than as one authoritative nanosecond value.

Correctness checks stay outside the timed interval where possible, and no test asserts that one traversal must be faster. A speed ordering can reverse across environments without either implementation being wrong. The test suite validates values; the optional experiment investigates performance. Mixing those roles creates fragile tests and encourages tuning to the test machine rather than to the real requirement.

20.2.2 Separating Algorithmic and Hardware Costs

A time difference may reflect cache misses, instruction count, vectorization, page translation, frequency changes, or interference. Timing alone does not identify the cause. A profile or counter measurement can support a causal hypothesis, but counter availability and interpretation are platform-specific.

Compare equal work and equal outcomes. A faster function that skips some rows or overflows differently has not optimized the same computation. The example's unsigned sixty-four-bit accumulation comfortably contains the values for its fixed dimensions. A general-purpose API must establish its own overflow policy rather than assuming every larger matrix remains within range.

Scaling experiments help distinguish causes. Sweep widths across several working-set sizes and examine both total time and time per element. Include dimensions that are not powers of two to investigate indexing and cache effects without declaring a universal explanation from a single favorable point.

The final engineering decision should connect measurement to a requirement. A five-percent change in an unused microbenchmark may be less valuable than removing an avoidable $O(n^2)$ stage from the real

request path. Preserve readable reference implementations, measure end-to-end effects, and document the cost of a more specialized layout.

Algorithmic cost and hardware cost answer different questions. Two traversals can both be $O(n)$ in the number of cells while exhibiting different elapsed times because their memory accesses behave differently. Conversely, a carefully tuned quadratic algorithm can beat a less polished $O(n \log n)$ implementation on tiny inputs and still become the wrong choice as the workload grows.

Measure across representative sizes rather than selecting one favorable point. A size sweep can reveal transitions when a working set no longer fits a cache level or when fixed setup costs stop dominating. Report the units carefully: a square matrix with width w contains $w*w$ elements, so doubling width quadruples the number of visited cells.

Compiler configuration is part of the experiment. Debug builds preserve different structures and usually disable many optimizations; sanitizer builds add diagnostic work. Use them for correctness investigations, then measure a separately tested optimized build. Record the compiler, standard library, flags, processor, operating system, and relevant input distribution. A result without those facts is difficult to interpret or reproduce.

Counters can explain timing but require caution. Cache-miss samples, branch events, or cycles are hardware- and tool-specific measurements. Some counters are unavailable, multiplexed, or restricted by system policy. Do not invent a cache explanation solely because one candidate took longer. State a hypothesis, inspect the access pattern, collect available evidence, and consider competing explanations.

The business boundary matters last as much as first. Saving half the kernel time is a small end-to-end improvement when that kernel accounts for only a tiny fraction of request latency. Include data conversion, allocation, synchronization, and result consumption in a second application-level measurement. Optimization is complete only when the required outcome improves without breaking correctness, resource limits, or maintainability constraints.

Repeated Timing with Correctness Check

Purpose. Performance measurement should pair a defined workload and correct result with repeated timing rather than relying on one stopwatch reading.

```
#include <algorithm>
#include <chrono>
#include <iostream>
#include <numeric>
#include <vector>
int main(){std::vector<int>v(100000);std::iota(v.begin(),v.end(),0);long long
check=0;std::vector<long long>times;for(int trial=0;trial<20;++trial){auto
s=std::chrono::steady_clock::now();long long
total=std::accumulate(v.begin(),v.end(),0LL);auto
e=std::chrono::steady_clock::now();check+=total;times.push_back(std::chrono::duration_cast<std::c
s).count());}std::sort(times.begin(),times.end());std::cout<<"sum="<<check<<"
median_ns="<<times[times.size()/2]<<"\n";}
```

How it works.

- `steady\_clock` measures elapsed time without wall-clock adjustments.
- Multiple trials reduce the risk of drawing conclusions from one noisy observation.
- Sorting durations allows a median to be reported while preserving the correctness result.

Expected behavior/output. The sum is deterministic; the median nanoseconds vary by machine, compiler, and system load.

Experiment. Compile with `-O0` and `-O2`, rerun the same workload, and report both result correctness and timing distribution.

Worked example — Investigate locality without turning timings into correctness tests

Requirement and design

Implement two equivalent matrix sums with different traversal orders and verify their equality. Keep a small deterministic default run for automated testing. Add an explicit benchmark mode that records paired median durations and a checksum, without enforcing an expected timing ratio.

Establish equality before comparing durations

The worked program first computes both traversal sums on the same bounded matrix and checks equality. The integer values and dimensions used by the example keep the total within the chosen unsigned accumulator. A general public matrix API would need explicit shape and overflow contracts; the compact helpers assume the controlled shape established by their caller.

The small array-of-structures and structure-of-arrays calculation similarly checks the mathematical result, not a performance claim. Its tiny input is unsuitable for inferring a meaningful layout speedup. Use it to

understand field placement, then design a separate experiment with realistic sizes and an observable output.

Run the default executable as a correctness exercise. Run the explicitly documented `--benchmark` path only in an optimized build when you intend to collect timing data. Preserve its raw output with environment metadata. The printed medians describe nine paired observations in that run; they are not a percentile guarantee for a production service or evidence of hard-real-time behavior.

Before adapting the example, state whether you are changing the problem size, layout, algorithm, compiler configuration, or measurement boundary. Changing several at once makes the result harder to attribute. A controlled experiment starts with one question, not with every available optimization switch.

Complete implementation

The complete, buildable chapter project is `source/book3/ch20/`. The following files are reproduced from that project; the publishing script inserts the source directly so the listing and companion remain synchronized.

main.cpp

```
#include "course_test.hpp"
#include <algorithm>
#include <chrono>
#include <cstdint>
#include <cstdint>
#include <iostream>
#include <string_view>
#include <vector>

using Sum = std::uint64_t;
Sum row_major(const std::vector<unsigned>& data, std::size_t width) {
    Sum sum{};
    for (std::size_t row = 0; row < width; ++row)
        for (std::size_t col = 0; col < width; ++col) sum += data[row * width + col];
    return sum;
}
Sum column_major(const std::vector<unsigned>& data, std::size_t width) {
    Sum sum{};
    for (std::size_t col = 0; col < width; ++col)
        for (std::size_t row = 0; row < width; ++row) sum += data[row * width + col];
    return sum;
}
struct Sample { float x, y, z; };

template<class Function>
long long trial(Function function, const std::vector<unsigned>& data, std::size_t
width,
                Sum& result) {
    const auto start = std::chrono::steady_clock::now();
    result = function(data, width);
    const auto stop = std::chrono::steady_clock::now();
    return std::chrono::duration_cast<std::chrono::nanoseconds>(stop -
start).count();
}

int main(int argc, char** argv) {
    const bool benchmark = argc == 2 && std::string_view{argv[1]} == "--benchmark";
    const std::size_t width = benchmark ? 1024 : 32;
    std::vector<unsigned> matrix(width * width);
```

```

    for (std::size_t i = 0; i < matrix.size(); ++i) matrix[i] =
static_cast<unsigned>(i % 100);
    CHECK(row_major(matrix, width) == column_major(matrix, width));
    const std::vector<Sample> aos{{1, 2, 3}, {4, 5, 6}};
    const std::vector<float> x{1, 4}, y{2, 5}, z{3, 6};
    CHECK(aos[0].x + aos[1].x == x[0] + x[1]);
    CHECK(y.size() == z.size() && z.size() == x.size());
    if (benchmark) {
        std::vector<long long> row_times, column_times;
        Sum checksum{};
        for (int i = 0; i < 9; ++i) {
            matrix[0] = static_cast<unsigned>(i);
            Sum row{}, column{};
            // Alternate order to reduce a simple first/second measurement bias.
            if (i % 2 == 0) {
                row_times.push_back(trial(row_major, matrix, width, row));
                column_times.push_back(trial(column_major, matrix, width, column));
            } else {
                column_times.push_back(trial(column_major, matrix, width, column));
                row_times.push_back(trial(row_major, matrix, width, row));
            }
            CHECK(row == column); checksum += row;
        }
        std::sort(row_times.begin(), row_times.end());
        std::sort(column_times.begin(), column_times.end());
        std::cout << "row_median_ns=" << row_times[4]
                  << " column_median_ns=" << column_times[4]
                  << " checksum=" << checksum << '\n';
    }
    course::report();
}

```

Build, run, and inspect

Run this block from the top-level extracted package. The fixture runner supplies input where needed and checks exit status and output in a temporary working directory.

```

cmake -S source/book3/ch20 -B build/b3-ch20 \
      -DCMAKE_BUILD_TYPE=Debug
cmake --build build/b3-ch20 --parallel 2
ctest --test-dir build/b3-ch20 --output-on-failure

```

For a multi-configuration generator on Windows, append `--config Debug` to the build command and `-C Debug` to CTest. The executable target is `b3_ch20`.

Evidence and interpretation

The default run uses a 32-by-32 matrix and checks row/column agreement plus a small record-layout equivalence. The benchmark mode uses width 1024 and prints `row_median_ns`, `column_median_ns`, and `checksum`. Those values depend on the executing environment and are not fixed expected output in the test suite.

To perform a local experiment after the normal tests, configure this chapter with `-DCMAKE_BUILD_TYPE=Release`, build it, and run the resulting `b3_ch20` executable with `--benchmark`. For a multi-configuration generator, select the Release configuration for both building and locating the executable. Run from a quiet machine and preserve the environment record with the output.

No timing result in this book is presented as a measured guarantee for the reader's hardware. The delivered test report distinguishes automated correctness checks from optional performance work. This boundary prevents a noisy benchmark from becoming an unsupported production claim.

Failure clinic — A faster benchmark computes less work

A developer moves result consumption out of a benchmark and stops checking the sum. The compiler can now prove that the computed value is unused and remove work. The apparent speedup measures the absence of the required computation. Restoring observable output is necessary, but the broader lesson is that benchmark code is still ordinary optimized C++ and must obey the same semantic reasoning as application code.

Another experiment initializes one candidate's data before timing but includes initialization for the other. The resulting comparison mixes layout cost with traversal cost asymmetrically. Either include preparation for both under an end-to-end contract or exclude it for both under a kernel-only contract. Both experiments can be useful; only the undisclosed mismatch is misleading.

A third mistake is treating a sanitizer slowdown as evidence that an algorithm has poor production performance. Instrumentation deliberately adds checks. Use the sanitizer run to validate exercised memory behavior, then rerun the corrected implementation under the intended release configuration. Keep those reports separate.

When results vary, do not discard inconvenient samples automatically. Record interruptions or policy-defined outliers, preserve raw observations, and explain any exclusion rule before selecting the final result. The goal is to understand the workload, not to manufacture a flattering number.

Practice and implementation lab

Add a blocked traversal with a configurable positive tile width and compare all three sums over non-square dimensions after redesigning the matrix interface. Sweep several matrix and tile sizes, record repeated whole-experiment results, and explain which conclusions are observations versus hypotheses. Preserve the original simple traversal as a reference.

Review answers and solution notes

A rectangular matrix needs separate rows and columns and a validated product matching storage size. Tile-end bounds must be clamped to the actual dimensions. Verify every element is visited exactly once before timing. A lower measured time is an observation for the recorded setup; attributing it specifically to cache behavior requires additional evidence. Do not choose a universal tile size from one processor and one input.

Engineering takeaway. Optimize a verified computation, report the measurement boundary, and keep hardware-specific observations separate from algorithmic guarantees.

Guided lab — Write a reproducible performance note

Begin with the unmodified correctness tests. Create a release build and record the compiler version and exact command line. Run the optional matrix experiment several times without other deliberate heavy workloads, retaining every output. Do not run privileged tuning commands or alter host-wide power settings merely to reproduce a teaching example.

Extend a separate experimental copy to accept a bounded list of matrix widths. Validate multiplication before allocating and reject sizes beyond your chosen memory budget. Include small, medium, and larger working sets while keeping the machine responsive. For each size, record element count, bytes occupied by the matrix, number of trials, correctness checksum, and timing summaries.

State two hypotheses before inspecting the measurements. One might be that contiguous traversal becomes more advantageous when the working set exceeds a relevant cache capacity. Another might be that timer overhead dominates at the smallest size. Compare the observations with both hypotheses and report uncertainty when the evidence cannot distinguish them.

For a layout experiment, define a pass that reads only the first field and another pass that reads all fields. Keep values and output work equivalent between the array-of-structures and structure-of-arrays versions. Include conversion cost in a separate end-to-end measurement if the application would actually convert layouts for each request.

Finish with a one-page engineering note: requirement, baseline, candidate, correctness evidence, environment, input distribution, measurement boundary, observations, tradeoffs, and a decision. A valid decision may be to keep the simpler implementation because the measured benefit is too small or too uncertain to justify additional complexity.

Review and solution reasoning

Locality describes access relationships, not an unconditional speed promise. Big-O remains useful because growth and memory representation are complementary layers of analysis. A monotonic clock helps measure durations but does not remove scheduler noise. An observable checksum supports benchmark validity but does not replace inspection when optimization behavior is central to the claim.

The strongest conclusion is bounded: on the recorded environment, for the stated inputs and measurement boundary, the candidate produced equal results and showed the reported distribution of costs. That sentence is more useful than “this algorithm is always faster” because another engineer can reproduce, challenge, and extend it.

Primary reference notes

Timing and data facilities: [chrono](#), [vector](#), and [fundamental types](#). Optimization and diagnostics: [GCC optimization options](#), [Clang AddressSanitizer](#), and [Clang UndefinedBehaviorSanitizer](#).

Appendix A — An Algorithmic Diagnostic Desk Reference

Start with the observed disagreement

A wrong result, a resource failure, and a slow result call for different first questions. When the value is wrong, reduce the input while preserving the failure and compare the public contract with the observed output. When an allocation fails, inspect the accepted size and ownership transition. When elapsed time is high, establish that the result is correct before interpreting performance. Changing a representation without identifying the violated requirement can hide one symptom while creating another.

A useful incident record contains the input, expected property, actual observation, build configuration, source revision, and the smallest known reproduction. “The tree fails sometimes” is not a reproduction. “After inserting 4,2,6,5 and deleting 4, inorder traversal contains 4 again” identifies an operation history that can be traced through the ownership links.

Locate the broken layer

Observation	First contract to inspect	Discriminating check
A search misses an existing key	Ordering and equality model	Compare lower boundary with a linear scan
A hash lookup fails after deletion	Probe-chain continuity	Force all keys into one collision path
A queue loses an element at wraparound	Head, size, and slot mapping	Compare every operation with a bounded deque model
A tree traversal is sorted but incomplete	Membership and ownership	Compare full contents with an independent set
A reported route has the right cost but invalid vertices	Witness feasibility	Check every consecutive edge and endpoint
A sorted result reorders equal records	Stability requirement	Retain arrival identities beside equal keys
A dynamic program agrees with itself but violates stock limits	State sufficiency	Exhibit two equal keys with different future choices
A benchmark is unexpectedly tiny	Observable computation	Check outputs and inspect optimized execution

The table suggests starting points, not automatic diagnoses. A missed lookup can also result from a dangling borrow or a changed key. Continue until the evidence identifies a specific violated assumption rather than stopping at the first plausible explanation.

Use operation histories, not only snapshots

Many data-structure defects need a sequence of actions. A tombstone bug requires insertion, collision, deletion, and a later lookup or update. A tail-pointer bug may require removing the only node and then inserting another. A circular buffer defect may appear only after several wraparounds with alternating full and empty states.

Represent these histories as replayable operations. Save the random seed and the actual minimized sequence when a generated test fails. A seed alone is not always a durable reproduction across changes to the generator or its operation distribution. Keep the operation grammar small enough that the sequence can be read and edited by a person.

After each action, check both the return value and a structural observation. For a map, compare all visible key/value associations, not just the most recent key. For a queue, compare size and logical order. For a search tree, compare membership and inorder contents, while recognizing that a valid inorder list alone does not prove height metadata or parent links.

Distinguish missing results from exceptional failures

An unsuccessful search is often an ordinary result. An invalid vertex index is a rejected request. Exhausted memory is an environmental failure. Treating all three as the same empty return value prevents callers from choosing an appropriate response.

The chapter interfaces vary intentionally to illustrate these choices. Read each contract before combining functions. In the supplemental application, an unreachable target is printed as `unreachable` with a successful process exit. A malformed input is an error with a different exit code. The application does not reinterpret disconnection as a parser failure merely because no route can be produced.

Keep four forms of evidence separate

A proof explains why a stated algorithm satisfies a property under assumptions. A reference comparison checks agreement with another implementation. A sanitizer run investigates exercised low-level behavior. A benchmark records performance for a specified workload and environment. A strong engineering argument often uses all four, but none automatically replaces the others.

For example, an invariant can establish that lower-bound search shrinks the interval correctly. Exhaustive small-vector tests can catch an implementation typo. Sanitizers can expose an out-of-bounds access in an executed path. A benchmark can compare repeated query costs after sorting. Reporting one of these as all four exaggerates what was established.

Decide whether to keep a custom structure

A custom teaching implementation makes hidden mechanisms visible. That is a learning benefit, not a reason to deploy it by default. Before replacing a standard container, write down the required operation mix, size range, ownership constraints, iterator guarantees, failure behavior, and measured bottleneck. Include maintenance and verification costs in the decision.

A fixed-capacity specialized queue may be appropriate when rejecting overflow is part of the protocol. A hand-written general tree may be inappropriate when its deletion, allocator behavior, exception guarantees, and long-term maintenance are unproven. The choice should follow requirements and evidence rather than the amount of code already invested.

Appendix B — Executable Invariant and Witness Laboratories

These four programs extend the chapter examples without changing their original contracts. Each is a separate CTest case in the root companion build. The printed check count is the number of internal assertions executed by that program, not the number of independent CTest cases. Complete source is included below from the same files that are compiled.

Force a collision history rather than hoping for one

The collision laboratory deliberately gives every key the same probe origin. Its seven-slot capacity is fixed, and it makes no average-case hashing-performance claim. This is an adversarial correctness arrangement: every lookup must respect the sequence of occupied, deleted, and never-used slots.

An update must continue past a deleted slot because the key may already exist later in the probe path. The implementation remembers the first reusable position but does not commit a new key until it knows whether an existing key should be updated. The independent `std::map` model expresses membership and values without sharing that probe mechanism.

The generated history mixes insertion, update, deletion, full-table rejection, and reads over thirteen possible keys. After each operation, every key in that bounded universe is compared. A full table must still permit updating an existing key. A failed insertion of a new key must leave all accepted associations unchanged.

```
#include "course_test.hpp"
#include <array>
#include <cstdint>
#include <map>
#include <optional>
#include <random>

// All keys deliberately share one starting bucket. No growing or concurrency.
class CollisionTable {
    enum class State { empty, occupied, deleted };
    struct Slot { State state{State::empty}; int key{}; int value{}; };
    std::array<Slot, 7> slots_{};
public:
    bool put(int key, int value) {
        std::optional<std::size_t> reusable;
        for (std::size_t i = 0; i < slots_.size(); ++i) {
            auto& slot = slots_[i];
            if (slot.state == State::occupied && slot.key == key) {
                slot.value = value;
                return true;
            }
            if (slot.state != State::occupied && !reusable) reusable = i;
            if (slot.state == State::empty) break;
        }
        if (!reusable) return false;
        slots_[*reusable] = {State::occupied, key, value};
        return true;
    }
}
```

```

        std::optional<int> get(int key) const {
            for (const auto& slot : slots_) {
                if (slot.state == State::empty) return std::nullopt;
                if (slot.state == State::occupied && slot.key == key)
                    return slot.value;
            }
            return std::nullopt;
        }
        bool erase(int key) {
            for (auto& slot : slots_) {
                if (slot.state == State::empty) return false;
                if (slot.state == State::occupied && slot.key == key) {
                    slot.state = State::deleted;
                    return true;
                }
            }
            return false;
        }
};

int main() {
    CollisionTable table;
    std::map<int, int> model;
    std::mt19937 random{317};
    for (int step = 0; step < 5000; ++step) {
        const int key = static_cast<int>(random() % 13) - 6;
        if (random() % 3 == 0) {
            CHECK(table.erase(key) == (model.erase(key) != 0));
        } else {
            const bool accepted = model.contains(key) || model.size() < 7;
            CHECK(table.put(key, step) == accepted);
            if (accepted) model[key] = step;
        }
        for (int candidate = -6; candidate <= 6; ++candidate) {
            const auto found = model.find(candidate);
            const auto expected = found == model.end()
                ? std::optional<int>{} : std::optional<int>{found->second};
            CHECK(table.get(candidate) == expected);
        }
    }
    course::report();
}

```

To extend this lab, record the operation list and add a reducer that removes contiguous blocks while preserving a failure. Keep the reference policy bounded to seven accepted distinct keys; comparing with an unbounded map without that policy would make full-table rejection look incorrectly like a defect.

Return a BFS witness and replace recursive DFS frames explicitly

The graph laboratory produces one shortest unweighted path and detects directed cycles without recursive calls. BFS records a predecessor only at first discovery. Marking discovery before enqueueing ensures that another edge cannot enqueue the same vertex as a fresh discovery. The selected path depends on adjacency order; the contract does not claim a unique path when several shortest paths exist.

The DFS frame stores both the current vertex and the next neighbor index. That index is the continuation state a recursive call would have retained. Gray means active in the current depth-first chain; black means

fully processed. An edge to a gray vertex establishes a directed cycle, while an edge to a black vertex can simply converge on already completed work.

The reference to the top frame is not used after a `push_back` that might reallocate the frame vector. This is an ownership and invalidation issue inside an algorithm often described only in graph terms. The twenty-thousand-vertex chain test demonstrates that this implementation does not consume one native recursive call per vertex; it still allocates explicit storage proportional to depth.

```
#include "course_test.hpp"
#include <algorithm>
#include <cstdint>
#include <optional>
#include <queue>
#include <stdexcept>
#include <vector>
using Graph = std::vector<std::vector<std::size_t>>;
void validate(const Graph& graph) {
    for (const auto& neighbors : graph)
        for (auto vertex : neighbors)
            if (vertex >= graph.size()) throw std::out_of_range("edge");
}
std::optional<std::vector<std::size_t>> path(
    const Graph& graph, std::size_t source, std::size_t target) {
    validate(graph);
    if (source >= graph.size() || target >= graph.size())
        throw std::out_of_range("vertex");
    std::vector<std::optional<std::size_t>> parent(graph.size());
    std::queue<std::size_t> pending;
    parent[source] = source;
    pending.push(source);
    while (!pending.empty()) {
        const auto current = pending.front(); pending.pop();
        for (auto next : graph[current]) if (!parent[next]) {
            parent[next] = current; pending.push(next);
        }
    }
    if (!parent[target]) return std::nullopt;
    std::vector<std::size_t> result;
    for (auto current = target; ; current = *parent[current]) {
        result.push_back(current);
        if (current == source) break;
    }
    std::reverse(result.begin(), result.end());
    return result;
}
bool cycle(const Graph& graph) {
    validate(graph);
    struct Frame { std::size_t vertex; std::size_t next; };
    std::vector<unsigned char> color(graph.size());
    std::vector<Frame> stack;
    for (std::size_t root = 0; root < graph.size(); ++root) {
        if (color[root] != 0) continue;
        color[root] = 1; stack.push_back({root, 0});
        while (!stack.empty()) {
            auto& frame = stack.back();
            if (frame.next == graph[frame.vertex].size()) {
                color[frame.vertex] = 2; stack.pop_back(); continue;
            }
            const auto next = graph[frame.vertex][frame.next++];
            if (color[next] == 1) return true;
            if (color[next] == 0) {
                color[next] = 1; stack.push_back({next, 0});
                // frame may now be invalid; it is never used after this push.
            }
        }
    }
    return false;
}
```

```

    }
    }
    return false;
}
int main() {
    const Graph dag{{1, 2}, {3}, {3}, {}, {}};
    CHECK(path(dag, 0, 3) == std::vector<std::size_t>({0, 1, 3}));
    CHECK(!path(dag, 0, 4));
    CHECK(path(dag, 2, 2) == std::vector<std::size_t>({2}));
    CHECK(!cycle(dag)); CHECK(cycle(Graph{{1}, {2}, {0}}));
    CHECK(cycle(Graph{{0}})); CHECK(!cycle(Graph{}));
    CHECK(course::throws<std::out_of_range>([] { cycle(Graph{{1}}); }));
    Graph chain(20000);
    for (std::size_t i = 1; i < chain.size(); ++i) chain[i - 1].push_back(i);
    CHECK(!cycle(chain));
    CHECK(path(chain, 0, chain.size() - 1->size() == chain.size());
    chain.back().push_back(0); CHECK(cycle(chain));
    course::report();
}

```

A useful extension is to return the cycle itself. Retain the active parent chain and verify that the reported sequence closes through actual directed edges. Do not substitute an arbitrary repeated vertex from the entire visited set: it may belong to a completed branch rather than the current ancestry.

Make stability observable with record identities

Sorting integers alone cannot reveal whether equal records exchanged positions. The stability lab pairs each key with an arrival identity. Its merge chooses from the left half whenever the keys are equal, preserving the relative order inherited from the original sequence.

The test compares complete records with `std::stable_sort`, then separately checks nondecreasing keys and increasing arrival identities within equal-key groups. The second check directly expresses stability rather than trusting only equality with a library result. Empty, repeated, negative, and mixed keys arise in the bounded deterministic trial set.

```

#include "course_test.hpp"
#include <algorithm>
#include <cstdint>
#include <random>
#include <vector>
struct Record {
    int key;
    int arrival;
    bool operator==(const Record&) const = default;
};
void merge_sort(std::vector<Record>& values) {
    std::vector<Record> temporary(values.size());
    const auto sort = [&](auto&& self, std::size_t first, std::size_t last) -> void {
        if (last - first < 2) return;
        const auto middle = first + (last - first) / 2;
        self(self, first, middle); self(self, middle, last);
        auto left = first, right = middle, output = first;
        while (left < middle && right < last) {
            // Choose the left input on equal keys to preserve arrival order.
            if (values[right].key < values[left].key)
                temporary[output++] = values[right++];
            else temporary[output++] = values[left++];
        }
    };
    sort(values, 0, values.size());
}

```

```

    }
    while (left < middle) temporary[output++] = values[left++];
    while (right < last) temporary[output++] = values[right++];
    for (auto i = first; i < last; ++i) values[i] = temporary[i];
};
sort(sort, 0, values.size());
}
int main() {
    std::mt19937 random{616};
    for (int trial = 0; trial < 300; ++trial) {
        std::vector<Record> actual;
        for (int i = 0; i < trial % 129; ++i)
            actual.push_back({static_cast<int>(random() % 9) - 4, i});
        auto expected = actual;
        std::stable_sort(expected.begin(), expected.end(),
            [](const Record& a, const Record& b) { return a.key < b.key; });
        merge_sort(actual);
        CHECK(actual == expected);
        for (std::size_t i = 1; i < actual.size(); ++i) {
            CHECK(actual[i - 1].key <= actual[i].key);
            if (actual[i - 1].key == actual[i].key)
                CHECK(actual[i - 1].arrival < actual[i].arrival);
        }
    }
    course::report();
}

```

Change the comparison in an experimental copy so equality selects the right half. The keys may remain sorted and the multiset may remain unchanged, while the stability check fails. This is a concrete demonstration that sortedness, preservation, and stability are distinct requirements.

Compare a recurrence with an independent state-space search

The coin laboratory compares bottom-up minimum-count tabulation with breadth-first traversal of reachable amounts. A BFS edge means adding one allowed denomination. Because every edge uses one coin, the number of layers to the target is the minimum count. This is a different evaluation model from the table recurrence, although both depend on the same stated unbounded positive-denomination problem.

The lab enumerates all subsets of denominations one through six and targets zero through forty. It also checks the familiar non-greedy example and invalid inputs. The table's sentinel is safe under the explicit target limit of one hundred; it must not be copied into an unbounded cost problem without reconsidering arithmetic and reachability.

```

#include "course_test.hpp"
#include <algorithm>
#include <queue>
#include <stdexcept>
#include <vector>
void validate(const std::vector<int>& coins, int target) {
    if (target < 0 || target > 100) throw std::invalid_argument("target");
    for (int coin : coins) if (coin <= 0 || coin > 100)
        throw std::invalid_argument("denomination");
}
int tabulate(const std::vector<int>& coins, int target) {

```

```

    validate(coins, target);
    std::vector<int> best(static_cast<std::size_t>(target) + 1, 101);
    best[0] = 0;
    for (int amount = 1; amount <= target; ++amount)
        for (int coin : coins) if (coin <= amount)
            best[amount] = std::min(best[amount], best[amount - coin] + 1);
    return best[target] == 101 ? -1 : best[target];
}

int breadth_first(const std::vector<int>& coins, int target) {
    validate(coins, target);
    std::vector<int> distance(static_cast<std::size_t>(target) + 1, -1);
    std::queue<int> pending; pending.push(0); distance[0] = 0;
    while (!pending.empty()) {
        const int amount = pending.front(); pending.pop();
        for (int coin : coins) {
            if (coin > target - amount) continue;
            const int next = amount + coin;
            if (distance[next] != -1) continue;
            distance[next] = distance[amount] + 1; pending.push(next);
        }
    }
    return distance[target];
}

int main() {
    for (unsigned mask = 0; mask < 64; ++mask) {
        std::vector<int> coins;
        for (int coin = 1; coin <= 6; ++coin)
            if ((mask & (1U << (coin - 1))) != 0) coins.push_back(coin);
        for (int target = 0; target <= 40; ++target)
            CHECK(tabulate(coins, target) == breadth_first(coins, target));
    }
    CHECK(tabulate({1, 3, 4}, 6) == 2);
    CHECK(breadth_first({2, 4}, 3) == -1);
    CHECK(course::throws<std::invalid_argument>([] { tabulate({0}, 3); }));
    CHECK(course::throws<std::invalid_argument>([] { breadth_first({1}, 101); }));
    course::report();
}

```

The enumeration is exhaustive over that selected small universe, not over every integer input. Its value is that boundary states, disconnected reachable sets, empty choices, and noncanonical denomination combinations are exercised systematically. Preserve those limits in the report so a finite experiment remains useful evidence rather than an exaggerated universal claim.

Appendix C — HarborRoutes: An Integrated Algorithmic Application

Requirement and deliberate boundaries

HarborRoutes reads a small directed weighted graph and one source/target query. It either prints one minimum-cost route or reports that the target is unreachable. The application combines bounded parsing, adjacency lists, a priority queue, shortest-path reasoning, predecessor reconstruction, independent witnesses, and deterministic command-line fixtures. It runs locally and performs no network communication.

The input is an ASCII whitespace-separated token stream. The first four fields are vertex count, edge count, source, and target. Each following edge contributes three fields: from, to, and nonnegative weight. Vertex count is between one and 128, edge count is at most 4,096, and weights are at most 1,000,000. Vertex identifiers are dense zero-based indices. Exactly the declared fields must be present, with no trailing token.

Tokens contain unsigned decimal digits only, with at most twelve characters. Leading zeros are allowed within that length. Signs, decimal points, non-ASCII numerals, and oversized fields are rejected. The parser bounds a token before appending another character. It does not first read an arbitrarily large line and only afterward check its size.

These limits bound retained graph and token storage for accepted inputs. They do not impose a total input-byte deadline: a stream containing endless whitespace can still keep the parser waiting. A network-facing version would need transport limits, cancellation, and time budgets. The present application is a local instructional tool, not a hostile-input service certification.

For example:

```
4 4 0 3
0 1 8
0 2 1
2 1 1
1 3 1
```

The route through vertices zero, two, one, and three costs three. Selecting the first discovered route to vertex one would retain cost eight and miss the improvement. This is why weighted routing needs more than the ordinary BFS layer rule.

Select the algorithm from the weight contract

Dijkstra's method is appropriate here because every edge weight is nonnegative. The priority queue chooses the smallest currently known tentative distance. Once that entry is current rather than stale, no path through a later nonnegative extension can improve a finalized smaller distance. This argument

would fail with unrestricted negative edges, so rejecting them is part of the algorithm selection, not merely a parser preference.

The implementation uses immutable (`distance`, `vertex`) queue entries instead of trying to mutate heap priorities in place. An improved distance pushes a new entry. When an older entry reaches the top, it is discarded if its stored distance differs from the current best distance for that vertex. The target can be returned early only after this stale-entry check.

Adjacency lists are sorted after parsing. Queue pairs also use the vertex identifier as a tie-breaker, and predecessor updates occur only on strict improvement. These choices provide deterministic results for reordered edge records in the tested contract. They do not promise the lexicographically smallest path among all equal-cost paths. Zero-weight edges are allowed; avoiding equal-cost predecessor rewrites also prevents casual creation of cycles in the predecessor chain.

The graph is validated again at the public algorithm boundary so programmatically constructed requests cannot bypass size, endpoint, and weight checks. The parser and solver therefore have related but independent responsibilities. Internal arithmetic includes an explicit overflow guard even though the accepted graph bounds keep a simple shortest path far below the maximum `uint64_t` value.

Define ownership and the commit point

`Request` owns its adjacency lists. `Route` owns its vertex sequence. Neither exposes a borrowed view whose lifetime depends on a local parsing buffer. The `load` function parses into a candidate request, then moves that completed candidate into the accepted state. A compile-time check verifies that this particular move assignment is nonthrowing.

If parsing or allocation fails before the commit, the previous accepted request remains unchanged. This guarantee concerns the in-memory request. It does not rewind the input stream or remove already consumed bytes. Similarly, detecting a failed output stream does not erase the prefix already written to an external destination. The regression tests demonstrate these limits explicitly.

Public interface

```
#ifndef HARBOR_ROUTES_HPP
#define HARBOR_ROUTES_HPP
#include <cstdint>
#include <cstdint>
#include <iosfwd>
#include <optional>
#include <stdexcept>
#include <vector>
namespace harbor {
using Cost = std::uint64_t;
inline constexpr std::size_t max_vertices = 128;
inline constexpr std::size_t max_edges = 4096;
inline constexpr Cost max_weight = 1000000;
struct Edge {
    std::size_t to;
    Cost weight;
    bool operator==(const Edge&) const = default;
};
using Graph = std::vector<std::vector<Edge>>;
```

```

struct Request {
    Graph graph;
    std::size_t source{};
    std::size_t target{};
    bool operator==(const Request&) const = default;
};
struct Route {
    Cost distance;
    std::vector<std::size_t> vertices;
    bool operator==(const Route&) const = default;
};
class InputError : public std::runtime_error {
public:
    using std::runtime_error::runtime_error;
};
Request parse(std::istream& input);
void load(std::istream& input, Request& accepted);
std::optional<Route> shortest_route(const Request& request);
void write(std::ostream& output, const std::optional<Route>& route);
}
#endif

```

Parsing, validation, shortest-route calculation, and rendering

```

#include "routes.hpp"
#include <algorithm>
#include <functional>
#include <istream>
#include <limits>
#include <ostream>
#include <queue>
#include <string>
#include <tuple>
#include <type_traits>
#include <utility>
namespace harbor {
namespace {
bool space(char c) {
    return c == ' ' || c == '\t' || c == '\n' || c == '\r'
        || c == '\f' || c == '\v';
}
int next(std::istream& input) {
    try {
        const int c = input.get();
        if (c == std::char_traits<char>::eof() &&
            (input.bad() || !input.eof()))
            throw InputError("input failure");
        return c;
    } catch (const std::ios_base::failure&) {
        if (input.eof() && !input.bad())
            return std::char_traits<char>::eof();
        throw InputError("input failure");
    }
}
std::optional<std::string> token(std::istream& input) {
    std::string value;
    for (;;) {
        const int raw = next(input);
        if (raw == std::char_traits<char>::eof())
            return value.empty() ? std::nullopt : std::optional{value};
        const char c = static_cast<char>(raw);
        if (space(c)) {
            if (!value.empty()) return value;

```

```

        continue;
    }
    if (value.size() == 12) throw InputError("token too long");
    value.push_back(c);
}
}
Cost number(std::istream& input, Cost limit) {
    const auto text = token(input);
    if (!text) throw InputError("missing field");
    Cost result{};
    for (char c : *text) {
        if (c < '0' || c > '9') throw InputError("unsigned integer required");
        const Cost digit = static_cast<Cost>(c - '0');
        if (digit > limit || result > (limit - digit) / 10)
            throw InputError("field exceeds limit");
        result = result * 10 + digit;
    }
    return result;
}
void validate(const Request& request) {
    const auto count = request.graph.size();
    if (count == 0 || count > max_vertices ||
        request.source >= count || request.target >= count)
        throw std::invalid_argument("invalid graph or query");
    std::size_t edges{};
    for (const auto& neighbors : request.graph) {
        if (neighbors.size() > max_edges - edges)
            throw std::invalid_argument("too many edges");
        edges += neighbors.size();
        for (const auto& edge : neighbors)
            if (edge.to >= count || edge.weight > max_weight)
                throw std::invalid_argument("invalid edge");
    }
}
}
}
Request parse(std::istream& input) {
    const auto vertices = static_cast<std::size_t>(number(input, max_vertices));
    if (vertices == 0) throw InputError("vertices must be positive");
    const auto edges = static_cast<std::size_t>(number(input, max_edges));
    const auto source = static_cast<std::size_t>(number(input, vertices - 1));
    const auto target = static_cast<std::size_t>(number(input, vertices - 1));
    Request candidate{Graph(vertices), source, target};
    for (std::size_t i = 0; i < edges; ++i) {
        const auto from = static_cast<std::size_t>(number(input, vertices - 1));
        const auto to = static_cast<std::size_t>(number(input, vertices - 1));
        const auto weight = number(input, max_weight);
        candidate.graph[from].push_back({to, weight});
    }
    if (token(input)) throw InputError("trailing field");
    for (auto& neighbors : candidate.graph)
        std::sort(neighbors.begin(), neighbors.end(), [](const Edge& a, const Edge&
b) {
            return std::tie(a.to, a.weight) < std::tie(b.to, b.weight);
        });
    return candidate;
}
void load(std::istream& input, Request& accepted) {
    auto candidate = parse(input);
    static_assert(std::is_nothrow_move_assignable_v<Request>);
    accepted = std::move(candidate);
}
std::optional<Route> shortest_route(const Request& request) {
    validate(request);
    const auto count = request.graph.size();
    const auto infinity = std::numeric_limits<Cost>::max();
    std::vector<Cost> distance(count, infinity);

```

```

std::vector<std::optional<std::size_t>> parent(count);
using Entry = std::pair<Cost, std::size_t>;
std::priority_queue<Entry, std::vector<Entry>, std::greater<Entry>> pending;
distance[request.source] = 0;
pending.push({0, request.source});
while (!pending.empty()) {
    const auto [cost, vertex] = pending.top(); pending.pop();
    if (cost != distance[vertex]) continue; // stale immutable entry
    if (vertex == request.target) break;
    for (const auto& edge : request.graph[vertex]) {
        if (cost > infinity - edge.weight)
            throw std::overflow_error("distance overflow");
        const auto candidate = cost + edge.weight;
        if (candidate < distance[edge.to]) {
            distance[edge.to] = candidate;
            parent[edge.to] = vertex;
            pending.push({candidate, edge.to});
        }
    }
}
if (distance[request.target] == infinity) return std::nullopt;
Route route{distance[request.target], {}};
auto current = request.target;
for (std::size_t steps = 0; steps < count; ++steps) {
    route.vertices.push_back(current);
    if (current == request.source) {
        std::reverse(route.vertices.begin(), route.vertices.end());
        return route;
    }
    if (!parent[current]) throw std::logic_error("missing predecessor");
    current = *parent[current];
}
throw std::logic_error("predecessor cycle");
}
void write(std::ostream& output, const std::optional<Route>& route) {
    if (!route) output << "unreachable\n";
    else {
        output << "distance=" << route->distance << "\npath=";
        for (std::size_t i = 0; i < route->vertices.size(); ++i) {
            if (i != 0) output << ',';
            output << route->vertices[i];
        }
        output << '\n';
    }
    if (!output) throw std::runtime_error("output failure");
}
}

```

Command-line boundary

```

#include "routes.hpp"
#include <exception>
#include <iostream>
int main() {
    try {
        const auto request = harbor::parse(std::cin);
        harbor::write(std::cout, harbor::shortest_route(request));
        std::cout.flush();
        if (!std::cout) throw std::runtime_error("output failure");
        return 0;
    } catch (const harbor::InputError& error) {
        std::cerr << "error: " << error.what() << '\n';
        return 2;
    }
}

```

```

    } catch (const std::exception& error) {
        std::cerr << "error: " << error.what() << '\n';
        return 3;
    }
}

```

The process returns zero for both a route and an unreachable result. Input errors return two; other caught standard exceptions return three. A successful output call is followed by a flush and another stream-state check. This detects reported local output failure, not whether a downstream person or remote storage system has durably received the result.

Validate a path independently of the priority queue

The test oracle uses an all-pairs dynamic-programming calculation on tiny graphs. It initializes each diagonal to zero, keeps the minimum weight for parallel direct edges, and considers each vertex as an allowed intermediate. Comparing all source/target pairs on generated graphs checks more than one preferred route from one source.

A witness checker separately verifies the returned endpoints, vertex bounds, absence of repeated vertices, existence of every consecutive edge, and agreement between the sum of minimum parallel-edge weights and the reported distance. The generated graph sizes and weight limits make the oracle's arithmetic safe. The oracle is intentionally not a replacement for the scalable application algorithm.

Failure-injection tests interrupt input at every position in one bounded valid request, including the attempted read after its last byte. Each interruption must raise an input error and preserve the previously accepted request. A separate stream with exception masks enabled tests ordinary end-of-input handling. An output buffer accepts only a short prefix, then reports failure; the test confirms both the exception and the retained prefix.

```

#include "routes.hpp"
#include "course_test.hpp"
#include <algorithm>
#include <limits>
#include <random>
#include <sstream>
#include <streambuf>
#include <string>
#include <utility>
using harbor::Cost;
constexpr Cost inf = std::numeric_limits<Cost>::max();
std::vector<std::vector<Cost>> reference(const harbor::Graph& graph) {
    const auto n = graph.size();
    std::vector<std::vector<Cost>> d(n, std::vector<Cost>(n, inf));
    for (std::size_t i = 0; i < n; ++i) {
        d[i][i] = 0;
        for (const auto& edge : graph[i])
            d[i][edge.to] = std::min(d[i][edge.to], edge.weight);
    }
    for (std::size_t k = 0; k < n; ++k)
        for (std::size_t i = 0; i < n; ++i)
            for (std::size_t j = 0; j < n; ++j)
                if (d[i][k] != inf && d[k][j] != inf)
                    d[i][j] = std::min(d[i][j], d[i][k] + d[k][j]);
    return d; // bounded tiny test graphs keep this arithmetic finite
}

```

```

void witness(const harbor::Request& request, const harbor::Route& route) {
    CHECK(!route.vertices.empty());
    CHECK(route.vertices.front() == request.source);
    CHECK(route.vertices.back() == request.target);
    CHECK(route.vertices.size() <= request.graph.size());
    Cost sum{};
    std::vector<bool> seen(request.graph.size());
    for (std::size_t i = 0; i < route.vertices.size(); ++i) {
        const auto vertex = route.vertices[i];
        CHECK(vertex < request.graph.size()); CHECK(!seen[vertex]);
        seen[vertex] = true;
        if (i == 0) continue;
        Cost edge_cost = inf;
        for (const auto& edge : request.graph[route.vertices[i - 1]])
            if (edge.to == vertex) edge_cost = std::min(edge_cost, edge.weight);
        CHECK(edge_cost != inf); sum += edge_cost;
    }
    CHECK(sum == route.distance);
}

class FailingInput : public std::streambuf {
    std::string text_;
    std::size_t position_{};
    std::size_t fail_at_;
    int_type underflow() override {
        if (position_ == fail_at_) throw std::ios_base::failure("injected");
        if (position_ == text_.size()) return traits_type::eof();
        return traits_type::to_int_type(text_[position_]);
    }
    int_type uflow() override {
        const auto value = underflow();
        if (!traits_type::eq_int_type(value, traits_type::eof())) ++position_;
        return value;
    }
public:
    FailingInput(std::string text, std::size_t position)
        : text_(std::move(text)), fail_at_(position) {}
};

class PrefixOutput : public std::streambuf {
public:
    std::string retained;
private:
    int_type overflow(int_type c) override {
        if (traits_type::eq_int_type(c, traits_type::eof()))
            return traits_type::not_eof(c);
        if (retained.size() == 5) return traits_type::eof();
        retained.push_back(traits_type::to_char_type(c)); return c;
    }
};

int main() {
    std::mt19937 random{901};
    for (int trial = 0; trial < 100; ++trial) {
        const std::size_t n = 1 + random() % 8;
        harbor::Request request{harbor::Graph(n), 0, 0};
        for (std::size_t from = 0; from < n; ++from)
            for (std::size_t to = 0; to < n; ++to)
                if (random() % 3 == 0)
                    request.graph[from].push_back({to, random() % 11});
        const auto expected = reference(request.graph);
        for (std::size_t from = 0; from < n; ++from)
            for (std::size_t to = 0; to < n; ++to) {
                request.source = from; request.target = to;
                const auto route = harbor::shortest_route(request);
                CHECK(route.has_value() == (expected[from][to] != inf));
                if (route) { CHECK(route->distance == expected[from][to]);
                    witness(request, *route); }
            }
    }
}

```

```

    }
    const std::string good = "4 4 0 3\n0 1 8\n0 2 1\n2 1 1\n1 3 1\n";
    std::stringstream initial(good);
    auto accepted = harbor::parse(initial);
    const auto original = accepted;
    CHECK(harbor::shortest_route(accepted)->distance == 3);
    for (std::size_t at = 0; at <= good.size(); ++at) {
        FailingInput buffer(good, at); std::istream input(&buffer);
        CHECK(course::throws<harbor::InputError>([&] { harbor::load(input,
accepted); }));
        CHECK(accepted == original);
    }
    std::stringstream exceptional(good);
    exceptional.exceptions(std::ios::failbit | std::ios::badbit);
    CHECK(harbor::parse(exceptional) == original);
    for (const std::string bad : { "", "0 0 0 0", "1 0 0 0 x", "1 1 0 0 0 0 -1",
        "129 0 0 0", "1 4097 0 0", "1 0 1 0",
        "1 1 0 0 0 0 1000001", "1234567890123" }) {
        std::stringstream input(bad);
        CHECK(course::throws<harbor::InputError>([&] { harbor::load(input,
accepted); }));
        CHECK(accepted == original);
    }
    std::vector<std::string> edges{"0 1 1", "0 2 1", "1 3 1", "2 3 1"};
    std::optional<harbor::Route> chosen;
    do {
        std::string text = "4 4 0 3\n";
        for (const auto& edge : edges) text += edge + '\n';
        std::stringstream input(text);
        const auto route = harbor::shortest_route(harbor::parse(input));
        if (!chosen) chosen = route; else CHECK(route == chosen);
    } while (std::next_permutation(edges.begin(), edges.end()));
    PrefixOutput buffer; std::ostream sink(&buffer);
    CHECK(course::throws<std::runtime_error>([&] { harbor::write(sink, chosen); }));
    CHECK(buffer.retained == "dista"); // External output is not rolled back.
    CHECK(course::throws<std::invalid_argument>([&] {
        harbor::shortest_route({harbor::Graph(1), 1, 0});
    }));
    course::report();
}

```

Build, exercise, and interpret

From the package root:

```

cmake -S source -B build/routes -DCMAKE_BUILD_TYPE=Debug
cmake --build build/routes --target harbor_routes harbor_routes_tests --parallel 2
ctest --test-dir build/routes -R '^harbor_routes_regression$' --output-on-failure
printf '4 4 0 3\n0 1 8\n0 2 1\n2 1 1\n1 3 1\n' |
    build/routes/supplemental/harbor_routes

```

The expected output is:

```

distance=3
path=0,2,1,3

```

The exact fixture suite additionally covers disconnected and directed graphs, parallel edges, zero cycles, ties, maximum accepted counts, malformed numbers, missing fields, and trailing fields. Read `source/`

`supplemental/fixtures/cases.json` for the complete inputs and expected process results. Its thirty command-line cases are separate from the one in-process regression case.

For a useful extension, add multiple queries over one immutable accepted graph. Separate graph parsing from query parsing and state which preparation can be reused. Retain an independent oracle for small inputs. Do not add negative weights while retaining Dijkstra's proof, and do not label repeated queries as concurrent execution unless synchronization and ownership have actually been designed and tested.

Appendix D — Working Glossary and Acceptance Review

Vocabulary for reasoning about algorithms

Abstraction. A model exposing relevant operations and guarantees while hiding some implementation details. A queue abstraction defines FIFO behavior; a ring buffer is one possible representation of it. The abstraction should not promise guarantees the representation does not maintain.

Amortized cost. A bound on the total cost of a sequence of operations expressed per operation. It is not a probability claim and does not mean each individual operation has the same latency. Geometric dynamic-array growth is the recurring example in this book.

Asymptotic bound. A statement about growth as an input measure increases. It requires a chosen cost model and input-size definition. Big-O is an upper bound; it does not automatically specify a tight bound or a particular best, average, or worst case.

Borrow. Access to an object owned elsewhere without taking ownership of its lifetime. References, pointers, iterators, and spans can act as borrows. The borrower must obey both lifetime and invalidation rules; non-nullness alone is not a lifetime proof.

Certificate or witness. Additional information allowing a claimed result to be checked. A route's vertices can demonstrate feasibility and cost. A feasible route alone does not demonstrate that no cheaper route exists.

Commit point. The transition at which a completed candidate replaces accepted state. Preparing a candidate first can preserve the old state on failure. The guarantee must name which state is protected; input consumption and external output are separate effects.

Comparator equivalence. Two values are equivalent under an ordering when neither compares before the other. This can differ from `operator==`. Ordered containers and sorting stability must be understood in terms of the comparator actually supplied.

Complete search. An exploration that accounts for every solution allowed by its model, either directly or through justified reductions. A heuristic that finds a useful result quickly is not automatically complete.

Differential testing. Comparing multiple implementations or models on the same inputs. Independence matters: two implementations sharing an incorrect recurrence may agree while both solve the wrong problem.

Invariant. A property maintained at a designated boundary, such as before every loop iteration or after every public mutation. An invariant is useful when it explains what the representation means and supports the next operation, not merely when it restates that the program should work.

Invalidation. The loss of a borrow's permitted relationship to a container after an operation. Reallocation can invalidate vector iterators even if a high-level operation later reports failure elsewhere. Logical value preservation and borrow preservation are distinct guarantees.

Optimal substructure. A property allowing an optimal solution to be assembled from appropriate optimal subproblem solutions. The chosen state must contain every constraint affecting future decisions.

Partition boundary. A position dividing a sequence according to a predicate. Lower-bound search returns such a boundary; it does not by itself prove that a matching element exists at that position.

Pruning. Omitting a search branch because a justified condition proves that it cannot contribute a required solution. Changing candidate order is a heuristic ordering decision, not necessarily pruning.

Stability. Preservation of relative order among records equivalent under the sorting comparator. It must be observed with identities or additional fields; sorting equal integers cannot expose an exchange of their indistinguishable positions.

State sufficiency. The property that equal state descriptions imply equal future subproblems under the fixed context. A memoization key that omits remaining inventory can violate this property even when its numeric amount matches.

Strict improvement. Replacing a candidate only when its objective value is better, not merely equal. In HarborRoutes this rule supports a simple deterministic predecessor policy and avoids rewriting equal-distance ancestry casually.

Tombstone. A deleted-slot marker that preserves an open-addressing probe path. It is available for future insertion but does not mean that lookup can stop as it would at a never-used slot.

Review an implementation before relying on it

Write the public contract in ordinary language and make the error model visible. State the accepted sizes, duplicate policy, ordering assumptions, graph direction, arithmetic bounds, and ownership relationships. Include the meaning of empty input and missing results. A precise small contract is more useful than a broad promise the implementation cannot defend.

Trace the representation through one successful operation and one rejected operation. Identify the invariant before and after each mutation. For ownership-changing code, draw which object owns each allocation and which pointers merely observe it. For a loop, name the progress measure and termination condition.

Inspect the tests as evidence rather than as a percentage. Look for independent expected results, edge cases, operation histories, failure injection where appropriate, and returned witnesses. Record the compiler and configuration under which they ran. A large count of repeated easy checks does not compensate for one omitted critical transition.

Finally review performance at the correct boundary. Include preparation and conversion costs when the application pays them. Separate correctness instrumentation from release measurement. A proposed optimization should preserve the required values and failure behavior, improve a relevant measured outcome, and leave a maintainable implementation. When the evidence does not justify a change, retaining the simpler version is a legitimate engineering result.

Primary references and further reading

Language references use the C++20 working draft N4861 and its HTML rendering. This preserves the selected language baseline instead of silently linking a rule to a changing future draft. Compiler and build documentation can change independently; consult the installed version for unsupported or altered options. The links below support the chapter mechanisms. Original examples, proofs, scenarios, and exercises are not reproduced from these sources.

C++20 working draft N4861 — Container adaptors [container.adaptors]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Algorithms library [algorithms]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

Clang AddressSanitizer documentation

Original project or institution documentation. [Open the primary reference.](#)

C++20 working draft N4861 — Associative-container requirements [associative.reqmts]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Binary-search operations [alg.binary.search]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Time library [time]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

CMake build-system manual

Original project or institution documentation. [Open the primary reference.](#)

C++20 working draft N4861 — Deque [deque]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Construction, destruction, and exception cleanup [except.ctor]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Numeric parsing with from_chars [charconv.from.chars]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Function declarations [dcl.fct]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Fundamental types [basic.fundamental]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

GCC: C dialect and language-standard options

Original project or institution documentation. [Open the primary reference.](#)

GCC: optimization options

Original project or institution documentation. [Open the primary reference.](#)

GCC: options controlling the kind of output

Original project or institution documentation. [Open the primary reference.](#)

C++20 working draft N4861 — Hash requirements [unord.hash]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — General principles [intro]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Iterator operations [iterator.operations]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Lambda expressions [expr.prim.lambda]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Object lifetime [basic.life]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Numeric limits [numeric.limits]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — List [list]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Minimum and maximum operations [alg.min.max]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

MIT 6.006, Spring 2020: algorithm lecture directory (further reading)

Original project or institution documentation. [Open the primary reference.](#)

Microsoft C++: selecting the language-standard version

Original project or institution documentation. [Open the primary reference.](#)

C++20 working draft N4861 — Nonmodifying sequence operations [alg.nonmodifying]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Generalized numeric operations [numeric.ops]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Optional objects [optional]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Queue and priority-queue adaptors [queue]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Set operations on sorted structures [alg.set.operations]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Sorting and related operations [alg.sorting]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Tuples [tuple]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

Clang UndefinedBehaviorSanitizer documentation

Original project or institution documentation. [Open the primary reference.](#)

C++20 working draft N4861 — Unique-ownership pointers [unique.ptr]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Unordered associative requirements [unord.req]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Utility components [utility]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Vector [vector]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Vector capacity [vector.capacity]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)

C++20 working draft N4861 — Vector modifiers [vector.modifiers]

C++ standard working draft; HTML rendering. [Open the primary reference.](#)